



Information Propagation Methods for Reliability, Reachability and Flow Analysis in Directed Acyclic Process Networks

Temitope Ohiani

Department of Civil and Environmental Engineering
University of Strathclyde, Glasgow

A thesis submitted in fulfilment of the requirements
for the degree of Doctor of Philosophy

August 2026

Supervised by Edoardo Patelli and Caroline Pyke

Declaration of Authenticity and Author's Rights

This thesis is the result of the author's original research. It has been composed by the author and has not been previously submitted for examination which has led to the award of a degree.

The copyright of this thesis belongs to the author under the terms of the United Kingdom Copyright Acts as qualified by University of Strathclyde Regulation 3.50. Due acknowledgement must always be made of the use of any material contained in, or derived from, this thesis.

Signed: _____

Date: _____

Previously Published Material

Parts of this thesis draw on the following work by the author, submitted in accordance with Regulation 265:

- T. Ohiani and E. Patelli, “The Information Propagation Method for Efficient Network Reliability Analysis,” in *2023 7th International Conference on System Reliability and Safety (ICSRS)*, Bologna, Italy: IEEE, 2023, pp. 580–584, doi: 10.1109/IC-SRS59833.2023.10381157. The information propagation method introduced there is developed further in Chapters 4 and 5.
- T. Ohiani and E. Patelli, “Information Propagation Algorithm for Exact Reliability Analysis in Directed Acyclic Process Networks,” submitted to *Reliability Engineering & System Safety*, 2026 (under revision). Its material on exact reliability under interval and probability-box inputs is developed in Chapter 5 and its validation.

For each of these publications the author conceived and developed the methods, implemented all software, designed and carried out all experiments and validation, and prepared the manuscripts; the co-author provided supervision, guidance on framing, and revision of the text.

Signed: _____

Date: _____

Abstract

Many civil infrastructure systems are process systems: a resource enters at one end, passes through connected stages, and leaves at the other. Where the direction of flow is fixed, it is a directed acyclic graph. An engineer asks three questions of it: with what probability supply reaches a point, how much it can deliver, and when a dependent sequence completes. Each question has its own method, its own model, and an assumption of exact inputs that real failure probabilities, capacities and durations rarely meet.

This thesis develops the Information Propagation Framework: one directed acyclic graph model supports reliability, capacity and schedule analysis; uncertain inputs are carried in the form they are stated in wherever the mathematics permits; and the analyses are delivered as a registered Julia package with a no-code interface that runs on the user's own machine. The three toolkits share one decomposition that identifies every reconvergent subgraph once.

The probability toolkit computes exact reachability reliability, returning the exact range for interval inputs and sound distributional bounds for probability boxes, validated against a decision-diagram oracle and two published benchmarks. The flow toolkit answers six questions about deliverable throughput, exact for point-valued capacities and validated against an external solver and exhaustive enumeration. The critical-path toolkit generalises the critical path method to a family of operator pairs and computes exact interval floats, validated against a path oracle and a scheduling benchmark. The framework is then applied end to end to Net3, a real water distribution network.

The thesis shows that one model can serve the three analyses without loss, that

exact reliability under interval and probability-box inputs is practical at moderate re-convergence, that exactness costs the same structural width as the established exact methods, and that the analyses reach a non-programming engineer without the system model leaving that engineer's machine.

Acknowledgements

This work was supported by the Nuclear Decommissioning Authority (NDA) in conjunction with the National Nuclear Laboratory (NNL) through the NDA Bursary Scheme (project number NNL/UA/118).

Firstly, thanks to both of my supervisors, Edoardo Patelli and Caroline Pyke, for their moral support and encouragement throughout this project. Edoardo, thank you especially for taking a chance on me: for looking past a background in robotics and software and seeing the value I could bring to this work, and for nurturing that. Thank you for your encouragement through the many bouts of imposter syndrome, and for your confidence in my research, especially through the middle years of this PhD when I needed it most.

Elizabeth De Braal, my mentor, gave so many last-minute meetings and so many encouraging words that I owe her a particular thank you.

Ander Gray's work on `ProbabilityBoundsAnalysis.jl`, and his help with the probability-box propagation in this thesis, made a real difference to this work, and I am grateful to him for both.

My research group carried me through more debugging investigations than I can count, with encouraging words, a ready ear, and suggestions and fixes offered freely in our group meetings.

To my friends: thank you for your support and understanding as I worked through this journey, for your accommodation of it, and for the words of encouragement and the check-ins along the way.

To my family: my mum, for always encouraging a love of learning and for being so

wonderfully supportive of me pursuing this degree; my sister, for being the proudest sister I could have asked for on this journey; and the rest of my family, for your support throughout.

Finally, to my fiancé Kameron: I would never have considered moving to Scotland, let alone undertaking this PhD, were it not for you. Thank you for being my partner in everything, through the many sleepless and stressful nights, for the celebrations and the gifts to cheer me up along the way, and for being willing to put life plans on hold so that I could have the time I needed to finish this work.

Contents

Declaration of Authenticity and Author's Rights	i
Previously Published Material	ii
Abstract	iii
Acknowledgements	v
List of Figures	xiv
List of Tables	xx
List of Algorithms	xxiv
Nomenclature	xxvii
1 Introduction	1
1.1 Motivation	2
1.2 Problem Statement and Research Questions	4
1.3 Objectives	5
1.4 Contributions	6
1.5 Outline of the Thesis	8
1.6 Publications	9
2 Background	11
2.1 Introduction	12

Contents

2.2	Network Reliability Computation	12
2.3	Network Flow	14
2.4	Scheduling on Networks	15
2.5	Representing Uncertain Inputs	16
2.5.1	Intervals and probability boxes	17
2.5.2	Neighbouring theories	18
2.5.3	Imprecision in the three analyses	19
2.6	Multi-Analysis Tools for Complex Infrastructure	19
2.7	Delivery of Engineering Analysis Software	21
2.8	Graph Tooling Across Languages	23
2.9	Summary	23
3	Complex Processes as Directed Acyclic Graphs	25
3.1	Introduction	26
3.2	From Process to Graph	26
3.3	Node Roles	27
3.4	Definitions and Notation	28
3.5	Layered Order and Closures	30
3.6	Information on the Network	32
3.7	From File to Model	33
3.7.1	Network structure files	33
3.7.2	Analysis input files	34
3.7.3	The graph object	36
3.8	Assumptions and Limits	36
3.9	Summary	37
4	Network Decomposition	38
4.1	Introduction	39
4.2	Background	39
4.3	A Generic Decomposition Module	40
4.4	The Diamond Subgraph	41

Contents

4.5	The Identification Algorithm	44
4.5.1	Shared ancestry and the earliest covering fork	45
4.5.2	Recursive flow	46
4.5.3	Subgraph construction	47
4.5.4	Context-aware identity	49
4.6	Structural Correctness	50
4.6.1	Global completeness	50
4.6.2	Determinism	51
4.7	The Unique Diamond Object	52
4.8	Worked Examples	53
4.8.1	The induced diamond	53
4.8.2	A factorised join	53
4.8.3	A nested decomposition	54
4.9	Discussion	57
4.10	Summary	58
5	Information as Probabilities: Probability Propagation Toolkit	59
5.1	Introduction	60
5.2	The Belief of a Node	61
5.3	Propagation in a Single Pass	62
5.4	Reconvergence Breaks the Local Update	63
5.5	Conditioning at a Join	65
5.6	Supernodes and Progressive Decomposition	68
5.6.1	A multi-level worked example	69
5.7	Validation	72
5.7.1	The corpus and the oracle	72
5.7.2	Two published benchmarks	73
5.7.3	Timing convention	74
5.8	Complexity Analysis	74
5.9	Propagating Uncertain Probabilities	77

Contents

5.9.1	Interval inputs are propagated exactly	77
5.10	Probability-Box Propagation	79
5.10.1	The conditioning combination is not a convolution	79
5.10.2	Two dependence bounds, one proved and one open	81
5.10.3	What the bounds are worth	81
5.10.4	The cost of the distribution	83
5.11	Case Study: A Medical Drone Logistics Network	84
5.11.1	Inputs, traced to their source	84
5.11.2	Three designs and the redundancy boundary	85
5.11.3	What the propagated bounds say	87
5.12	Discussion	89
5.13	Summary	90
	Publications	90
6	Information as Resources: Capacity Flow Toolkit	92
6.1	Introduction	93
6.2	Background	94
6.3	The Flow Model	94
6.3.1	Several sources and sinks	96
6.3.2	Node capacities by node splitting	98
6.4	Maximum-Flow Solvers	101
6.4.1	Augmenting-path solvers	101
6.4.2	Push-Relabel Solver	103
6.5	The Questions the Toolkit Answers	104
6.5.1	The binding cut and its lattice of equivalent cuts	104
6.5.2	Redundancy and single points of failure	107
6.5.3	Throughput-limiting edges and their marginal values	108
6.5.4	Failure impact and capacity degradation	110
6.5.5	Degradation and upgrade thresholds	111
6.6	Cost	112

Contents

6.7	Validation	114
6.8	Case Studies	115
6.8.1	A worked example on a synthetic network	116
6.8.2	The IEEE Reliability Test System	127
6.8.3	Scale	129
6.9	Discussion	130
6.10	Summary	131
7	Information as Schedule and Cost: Critical Path Toolkit	132
7.1	Introduction	133
7.2	Background	134
7.3	Analysis Modes	134
7.4	The Propagation Kernels	135
7.4.1	The order-based kernel	135
7.4.2	The linear kernel	136
7.4.3	Cost	139
7.5	Interval Durations	139
7.6	Exact Interval Floats by the Domination Split	141
7.7	Validation	145
7.8	Case Studies	146
7.8.1	A benchmark project network	146
7.8.2	The boundary of the split	149
7.9	Discussion	149
7.10	Summary	150
8	Package Implementation in Julia	152
8.1	Introduction	153
8.2	The Language	153
8.3	Package Structure	154
8.4	The Graph Object	156
8.5	The Three Value Forms in Code	157

Contents

8.6	Caches and Memory	159
8.7	Validation Infrastructure	160
8.8	The Server	162
8.9	Distribution and Reproducibility	163
8.10	Limitations	164
8.11	Summary	164
9	The No-Code Web Interface	166
9.1	Introduction	167
9.2	Who the Interface Is For	167
9.3	The Workflow	168
9.4	What the Interface Enforces	175
9.5	Architecture	176
9.6	The Contract as the Boundary	177
9.7	Evaluation	179
9.8	Limitations	179
9.9	Summary	180
10	An End-to-End Application: One Network, Three Analyses	181
10.1	Introduction	182
10.2	The Network	182
10.3	Inputs	183
10.3.1	Reliability	183
10.3.2	Capacity	184
10.3.3	Schedule	185
10.4	Reliability	185
10.5	Capacity	189
10.6	Schedule	190
10.7	Two Result Sets on One Drawing	192
10.8	What the Run Showed About the Framework	194
10.9	Summary	196

11 Conclusions and Future Work	197
11.1 What Was Set Out and What Was Done	198
11.2 Answers to the Research Questions	198
11.2.1 One model, three analyses	198
11.2.2 Reconvergence identified once	199
11.2.3 Exact reliability under imprecision, and its cost	199
11.2.4 Schedule quantities under interval durations	200
11.2.5 Delivery without code and without leaving the machine	200
11.3 Contributions	201
11.4 Limitations	201
11.5 Future Work	202
11.6 Closing Remark	204
Appendices	205
A Case-Study Network Specifications	206
A.1 Networks at a glance	207
A.2 power-network (23 nodes, 27 edges)	208
A.3 grid-graph (16 nodes, 24 edges)	208
A.4 KarlNetwork (26 nodes, 74 edges)	209
A.5 counterexample-n15 (15 nodes, 23 edges)	209
A.6 PSPLIB j301_1 (32 activities, 48 precedences)	209
A.6.1 Durations and successors	209
A.7 IEEE RTS-24 (24 buses, 34 edges as oriented)	211
A.8 Medical drone delivery designs (217/263, 242/1753, 230/1648)	211
A.9 metro_directed_dag (306 nodes, 350 edges)	212
A.10 bnlearn structural benchmarks	212
A.11 Net3 (97 nodes, 119 edges)	212
A.12 The random and adversarial corpus	213

B Full Numerical Results	215
B.1 Timing convention	216
B.2 Probability propagation	216
B.2.1 Exactness on the 129-graph corpus	216
B.2.2 Named-network reproductions	216
B.2.3 Adversarial families	217
B.2.4 Interval and probability-box bounds	217
B.3 Capacity flow	218
B.3.1 Twelve-network validation summary	218
B.3.2 DIMACS-scale timing	218
B.3.3 Configuration comparison, sensitivity and min-cut enumeration	219
B.4 Critical path	219
B.4.1 Ten-network float validation	219
B.4.2 PSPLIB j301_1 per-node float bounds	220
B.5 Net3 integrated case study	220
B.6 Figure and table to script and data map	221
Bibliography	223

List of Figures

1.1	The layers of the Information Propagation Framework, with the value forms each analysis toolkit supports.	4
3.1	A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.	27
3.2	Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.	29
4.1	Induced diamonds. Every diamond path originates from the single fork and the subgraph carries one diamond join.	42
4.2	Non-induced diamonds. In (b) two diamonds overlap through the shared fork g , and in (c) parent p is itself the fork of the inner pattern.	42
4.3	Edge isolation at a nested join	48
4.4	A factorised join. Two correlated groups meet at join 7, one through each fork, and source 8 enters from outside both. Bold edges mark the two constituent diamonds.	53
4.5	Frame 1. The network, with forks outlined in bold and joins shaded. . .	54
4.6	Frame 2. The maximal diamond at join 8, with fork 1 fixed (filled) and the span covering the whole network.	55

List of Figures

4.7	Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 fixed, the inner diamond in bold, and edge (1, 3) cut.	56
4.8	Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.	57
5.1	The simple diamond. Source 1 feeds fork 2 (bold), whose two paths reconverge at join 5 (shaded).	64
5.2	The multi-level worked network. Sources are double-bordered, forks 1, 7 and 14 bold, joins 4, 6, 11, 12 and the target T shaded.	70
5.3	Maximum conditioning-set size against the logarithm of the sifted decision diagram's node count across the complexity-validation networks. Both quantities measure the same structural width.	75
5.4	Cost on the two adversarial families: the propagation's deduplicated subproblem count and the sifted decision diagram's node count against the family parameter, on a logarithmic vertical axis. Left, fan-in- k ; right, mesh- w	77
5.5	Cumulative distribution of one sink's belief under uniform component inputs on a twelve-node random DAG. The toolkit's band brackets the Monte Carlo distribution; the band from a convolution operator is shifted to higher reliability and places mass where the true distribution has none.	80
5.6	Tightness envelope of the propagated probability-box bound on the benchmark grid: band width against component uncertainty width, across input-distribution shapes and reconvergence regimes.	82
5.7	Reliability map of the dense decentralised design: each facility coloured by the lower bound of its propagated reachability (left, darker is less certainly reachable) and by the width of its reliability band (right, darker is more uncertain). Hubs, drawn as diamonds, are assumed always available. Uncertainty concentrates at island and far-northern facilities, and the worst-served facility, Islay Hospital, is bounded at $[0.56, 0.72]$	88
6.1	Minimal single-source single-sink capacitated DAG network.	95

List of Figures

6.2	Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.	97
6.3	Node-splitting transformation.	99
6.4	Runtime workflow for the flow-capacity toolkit with available modules. .	105
6.5	Single-edge sensitivity in Configuration A and cross-configuration rank comparison (A vs B).	121
6.6	Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.	124
6.7	Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.	125
6.8	Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.	126
6.9	IEEE RTS-24 oriented by DC power flow at peak load. Edge labels are line ratings in MVA; double-circuit pairs are one edge with the summed rating. Bold outlines mark the eleven generator buses, and filled nodes mark the four generators (buses 1, 2, 13 and 16) that are at their nameplate limit in the maximum flow found under the net-injection model. The bold line 7–8 has 4 MVA of headroom, the least of any line. Dashed lines form the nine-line binding cut of the structural-subset model. . . .	127
7.1	The running example. Node durations 1, 5, 2, 1. LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4.	137
7.2	Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.	142

List of Figures

7.3	PSPLIB j301_1. Bold nodes and edges form the LongestPath critical chain (completion 38); dashed nodes and edges form the ShortestPath chain (completion 18). The filled nodes are those the interval analysis of Figure 7.4 finds necessarily critical under ± 20 per cent duration uncertainty.	147
7.4	Exact float bounds on PSPLIB j301_1 with every duration at plus or minus 20 per cent. Black bars are the possibly critical nodes, whose lower bounds reach zero. Grey bars are the activities whose floats stay positive everywhere in the box. The necessarily critical nodes are drawn as points at zero, since both bounds of their float are zero.	149
8.1	Structure of the registered package. Input builds the graph object; GraphValidation and GraphTraversal support it internally; Diamonds, Probability, CriticalPath and Flow are the four analysis-facing modules described in Section 8.3.	155
9.1	The upload page. Left, the folder chosen, with the scenarios it holds and the analyses each supports listed before upload. Right, the expected folder layout.	169
9.2	The workflow a session follows: load the network, then read its structure, explore its reconvergence, and apply the three toolkits, before comparing scenarios.	169
9.3	The network page after upload: structural summary of the graph object and the network drawn by layer.	170
9.4	The reliability page, Belief tab. Top, the value-form filter and the scenario chips, each chip labelled with its value form. Below the tabs, summary tiles and the per-node belief table with its role filters.	171
9.5	A diamond's detail view: the subgraph, its conditioning set and local sources, the override fields for the local sources, and the two actions, run in isolation and analyse as a new network.	172

List of Figures

9.6	The flow page, configuration tab: scenario, solver, tolerance, the failure combination size, enumeration limits, and the optional target throughput and degradation multipliers.	173
9.7	The schedule page under an interval scenario. The value-form selector shows the form the file declares and marks the probability-box form unavailable for this toolkit; the advanced control exposes the pass modes; the result tiles carry the method tag.	173
9.8	The Compare tab: scenario selection, the run control for scenarios not yet run, and the side-by-side table with a baseline selector.	174
9.9	The system-profile page: the result sets available on the network, one selected and drawn by layer, with a second selectable for overlay.	174
9.10	The deployment, contained entirely within the user's machine. The front end reaches the analysis service over HTTP on localhost; the service alone calls the framework's modules and reads and writes the session's files and decomposition store.	178
10.1	The network page after upload: 97 nodes, 119 edges, 3 sources, 18 sinks, 35 forks, 23 joins and 28 layers, with the four scenario folders and the analyses each supports.	183
10.2	Reliability, deterministic scenario, filtered to the eighteen sinks: the belief at each, with the summary tiles above. Conditioning width 12; minimum belief 0.806 at node 76.	186
10.3	Reliability, interval scenario, the same eighteen sinks: each belief as a range. Mean band 0.363, maximum band 0.636 at node 76.	187
10.4	The Diamonds tab: 21 maximal diamonds, 307 unique, width 12. Each entry gives the conditioning set, size and local sources, and the number of nested sub-diamonds.	187
10.5	The detail view of the diamond at join 77: 46 nodes, 56 edges, conditioning set {95}, eleven sub-diamonds, and the controls to run it in isolation or promote it.	188

List of Figures

10.6	Flow summary, baseline: throughput 1,837.9 litres per second, unique minimum cut of four edges, no single point of failure, and the allocation per sink.	189
10.7	Flow bottlenecks, baseline: the four edges of the minimum cut with their flows, the lattice's empty free zone, and the per-edge redundancy scores.	190
10.8	Flow sensitivity and failure impact, baseline. The marginal range of edge 95 to 96 is unbounded. The k -edge table gives the combined loss of each pair of cut edges; the degradation table gives the capacity each edge can lose before a target of 1,654 litres per second is missed.	191
10.9	Flow summary, degraded scenario: 954.6 litres per second, a 48 per cent drop, with the same four-edge cut and five saturated edges.	191
10.10	Schedule, baseline, LongestPath: project value 1,773.3 hours, 28 critical and 27 near-critical activities, with the critical chain and the per-activity schedule quantities.	192
10.11	Schedule, interval scenario: project value [1,418.7, 2,128.0] hours. The method tag reads conservative enclosure and the banner states why the exact split was declined; 0 activities necessarily critical, 61 possibly critical.	193
10.12	Schedule, MaxScaling mode with the reliability probabilities as factors, showing the table rows around node 76: the best route in the network compounds to 0.9928 and the weakest best route reaches node 76 at 0.7515.	193
10.13	The schedule Compare tab across the four scenarios, with the baseline selected for differences. The interval row reports necessarily and possibly critical counts separately.	194
10.14	The profile page: the edges of the minimum cut (baseline flow) overlaid with the critical path (baseline schedule) on the network drawn by layer. Zero nodes in both.	195

List of Tables

3.1	Notation established by the network model.	29
3.2	Input contracts by layer and analysis class.	36
4.1	Fields of the unique diamond object	52
5.1	Conditioning trace of the worked network. w is the conditioning node's contextual belief, $\psi(1)$ and $\psi(0)$ the conditional signal probabilities the diamond's sub-runs return at the join, and $P(E)$ the group's recombined signal $w\psi(1) + (1-w)\psi(0)$	71
5.2	Exactness and structure on four named networks against the sifted decision-diagram oracle. Maximal and unique are the counts of maximal and unique diamonds; $\max C $ is the largest conditioning set; the last column is the worst per-node deviation from the oracle.	72
5.3	Reproduction of the published power-network reliabilities of Tong and Tien [32], sink 23, node priors 1, every link at R_l . The oracle column is the sifted decision diagram on the same topology.	74
5.4	Complexity on adversarial families. Operations are the propagation's deduplicated sub-problem count; nodes are the sifted decision diagram's size. Small instances were validated exact against path enumeration. . .	76
5.5	Certified bound on the requirement-violation probability $P(b(v) \leq 0.95)$ at the benchmark grid's target node, from one propagation, against the Monte Carlo sample count required to match the certified precision under worst-case a-priori planning.	83

List of Tables

5.6	Probability-box propagation time by discretisation level, on the fifteen-node reference network and on the 23-node power network. Identification is under 0.01 s at every level on both.	84
5.7	The three proxy designs: structure, conditioning width, and propagated interval reliability bounds. Every belief is contained in $[0, 1]$ with lower bound at most upper bound. The worst-served facility in both dense designs is Islay Hospital.	87
6.1	The three maximum-flow solvers.	101
6.2	Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm. . .	114
6.3	Configuration comparison summary.	117
6.4	Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore <code>sink_1</code> service under the node-capacitated solve.	119
6.5	Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).	120
6.6	Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).	122
6.7	Parametric threshold and marginal range comparison for key edges. . . .	123
6.8	Min-cut enumeration for Configuration A.	123
6.9	IEEE RTS-24 under the net-injection model at peak load.	129
6.10	Warm second-call solve times on DAG variants of the <code>genrmf</code> mesh family. Edmonds-Karp was not run above 50,000 edges.	130
7.1	The modes provided by the toolkit. In the three order-based modes the backward quantities are computed by the same fold on the reversed graph and the margin is the gap between the project value and the value through the node. In the accumulation mode the backward quantity is the adjoint, a path count.	135

List of Tables

7.2	LongestPath on PSPLIB j301_1: the critical chain and the three nearest non-critical activities.	147
7.3	Accumulation on PSPLIB j301_1 under a budget of 398.2 at the end node (current total 362). Allowance is headroom divided by path multiplicity. The first eight rows are the largest contributors; node 6 is appended below the rule because the text discusses its allowance despite a smaller contribution.	148
8.1	Modules of the registered package, the development name each carried, the chapter each realises, and their size.	155
8.2	Fields of the graph object and their types.	156
8.3	Value helpers over which the probability propagation is generic.	158
A.1	Every named case-study network, reproduction target and benchmark family in the thesis. Counts are of the directed acyclic graph after orientation. The drone designs use non-contiguous node identifiers; the count is of nodes actually present.	208
A.2	PSPLIB j301_1 activity durations and precedence successors. Dummy start (node 1) and end (node 32) carry duration 0 and are omitted. Source: <code>validation/critical-path/thesis_writing_pack/appendix_a_durations_successors.csv</code> , parsed directly from <code>j301_1.sm</code>	210
B.1	Reproduction of published exact reliabilities. The grid residuals are within the paper’s own five-decimal print precision. The two power-network residuals at $R_l = 0.9$ and 0.3 are order 10^{-3} , larger than print rounding; the toolkit and the decision diagram agree with each other to machine precision at all three points, so the gap is a modelling difference from the paper, flagged and not resolved. Scripts: <code>validation/probability/asce_grid_reproduction.jl</code> , <code>asce_power_reproduction.jl</code>	217

List of Tables

B.2	Flow toolkit validation. F^* is the maximum flow; every solver and every extended check agrees. Full per-check results: <code>validation/flow/flow_validation_summary.csv</code> and <code>flow_validation_extended_summary.csv</code> ; script: <code>validation/flow/run_flow_validation.jl</code>	218
B.3	Generated GENRMF directed-acyclic max-flow instances [80]. Warm second-call times, Dinic solver. Full table (all three solvers): <code>validation/flow/dimacs/timings.csv</code> ; generator: <code>validation/flow/dimacs/generate_genrmf_dag.py</code> (seeded).	219
B.4	Critical-path float validation. Script: <code>validation/critical-path/run_float_validation.jl</code> with <code>oracle.jl</code> . The interval-split engine is checked separately by <code>run_interval_validation.jl</code> and, on PSPLIB j301_1, by a 50,000-sample Monte Carlo soundness and attainment check (<code>run_mc_check_psplib_j301_1.jl</code> , 0 bound violations).	220
B.5	Net3 headline numbers. The interval schedule is the <i>conservative enclosure</i> : the exact domination split is screened out before any cost is computed because Net3’s 122 non-degenerate interval durations exceed the fixed threshold at which the split is attempted (a stated limitation, Ch. 7 and 11). Every request and response JSON, the run summary, and the full write-up with provenance are in <code>case-studies/net3/</code> ; driver: <code>net3-scenarios/run_net3_scenarios.py</code>	220
B.6	Every data-driven figure and validation table in the thesis, its generating script, and its source data, all relative to the archived data repository. Diagrams and diamond renders are regenerated from the network data, not archived; interface figures are screenshots and are not reproducible artefacts. Full logs and scripts are archived at the DOI above.	222

List of Algorithms

1	Layered order with ancestor and descendant closure	31
2	Recursive diamond identification (skeleton)	45
3	Ford-Fulkerson Min-Cut Max-Flow	102
4	Push-Relabel Min-Cut Max-Flow	104
5	Order-based kernel (one mode)	137
6	Linear kernel (accumulation mode)	138
7	Domination split for exact interval floats (LongestPath, point-valued edge values)	144

Nomenclature

Abbreviations

API	Application Programming Interface
BDD	Binary Decision Diagram
BFS	Breadth-First Search
CPM	Critical Path Method
DAG	Directed Acyclic Graph
DFS	Depth-First Search
DIMACS	DIMACS implementation challenge (maximum-flow benchmark series)
DOI	Digital Object Identifier
EPANET	Water distribution system modelling software (US EPA)
GIS	Geographic Information System
HTTP	Hypertext Transfer Protocol
IPF	Information Propagation Framework
JSON	JavaScript Object Notation
MC	Monte Carlo
MPM	Metra Potential Method
NDA	Nuclear Decommissioning Authority
NP	Nondeterministic Polynomial time (as in NP-hard, NP-complete)
OpenAPI	Machine-readable REST interface description format
PERT	Program Evaluation and Review Technique
p-box	Probability Box
PPA	Probability Propagation Algorithm
PSPLIB	Project Scheduling Problem Library
RTS	Reliability Test System (IEEE) ^{xxvii}
SPOF	Single Point of Failure
WNTR	Water Network Tool for Resilience

List of Algorithms

Units

MW megawatt

MVA megavolt-ampere

Chapter 1

Introduction

1.1 Motivation

Many civil infrastructure systems are process systems. They take a resource in at one end, transform or transport it through a sequence of connected stages, and deliver it at the other. A water treatment works, for instance, abstracts raw water from a river, passes it through screening, coagulation, filtration and disinfection, and pumps the treated water into a distribution network that carries it to consumers. Each stage depends on the stages upstream of it, and the output of the works depends on all of them. Systems of this kind, in which a resource moves through a sequence of stages towards a delivery goal, are referred to in this thesis as goal-oriented process systems. Where the direction of flow through such a system is fixed and there is no feedback in normal operation, the system can be modelled as a directed acyclic graph (DAG), with components as nodes and dependencies as directed edges.

Given such a model, several different questions can be asked of it, and three are addressed in this thesis. The first is a question of reliability: with what probability does supply reach a given point when components can fail? The second is a question of capacity: how much can the system deliver, and which components limit it? The third is a question of schedule: when does a dependent sequence of operations complete, and which operations can be delayed without delaying the whole? Each question has a method of its own. Network reliability analysis answers the first [1], the maximum-flow and minimum-cut theory of the 1950s answers the second [2], and the critical path method of the same decade answers the third [3]. The three methods have separate literatures and separate software, and each is normally applied to its own model of the system. However, the topology is the same in all three. What changes with the analysis goal is the meaning of the value attached to a component: a failure probability, a throughput limit or a duration. In the framework developed here the three analyses are therefore independent interpretations of one network model, each with its own inputs and its own algorithm, and the model is entered once.

The values attached to the components are rarely known precisely. Aleatory uncertainty is the inherent variability of a quantity, such as the time to failure of a pump

across a population of identical pumps, and is well described by a probability distribution. Epistemic uncertainty is a lack of knowledge about the quantity, and can in principle be reduced by collecting more data. For example, a failure probability estimated from three recorded failures, a capacity quoted as a nameplate rating with a tolerance, or a duration elicited from an engineer who will commit to a range but not to a number are all cases of the second kind. Describing epistemic uncertainty with a single value, or with a precise probability distribution, requires assumptions that the data do not support. Generalised probabilistic methods have been developed to avoid this. An interval bounds an unknown quantity between two values without assuming anything about where it lies between them. A probability box, or p-box, bounds an unknown distribution between a lower and an upper cumulative distribution function, and so carries partial statistical knowledge without fixing a distribution [4, 5]. However, all three methods named above consume a single value for each input. In practice an analyst therefore chooses a point value and treats the remaining uncertainty in a separate sensitivity study, if at all. This thesis instead propagates intervals and p-boxes through the network computation itself, so that the result carries the same kind of bound as the inputs. For the reliability question in particular this has rarely been done, because exact network reliability is $\#P$ -hard [6, 7] and every exact method in common use, from reliability block diagrams to binary decision diagrams, is built on point-valued component probabilities.

This thesis develops the Information Propagation Framework (IPF). The framework provides one DAG model of a system on which reliability, capacity and schedule analysis are computed, and it propagates interval and p-box inputs through each analysis as far as the mathematics of that analysis allows. It is provided as a Julia package together with a no-code user interface for non-technical users. Figure 1.1 shows the framework as a set of layers. The network model, the input processing that builds it and the decomposition module that identifies its reconvergent structure are built once per system. The three analysis toolkits read the same graph object. The probability toolkit propagates deterministic values, intervals and p-boxes; the critical path toolkit propagates deterministic values and intervals; the capacity toolkit is point-valued in the

present work. The package and the interface are the delivery layer. The three toolkits are the analyses developed in this thesis. A further analysis would read the same graph object.

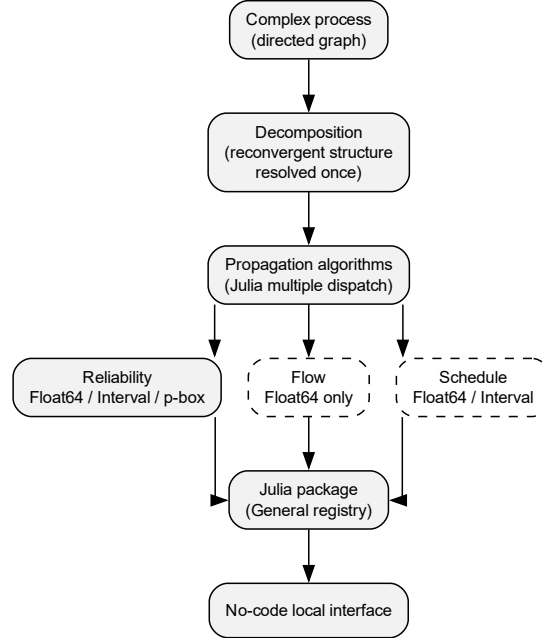


Figure 1.1: The layers of the Information Propagation Framework, with the value forms each analysis toolkit supports.

1.2 Problem Statement and Research Questions

Several engineering software tools already run more than one kind of analysis against one shared network object, as reviewed in Chapter 2. No tool or method found in that review combines source-to-node reliability, capacitated flow and activity scheduling on one network representation. The closest work pairs two of the three. Huang, Huang and Lin compute the exact probability that a project completes within a time and a budget constraint, which pairs reliability with schedule, from discrete point-valued activity states [8]. Tong’s dissertation investigates infrastructure flow networks in terms of both connectivity and flow capacity, which pairs reliability with flow, using a point-valued

propagation method throughout [9]. Furthermore, in neither case, and in none of the commercial or open-source tools reviewed, is an interval or a p-box propagated natively through the computation, and none of them is delivered in a form that a user can operate without writing code.

Following these considerations, the research questions addressed in this thesis are the following:

1. Can one DAG model of a complex process serve reliability, capacity and schedule analysis of the same system, so that the topology is entered once and read three times?
2. The redundancy that makes a system robust is also what makes its exact analysis expensive, because routes that diverge and reconverge share components. Can this structure be identified once, independently of any particular analysis, and reused?
3. Can exact source-to-node reliability be computed when component probabilities are given as intervals or as p-boxes, what is guaranteed of the result in each case, and what does the computation cost relative to decision-diagram methods?
4. Which of the quantities of critical path analysis remain exactly computable when durations are intervals, and at what cost, given that the general problem is NP-hard [10]?
5. Can these analyses be provided to an engineer who does not program, without the system model leaving the engineer's machine?

1.3 Objectives

The aim of this work is a framework for the analysis of complex processes modelled as DAGs, in which the treatment of uncertain inputs is chosen by the state of knowledge about the system. The limits of the tool should not force that choice. The objectives are:

- To define a network model general enough that reliability, capacity and schedule analysis are each an interpretation of the same object, and to build that object from the file formats in which infrastructure data commonly arrives.
- To develop exact analysis methods for each of the three interpretations, and to extend each to interval and p-box inputs as far as the mathematics of that interpretation allows.
- To validate every exact result against an independent computation, and to measure where each method stops being practical.
- To provide the framework as a documented Julia package and as a no-code user interface built on that package.

1.4 Contributions

The main contributions of this thesis are the following.

1. **A network model and its graph object.** A system is modelled as a finite DAG with sources, sinks, forks and joins. The graph object built from it carries, in addition to the topology, a layered topological order, the ancestor and descendant closure of every node, and the node roles, all computed in one pass. Every analysis in the framework reads this object. An input module builds it from edge lists and adjacency matrices, and attaches analysis inputs from a JSON contract that declares the value form (deterministic, interval or p-box) and the analysis class.
2. **A network decomposition module.** The module identifies every instance of a subgraph pattern in the network, with the diamond, a fork whose paths reconverge at a downstream join, as its default pattern. Each unique diamond is stored once, as a graph object of the same form as the network, and the identification is proved to be complete, deterministic and terminating. The probability toolkit is the consumer of this module in the present work.

3. **The Probability Propagation Toolkit.** Its algorithm, the Probability Propagation Algorithm (PPA), computes for every node the exact probability that the node operates and is reached from a source, in one pass over the layered order, resolving each reconvergence by conditioning on the diamond identified there. The same propagation is applied to deterministic, interval and p-box inputs. For deterministic inputs it returns the exact reliability; for intervals it returns the exact range, by a monotonicity argument; for p-boxes it returns sound bounds on the distribution of the reliability. The results are validated against binary decision diagrams, exhaustive path enumeration and Monte Carlo simulation on a corpus of 129 random networks, structured families, benchmark networks from the literature and a real medical logistics network.
4. **The Capacity Flow Toolkit.** For point-valued capacities the toolkit computes maximum throughput, minimum cuts, saturated edges, single points of failure, degradation and upgrade thresholds and path-disjoint redundancy, with every solution checked for feasibility, conservation and max-flow min-cut agreement.
5. **The Critical Path Toolkit, generalised over the propagated quantity.** The classical critical path method is one instance of a family of analyses defined by a pair of operators. A backward pass exists only for pairs whose algebra supports one. For interval durations the forward quantities are computed exactly by two runs at the corners of the input box. The floats are computed exactly by a domination split that reduces the enumeration from every uncertain duration to the durations that can bypass the node in question.
6. **Delivery as a package and an interface.** The framework is a Julia package, and the package is the server behind a browser interface that runs entirely on the user's machine. The interface accepts the same files the package reads and returns results in the same value form they were given in.

1.5 Outline of the Thesis

Chapter 2 reviews the representations of uncertainty used in engineering analysis and the exact methods for network reliability, flow and scheduling. It then examines the tools and methods that combine more than one of these analyses on one network, and the patterns in which engineering analysis software is delivered to its users.

In Chapter 3 the network model is defined. Components become nodes and dependencies become directed edges, the acyclicity assumption is discussed, and the roles of sources, sinks, forks and joins are given their physical meaning. The chapter then establishes the layered order and the closures that every later algorithm depends on, describes how a model is built from the heterogeneous files in which real systems are described, and states the assumptions that all later results are conditional on.

Chapter 4 presents the network decomposition module. The diamond subgraph is defined, the identification algorithm is described, and its completeness, determinism and termination are proved. Worked examples trace the algorithm on nested and factorised reconvergence.

Chapter 5 presents the Probability Propagation Toolkit. The chapter shows why a local update fails at a reconvergent join, proves that conditioning on the diamond repairs it, and extends the propagation to intervals and p-boxes. The method is validated against an independent exact oracle, its cost is analysed and measured against binary decision diagrams, and it is applied to a conceptual medical drone delivery network for Scotland [11].

Chapter 6 presents the Capacity Flow Toolkit, organised around the questions it answers: throughput, binding cuts, the most damaging failures, degradation and upgrade thresholds, redundancy and single points of failure. The chapter states the point-valued scope of the toolkit, demonstrates it on a synthetic worked example under two capacity configurations, and applies it to the 24-bus IEEE Reliability Test System as a real-network case study. Timings on generated instances of a DIMACS mesh family are also reported.

Chapter 7 presents the Critical Path Toolkit. Analysis modes are defined by operator

pairs, the forward and backward kernels are given, and the interval scheme and the domination split are proved. The toolkit is applied to a benchmark project network from the PSPLIB library under three interpretations of one set of inputs, and the exact interval floats are compared with the conservative enclosure on the same instance.

Chapter 8 describes the implementation of the framework in Julia: the types of the graph object, the value helpers over which the propagation is generic, and the server. It also records why a purpose-built graph object was preferred to the language’s general graph library, and how the package is delivered through the Julia General registry.

Chapter 9 presents the no-code interface, its local client and server architecture, the contract between the interface and the package, and the commitment that no data leaves the user’s machine.

In Chapter 10 the complete framework is applied to Net3, running reliability, capacity and schedule analysis on one model through the interface. Chapter 11 summarises the work and discusses the directions in which it can be extended.

1.6 Publications

The following papers have been published, submitted or are in preparation on the basis of the research reported in this thesis:

- Ohiani, T. and Patelli, E. (2023). “The Information Propagation Method for Efficient Network Reliability Analysis.” *2023 7th International Conference on System Reliability and Safety (ICSRS)*, 580–584. The method published under this name is the Probability Propagation Algorithm of the Probability Propagation Toolkit, Chapter 5.
- Ohiani, T. and Patelli, E. Journal paper on exact and imprecise network reliability by probability propagation, submitted to *Reliability Engineering & System Safety* (under revision).
- Ohiani, T. and Patelli, E. Conference paper on the no-code local interface, (in preparation for ESREL 2026).

Chapter 1. Introduction

- Conference paper on the Julia package, (in preparation).

Chapter 2

Background

2.1 Introduction

Chapter 1 introduced three questions that can be asked of a process system modelled as a directed acyclic graph, and named the method that answers each. This chapter reviews those methods and the literature around them. Sections 2.2 to 2.4 cover network reliability, network flow and scheduling in turn, with attention to the exact methods and to where their cost comes from. Section 2.5 reviews the representations available for uncertain inputs, defines the two that this thesis uses, and records how each of the three literatures has treated imprecision. Section 2.6 examines the tools and methods that run more than one of the three analyses on a single network, and Section 2.7 the forms in which engineering analysis software reaches its users. Section 2.8 notes the state of graph tooling in the languages the work could have been written in. The chapter closes with the gap that the rest of the thesis addresses.

2.2 Network Reliability Computation

The reliability of a network in this work is the probability that the network performs its function when its components can fail. For the kind of systems the framework addresses, the function is delivery from a source to a node, so the quantity of interest is the probability that at least one path from a source to the node has all of its components operating. Computing this quantity exactly is #P-hard in general [6, 7], so every exact method is exponential in some property of the network, and the methods differ in which property that is.

The earliest formulations are structural. A reliability block diagram arranges components in series, parallel and k -out-of- n groups, each of which has a closed-form system reliability [1]. Fault trees and event trees build the failure mechanism explicitly as a tree of logic gates [12]. Both are efficient as far as the failure logic reduces to a tree or a series-parallel structure. However, both stop being efficient at the same point, which is where routes in the network diverge and reconverge with shared components upstream. This structure follows from the design of the system. A robust network provides more than one route to the same destination, and two routes to one destination begin at a

common component. Hence the redundancy that makes a system worth analysing is also the structure these formalisms cannot hold [13, 14].

Graph-theoretic methods compute on the network directly. Since reaching a node is the union of its path events, the minimal path sets and the minimal cut sets give the reliability exactly once the union is made disjoint, and recursive decomposition improves on raw enumeration by splitting the network into subproblems [15, 16]. However, the collections of paths and cuts grow combinatorially with the same redundancy that motivates the analysis. Series-parallel reduction collapses independent stretches of the network pair by pair [17], and where the network is not series-parallel, factoring conditions on a single component and recurses on the working and failed cases until the remainder becomes reducible [18–20]. Polygon-to-chain reductions widen what can be collapsed in one step beyond a plain series or parallel pair [21]. Tree decomposition generalises the idea to graphs that no sequence of such reductions can collapse. The network is arranged into small overlapping bags connected as a tree [22], and the cost of exact computation is then bounded by the width of that decomposition, whatever the size of the graph. This is what makes exact reliability feasible on networks as large as metropolitan transit systems [23]. Monte Carlo simulation does not depend on structure and handles any network, but it is not exact, and the sample count it needs grows large when failures are rare or the required precision is high [24].

Binary decision diagrams (BDDs) compile the Boolean structure function of the network into a canonical decision graph on which reliability is a single traversal [25, 26]. The approach handles systems with thousands of variables. However, its cost is concentrated in the size of the diagram, which depends on the order in which the variables are decided, and finding a good ordering is itself intractable in general [27]. Reconvergent fan-out is the hard case for a diagram, since intermediate nodes must remember which shared ancestors have already been decided, and on a densely reconvergent network the construction can exhaust memory under one ordering while staying compact under another. Diagram implementations store each distinct sub-diagram once, a technique known as hash-consing [28, 29], and the same technique is used in Chapter 4 for a different unit of structure.

The probabilistic-inference literature reaches the same computation from another direction. Reachability from a source is marginal inference on a directed graphical model, and the exact inference methods are conditioning and clustering. Cutset conditioning enumerates the states of a set of nodes whose fixing renders the remaining structure singly connected [30], and the junction-tree algorithm clusters the network into a tree of overlapping bags [31]. Both are exact and both cost exponential in a width of the graph, the cutset size for the one and the treewidth for the other. Message passing makes the same inference local and scalable when exactness is relaxed. The directed probability propagation method of Tong and Tien is a reliability-specific variant that tracks marginal and pairwise probabilities to approximate the dependence between paths [32]. It gains scale at the cost of approximation error and of pairwise bookkeeping that grows with the number of dependent interactions. Its sixteen-node benchmark grid is used again in Chapter 5.

Every method in this section, from the block diagram to the junction tree, takes a single number for each component probability. Section 2.5 will further discuss this.

2.3 Network Flow

Flow along capacitated edges is the oldest of the three analyses in its deterministic form. The maximum-flow minimum-cut theorem of Ford and Fulkerson states that the largest amount that can be sent from a source to a sink equals the smallest total capacity of any set of edges whose removal separates them [2, 33]. Hence the theorem gives a certificate of optimality, the minimum cut, alongside the flow itself, and the augmenting-path, blocking-flow and push-relabel families of algorithms that followed all compute both. Two further results from the same theory are used in Chapter 6. A node with its own capacity is handled by splitting it into an entry node and an exit node joined by an edge of that capacity, which preserves the correspondence between flows in the two networks. Any feasible flow decomposes into a set of source-to-sink path flows that sum to the total. The minimum cuts of a network are not in general unique, and their structure is characterised by the sets of nodes reachable from the source in the

residual graph and reachable to the sink in the reverse residual graph, between which every minimum cut lies. The Birnbaum importance of a component, its marginal effect on system performance, was defined in the reliability setting [34] and carries over to flow as the marginal value of a unit of capacity.

Deterministic flow analysis does not address what happens when the capacities themselves are uncertain. Two different approaches have been developed for this case. In robust optimisation an uncertain capacity is treated adversarially: the capacity is taken from a set of possible values, no probability is assigned within that set, and the network is evaluated against the worst case in the set. The robust maximum-flow problem is solvable in polynomial time, while the corresponding robust minimum-cut problem is NP-hard [35]. The stochastic-flow network reliability problem, introduced by Doulliez and Jamouille, holds each arc capacity as a discrete random variable and asks for the probability that the network can deliver a stated demand, computed by decomposing the space of feasible capacity configurations [36]. The family of methods that followed, state-space decomposition, boundary-point enumeration and sum-of-disjoint-products techniques, mirrors the path and cut-set methods of Section 2.2 closely, and the same formalism carries into project scheduling, where an activity’s duration and cost take the place of an arc’s capacity states (Section 2.6).

2.4 Scheduling on Networks

The critical path method models a project as a network of activities and computes the longest path through it. The project cannot finish earlier than that path allows, and the activities on it have no room to slip. Every other activity carries a float, equal to how far it can slip before it delays the project. The method and its float calculus date to the late 1950s [3]. The question of duration uncertainty was raised almost at once. PERT replaced each fixed duration with a three-point estimate and a distributional assumption [37], and its descendants in modern practice replace the assumption with Monte Carlo sampling.

A distribution-free treatment of the same uncertainty came much later. When each

duration is known only as an interval, the forward quantities remain easy, since the earliest completion of every node is a monotone function of the durations and its bounds are found at the two corners of the input box. However, the backward quantities change character. Deciding whether an activity is possibly critical is NP-complete, and computing float bounds is NP-hard [10], even on planar networks [38]. Exact methods exist for latest starting times and for one of the two float bounds [39], the consolidated picture of what is and is not tractable is given by Fortin, Zieliński, Dubois and Fargier [40], and the recognised tractable class is the series-parallel networks [41]. The backward quantities are hard because a float is the difference between two quantities, the project completion and the completion forced through the activity, which are both monotone in the durations but share the same inputs. Their difference is therefore not monotone, and the corner argument that works for the forward quantities does not apply.

Furthermore, none of the machinery is specific to time. The forward recursion combines incoming values and compounds them along edges, and any quantity with those two operations propagates the same way. With combination as maximum and compounding as addition the recursion is the critical path method. With other pairs it is cost roll-up or route selection. Max-plus linear systems theory treats the classical forward and backward passes as linear algebra over an idempotent semiring, with the backward pass arising as residuation [42]. A forward recursion exists for any pair of operations, but a backward one is a property the pair either has or lacks, and this is the discipline Chapter 7 adopts.

2.5 Representing Uncertain Inputs

The methods reviewed so far take a component failure probability, an edge capacity and an activity duration as known numbers. In practice each is estimated from sparse failure data, elicited from expert judgement, or quoted as a manufacturer's tolerance. Chapter 1 distinguished aleatory uncertainty, the variability of a quantity across a population or over repeated trials, from epistemic uncertainty, a lack of knowledge about a quantity that more data would reduce. Probability theory is the accepted framework

for the first. For the second it is less satisfactory, because it requires a precise probability to be assigned even when the evidence supports only a range, so that a quantity about which little is known is modelled in the same way as one that is known well. Generalised probabilistic methods have been developed to describe epistemic uncertainty with fewer assumptions, and they can be applied to any computational model without modifying it.

2.5.1 Intervals and probability boxes

An interval $[x, \bar{x}]$ states that a quantity lies between two bounds and states nothing else. It is the appropriate form when a datasheet gives a tolerance or when a value is known only to a measurement precision. Interval arithmetic returns an enclosure of the true range of a function over the interval inputs. The enclosure is exact only when each input variable appears once in the expression. Otherwise the arithmetic treats repeated occurrences as if they were independent, and the enclosure is wider than the true range. This is known as the dependency problem. For a function that is monotone in each of its inputs the exact range is available without interval arithmetic at all, from two evaluations at the lower and upper corners of the input box. This observation is used in Chapters 5 and 7.

A probability box, or p-box, bounds a distribution instead of a value. Following Ferson et al. [4], a p-box on a random variable X is a pair of cumulative distribution functions $\underline{F}$ and $\overline{F}$ with $\underline{F}(x) \leq \overline{F}(x)$ for all x , together with optional constraints on the moments and on the family of the distribution. A distribution F belongs to the p-box when $\underline{F}(x) \leq F(x) \leq \overline{F}(x)$ for all x , its mean and variance lie in the stated intervals, and it belongs to the stated family if one is given. When the two bounding functions coincide the p-box is a single distribution and represents aleatory uncertainty alone. When they are the step functions at the two ends of an interval the p-box is that interval and represents epistemic uncertainty alone. Between these two cases the p-box carries both. A p-box therefore covers the situation of a distribution whose family is known but whose parameters are not, and the situation of a distribution estimated from samples measured with limited precision.

Williamson and Downs established the arithmetic of p-boxes [5]. For two random variables with p-boxes A and B and a binary operation, the p-box of the result depends on what is known about their dependence. Under independence the result is a convolution of the bounding functions, evaluated on a discretisation of each into a fixed number of levels, so that the cost of every operation grows with the discretisation. Under no assumption about dependence the result is obtained from the Fréchet–Hoeffding inequalities, which bound the joint distribution of any pair of random variables by functions of their marginals alone. The Fréchet bound is valid whatever the true dependence and is therefore sound. For the same reason it is wider than the independence bound. Which of the two applies at a given step depends on whether the quantities being combined are functions of disjoint sets of underlying variables, and this question governs the p-box propagation of Chapter 5. Software for p-box arithmetic exists in R [43], Python [44] and Julia [45]; the Julia library is one of the libraries this framework builds on.

2.5.2 Neighbouring theories

Two other theories represent epistemic uncertainty without a precise probability assignment. Fuzzy set theory represents a quantity by a membership function, assigning each possible value a graded degree of membership instead of a frequency [46], and possibility theory develops the same idea into a pair of measures, possibility and necessity. Evidence theory, also called Dempster–Shafer theory, assigns belief mass to sets of values directly and combines evidence from independent sources without assigning a probability to individual outcomes [47]; its belief and plausibility functions bound the probability of an event from below and above. Both theories can be placed within the wider field of imprecise probability, and a Dempster–Shafer structure on the real line can be converted to a p-box and back. Neither theory is used in this thesis. The quantities propagated here are probabilities, capacities and durations. The information available on them in practice is stated directly as an interval or a p-box without translation into membership functions or belief masses. In addition, the operations the framework performs on its values are products, complements and weighted combinations, for which the p-box arithmetic above gives results that are again p-boxes, so the

same representation is carried from input to output without conversion.

2.5.3 Imprecision in the three analyses

The exact reliability methods of Section 2.2 all consume a point-valued component probability, and none of them propagates an interval or a p-box natively. For example, an interval query against a decision diagram means evaluating the diagram at chosen input corners, and there is no analytic route from a diagram to a bound on a distribution. In flow analysis, Section 2.3 showed the two established responses to uncertain capacities, the adversarial uncertainty set of robust optimisation and the point-valued discrete random variable of stochastic-flow reliability, and neither is an interval or a p-box in the sense above. In scheduling, the interval treatment of durations exists and its complexity is known, and the difficulty is confined to the backward quantities. Across the three analyses, the same small set of representations recurs, point values, distributions, adversarial sets and intervals, but no single representation is used consistently across all three. Each analysis developed its own treatment of uncertainty independently of the other two.

2.6 Multi-Analysis Tools for Complex Infrastructure

The same network is often analysed for several different behaviours, and several engineering software tools run more than one kind of analysis against a single network representation. **pandapower**, an open-source power-system tool, runs power flow, optimal power flow, state estimation, topological graph search and short-circuit calculation against one explicit graph of buses, lines and transformers, a range of analyses matched by commercial suites such as DlgSILENT PowerFactory and PSS/E [48]. The ArcGIS Utility Network models power, gas and water networks as a graph of features connected through topology and runs eight distinct trace types, connectivity, isolation, subnetwork discovery and loop detection among them, against that shared topology [49]. Neither tool’s analyses are source-to-node reliability, capacitated flow or activity scheduling, and neither propagates interval or p-box uncertainty.

Whether an existing tool or method combines these three analyses specifically, on one network representation, is a narrower question. A search for such a tool found three partial answers among tools and methods and one comparable piece of doctoral research, and none of the four meets the description fully.

Commercial reliability software has, in practice, paired reliability with flow. Several of the block diagram and fault tree suites in active use, ReliaSoft’s BlockSim, Isograph’s Availability Workbench and ITEM Software’s ITEM ToolKit among them, bundle a throughput or process-flow module alongside their reliability engine [50–52]. These modules model multi-flow-type capacity, bottlenecks and resource allocation on the same diagrams the reliability analyses use. None of their public documentation describes a project-scheduling or critical-path module, and all three represent component life and failure with classical distributions, fitted from data or specified by the analyst.

Open-source infrastructure simulation platforms have paired flow with a form of scheduling. InfraRisk integrates power-flow, water-hydraulic and traffic-flow simulators for interdependent power, water and road networks under a resilience framing [53]. Its recovery module sequences repair-crew actions with start and end times computed from travel time and repair duration, by heuristic prioritisation or by model-predictive-control optimisation. This sequencing is a logistics schedule, since it optimises the order and timing of repair actions against a performance-loss metric. It is not a critical-path computation over a fixed activity network. Similarly, its reliability side is not a source-to-node reachability computation. The hazard module derives point component-failure probabilities from exposure and intensity to drive Monte Carlo disaster scenarios, and the platform reports its results as simulated performance loss.

A third direction pairs reliability with schedule directly, inside project-network reliability itself. Modelling a project’s activities as the arcs of a multi-state network and asking for the probability that the project completes within both a time and a budget constraint is an established line of work. Huang, Huang and Lin compute this project reliability exactly. The space of feasible completion-time and budget combinations is split into subsets via the project’s upper and lower boundary points, each subset is evaluated by inclusion-exclusion, and the method is validated against full enumeration

on a thirteen-activity manufacturing project [8]. However, the method has no notion of a physical arc capacity carrying a transportable flow, since activity duration and cost stand in its place, and every input is a discrete state with a fixed probability.

Tong’s doctoral thesis comes closest to combining two of the three exactly. The author develops new methods for infrastructure flow networks and investigates their reliability in terms of both connectivity and flow capacity, using the probability propagation method reviewed in Section 2.2 [9]. Schedule and critical-path analysis are absent from it, and the probability treatment throughout is point-valued.

No candidate reviewed above combines all three analyses on one network. Commercial software pairs reliability and flow, with no schedule dimension. Open-source simulation platforms combine flow and a form of scheduling without formal reliability computation. Project-network theory brings together reliability and schedule; it has no flow network. Tong’s thesis reaches reliability and flow again, this time exactly, and omits schedule entirely. Every one of the four represents its uncertain inputs precisely or adversarially.

2.7 Delivery of Engineering Analysis Software

The gap between an analysis existing in a library and an engineer being able to use it is sometimes referred to as the last-mile problem [54, 55]. Engineering analysis software is generally delivered to its users in one of three ways, as a browser-based service, as a licensed desktop installation, or as a web-based geographic information system. None of these is what the business-software industry calls no-code, and this section reviews all four.

The no-code and low-code software category defines itself around business application development. A systematic review of 383 definitions across 306 publications from 2014 to 2025 characterises no-code platforms as visual, drag-and-drop environments that let a non-programmer build software without writing code, and low-code platforms as the same idea with a professional-developer audience layered on top [56]. The same review maps the identified platforms onto ten functional classes, such as enterprise re-

source planning, customer relationship management and business process automation, with commercial examples including Salesforce and UiPath. Engineering or scientific analysis does not appear among them.

Within engineering analysis, browser-delivered simulation as a service is one established pattern. SimScale, launched in 2013, runs computational fluid dynamics, finite element, thermal and electromagnetic analysis in the browser with no local installation [57], with the computation running in the vendor’s cloud. The licensed desktop suite is a second and older pattern. The commercial reliability software named in Section 2.6 requires no programming and runs as a local installation, so it has the local computation that SimScale lacks and lacks the browser delivery that SimScale has. Web-based geographic information systems are a third. A PRISMA-based systematic review of web GIS for hazard and infrastructure risk assessment screened 1,775 articles and admitted 65. It organises the field around themes including visualisation, user-interface design and decision support [58]. The platform of Repetto et al. is a representative member [59]. It delivers wind monitoring, forecasting and statistical risk mapping through a web GIS built on open geospatial software, and it is operated by port authorities who never touch the underlying numerical models. Most platforms in this category address monitoring and forecasting. They do not perform a structural reliability or component-failure computation. Similarly, network visualisation environments such as Gephi [60] and Cytoscape [61] make graphs visual and explorable but carry no quantitative reliability analysis. The analytical capability lives in libraries, e.g. in Matlab [62], R [43], Julia [45] and Python [44], which a non-programmer cannot use.

A separate obstacle is data sovereignty. The accessible form of modern software is the web application, and the usual web application is a cloud service, which raises security and custody questions for proprietary system models [63], and in domains such as nuclear decommissioning is not acceptable at all. Local-first software keeps the usability of the web while storing and computing its data on the user’s own machine [64]. SimScale is browser-based but not local; the desktop suites are local but not browser-based; the web GIS platforms are browser-based but run on a remote server. No tool found in this search combines browser delivery, no programming, and local computation

and data.

2.8 Graph Tooling Across Languages

General-purpose graph libraries exist in every language the work could have been written in. MATLAB has offered shortest-path, maximum-flow, centrality, connected-component and spanning-tree functions on its native graph classes since the mid-2010s [65], and R’s `igraph` provides the same over one shared graph object [66]. Python’s `NetworkX` covers simple, directed and multi-edge graphs over a single native numeric type [67], and Rust’s `petgraph` is a newer and smaller equivalent [68]. Julia’s ecosystem, centred on `Graphs.jl` and its satellite packages for weighted and property graphs, occupies the same position [69]. Dedicated reliability tooling is rarer. R carries a package built for structural reliability, computing system and survival signatures over a network’s topology [70]. No single Python package built for network reliability in the block diagram, path-set or decision-diagram sense was found; the closest candidate, `pgmpy`, is a general Bayesian-network toolkit. This is a gap in packaged tooling only, since network reliability is an actively reviewed field with applications across communications, transportation and energy distribution [71]. Furthermore, none of the packages named above, in any of the five languages, carries a native notion of an interval- or p-box-valued edge or node, because none is built for propagating uncertain quantities across a network. The reasons for choosing Julia, and the language features the implementation depends on, are given in Chapter 8.

2.9 Summary

Network reliability, network flow and scheduling analysis developed separately, each with its own exact methods and its own treatment of uncertain inputs. The exact reliability methods are all exponential in some width of the network and all consume a point-valued component probability. Interval and p-box representations of epistemic uncertainty exist, with an established arithmetic, but are not propagated natively by any of the exact methods reviewed. Several engineering software tools run more than

Chapter 2. Background

one analysis on one network object, but no tool, method or thesis found in this search combines source-to-node reliability, capacitated flow and activity scheduling on one network representation, and none of the closest candidates propagates imprecision. No engineering analysis tool found combines browser delivery, no programming and local computation. The framework developed in the following chapters addresses each of these points.

Chapter 3

Complex Processes as Directed Acyclic Graphs

3.1 Introduction

The framework rests on a single model of the systems it analyses. This chapter defines that model and the object built from it, which every later chapter reads. Section 3.2 describes how a process system is mapped to a directed acyclic graph and discusses the acyclicity assumption. Section 3.3 gives the physical meaning of the node roles. Section 3.4 states the definitions and notation used throughout the thesis. Section 3.5 establishes the layered processing order and the ancestry closures on which every algorithm in the framework depends. Section 3.6 describes the values that an analysis attaches to the network, the three interpretations of those values, and the three forms in which a value can be known. Section 3.7 describes how a model is built from the files in which real systems are described, and Section 3.8 states the assumptions that all later results are conditional on.

3.2 From Process to Graph

A component is any unit of the system that can carry a value of its own, such as a pump or a treatment stage. A dependency is the relation in which one component feeds, supplies or precedes another. The model represents each component as a node and each dependency as a directed edge. In a goal-oriented process system, one in which a resource moves through the components towards a delivery point, the relation between two connected components is already directed. For example, water leaves the intake and arrives at the pump, and the pump does not supply the intake. Hence the direction of an edge is read off the system.

The model further requires that the directed edges form no cycle. For many of the systems of interest this holds without further assumption. For example, treatment trains and project schedules move material or work strictly forward, and a dependency cycle in a schedule would be a planning error. For other systems, however, acyclicity is imposed by orienting the network. A distribution pipe can in principle carry water in either direction, but a network operating around its design flows has a dominant direction on every pipe, and orienting each edge by that dominant flow gives a directed

acyclic snapshot of the operating regime. The orientation excludes regimes far from the snapshot, such as flow reversal under unusual demand, recirculation loops and feedback control, and Section 3.8 returns to what that exclusion means for the results. Figure 3.1 shows the mapping on a small treatment process. The process duplicates its treatment stage, and the duplication appears in the graph as a pair of parallel routes that later reconverge, which is the structural pattern Section 3.3 names.

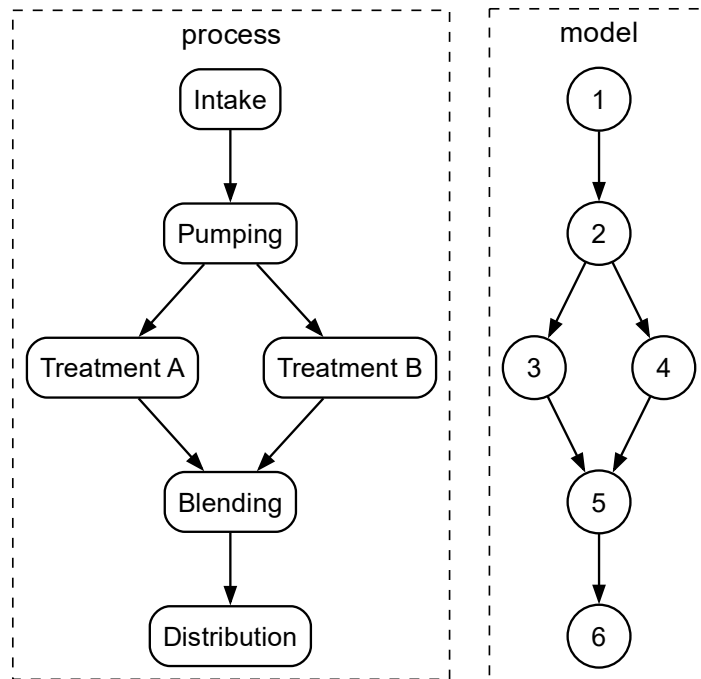


Figure 3.1: A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.

3.3 Node Roles

The position of a node in the network gives it a role, and each role has a physical meaning. A node with no incoming edge is a source, the point where supply, material or work enters the system (e.g. an intake or the start of a project). Similarly, a node with

no outgoing edge is a sink, where the system delivers (e.g. a demand point or a completed handover). A node that feeds two or more downstream components is a fork. Forks are how a system distributes and, in particular, how it builds in redundancy, because providing two routes to the same destination begins with one component supplying two. Finally, a node fed by two or more components is a join. This is where distributed flows gather and where redundant routes come back together.

A robust system duplicates its critical routes, and duplicated routes must reconverge at the destination they protect. A well-designed network therefore contains many pairs of paths that separate at a fork and meet again at a join while sharing the components upstream of the fork. Whether that sharing matters depends on the question being asked. Chapter 4 makes the pattern precise, and Chapter 5 shows that for the probabilistic interpretation of the model, ignoring it gives wrong answers. For the present chapter it is enough to note that reconvergence is common in the systems being modelled. Figure 3.2 shows it on a small network.

3.4 Definitions and Notation

Definition 3.1 (Network model). *A system is modelled as a finite directed acyclic graph $G = (V, E)$ with node set V and edge set $E \subseteq V \times V$. The sources $S \subseteq V$ are the nodes with no incoming edge and the sinks $T \subseteq V$ the nodes with no outgoing edge. For a node v , $\text{Pa}(v)$ denotes its parents, the nodes with an edge into v , and $\text{Ch}(v)$ its children.*

Definition 3.2 (Ancestry). *A node u is an ancestor of v when a directed path leads from u to v , and a descendant when one leads from v to u . The sets $\text{anc}(v)$ and $\text{desc}(v)$ collect them. Throughout the framework both are taken self-inclusively, so that $v \in \text{anc}(v)$ and $v \in \text{desc}(v)$.*

Definition 3.3 (Forks and joins). *The forks of G are $F = \{v : |\text{Ch}(v)| \geq 2\}$ and the joins are $J = \{v : |\text{Pa}(v)| \geq 2\}$.*

These definitions are the shared vocabulary of the thesis. The chapters that follow refer to them without restating them, and Table 3.1 collects the notation in one place.

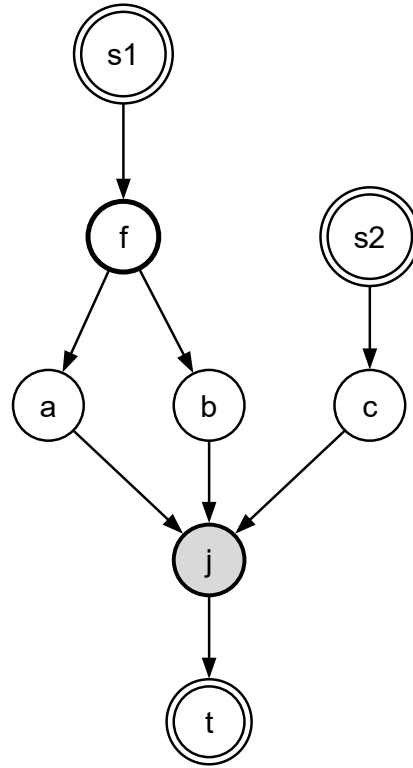


Figure 3.2: Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.

Table 3.1: Notation established by the network model.

Symbol	Meaning
$G = (V, E)$	the system as a finite DAG
S, T	sources (no incoming edge), sinks (no outgoing edge)
$\text{Pa}(v), \text{Ch}(v)$	parents and children of v
$\text{anc}(v), \text{desc}(v)$	ancestors and descendants, self-inclusive
F, J	fork nodes (fan-out ≥ 2), join nodes (fan-in ≥ 2)
$L = \{L_1, \dots, L_k\}$	layered order (Section 3.5)

3.5 Layered Order and Closures

All of the framework's analyses visit the nodes of the network in an order in which each node is processed after all of its parents. For a DAG such an order always exists. A topological sort of G is a linear ordering of V in which u precedes v for every edge $(u, v) \in E$. The framework uses a coarser form of the same order, in which nodes at the same depth from the sources are grouped into a layer.

Theorem 3.4 (Layered order). *For every finite DAG $G = (V, E)$ there exists a partition of V into non-empty layers $L = \{L_1, L_2, \dots, L_k\}$ such that, for every node $v \in L_i$, every parent of v lies in a layer L_j with $j < i$. An algorithm that processes the layers in order, and within a layer processes each node once, visits every node exactly once after all of its parents have been visited, and completes in exactly k layer steps.*

Proof. By induction on $|V|$. If $|V| = 1$ the single node is a source and $L = \{L_1\}$ with $L_1 = V$. Suppose the statement holds for every DAG with fewer than n nodes, and let G have n nodes. Since G is acyclic it has at least one source; let L_1 be the set of all its sources. Removing L_1 and the edges leaving it gives a DAG G' with fewer than n nodes, which by the inductive hypothesis has a layered order $\{L_2, \dots, L_k\}$ in which every parent within G' of a node in L_i lies in an earlier layer. A node of G' may also have parents in L_1 , and these lie in the earliest layer. Prepending L_1 therefore gives a layered order of G . The processing claim follows directly: when layer L_i is processed, every parent of every node in L_i lies in a layer already processed, so each node is visited once with its parents resolved, and there are k layers. $\square$

In the following chapters, every analysis is an exact method with no iterative convergence loop. Hence a fixed processing order declared once serves all of them, and the number of steps is known from the structure before any analysis runs. Nodes within a layer also share no dependency, so a layer is a natural boundary for parallel processing by any algorithm that supports it. The layers are computed by a depth-partitioned form of Kahn's algorithm, shown as Algorithm 1, which takes the sources as the first layer, removes them, and takes the nodes that become sources as the next. In the implementation the layers are called iteration sets.

Alongside the layers, later algorithms rely on cheap answers to the question of whether one node is an ancestor of another. The adjacency indices alone do not provide this, since a traversal per query would cost $O(|V| + |E|)$ each time. Algorithm 1 therefore accumulates the self-inclusive ancestor and descendant closure of every node during the same pass. As each node is dequeued, its ancestor set is passed to its children and it is recorded as an ancestor of each of them, and the descendant sets are populated in the same step. Computing the layers alone costs $O(|V| + |E|)$. Carrying the closures adds a set-union cost that depends on the transitive density of the graph, and in the worst case each node's closure contains every other node, giving $O(|V|^2)$ space. In return, any reachability query is one set membership test. The trade of space for time is justified by the number of such queries the decomposition and propagation algorithms make.

Algorithm 1 Layered order with ancestor and descendant closure

Require: DAG as edge list E (acyclicity validated by the parser), outgoing index Out , incoming index In

Ensure: layers L , anc, desc

```

1: compute indeg[v] for every node v
2: initialise anc[v] ← {v} and desc[v] ← {v} for every node v
3: enqueue every node with indeg = 0 into queue Q; L ← ⟨⟩
4: while Q not empty do
5:   ℓ ← |Q|                                     ▷ the current layer is the current queue
6:   Lcur ← ∅
7:   for i = 1 to ℓ do
8:     x ← dequeue(Q); add x to Lcur
9:     for all y ∈ Out[x] do
10:      anc[y] ← anc[y] ∪ anc[x]
11:      add y to desc[a] for every a ∈ anc[x]
12:      indeg[y] ← indeg[y] − 1
13:      if indeg[y] = 0 then enqueue y into Q
14:    end if
15:  end for
16: end for
17:  append Lcur to L
18: end while
19: return L, anc, desc

```

3.6 Information on the Network

The topology of the network and the values attached to it are held separately. The topology of Definition 3.1 is fixed once per system, and an analysis attaches values to its nodes and edges whose meaning depends on the question. Under the probabilistic interpretation a node's value is the probability that the component operates and an edge's value the probability that the handover between its endpoints succeeds, and the question is with what probability each node is reached from the sources despite component failures. This question is answered by the Probability Propagation Toolkit of Chapter 5. Under the capacity interpretation the values are throughput limits, and the question is how much the network can deliver, where the limits bind and what relieving them would gain. This question is answered by the Capacity Flow Toolkit of Chapter 6. Under the schedule and cost interpretation the values are durations or costs, of the component's own processing and of the transfer along each dependency, and the question is when the network completes, which dependencies set that outcome and which have room to move. This question is answered by the Critical Path Toolkit of Chapter 7. The three interpretations are three different analyses of one topology. Because the topology is common to them, a single system model answers questions that are usually asked of three separate models [13].

How well a value is known is a further property, independent of what it means. Chapter 2 defined the three forms the framework accepts. A deterministic value asserts full knowledge and is appropriate for design constants and well-measured components. An interval asserts only bounds and is appropriate when data is sparse, when a manufacturer quotes a tolerance, or when experts will commit to a range but not to a number. A probability box bounds a distribution and carries partial statistical knowledge, for example measurements sufficient to constrain a curve without fixing it. The framework propagates each form through the analyses that support it, as set out in Chapter 1, so that the choice among them can be made on the state of knowledge about the system.

Before any of the three questions can be asked, the model must be built from the descriptions in which real systems arrive, which is the subject of the next section. For

the probabilistic interpretation the reconvergent structure of Section 3.3 must also be identified, which is the subject of Chapter 4. Both are preprocessing steps performed once per system.

3.7 From File to Model

The data describing an infrastructure system is rarely held in one place or in one format. Different objectives are pursued on the same system, and each workflow maintains the parameters relevant to its own purpose. Furthermore, the tools used across these workflows implement different schemas and export conventions, and a dataset is often shaped to fit a particular tool. Beneath these differences, however, any DAG description of a system reduces to a node list and an edge list, in which nodes are point features such as junctions or demand points and edges are the relationships between them. Infrastructure formats such as GraphML, EPANET `.inp` and MATPOWER `.m` all reduce to node and edge records beneath their schemas, and the framework asks the user to perform that reduction, supplying the structure in one of the two forms described in Section 3.7.1. Values describing the behaviour of the system, such as a capacity or a failure probability, are a separate layer of information over that structure, and a source dataset may combine the two. The input module treats structure and analysis inputs as two layers of one model, parses each from its own file, and produces the graph object every analysis reads. It is the entry point of the framework, and no analysis can be invoked without it.

3.7.1 Network structure files

Components in a physical system usually have descriptive identifiers, such as `Pump_A` or `Substation_North`. However, the framework requires integer identifiers, because integers are compact to store and fast to index in the repeated lookups and set operations that every algorithm performs. The integers need not be contiguous, but the user is responsible for mapping domain identifiers to unique integers before parsing.

The network parser accepts an edge list or an adjacency matrix, in a `.edges` or a

.`csv` file, and detects which it has been given in stages. A `.edges` extension selects the edge-list reader directly. A `.csv` file is ambiguous, so the parser first scans the header for source and destination keywords that indicate an edge list. Failing that, it scans the content for the integer, delimiter, integer pattern of an edge record. If neither test succeeds, the parser treats a two-column table as an edge list and anything else as an adjacency matrix. An adjacency matrix is streamed row by row into an edge list. Header rows and comment lines are skipped in either case.

As edges are read, self-loops are rejected at once and cycle-forming edges are detected during ingestion, since both violate the acyclicity of Definition 3.1. For every valid edge both nodes and both directions of the relationship are recorded in the same pass, so that the incoming and outgoing indices are built together and no second pass over the edges is needed to construct the reverse adjacency. Ingestion is therefore linear in the number of edges. The parser returns the minimal graph, which is the edge list, the two indices and the sources.

3.7.2 Analysis input files

The values that drive an analysis are usually observed and managed separately from the network structure, in manufacturer specifications or planning estimates, and the framework keeps them in separate files. Embedding them in the structure file would produce a file that is large and hard to maintain. It would also force a user who wants to evaluate the same network under several interpretations, or under several scenarios of one interpretation, to keep a duplicate topology file for each.

All analysis inputs are given as JSON files. JSON is human-readable and platform-independent, and it maps directly onto the key-value form in which component attributes are described. A flat format such as CSV suits the rigid tabular shape of an edge list; JSON's nesting suits the values. An interval or a p-box is written as a nested object, without column conventions and without flattening a structured value into several cells. No specialist tool is needed. A data owner who knows the node and edge identifiers can contribute values without knowledge of graph theory.

Every analysis input file declares a `data_type`, which is `Float64`, `Interval` or a

probability-box construction, and the framework selects the deserialiser from that declaration. The analysis class is not declared in the file; it is fixed by the file's role, since each analysis reads its own input file from the session. Every key must map to a valid node identifier or edge identifier of the parsed graph, and every value must be parseable into the declared type. The parser is driven by the values and the declared type. Descriptive labels are ignored, so the JSON can carry whatever naming convention the data owner uses. Concentrating the deserialisation of uncertain values in one place, instead of in a separate reader per analysis, keeps the handling of intervals and p-boxes consistent across the framework.

The three analysis classes differ in what they accept. Probability inputs are given in two files, one carrying a prior for each node and one carrying a probability for each edge. All values must lie in $[0, 1]$, and values outside that range are rejected at parse time. This is the most expressive class and accepts all three data types. An interval is written as a lower and an upper value. When the file declares the `pbox` data type, each value is an object that names its own construction, so that values in one file may use different constructions. The supported constructions mirror the constructor families of the `ProbabilityBoundsAnalysis.jl` library: a scalar or an interval as a degenerate box, a parametric distribution with point or interval parameters, an envelope of distributions, or a distribution-free box from bounds on the moments. Flow capacity inputs are given in one file as a list of edge records, each carrying a source, a destination and a capacity, with an optional list of node capacities. An older nested form keyed by edge string is also accepted, and the parser selects the reader from the file's top-level key. Since capacity is point-valued in the present framework, only the `Float64` data type is accepted. Unconstrained capacity is written with an infinity token, and every capacity is checked to be non-negative and not NaN at parse time. Critical path inputs are given in one file with a time section carrying a duration for each node, a delay for each edge and an initial time, and an optional cost section of the same shape. This class accepts `Float64` and `Interval`. The framework is unit-agnostic for it: the values may be durations, costs or any other quantity that accumulates along a path, provided the interpretation is consistent within a run, and the analysis mode that reads them is

chosen at invocation as described in Chapter 7. Table 3.2 summarises the contracts.

Table 3.2: Input contracts by layer and analysis class.

Layer or class	Contract	Data types	Validation
Network structure	Edge list or adjacency matrix over integer node identifiers	Integers; directed edges	Format detected in stages; self-loops and cycles rejected
Probability	Node prior file and edge probability file, keyed to graph identifiers	<code>Float64</code> , <code>Interval</code> , p-box	Values in $[0, 1]$; <code>data_type</code> selects the deserialiser
Flow capacity	Edge records (source, destination, capacity); optional node capacities	<code>Float64</code> only	Non-negative and not NaN; infinity token accepted
Critical path	Time section (node durations, edge delays, initial time); optional cost section	<code>Float64</code> , <code>Interval</code>	Unit-agnostic; mode chosen at invocation

3.7.3 The graph object

The output of input processing is the framework’s graph object, assembled once from the validated outputs of the parser and of Algorithm 1. It carries the edge list, the outgoing and incoming indices, the sources, the sinks, the sorted node list, the fork and join sets, the layers, and the ancestor and descendant closures. These fields are built once and shared by every analysis. The analysis inputs, node priors and edge probabilities, edge and node capacities, and node and edge values, are attached to the same object per invocation. One instance of a network can therefore be evaluated under different interpretations and under different scenarios of one interpretation without rebuilding the structure. Every argument that any analysis in the framework requires can be taken from this object, and it is the point of handover from preprocessing to analysis. The concrete types chosen for its fields, and the reasons for building a purpose-specific object instead of using the language’s general graph library, are discussed in Chapter 8.

3.8 Assumptions and Limits

The model rests on four assumptions, and each is a limit on what the results mean.

1. Component behaviours are independent. A common cause that fails several components together is not represented, and where it matters it must be modelled explicitly as a shared upstream node.
2. Under the probabilistic interpretation a component is binary, operational or failed, and degraded intermediate states are outside the model.
3. The topology is static over the analysis horizon, so reconfiguration during operation is treated as a sequence of separate analyses.
4. The acyclicity of Section 3.2 excludes feedback and flow reversal. This is a property of the operating snapshot being analysed, since the physical assets may carry flow in either direction.

This list applies to every analysis in the following chapters, and none of the algorithms introduces an assumption beyond it. Where a later chapter relaxes or adds to the list for a particular interpretation, it says so.

3.9 Summary

A process system is modelled as a finite directed acyclic graph in which components are nodes, dependencies are directed edges, and the roles of source, sink, fork and join have physical meanings. The graph object built from the model carries the topology, a layered processing order whose existence Theorem 3.4 guarantees, and the ancestor and descendant closure of every node, all computed in one pass. An analysis attaches values to the object under one of three interpretations, probabilistic, capacity, or schedule and cost, in one of three forms, deterministic, interval or probability box. The input module builds the object from edge-list or adjacency-matrix structure files and JSON analysis files whose contracts were given in Section 3.7, validating acyclicity, identifier mapping and value domains at parse time. Four assumptions, listed in Section 3.8, apply to every later result.

Chapter 4

Network Decomposition

4.1 Introduction

Chapter 3 produced a validated graph object that carries the network topology together with its layered iteration sets and ancestry closures, and every analysis in the framework reads that object as it is. This chapter presents the Network Decomposition Module, a second pre-processing step which, when invoked, extends the graph object with the network's decomposition structure. Its objective is to identify every instance of a specified subgraph pattern in the input DAG. The default pattern is the diamond, the fork-and-join structure that Chapter 3 identified as the redundancy of engineered systems. In the present work the module is consumed by the Probability Propagation Toolkit of Chapter 5, which conditions on diamonds to compute exact reachability. The Critical Path Toolkit of Chapter 7 does not use it. Since each identified subgraph is returned as a graph object of the same form as the network, the module is available to any further analysis that a user adds.

Section 4.2 places the module among the decomposition strategies used in network analysis. Section 4.3 describes the three discriminators that define a pattern, and Section 4.4 defines the diamond in their terms. Section 4.5 presents the identification algorithm and proves its grouping, termination, edge-isolation and identity properties. Section 4.6 proves global completeness and determinism. Section 4.7 describes the unique diamond object, and Section 4.8 traces the algorithm on three small networks. Section 4.9 gives the module's limitations and the extension that follows from them.

4.2 Background

Real infrastructure networks are rarely trees. Their components connect to and depend on many others at once, and the interactions between them overlap instead of branching cleanly [13, 14]. Redundancy is one cause. A robust network supplies more than one route to the same goal, so paths diverge and reconverge. Process networks and hierarchical flows behave the same way, with a component feeding several downstream dependencies while itself depending on shared upstream outputs. As the system grows, overlapping paths accumulate.

Most graph-based analysis methods decompose. For large networks with complicated connectivity they must [72]. Broadly, decomposition breaks a network into substructures that are easier to handle than the whole, and methods differ in what they do with those substructures. Some strategies aim to reduce: a substructure is replaced by a single equivalent element, so the network shrinks while the analysis stays exact. Other decomposition methods work by partition: the network is divided into units of work sized for the solver, and the pieces are processed in turn. Beyond either, decomposition can give a more compact representation of the network, storing a repeated substructure once instead of every time it appears, without losing the overall structure. Depending on how the decomposed units are defined, they can also support localised investigation, such as confining a sensitivity study to one part of the system without needing to run analysis against the full network.

There is no universal unit of decomposition. Each method defines its own. In the series-parallel reduction reviewed in Chapter 2 the units are the series and parallel pairs themselves [17]. In factoring the unit is a single component, on which the method conditions to split the problem into the working and failed cases [19]. In tree decomposition the unit is a bag of nodes, and a tree of overlapping bags is the subproblem structure a dynamic program can process in a single sweep [22, 23].

The framework’s default decomposition unit is the *diamond*, a fork-join subgraph in which paths fan out from a common node and fan back in at a node further downstream. The decomposition is partition-forward, that is, each identified diamond is returned as a localised graph object of the same form as the full network and can be handed to any analysis module in the framework on its own. Secondly, a diamond can serve as a reductive unit, since a solved diamond can stand in for its subgraph as a single equivalent component. This is how the Probability Propagation Toolkit uses it.

4.3 A Generic Decomposition Module

The network decomposition module identifies and returns every instance of a subgraph pattern discovered in the input DAG. The specific subgraph pattern is decided by three

discriminator functions:

- an *eligibility* discriminator, which decides which nodes may take part in a pattern at all;
- an *anchoring* discriminator, which decides what structure a pattern forms around;
- a *membership* discriminator, which decides when two paths into a node belong to the same pattern.

The identification algorithm is the same whatever the discriminators say.

A discriminator can be defined on any property of the graph object, or on any network characteristic mapped to a property of it during input processing, and each discriminator can be overridden or modified to fit the required definition. For example, the membership threshold can be raised from two paths to three, or the anchoring structure can be changed from fan-out-then-fan-in to fan-in-then-fan-out. A discriminator can also be defined on a characteristic that is not already a graph object property, provided it is mapped onto one during input processing. A component type supplied with the raw network, for instance, can be mapped to a node property so that only one class of component can anchor a pattern.

In the Julia implementation of the framework, the eligibility discriminator is exposed as a runtime override, and the other two are small internal functions that a replacement definition would replace.

While the module stands as a generic subgraph decomposition tool, the discussion in the rest of this chapter mainly focuses on the framework’s default target, the *diamond* subgraph.

4.4 The Diamond Subgraph

As introduced in Section 4.2, the decomposition unit in this work is the diamond subgraph. Diamond patterns are formed when paths diverge from a common fork and reconverge downstream with overlapping ancestries.

An *induced* diamond is a diamond made of one fork and one diamond-forming join. Every one of its paths originates from that single fork, whether there are two of them or four, and the subgraph carries no second diamond join (Figure 4.1). However, diamonds are not always induced. Non-diamond paths may still reach the same join node, as in Figure 4.1c. As network interconnectivity increases, so does the likelihood of diamond patterns and their complexity, with diamonds becoming more nested and overlapping each other's paths (Figure 4.2).

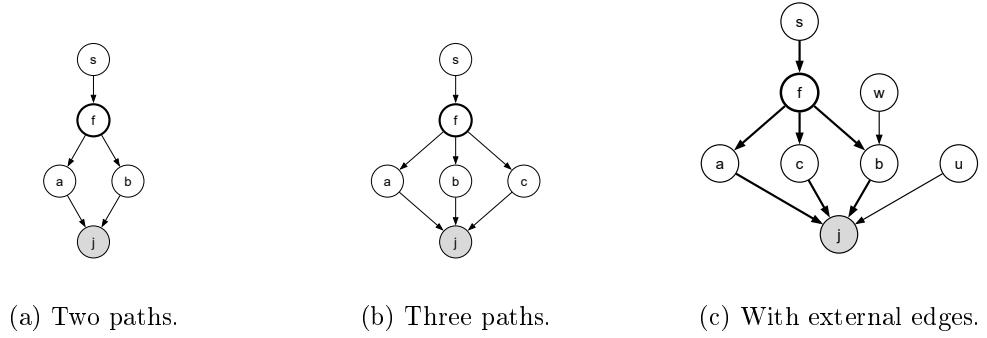


Figure 4.1: Induced diamonds. Every diamond path originates from the single fork and the subgraph carries one diamond join.

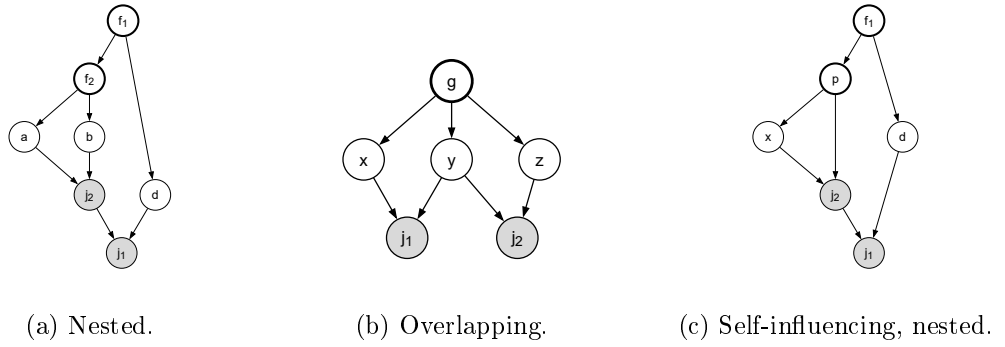


Figure 4.2: Non-induced diamonds. In (b) two diamonds overlap through the shared fork g , and in (c) parent p is itself the fork of the inner pattern.

The decomposition module consumes the DAG $G = (V, E)$ and the iteration sets $L = \{L_1, \dots, L_k\}$ from the graph object of Chapter 3. Given the iteration sets, the topological index $\text{topi} : V \rightarrow \{1, \dots, |V|\}$ follows by numbering nodes in increasing order across layers and in sorted node order within each layer, so every node receives a

distinct integer and topi is a strict total order on V refining the layered order.

The diamond in the framework is thus defined by the three discriminators.

- Eligibility criterion: the node's declared value must leave its reachability unsettled.
- Anchoring structure: the shared fork from which the reconvergent paths originate.
- Membership criterion: two parents belong to the same pattern when they share ancestry.

The eligibility criterion is the one discriminator not drawn from the diamond's definition. In the implementation, `is_det` is the only place where identification reads declared values. Since the module was built with probabilistic propagation in mind, the default criterion excludes nodes whose prior probability is deterministic. There are two such cases: an inactive node, with prior 0, and a perfect source, with prior 1. The second case requires both conditions. A non-source node with prior 1 may still be uncertain, because its incoming edges may fail, so excluding every node valued 0 or 1 would wrongly disable every diamond in a network where all priors happen to be 1. For interval-valued and p-box-valued networks the two cases apply only to degenerate values, an interval or p-box collapsed exactly onto 0 or 1.

Definition 4.1 (Diamond join). *A join node is a diamond join when an unsettled fork lies in the shared ancestry of two of its parents, or one unsettled parent is itself an ancestor of another, so that a diamond forms there. A diamond join can itself be a fork of a different diamond elsewhere in the network.*

Each parent arriving at a diamond join carries its *influencing set*, its ancestry under the self-inclusive closure, the nodes with a path to it and the parent itself. A diamond join can also receive paths that originate outside the fan-out, and these form its non-influencing set.

Definition 4.2 (Non-influencing set). *The non-influencing set of a diamond join is the set of its parents whose influencing sets meet no other parent's. Their edges enter the join from outside every reconvergent pattern there and are carried alongside the join's diamonds as independent contributions.*

Given this, the diamond subgraph is defined formally.

Definition 4.3 (Diamond). *A diamond D is a subgraph of G whose relevant node set $R \subseteq V$ spans a diamond join, its correlated parents, and everything upstream of them, together with a set $C \subseteq V$ of fixed nodes that isolate the pattern.*

The framework exposes two views of the network’s decomposed subgraphs.

Unique diamonds are the complete set of distinct diamonds found in the network, each stored once, in `unique_subgraphs`. Any node can participate in any number of unique diamonds, whether as a fan-out point, a fan-in point, or an intermediate node along the paths.

The maximal diamond is the network-level view of a diamond join. It is the full subgraph the join’s reconvergence forms, made up of the diamond of every group of the join’s parents that forms one, spanning the join, the groups, and everything upstream of them, together with the edges of the join’s non-influencing set. Therefore every diamond join has at most one maximal diamond, while a fork can participate in several. Every diamond found at a deeper level inside a maximal diamond is a **sub-diamond**, and its edge list is contained in the enclosing diamond’s.

Two consequences follow from these definitions. A unique diamond with no inner sub-diamond has one fork and one diamond join, so it is an induced diamond, the smallest a diamond can be. Furthermore, since every diamond is either maximal at its join or a sub-diamond of a maximal one, the set of unique diamonds is the set of distinct maximal diamonds together with their inner sub-diamonds.

4.5 The Identification Algorithm

`identify_diamonds` is a recursive set-membership algorithm over the closures of the graph object. For every join node of G it identifies the join’s maximal diamond, pairing the join with the earliest covering forks in its parents’ shared ancestry, and it recurses inside each diamond it forms for the sub-diamonds. Each call works on a join v , a *fixed context* $E \subseteq V$ of nodes already fixed by enclosing diamonds, and the parents P of v in view. The recursion starts from $E = \emptyset$, and Algorithm 2 shows the skeleton.

Under a context each parent's influencing set shrinks to

$$\text{infl}(p, E) = \text{Anc}(p) \setminus E,$$

with $\text{Anc}(p)$ self-inclusive so that $p \in \text{infl}(p, E)$ whenever $p \notin E$. Two parents p, q of a join are *structurally correlated under E* when $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$, that is, when some node upstream of both is not held fixed. Extending the context with a node removes it from every influencing set, as fixing a component does in factoring.

Algorithm 2 Recursive diamond identification (skeleton)

Require: join node v , fixed context E , parents P of v in view

Ensure: the diamonds at v under E

- 1: partition P into groups by pairwise influencing-set intersection
 - 2: **for all** groups G with $|G| \geq 2$ **do**
 - 3: $f \leftarrow$ eligible covering fork of G with minimum topi
 - 4: **if** no eligible covering fork exists **then**
 - 5: members of G join the non-influencing set of v ; skip the group
 - 6: **end if**
 - 7: build diamond D for (v, G, f) under $B = E \cup \{f\}$
 - 8: reuse the stored entry if the identity key of D is already in `unique_subgraphs`
 - 9: **for all** join nodes j within D **do**
 - 10: recurse on $(j, B, \text{parents of } j)$, restricted to G when $j = v$
 - 11: **end for**
 - 12: **end for**
 - 13: singleton groups join the non-influencing set of v
-

4.5.1 Shared ancestry and the earliest covering fork

At a join v with parents P under context E , any two parents whose influencing sets intersect are merged into one group by union-find.

Theorem 4.4 (Grouping correctness). *Fix a join v , context E , and parent set P . Let H be the undirected graph on vertex set P with an edge between p and q if and only if $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$. Then `components`(P, E) returns exactly the connected components of H .*

Proof. The intersection $\text{infl}(p, E) \cap \text{infl}(q, E)$ is evaluated for every unordered pair of P , so the pairs that intersect are exactly the edges of H , and union-find merges exactly

those pairs. Union–find over a vertex set and its edge relation returns the connected components, which for P under this edge relation is the component partition of H . $\square$

Theorem 4.5 (Group disjointness). *If $G_i \neq G_j$ are distinct groups returned by $\text{components}(P, E)$, then*

$$\left(\bigcup_{p \in G_i} \text{infl}(p, E) \right) \cap \left(\bigcup_{q \in G_j} \text{infl}(q, E) \right) = \emptyset.$$

Proof. Let $p \in G_i$ and $q \in G_j$. By Theorem 4.4 they lie in different components of H , and since every pair was evaluated, $\text{infl}(p, E) \cap \text{infl}(q, E) = \emptyset$. The intersection of the two unions expands to the union of these pairwise intersections, each of which is empty. $\square$

Disjointness here is a statement about the influencing sets of the diamond groups, namely that each group decomposes separately. Whether separately computed values recombine correctly at the join is a different claim, which Chapter 5 makes and proves for the probabilistic interpretation. Parents in singleton groups fall into the join’s non-influencing set.

Each group of two or more is then searched for its eligible covering forks, those lying in the shared ancestry of at least two members and not already in the fixed context, with a member that is an ancestor of another counting as its own cover. The diamond anchors on the earliest of them, the covering fork of minimum topological index, which leaves the largest part of the group’s shared structure intact downstream for reuse, and the strict total order makes the choice unique. A group whose shared ancestry offers no eligible covering fork forms no diamond, and its members join the non-influencing set.

4.5.2 Recursive flow

Theorem 4.6 (Termination). *The recursion of Algorithm 2 terminates after at most $|V|$ nested context extensions along any recursive path.*

Proof. Along any path of the recursion, each call either returns without recursing, which happens when every group at the join in view is a singleton or offers no covering fork, or fixes a fork f and recurses with $B = E \cup \{f\}$. Covering forks are never already in

E (Section 4.5.1), so $f \notin E$ and $|V \setminus B| = |V \setminus E| - 1$ strictly. The quantity $|V \setminus E|$ is a non-negative integer that strictly decreases along the recursive path, and from its initial value $|V \setminus \emptyset| = |V|$ it admits at most $|V|$ decrements. $\square$

Fixing fork f for group G at join v yields the diamond with boundary $B = E \cup \{f\}$. Paths within the diamond can share further unfixed ancestry, so the recursion runs again at every join within it, the diamond join itself included, under the extended context. Sub-diamonds form inside maximal diamonds this way until no covering fork and join pair remains anywhere.

4.5.3 Subgraph construction

The diamond for group G at join v under boundary B spans

$$R = \{v\} \cup G \cup \bigcup_{p \in G} \text{Anc}(p),$$

the join, the group, and everything upstream of the group, with induced edge list

$$\mathcal{E}_D = \{(u, w) \in E : u, w \in R, w \notin B, (w \neq v \text{ or } u \in G)\}.$$

Write $V(\mathcal{E}_D)$ for the set of its endpoints. The first two clauses keep the edge list on the relevant set R . The third removes every edge into a fixed node, whose upstream structure is already settled, while its outgoing edges stay and act as local sources of the diamond. The fourth admits an edge into the diamond join only from the group. The reason for the fourth clause is given after the lemma.

Lemma 4.7 (Edge isolation). *Let D be constructed for group G at join v under boundary B as above. Then $(u, w) \in \mathcal{E}_D$ if and only if $(u, w) \in E$, $u, w \in R$, $w \notin B$, and $(w \neq v \text{ or } u \in G)$. Consequently every directed path within $\mathcal{E}_D$ that terminates at v has its final edge originating in G , and no node of B appears on any path within $\mathcal{E}_D$ except as its first node.*

Proof. The membership characterisation restates the filter defining $\mathcal{E}_D$. For the path properties, induct on path length. A length-one path into v is a single edge $(u, v) \in \mathcal{E}_D$,

which the fourth clause forces to have $u \in G$. Any longer path has a final edge satisfying the same property, and each preceding edge (u, w) has $w \notin B$ and both endpoints in R by the first three clauses, so no node after the first lies in B and no edge of the path enters from outside R . $\square$

Consider a nested level at which an enclosing diamond has already fixed a fork r , so $r \in E$, and suppose r is both an ancestor of some group member p and a direct parent of the join v . Partitioning places r by its remaining free ancestry, since fixing removed r from every influencing set. In the minimal case of r a source with no free ancestry its influencing set is empty, and r lands in a singleton group whose contribution to v enters separately.

However, the span R is built from the raw closure $\text{Anc}(p)$ and contains r whatever the context says, so the partition and the span disagree wherever $E \neq \emptyset$. The edge (r, v) then has both endpoints in R and a head outside B , so the first three clauses admit it, and a diamond meant to capture v as reached through G alone gains a second, unrelated entry into its own join. The fourth clause blocks this edge. Figure 4.3 shows the minimal instance.

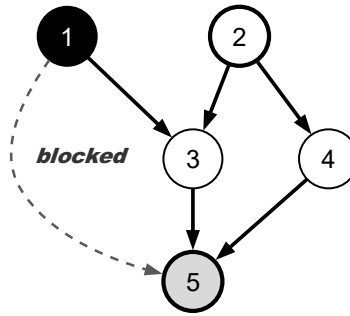


Figure 4.3: Edge isolation at join 5 in context $E = \{1\}$. Node 1 (filled) was fixed by an enclosing diamond and is excluded from the group $G = \{3, 4\}$ with fork 2. Its direct edge $(1, 5)$, drawn dashed, is admitted by the first three filter clauses but blocked by the fourth. Bold edges form the nested diamond's edge list.

4.5.4 Context-aware identity

An edge list alone cannot place a diamond in the nesting hierarchy.

Proposition 4.8. *(i) There exist graphs and contexts $E_A = \emptyset$ and $E_B = \{g\}$, with g a source fork among the endpoints of $\mathcal{E}_D$, under which the construction of Section 4.5.3 anchors the same fork f and yields the same edge list $\mathcal{E}_D$, while one diamond is maximal and the other a sub-diamond. The pair $(\mathcal{E}_D, \{f\})$ therefore conflates distinct diamonds. (ii) Under $\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D)))$, equal keys imply the same diamond join, the same group, the same edge list, and the same set C , with the two fixed contexts agreeing on every node of $V(\mathcal{E}_D)$ and every stored field equal.*

Proof. For (i), take a join v whose group G anchors on fork f , and a source fork g that is an ancestor of exactly one member of G and keeps one edge whose head lies outside both boundaries. Coverage of one excludes g from the covering-fork search, so the search offers the same forks in both cases and f is anchored in both, and removing g from a single influencing set cannot split the group, since the correlation runs through f . The cut into g removes nothing, as a source has no incoming edges, and g 's retained edge passes the filter in both cases, so $g \in V(\mathcal{E}_D)$. The group, the fork, the span, and the edge list therefore coincide, while the diamond is maximal under E_A and a sub-diamond of the one that fixed g under E_B . The keys differ, since $E_A \cap V(\mathcal{E}_D) = \emptyset$ and $g \in E_B \cap V(\mathcal{E}_D)$.

For (ii), equal keys give equal $\mathcal{E}_D$ and equal C , since both are hashed. The diamond join is recoverable as the unique endpoint of $\mathcal{E}_D$ that every other endpoint precedes in the closure; acyclicity makes it unique. The group is the in-neighbourhood of the join in $\mathcal{E}_D$, which the fourth clause admits from the group alone. Every stored field is computed from $\mathcal{E}_D$, $V(\mathcal{E}_D)$, C , and the global closures, so equal inputs give equal fields, and $B \cap V(\mathcal{E}_D) = (\{f\} \cap V(\mathcal{E}_D)) \cup (E \cap V(\mathcal{E}_D)) = C \cap V(\mathcal{E}_D)$ in both constructions. The two fixed contexts can still disagree away from the diamond, but only narrowly. A node outside the span is never consulted, since every membership test in the recursion concerns a parent of a join within the diamond or one of its ancestors, all inside the span by transitivity. A span node x in one fixed context and not the other has no incoming

edge, since such an edge would have its tail in the span and would pass the filter where x is free, forcing $x \in V(\mathcal{E}_D)$, and each outgoing edge of x inside the span is absent from the shared $\mathcal{E}_D$, so its head lies in each construction's boundary or it is a direct edge into the join excluded by the fourth clause. Every path from such a node towards the diamond therefore begins at a node in both fixed contexts or at an excluded edge. $\square$

Since a diamond cannot be unique by its edge list and fork alone, every unique diamond is keyed by

$$\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D))),$$

where $\mathcal{E}_D$ is the diamond's edge list, f its covering fork, and $E \cap V(\mathcal{E}_D)$ the fixed context restricted to the diamond's endpoints.

Before a new entry is built, the key is looked up in `unique_subgraphs`, and a structurally identical diamond already resolved anywhere in the recursion is referenced instead of rebuilt. This is hash-consing [28, 29], a table keyed by structural identity so that equal structures are stored once and identity comparison becomes a key comparison, sometimes used in BDD algorithms. Without the store the recursion is a tree and every call site is distinct. With it, structurally identical call sites collapse into shared entries and the representation becomes a DAG. Diamonds with the same identity share one stored entry regardless of the outer context, since Proposition 4.8 bounds whatever influence a node outside the identity carries towards the diamond, and the shared entry meets the same fixed context within.

4.6 Structural Correctness

4.6.1 Global completeness

Theorem 4.9 (Global completeness). *Let v be a join node of G with parent set P , and suppose there exist distinct $p, q \in P$ and an unsettled node a such that either a is a fork node in $\text{Anc}(p) \cap \text{Anc}(q)$, or $p \in \text{Anc}(q)$ (the asymmetric case, taking $a = p$). Then the top-level pass of `identify_diamonds` stores a maximal diamond for v . Conversely, a*

join admitting no such pair receives none.

Proof. At the top level $E = \emptyset$, so in either case $a \in \text{infl}(p, \emptyset) \cap \text{infl}(q, \emptyset)$ and the intersection is non-empty. By Theorem 4.4, p and q lie in a common group G with $|G| \geq 2$. In the covering-fork search, a counts once for each member it covers. In the symmetric case a is a fork in both ancestries and covers p and q . In the asymmetric case $a = p$ is necessarily a fork, since it has an edge to v and a path to q that cannot pass through v , and with the self-inclusive closure it covers itself as well as q . Either way the coverage reaches 2 and the set of covering forks is non-empty. A non-empty finite set has a topi-minimal element, so the search anchors some fork, not necessarily a itself, Algorithm 2 constructs a diamond for G , and the top-level pass records it under v . For the converse, suppose no such pair exists. Any covering fork the search admits would yield one, since a symmetric cover is a fork in two members' ancestries, an asymmetric cover is a parent in another member's ancestry, and the search admits only unsettled nodes. Every group therefore offers no covering fork, whether its members are unrelated or related only through settled structure, and the algorithm constructs nothing at v . That is the correct outcome, because whatever shared ancestry remains at v is already settled. $\square$

4.6.2 Determinism

Theorem 4.10 (Determinism). *For a fixed input graph and iteration sets (G, L) , join v , context E , and admissible parent set, the diamonds returned by Algorithm 2 are uniquely determined, independent of set or dictionary iteration order, thread scheduling, or any other accident of execution.*

Proof. By induction on recursion depth, it suffices that each step of one call is a function of its arguments alone. This holds for the partition step because, by Theorem 4.4, its output is the connected-component partition of H , a graph invariant that does not depend on the order in which pairs are examined or merges performed. It holds for the selection step because the set of covering forks is itself a function of the arguments, and either it is empty, in which case the group deterministically produces no diamond,

or the diamond anchors on its minimum under topi , which a strict total order makes unique. The boundary $B = E \cup \{f\}$ and span R are then determined, hence so is $\mathcal{E}_D$, and hence so are the joins passed to the recursive calls, which are deterministic by the inductive hypothesis. The store adds one further mechanism, reuse of an earlier entry on a key hit. Proposition 4.8(ii) makes equal keys carry the same join, group, edge list, and stored fields, and confines any residual difference between merged call sites to nodes outside the identity, so reuse introduces no dependence on visit order. $\square$

This guarantee is similar to the canonicity guarantee of reduced ordered binary decision diagrams [25]. While BDD canonicity depends on an externally supplied variable ordering, topi is derived from a property of the input graph.

4.7 The Unique Diamond Object

The algorithm builds each unique diamond as a graph object of its own. The unique diamond object carries the diamond’s characteristics together with its subgraph in the same form as the network’s graph object, so a diamond can be treated as a standalone DAG network for any other type of analysis in the framework. Table 4.1 lists its fields.

Table 4.1: Fields of the unique diamond object

Field	Role
<code>sub_outgoing_index</code>	Forward adjacency of the diamond
<code>sub_incoming_index</code>	Reverse adjacency of the diamond
<code>sub_sources</code>	Local sources
<code>sub_fork_nodes</code>	Local forks
<code>sub_join_nodes</code>	Local joins
<code>sub_ancestors</code>	Ancestor closure restricted to the diamond’s nodes
<code>sub_descendants</code>	Descendant closure restricted to the diamond’s nodes
<code>sub_iteration_sets</code>	Layered order restricted to the diamond’s nodes
<code>sub_node_priors</code>	Value map restricted to the diamond’s nodes
<code>sub_diamond_structures</code>	For each join within this diamond, its sub-diamonds, held by their keys into <code>unique_subgraphs</code> , and its non-influencing set
<code>is_maximal</code>	Whether the diamond is maximal
<code>diamond</code>	The span R , the set C , and the edge list $\mathcal{E}_D$

Invoking the decomposition module on the graph object of Chapter 3 extends the

object with the field `unique_subgraphs`, holding the network’s unique subgraphs. The Probability Propagation Algorithm of Chapter 5 reduces each solved diamond to a supernode. The concrete Julia types behind the fields of Table 4.1 are described in Chapter 8.

4.8 Worked Examples

4.8.1 The induced diamond

Figure 4.1a is the minimal pattern, a source s feeding a fork f , two internally disjoint paths, and a join j where they meet. At j both parents’ influencing sets contain f , so the parents form one group. Fork f is the only covering fork and is fixed, and the diamond spans the whole figure with $C = \{f\}$.

4.8.2 A factorised join

The nine-edge network of Figure 4.4 has forks 1 and 2 and a single join at node 7, which source 8 also feeds directly. The parents split into two correlated groups, $\{3, 4\}$ through fork 1 and $\{5, 6\}$ through fork 2, with parent 8 related to neither. Two constituent diamonds arrive at join 7, one anchored on each fork, and node 8 is carried in the non-influencing set. Together they are the join’s maximal diamond.

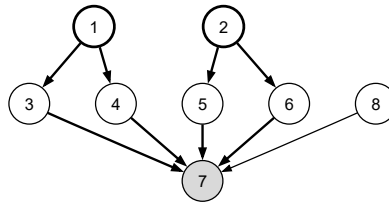


Figure 4.4: A factorised join. Two correlated groups meet at join 7, one through each fork, and source 8 enters from outside both. Bold edges mark the two constituent diamonds.

4.8.3 A nested decomposition

The eight-node network of Figure 4.5 has forks 1 and 3 and joins 6 and 8. An outer reconvergence runs through fork 1 and meets at join 8, and on one of its branches an inner reconvergence runs through fork 3 and meets at join 6. The decomposition below is the machine-checked output of the implementation on this edge list, and the examples of Figures 4.3 and 4.4 were verified in the same run.

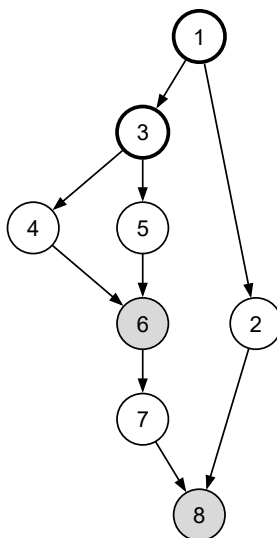


Figure 4.5: Frame 1. The network, with forks outlined in bold and joins shaded.

Join 8, top level. The parents are $\{2, 7\}$, with $\text{infl}(2, \emptyset) = \{1, 2\}$ and $\text{infl}(7, \emptyset) = \{1, 3, 4, 5, 6, 7\}$. The sets intersect in $\{1\}$, so the parents form one group. Fork 1 is the only covering fork of the pair and is fixed, and the maximal diamond D_1 spans all nine edges with $C = \{1\}$ (Figure 4.6).

Join 6, inside D_1 . Under the boundary $B = \{1\}$, join 6 has parents $\{4, 5\}$ with $\text{infl}(4, \{1\}) = \{3, 4\}$ and $\text{infl}(5, \{1\}) = \{3, 5\}$. The parents are still correlated through fork 3, so fixing 1 did not resolve the inner reconvergence. The recursive call fixes fork 3 and produces the inner diamond D_3 with $C = \{3\}$ and $\mathcal{E}_D = \{(3, 4), (3, 5), (4, 6), (5, 6)\}$. The edge $(1, 3)$ is cut by the boundary clause because its head is fixed at this level (Figure 4.7).

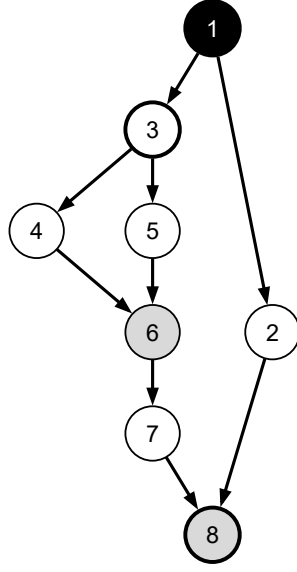


Figure 4.6: Frame 2. The maximal diamond at join 8, with fork 1 fixed (filled) and the span covering the whole network.

Join 6, top level. Join 6 is also a join of the network itself, so the top-level pass visits it in its own right. With $E = \emptyset$, forks 1 and 3 both cover parents 4 and 5, and the earliest covering fork rule fixes fork 1 and not fork 3. The result is a second maximal diamond D_2 spanning $\{(1, 3), (3, 4), (3, 5), (4, 6), (5, 6)\}$ with $C = \{1\}$, and the recursion inside it, now in context $\{1\}$, again produces exactly D_3 .

The emitted hierarchy. In total the module returns two maximal diamonds and three unique diamonds (Figure 4.8). D_1 sits at join 8 and D_2 at join 6, each with $C = \{1\}$. The inner D_3 at join 6 with $C = \{3\}$ is stored once and referenced from the substructure tables of both D_1 and D_2 under the same key. Two different enclosing recursions posed the same sub-problem, with the same edge list, the same fork, and contexts agreeing on the span. Proposition 4.8(ii) is the reason the single shared entry is correct.

The trace also confirms the edge-isolation case of Section 4.5.3 end to end. In the network of Figure 4.3, the nested diamond at join 5 under $E = \{1\}$ is emitted with $C = \{2\} \cup (\{1\} \cap V(\mathcal{E}_D)) = \{1, 2\}$, node 1 staying among the edge list’s endpoints

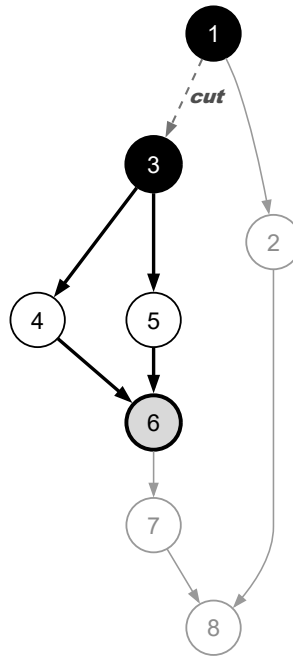


Figure 4.7: Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 fixed, the inner diamond in bold, and edge $(1, 3)$ cut.

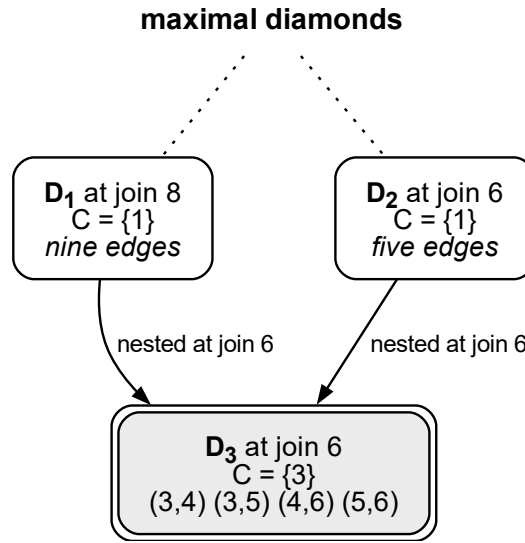


Figure 4.8: Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.

through its surviving edge (1,3), which is the context-aware identity of Section 4.5.4 visible in the output. Node 1 is recorded in the nested diamond’s non-influencing set, and the edge (1,5) appears nowhere in its edge list. At the top level node 1 is a member of the group itself, so the maximal diamond at join 5 carries (1,5) legitimately.

4.9 Discussion

Decomposition runs on a directed acyclic input. A network carrying cycles is decomposed only after the orientation of Chapter 3 has broken them, and the general cyclic case is outside the module’s scope.

Of the three discriminators, only eligibility is configurable at run time. An analysis that needs a different account of which nodes may take part supplies its own test through the override of Section 4.3. The anchoring and membership discriminators are internal functions of the implementation. Changing either is a change to the source, and neither has a configuration route.

The diamond is the only pattern for which a set of discriminators has been defined. The module’s genericity over patterns rests on the separation between the identification algorithm and the three discriminators, stated in Section 4.3. No second pattern has been defined here to exercise it.

A further analysis could define its own three discriminators and obtain every instance of a different subgraph pattern from the same algorithm, each instance returned as a graph object of the form given in Section 4.7. Chapter 11 lists this with the framework’s other extensions.

4.10 Summary

The module extends the graph object of Chapter 3 with `unique_subgraphs`, storing every diamond found at any level exactly once, each entry a graph object of the same form as the network it came from, with the maximal diamonds flagged. The decomposition terminates within $|V|$ nested steps and is canonical for a given input (Theorems 4.6 and 4.10). It groups a join’s parents exactly by shared unfixed ancestry and produces a diamond wherever unsettled reconvergence exists and only there (Theorems 4.4–4.9). Nothing outside a diamond’s span enters its edge list (Lemma 4.7), and the context-aware identity of Section 4.5.4 distinguishes a maximal diamond from a sub-diamond with the same edge list and fork (Proposition 4.8).

Chapter 5

Information as Probabilities: Probability Propagation Toolkit

5.1 Introduction

Under the probabilistic interpretation of the network model, the value on a node is the probability that the component operates and the value on an edge the probability that the handover between its endpoints succeeds. The question this interpretation makes precise is with what probability each node is reached from the sources despite component and handover failures. The Probability Propagation Toolkit answers that question exactly, for every node of the network at once. Its algorithm, the Probability Propagation Algorithm (PPA), was first published as the Information Propagation Method [73]; the name used in this thesis distinguishes the algorithm from the framework that carries it.

The toolkit reads the graph object of Chapter 3 and, wherever the network reconverges, the decomposition of Chapter 4. It is the analysis the decomposition module was built for, because reconvergent paths are where local probability calculations stop being valid. A local update at a join treats its parents as independent, and when the parents share an upstream component that treatment overstates the join’s belief. The algorithm repairs this by conditioning on the shared fork, computing each branch separately and recombining by total probability, and it applies the same recursion to nested and overlapping diamonds through the supernode reduction of Section 5.6.

The chapter makes four claims and supports each. The propagation is exact for point-valued inputs, proved in Sections 5.5 and 5.6 and checked against an independent decision-diagram oracle on a corpus of 129 graphs, on named and adversarial families, and on two published benchmarks whose exact values are reproduced (Section 5.7). Its cost is governed by the same structural width that governs a well-ordered decision diagram, and the two methods are competitive for exact point-valued reliability with neither dominating (Section 5.8). For interval inputs the propagation returns the exact range at machine precision, at lower one-shot cost than the decision-diagram route (Section 5.9.1). For probability-box inputs it returns sound bounds on the distribution of every belief, whose tightness is characterised by an envelope over network structure and input uncertainty (Section 5.10). Section 5.11 applies the toolkit to a medical

drone delivery network, Section 5.12 states the limits of the method, and Section 5.13 concludes. Before the method, Section 5.2 defines the quantity computed and Section 5.3 the single pass on which everything is built.

Chapter 2 reviewed the exact reliability methods against which this toolkit is placed. Three points from that review are used here. Every exact method is exponential in some width of the network, because the problem is #P-hard, and the question is which width and at what constant. The binary decision diagram, with a variable ordering found by sifting, is the standard this chapter compares against, since an unordered diagram is not a fair baseline for any method. And every exact method reviewed consumes a point-valued component probability; the interval and probability-box forms of Chapter 2 are what this toolkit propagates that the others do not.

5.2 The Belief of a Node

Every component of the network can fail. A node v carries its prior π_v , the probability that the component operates, and an edge (u, v) carries its probability ρ_{uv} , the probability that the handover from u to v succeeds, both attached to the graph object during input processing. Writing X_v and Y_{uv} for the corresponding indicator variables, the independence assumption of Chapter 3 makes all of them mutually independent.

Information reaches a node when some path delivers it, and whether it does is itself an indicator variable. The reachability of v is

$$R_v = X_v \wedge \left(v \in S \vee \bigvee_{u \in \text{Pa}(v)} (R_u \wedge Y_{uv}) \right), \quad (5.1)$$

where S is the source set and $\text{Pa}(v)$ the parents of v . The toolkit computes the *belief* of every node,

$$b(v) = P(R_v = 1), \quad (5.2)$$

the probability that v operates and is reached from at least one source. Unrolling the recursion writes R_v as a disjunction over all source-to- v paths of the conjunction of each path's node and edge indicators, so the belief is equally the probability that at least

one path is viable end to end.

Computing $b(v)$ subsumes two-terminal network reliability and is therefore #P-hard [6, 7]. No exact method escapes exponential cost on every network, and the aim is exactness at a cost set by the structure of the particular network. R_v is also a monotone Boolean function of the component indicators, since restoring a failed component can only create source-to-node connectivity, never destroy it. Both facts are used throughout the chapter.

Proposition 5.1 (Monotonicity). *$b(v)$ is non-decreasing in every node prior and every edge probability.*

Proof. Realise each indicator from its own independent uniform variable, $X_w = \mathbf{1}\{U_w \leq \pi_w\}$ and $Y_{uw} = \mathbf{1}\{U_{uw} \leq \rho_{uw}\}$. Raising any single π_w or ρ_{uw} can switch its indicator from 0 to 1 but never from 1 to 0, and R_v is monotone in every indicator, so on every realisation R_v can only switch from 0 to 1. Hence $P(R_v = 1)$ cannot decrease. $\square$

5.3 Propagation in a Single Pass

A source is reached exactly when it operates, so its belief is its prior. Every other node receives information through its parents. The *signal* through a parent p of v is the event A_p that a viable path reaches p and the handover to v succeeds, with probability

$$S_p = b(p) \rho_{pv}, \quad (5.3)$$

and v is reached when at least one signal arrives, so

$$b(v) = \pi_v P\left(\bigcup_{p \in \text{Pa}(v)} A_p\right). \quad (5.4)$$

When the signal events are mutually independent the union has the closed form

$$b(v) = \pi_v \left(1 - \prod_{p \in \text{Pa}(v)} (1 - b(p) \rho_{pv})\right), \quad (5.5)$$

which for a single parent collapses to the chain rule $b(v) = \pi_v b(p) \rho_{pv}$.

The iteration sets of Chapter 3 order the computation so that every parent’s belief exists before it is consumed. The algorithm initialises the sources, then processes each layer in turn, applying the local update once per node, and the layered order of Chapter 3 makes the single forward pass complete in exactly $|L|$ layer steps. There is no iterative refinement to converge and no sampling to average, because every update is an exact probability calculation.

Equation (5.5) evaluates the union as a product of complements instead of by the alternating inclusion-exclusion sum. Under independence the two are identical, but in the product form each signal probability occurs exactly once, which keeps the evaluation numerically stable and, once values become intervals, avoids the artificial widening that repeated occurrences of a variable inflict on interval arithmetic (Section 5.9.1).

The update rests on one hypothesis. The update of Eq. (5.5) is exact when the signal events are independent, and they are independent when the paths feeding the parents share no component upstream. In a tree that is always true. In the systems this framework serves, however, it is false, because Chapter 3 established that engineered redundancy is reconvergence, with duplicated routes sharing the components upstream of the fork that created them.

5.4 Reconvergence Breaks the Local Update

Figure 5.1 shows the smallest network on which the local update fails, a diamond in the sense of Chapter 4. Source 1 has prior 1, every other node has prior 0.9, and every edge has probability 0.9. Every worked number in this chapter is the output of the implementation on the stated inputs, checked in the same run against an independent exact computation, exhaustive enumeration of all component states here and exhaustive path enumeration with inclusion-exclusion for the larger example of Section 5.6.1.

The forward pass runs cleanly until the join. The fork’s belief is the chain $b(2) = 1 \times 0.9 \times 0.9 = 0.81$, and each intermediate node follows as $b(3) = b(4) = 0.81 \times 0.9 \times 0.9 = 0.6561$, so each signal into the join carries $S = 0.6561 \times 0.9 = 0.59049$. Treating the

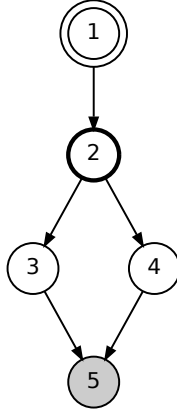


Figure 5.1: The simple diamond. Source 1 feeds fork 2 (bold), whose two paths reconverge at join 5 (shaded).

two signals as independent, Eq. (5.5) gives

$$b(5) = 0.9 (1 - (1 - 0.59049)^2) = 0.749071,$$

while exhaustive enumeration over all component states gives $b(5) = 0.675462$. The local update overestimates the join's belief by eleven per cent of its true value, and it does so because both signal events contain the fork's reachability. The two events are positively dependent, their intersection is larger than the product of their probabilities, and a union rule that understates the intersection overstates the union. On a network built out of redundancy the error is the norm, and it always overstates the reliability of the system.

However, conditioning on the shared fork repairs the calculation. Given $R_2 = 1$ the two paths share nothing further, so each transmits independently with probability $0.9 \times 0.9 \times 0.9 = 0.729$ and the join receives a signal with probability $1 - (1 - 0.729)^2 = 0.926559$. Given $R_2 = 0$ nothing arrives, since every path into the join runs through

the fork. Weighting the two cases by $P(R_2 = 1) = b(2) = 0.81$,

$$P(\text{signal at } 5) = 0.81 \times 0.926559 = 0.750513, \quad b(5) = 0.9 \times 0.750513 = 0.675462,$$

the exact value. Two conditioned sub-problems replace an enumeration over 2^{10} component states, because conditioning is applied only to the one variable whose sharing created the dependence. On real networks, where diamonds nest, overlap and arrive several to a join, the same economy requires the machinery of Chapter 4.

5.5 Conditioning at a Join

The example used two facts without naming them, that a belief can be split over the states of other nodes and that fixing the right node makes the parents' signals independent. A third governs the joins the example was too small to show, where several groups of parents arrive sharing nothing between them and each group's contribution must recombine with the others. All three hold in general, and together they justify the algorithm's treatment of any join. The structural objects come from Chapter 4, whose decomposition partitions each join's parents into groups of shared unfixed ancestry, anchors each group's diamond, and sets aside the non-influencing parents. A diamond arrives carrying its set C of fixed nodes, and since the propagation resolves the diamond by conditioning on the reachability states of exactly those nodes, this chapter calls C the *conditioning set*.

Lemma 5.2 (Conditional invariance). *For any set $A \subseteq V$ and any node v ,*

$$b(v) = \sum_c P(R_A = c) P(R_v = 1 \mid R_A = c),$$

where R_A is the vector of reachability indicators of A and the sum runs over its states $c \in \{0, 1\}^{|A|}$. Every term is well defined because each R_u is a deterministic function of the mutually independent component indicators.

Proof. The events $\{R_A = c\}$ partition the sample space, and the display is the law of total probability over that partition. $\square$

Lemma 5.3 (Separator sufficiency). *Let v be a join evaluated within a network in which every node of a fixed set C enters only as a local source whose state is pinned, and suppose the influencing sets $\text{infl}(u, C) = \text{anc}(u) \setminus C$ of v 's parents within that network are pairwise disjoint. Then, conditional on $R_C = c$, the parents' signal events are mutually independent, and*

$$b(v \mid R_C = c) = \pi_v \left(1 - \prod_{u \in \text{Pa}(v)} (1 - P(R_u = 1 \mid R_C = c) \rho_{uv}) \right).$$

Proof. Every path into a parent u either starts at a node of C , whose pinned state replaces everything upstream of it, or runs through components of $\text{infl}(u, C)$ alone. Conditional on $R_C = c$, the signal event through u is therefore a function of the constant c , of the indicators of the nodes of $\text{infl}(u, C)$, of the edges among them, and of the edge (u, v) . The influencing sets are pairwise disjoint, so these indicator sets are pairwise disjoint and mutually independent, and functions of disjoint sets of independent variables are independent. The display is Eq. (5.5) applied conditionally. $\square$

Chapter 4 establishes both hypotheses at every site where the propagation applies the independent update. Its edge isolation lemma guarantees that a fixed node appears in a diamond's edge list only as a local source, with its outgoing edges kept and its upstream structure cut. Its grouping theorem certifies disjoint influencing sets, and the recursion extends the conditioning one covering fork at a time until the parents in view are disjoint in this sense. The conditional beliefs the display consumes are the *contextual beliefs*, the values each node holds within the sub-network currently being evaluated, which are conditional on the ambient states by construction because they were computed there.

The remaining question is the one Chapter 4 deferred, whether the contributions of separately decomposed groups recombine correctly at the join. Its group disjointness theorem gives distinct groups pairwise disjoint influencing sets under the current context, and disjoint influence is exactly independence.

Lemma 5.4 (Recombination across groups). *Fix a join v and a context E whose nodes' states are given, and let $G_1, \dots, G_m$ be the parent groups of the decomposition's partition*

at v , together with the singleton groups forming the non-influencing set. Write E_k for the event that some parent in G_k delivers a signal to v . The events $E_1, \dots, E_m$ are mutually independent, so

$$P(\text{signal at } v) = 1 - \prod_{k=1}^m (1 - P(E_k)),$$

and each group is resolved on its own conditioning set alone.

Proof. Each E_k is a function of the indicators of the nodes in $\bigcup_{p \in G_k} \text{infl}(p, E)$, of the edges among them, and of the edges from G_k into v . By group disjointness the node sets are pairwise disjoint, hence so are the edge sets they induce, and the given context enters every E_k only as fixed constants. The events are therefore functions of pairwise disjoint sets of mutually independent indicators, hence mutually independent, and the display is the union rule for independent events. $\square$

The lemma also covers the non-influencing set of Chapter 4. A non-influencing parent is a singleton group, its influencing set meeting no other's, so its signal enters the product as one more independent factor at the cost of a single multiplication, with no conditioning at all. A join can therefore gather several diamonds and several plain parents, and the recombination stays a one-line product over independent contributions.

Recombination across groups is where the decomposition reduces the cost. A join whose m groups carry conditioning sets $C_1, \dots, C_m$ is resolved in $\sum_k 2^{|C_k|}$ conditioned sub-problems, where conditioning on all the groups jointly would take $2^{\sum_k |C_k|}$. The gap is widest at high fan-in over independent structure. On an adversarial family in which k independent reconvergent branches meet at one join, the implementation resolves exactly $2k + 1$ conditioned sub-problems for every k tested up to 16, linear where joint conditioning would be exponential.

What remains at a single group is the enumeration itself. Writing $C = \{a_1, \dots, a_m\}$ for the group's conditioning set, total probability gives

$$P(E_k) = \sum_{c \in \{0,1\}^m} \left(\prod_{i=1}^m b(a_i)^{c_i} (1 - b(a_i))^{1-c_i} \right) \psi(c), \quad (5.6)$$

where the weights are built from the conditioning nodes' contextual beliefs and $\psi(c)$ is the probability that the group delivers a signal to v given the state c . A conditioning state does not simplify the diamond to a formula. It poses a smaller instance of the whole problem, which the algorithm solves in the same way.

5.6 Supernodes and Progressive Decomposition

A diamond arrives from Chapter 4 as a graph object of the same form as the network itself, and the propagation exploits that self-similarity directly. For each state c of the conditioning set, the algorithm poses the diamond as a network in its own right and runs on it the same single pass it runs on the whole. The local sources of that sub-network take their values from the enclosing computation, a conditioning node pinned to its state and every other local source keeping its contextual belief. The join's own prior is deferred, set to 1 inside the sub-problem so that the sub-run returns the probability that a signal arrives, with the join's prior applied exactly once by the enclosing update. The value the sub-run returns at the join is $\psi(c)$.

The map $c \mapsto \psi(c)$ is all the enclosing computation needs from the diamond. The solved diamond is therefore replaced by a *supernode*, a single equivalent component that delivers a signal to the join with probability $\psi(c)$ under each state of its conditioning set.

Lemma 5.5 (Supernode equivalence). *Let D be a diamond at join j with conditioning set C , solved in its context to the map ψ_D . Replacing D by a supernode that delivers a signal to j with probability $\psi_D(c)$ under each state c leaves the belief of every node downstream of j unchanged.*

Proof. Fix a state c . Conditional on $R_C = c$, the event that D delivers a signal to j is a function of the indicators internal to D , which are independent of every indicator outside D . The enclosing computation only ever conditions on the nodes of its own conditioning sets, each of which enters the sub-network containing D as a local source, upstream of D or beside it, so each such conditioning event is a function of indicators outside D and leaves $P(\text{signal at } j \mid R_C = c) = \psi_D(c)$ unchanged. A downstream belief

depends on D only through the signal event at j , entering by Lemma 5.2 as the pair of the weights $P(R_C = c)$ and the conditionals $\psi_D(c)$, and the supernode preserves both. $\square$

The proof’s independence step needs the enclosing computation to fix no node internal to D , and the decomposition guarantees this. However, when an outer conditioning does reach inside a subgraph, the context-aware identity of Chapter 4 makes the inner structure a different diamond with its own stored entry, so the sub-problem solved is always the one whose context holds.

Nesting makes the reduction progressive. Within a state’s sub-run, an inner join with unresolved shared ancestry carries its own diamond, recorded by the decomposition in the enclosing diamond’s substructure table, and the sub-run resolves it by the same conditioning, from the inside out. Termination is inherited from Chapter 4, whose recursion bounds the nesting by strictly growing the fixed context, and exactness follows by induction on nesting depth, each level applying Lemmas 5.2 to 5.5 within its own sub-network, which satisfies the same model assumptions as the whole.

Two mechanisms keep the recursion from repeating itself. A conditioned sub-problem is identified by its diamond’s edge list together with the local source values its state assigns, and every solved sub-problem is stored under that identity, so a structurally identical sub-problem posed anywhere else in the recursion is answered from the store. Separately, a conditioning node whose contextual belief is already pinned to 0 or 1 by the enclosing state carries no remaining uncertainty, so its enumeration collapses to the single state of non-zero weight. The collapse is exact because the discarded state has weight exactly $b=0$ or $1-b=0$ in Eq. (5.6).

5.6.1 A multi-level worked example

The network of Figure 5.2 contains every structure of the preceding sections: three sources, three forks, nested diamonds, a join fed by a diamond and a non-influencing parent together, and a target whose reconvergence encloses every other structure. Sources S_1, S_2, S_3 have prior 1, every other node prior 0.9, and every edge probability 0.9.

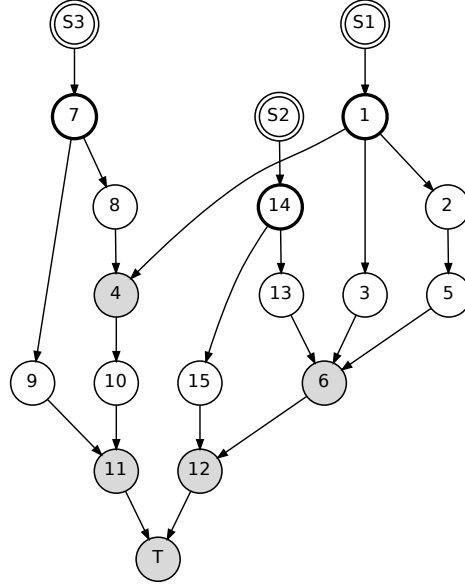


Figure 5.2: The multi-level worked network. Sources are double-bordered, forks 1, 7 and 14 bold, joins 4, 6, 11, 12 and the target T shaded.

The decomposition of Chapter 4 returns four maximal diamonds. D_1 at join 6 anchors on fork 1, covering the reconvergent parents 5 and 3, while parent 13 arrives from S_2 's side sharing nothing with them and lands in the join's non-influencing set. D_2 at join 11 anchors on fork 7 and D_3 at join 12 on fork 14. D_4 at the target anchors on fork 1, because the target's two parents, 12 and 11, share only node 1 in their unfixed ancestry, and within D_4 's span the reconvergences at joins 11 and 12 recur as sub-diamonds. Join 4 receives parents 1 and 8 whose ancestries are disjoint, so it needs no diamond anywhere, and the plain update of Eq. (5.5) handles it at every level.

Table 5.1 traces the four resolutions in the terms of Sections 5.5 and 5.6. Each is a two-state enumeration whose weight is the conditioning node's contextual belief, here 0.81 in all four cases since forks 1, 7 and 14 each sit one perfect source, one edge and one prior away.

At join 6 the diamond's contribution recombines with the non-influencing parent by Lemma 5.4: parent 13 delivers its signal with probability $0.6561 \times 0.9 = 0.590490$ independently of the group, so the join receives a signal with probability $1 - (1 -$

Table 5.1: Conditioning trace of the worked network. w is the conditioning node’s contextual belief, $\psi(1)$ and $\psi(0)$ the conditional signal probabilities the diamond’s sub-runs return at the join, and $P(E)$ the group’s recombined signal $w\psi(1) + (1-w)\psi(0)$.

Diamond	Join	C	w	$\psi(1)$	$\psi(0)$	$P(E)$
D_1	6	$\{1\}$	0.81	0.889023	0	0.720108
D_2	11	$\{7\}$	0.81	0.893745	0.478297	0.814810
D_3	12	$\{14\}$	0.81	0.931860	0.583288	0.865631
D_4	T	$\{1\}$	0.81	0.913943	0.818038	0.895721

$0.720108)(1 - 0.590490) = 0.885382$ and $b(6) = 0.9 \times 0.885382 = 0.796843$. The other three joins carry a single group each, so their beliefs follow directly from the table, $b(11) = 0.733329$, $b(12) = 0.779068$, and $b(T) = 0.806149$.

The dead states of D_2 , D_3 and D_4 show that the conditioning weights the two branches; it does not switch one off. With fork 7 down, join 11 is still reached through node 1’s side at probability 0.478297, with fork 14 down, join 12 is still reached through S_1 ’s side at 0.583288, and with fork 1 down the target still collects the S_2 and S_3 routes at 0.818038. Each of those values is itself the output of a full sub-network propagation in which the fork is a pinned local source, and inside D_4 ’s sub-runs the reconvergences at joins 11 and 12 are resolved by their own nested diamonds under the extended context.

The store is visible in the run, which poses twenty conditioned sub-problems of which sixteen are distinct. The four savings are the inner reconvergence at join 6, which D_3 ’s recursion meets again inside each of its states in the form D_1 already solved, because node 14 lies outside that sub-problem’s span, so the extended context is invisible to its identity and the store answers all four posings. The nested diamonds inside D_4 show the opposite case: node 1 lies inside their spans, its pinned state is part of their identity, and each is stored under both outer states, four entries apiece instead of two. The same subgraph is reused where the context cannot reach it and kept apart where it can, which is the context-aware identity of Chapter 4 applied at propagation level. Every belief above agrees with exhaustive path enumeration under inclusion-exclusion to floating-point round-off.

5.7 Validation

5.7.1 The corpus and the oracle

Every guarantee in this chapter was checked against implementations that share none of the propagation’s machinery. The exactness oracle is a reduced ordered binary decision diagram with dynamic variable reordering, built independently for each network; exhaustive path enumeration is used where it is tractable and Monte Carlo sampling beyond that. The corpus comprises 129 random and mutated DAGs (120 random graphs across five sizes from 10 to 25 nodes and three densities, eight mutants of a 28-node base graph, and one hand-built adversarial counterexample), six structured families, larger random graphs of 30 nodes, and the named networks of Table 5.2. The provenance of every named network is given in Appendix A; the corpus graphs are generated from recorded seeds and can be rebuilt byte for byte.

Table 5.2: Exactness and structure on four named networks against the sifted decision-diagram oracle. Maximal and unique are the counts of maximal and unique diamonds; $\max |C|$ is the largest conditioning set; the last column is the worst per-node deviation from the oracle.

Network	$ V $	$ E $	Src	Maximal	Uniq	Max $ C $	Deviation
power-network	23	27	3	5	9	3	8.3×10^{-17}
grid-graph	16	24	3	8	41	7	1.1×10^{-16}
KarlNetwork	26	74	1	18	147	11	5.6×10^{-17}
counterexample-n15	15	23	1	4	13	4	5.6×10^{-17}

Two regimes were run on every graph: a perfect regime with node priors 1 and link probability 0.9, and an imperfect regime with random priors and link probabilities. In both, the propagated point beliefs agree with the oracle to a worst per-node deviation of 1.1×10^{-16} , and the propagated intervals coincide with the oracle’s corner evaluations to at worst 2.8×10^{-16} . The corpus was run twice: once during the development of the method and once after the correction of the eligibility test described in Chapter 4. The second run reproduced all 129 results to within 10^{-9} , which is expected, since the corpus draws every prior from the open interval (0.3, 0.99) and so never contains the prior of exactly one on a non-source node that the original test mishandled. The

correction changed nothing on this corpus and was found instead on the real networks of Section 5.11, where hub priors of exactly one occur.

Naive topological ordering of the decision diagram exceeds 2.5×10^6 nodes on the densest corpus graphs, whereas sifting keeps every graph in the corpus to at most 20,323 nodes. Every comparison below is against the sifted diagram.

5.7.2 Two published benchmarks

Two networks in the corpus come with published exact reliabilities, and both are reproduced. The first is the sixteen-node directed grid of Tong and Tien [32], sources 1, 3 and 13, node priors 1, under link reliabilities 0.9 and 0.1. The source reports exact sink reliabilities to five decimal places at thirteen sinks for each regime. The toolkit's beliefs agree with every one of the twenty-six published values to within 5×10^{-6} , which is the rounding of a five-decimal table; the largest differences, 4.8×10^{-6} at node 12 under 0.9 and 4.3×10^{-6} at node 16 under 0.1, are both below half a unit in the last published digit. The source's own approximate method reports a gap of 4.8×10^{-4} at node 16 under 0.9 between its lower and upper bounds; the toolkit's value is exact and lies within that gap.

The second is the 23-node directed power distribution network of the same source, Figure 11 there, with sources 1, 7 and 18 and sink 23, under a single link reliability R_l on every link and node priors 1. Table 5.3 compares the toolkit's sink belief with the published exact value at the three regimes reported. At $R_l = 0.99$ the two agree to all five published digits. At $R_l = 0.9$ and 0.3 the toolkit's values are higher by 1.8×10^{-3} and 5.1×10^{-4} . These residuals are too large to be print rounding and too small to be a missing or extra path: adding the one edge that the figure could be read as containing beyond the 27 used here makes every regime worse, by 2.8×10^{-2} at $R_l = 0.9$. The decision-diagram oracle agrees with the toolkit to 1.1×10^{-16} at all three regimes on this topology, so the residual is not in the computation. It is attributed to a difference in how the network was transcribed from the figure or in the source's own evaluation, and it is left open. The same network is the demonstration case of Chapter 9.

Table 5.3: Reproduction of the published power-network reliabilities of Tong and Tien [32], sink 23, node priors 1, every link at R_l . The oracle column is the sifted decision diagram on the same topology.

R_l	Published	PPA	Oracle	PPA – published
0.90	0.85741	0.85917	0.85917	+0.00176
0.99	0.98969	0.98969	0.98969	0
0.30	0.00221	0.00272	0.00272	+0.00051

5.7.3 Timing convention

Every runtime in this chapter is a wall-clock measurement on one core after a discarded warm-up run, so that just-in-time compilation is excluded; the convention is stated once in Appendix B. The decision-diagram times were measured in the same process and the same language, so that no cross-language artefact enters the comparison.

5.8 Complexity Analysis

The cost of the propagation is concentrated in the diamonds, because everything outside them is the linear-time update of Eq. (5.5) applied once per node. A diamond with conditioning set C and edge list E_D is resolved by $2^{|C|}$ sub-runs, each a single pass over its edge list, so one resolution costs $2^{|C|} \cdot O(|E_D|)$, and the total work sums this over every sub-problem the recursion poses. The sum ranges over posings, not over unique diamonds; nesting re-poses the same diamond under many enclosing states, and on deeply nested networks the posing count exceeds the unique-diamond sum by orders of magnitude.

Both refinements of Section 5.6 act on this sum. The store collapses the posings to the distinct sub-problems, which across the corpus is between one and five per cent of the unmemoised bound, so the posing bound is a sound a-priori budget, computable from the decomposition before any propagation runs, and the realised cost is far below it. The factorisation of Lemma 5.4 acts on the exponents, replacing one joint enumeration by a sum of small ones. On the adversarial fan-in family, in which k independent reconvergent branches meet at one join, the implementation resolves exactly $2k + 1$

sub-problems for every k tested up to 16 (Table 5.4), where joint conditioning would resolve 2^k .

However, no refinement removes the width. The governing parameter is the largest conditioning set the network forces. The propagation is a specialisation of cutset conditioning [30] to source-to-node reachability, with two departures. Where cutset conditioning selects one global set whose enumeration renders the whole network singly connected, the propagation identifies a local separator at each reconvergence it meets and recurses, so the realised cost is a sum of local exponentials, and tree-shaped regions are crossed at linear cost. And where the classical scheme conditions jointly, the factorisation refuses to enumerate independent forks together.

The measurements place the method. Figure 5.3 plots the maximum conditioning set against the logarithm of the sifted decision diagram’s node count across the corpus, and the two track each other, seven against $\log_2 290 \approx 8.2$ on the sixteen-node grid, because both measure the same structural width. Realised work tells the same story with no winner: on some networks the propagation resolves fewer sub-problems than the diagram holds nodes and on others more, and no family in the corpus systematically favours either method for exact point-valued computation.

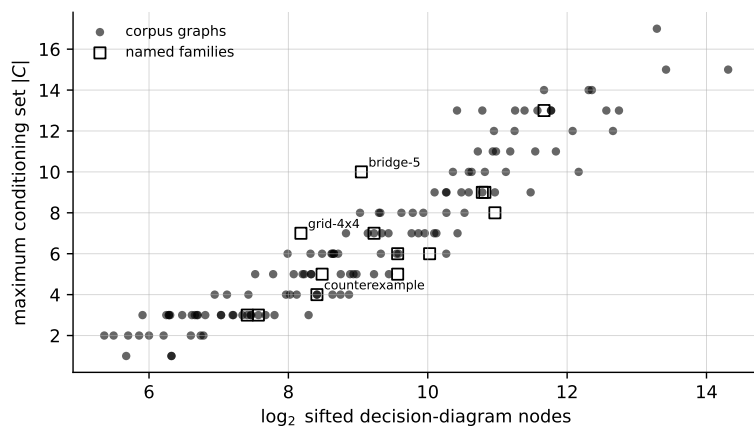


Figure 5.3: Maximum conditioning-set size against the logarithm of the sifted decision diagram’s node count across the complexity-validation networks. Both quantities measure the same structural width.

Two adversarial families locate where the two methods agree and where they part

(Table 5.4 and Figure 5.4). On fan-in- k , k independent gadgets reconverging at one sink, factorisation makes the propagation linear and the diagram is linear too; the two tie. On mesh- w , a width- w lattice with one correlated block and no independent partition, the propagation’s conditioning nests and its operation count grows far faster than the diagram’s node count, which stays compact under sifting; at $w = 8$ the propagation resolves 3.9×10^6 sub-problems against a diagram of 2,621 nodes. On square meshes both grow exponentially with the diagram’s exponent the smaller. Hence the propagation is never structurally better than a well-ordered diagram for exact point-valued reliability: the two tie on independent structure and the diagram wins on dense correlated structure. The toolkit’s case rests on the two capabilities that follow, which a decision diagram does not offer.

Table 5.4: Complexity on adversarial families. Operations are the propagation’s deduplicated sub-problem count; nodes are the sifted decision diagram’s size. Small instances were validated exact against path enumeration.

Family	Parameter	PPA operations	Max $ C $	Diagram nodes	Regime
fan-in- k	4	9	1	126	tie
fan-in- k	8	17	1	474	tie
fan-in- k	12	25	1	636	tie
fan-in- k	16	33	1	802	tie
mesh- w	2	78	6	315	–
mesh- w	4	20,603	8	700	diagram
mesh- w	6	832,922	10	1,487	diagram
mesh- w	8	3,895,252	11	2,621	diagram

On a set of 17 Bayesian-network benchmarks from the bnlearn repository, unrelated to the infrastructure networks that motivate this thesis, identification completed on every network in under two seconds and propagation completed in point and interval form on 16 of the 17. The exception, with a maximum conditioning width of about 21, exhausted memory at identification; the sifted diagram on the same network exhausted its time budget during construction. The two methods fail on the same network for the same reason, which is the width, and the toolkit’s applicability extends to probabilistic graphical models generally.

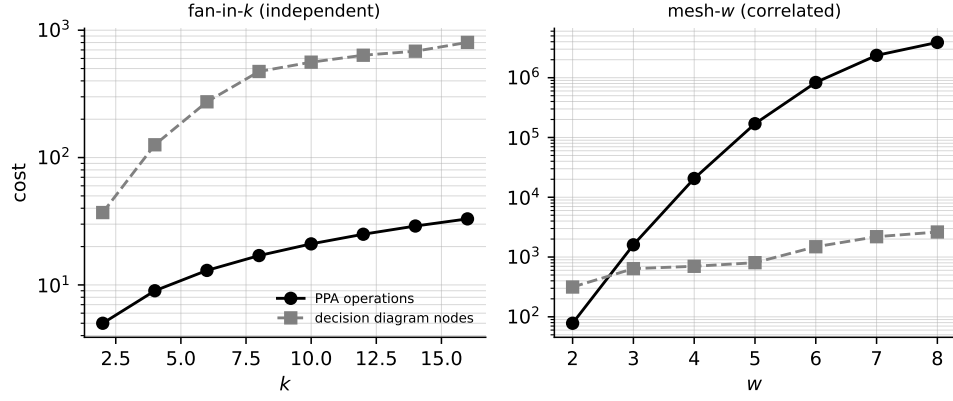


Figure 5.4: Cost on the two adversarial families: the propagation’s deduplicated subproblem count and the sifted decision diagram’s node count against the family parameter, on a logarithmic vertical axis. Left, fan-in- k ; right, mesh- w .

5.9 Propagating Uncertain Probabilities

The inputs of a reliability analysis are rarely known exactly. Failure probabilities come from sparse operational records and expert judgement, and Chapter 3 gave the framework three value forms to carry those states of knowledge: a deterministic value asserting full knowledge, an interval asserting only bounds, and a probability box bounding a distribution. However, in practice these forms are usually collapsed before the analysis begins: a point estimate is chosen and the uncertainty is deferred to a sensitivity study afterwards. The toolkit propagates the stated form itself, because everything the propagation does to a value is one of three operations, complement against one, product, and the weighted combinations of Eq. (5.6), and none of the machinery of Sections 5.5 and 5.6 requires the values to be numbers. The framework realises the propagation once over all three forms. What is guaranteed of the output differs by form: the exact range for intervals (Section 5.9.1) and sound distributional bounds for probability boxes (Section 5.10).

5.9.1 Interval inputs are propagated exactly

An interval input asserts that a component’s reliability lies between two bounds and nothing more, and the correct output for a node is then the exact range of beliefs

attainable over all inputs consistent with the stated bounds. The monotonicity of Section 5.2 makes that range available outright.

Proposition 5.6 (Corner exactness). *Let every node prior and edge probability be given as an interval, $\pi_v \in [\underline{\pi}_v, \bar{\pi}_v]$ and $\rho_{uv} \in [\underline{\rho}_{uv}, \bar{\rho}_{uv}]$. The exact range of $b(v)$ over all inputs consistent with the intervals is*

$$[b(v) \text{ at all lower endpoints}, b(v) \text{ at all upper endpoints}].$$

Proof. By Proposition 5.1, $b(v)$ is non-decreasing in every input coordinate, so over a box its minimum is attained where every coordinate sits at its lower endpoint and its maximum where every coordinate sits at its upper endpoint. $\square$

No dependency problem arises and no corner search is needed. Neither is typical of interval computations. A general function over an input box takes its extremes at unknown corners among the 2^n available, and naive interval arithmetic on it overapproximates whenever a variable occurs more than once. Here monotonicity identifies the two extreme corners, and the propagation is arranged so that the arithmetic between them stays exact. Its complement and product are exact interval operations, and the product form of Eq. (5.5) presents each signal to the arithmetic exactly once. The conditioning combination is evaluated at the endpoint values of each conditioning belief, which suffices because Eq. (5.6) is multilinear in each of them and a multilinear function attains its extremes over a box at its corners. Every step returns an enclosure of the true range, and across the full validation corpus the propagated interval coincides with the two corner propagations to machine precision, at worst 2.8×10^{-16} in either bound over all 129 graphs. On the worked network with every non-source prior and every edge widened to $[0.85, 0.95]$, the propagated belief at the target is $[0.634002, 0.931254]$ and agrees with the two corner runs to the last floating-point digit. Structural certainties are kept degenerate in such a widening, a perfect source stays $[1, 1]$, because widening a certainty into an uncertainty changes which nodes can carry conditioning and poses a different and harder problem.

The conditioning machinery matters as much here as for point values. For example,

an interval propagation that ignores reconvergent dependence stays sound but overwidens, by up to 0.45 of the unit range on the corpus, wide enough to turn an informative bound into a vacuous one. Exactness of the range comes from the conditioning, and interval arithmetic alone does not give it.

Carrying intervals costs little. The propagation costs a factor of about 1.2 over point propagation, a few extra floating-point operations per step. For a one-shot query, producing a network’s belief range from scratch, it completed faster than the decision-diagram route to the same range, which builds the diagram once and evaluates its two corners, on every one of eight corpus families tested, by factors between 3.9 and 99.9. However, the claim is scoped to the one-shot cost, where the diagram’s construction counts against it, and a workload that amortises one diagram across many repeated queries on a fixed network is a different and untested scenario.

5.10 Probability-Box Propagation

A probability box is the strongest of the three input forms, a pair of bounding distribution functions carrying partial statistical knowledge of an uncertain reliability [4], and propagating it gives the strongest output, bounds on the whole distribution of a node’s belief, from which any decision quantity over that distribution can be bounded in turn. The extension raises one new mathematical difficulty at the conditioning step, and this section locates it, resolves it, and states what is proved and what is not.

5.10.1 The conditioning combination is not a convolution

The chain and union steps of the propagation raise no new issue for distributional values, because the quantities they combine are independent for the same structural reason that makes the point computation exact, and p-box arithmetic under independence is standard [5]. The difficulty sits at the conditioning combination. Taking the conditioning nodes one at a time, Eq. (5.6) is the convex combination

$$W \cdot A + (1 - W) \cdot B,$$

where W is the conditioning node’s contextual belief and A, B the conditional results of its two states. All three are now uncertain quantities, and two dependencies bind them. A and B are functions of the same upstream reliabilities, so they are dependent on each other, and the two coefficients are one random variable, not two. Convolving the terms as if independent ignores both bindings, and the consequence is unsoundness: probability mass escapes the unit interval, by as much as 0.34 on the benchmark grid of Section 5.8. A distributional result that assigns mass outside $[0, 1]$ bounds nothing.

The framework’s operator integrates over the weight’s own uncertainty. At each discretisation level of W ’s bounding functions it scales the two branch distributions by w and $1 - w$ and combines them under an explicit bound on their dependence, mixes the results across levels, and envelopes the mixture built on W ’s lower bounding function with that built on its upper. Applied one conditioning node at a time, the nested two-way combination reproduces the full mixture over all $2^{|C|}$ states. The output is finally intersected with $[0, 1]$, a projection that discards only values a probability cannot take and so can tighten but never invalidate a bound. The projection is guarded: it raises an error if the excursion ever exceeds floating-point round-off by orders of magnitude, so that a soundness regression cannot hide inside it.

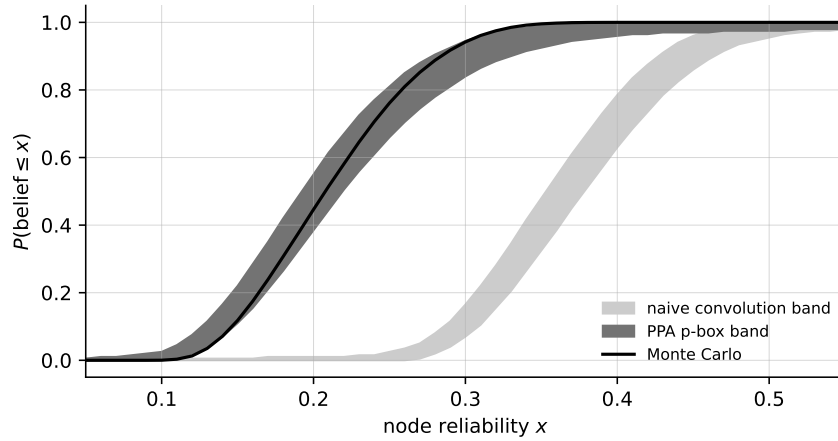


Figure 5.5: Cumulative distribution of one sink’s belief under uniform component inputs on a twelve-node random DAG. The toolkit’s band brackets the Monte Carlo distribution; the band from a convolution operator is shifted to higher reliability and places mass where the true distribution has none.

5.10.2 Two dependence bounds, one proved and one open

The dependence bound between the branches is where provability and tightness separate. Neither is the same as soundness. Sound means the computed pair of bounding functions contains the true distribution, and a sound result is never wrong. Tight means the pair is narrow, and a sound result can still be too wide to use. The choice of bound trades one against the other.

Bounding the branches by the Fréchet–Hoeffding inequalities is valid under any joint dependence whatsoever, so the resulting belief bounds inherit soundness from a classical theorem [5] with no proof needed here. The cost is width, and on hard cases the width can approach vacuity. The framework’s default operator instead restricts the dependence bound to non-negative dependence, on the ground that A and B are both non-decreasing functions of the shared upstream reliabilities and so cannot move against each other. The restricted operator is tighter and never vacuous in practice, and it has produced no soundness violation in any of the fifty validation configurations checked against Monte Carlo ground truth. But the motivating argument is not a proof. The dependence-bound literature offers a strict trichotomy, independence, a fully specified dependence, or no assumption at all, with no theorem for positive-but-otherwise-unknown dependence, and an attempted derivation through the natural ordering-of-dependence route meets an obstruction that is not a technicality: the comparison function is not uniformly ordered, and independent against comonotone uniform inputs gives combined distributions that cross. Soundness of the restricted operator therefore stands as an open question, stated as such, and the framework keeps the distinction operational: the restricted operator is the default, and the Fréchet operator is used wherever a claim must rest on proof alone.

5.10.3 What the bounds are worth

Soundness never varies across the validation sweep, and tightness does, so the output is characterised by an envelope of tightness values. Figure 5.6 sweeps input-distribution shapes, uncertainty widths and reconvergence regimes on the benchmark grid. The bound band is near 0.18 of the unit range under high component reliability and weak

reconvergence, and about 0.70 under deep reconvergence with broadly uncertain components. It is driven by the regime and the input width; the shape of the input distributions has little effect. There are no violations anywhere in the sweep. Deep reconvergence under broad uncertainty is where the branch dependence is strongest and any distribution-free bound on it loosest, so the shape of the envelope follows from the dependence structure and is not an artefact of the implementation.

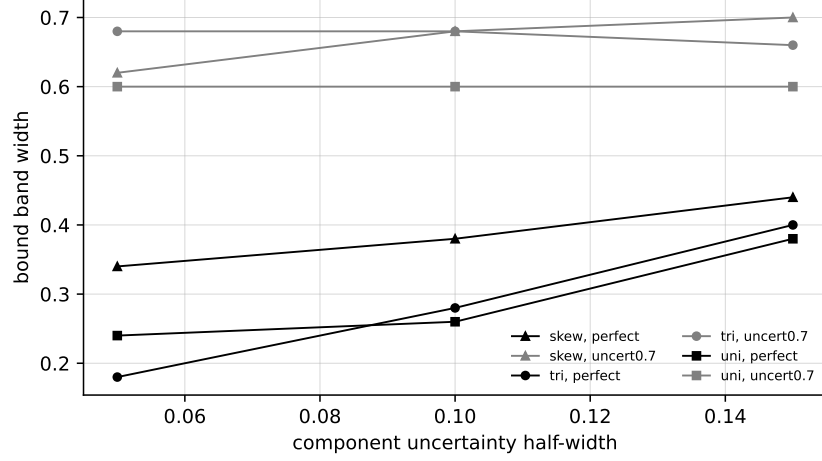


Figure 5.6: Tightness envelope of the propagated probability-box bound on the benchmark grid: band width against component uncertainty width, across input-distribution shapes and reconvergence regimes.

The output is a certificate that no point-valued exact method produces. The output bounds the distribution of $b(v)$, so for a stated reliability requirement x^* it bounds $P(b(v) \leq x^*)$, the probability that the node fails its requirement, analytically and from a single propagation. Table 5.5 shows the certified band on that probability at a representative requirement of $x^* = 0.95$ on the benchmark grid, between 0.02 and 0.12 wide across the regimes tested. Matching that precision by simulation, planned conservatively without foreknowledge of the answer, costs between 267 and 9,604 samples of the network’s uncertain inputs, and the result is a statistical estimate, not a certified bound. The planning figure is the one to quote, because the retrospective sample requirement collapses whenever the true probability sits near 0 or 1, correct statistics but knowable only in hindsight. A decision diagram has no analytic route to this quantity and would

fall back on simulation.

Table 5.5: Certified bound on the requirement-violation probability $P(b(v) \leq 0.95)$ at the benchmark grid’s target node, from one propagation, against the Monte Carlo sample count required to match the certified precision under worst-case a-priori planning.

Regime	Width	Certified band	Band width	MC samples
perfect components	0.05	[0.00, 0.02]	0.02	9,604
perfect components	0.10	[0.00, 0.10]	0.10	385
perfect components	0.15	[0.00, 0.12]	0.12	267
uncertain components	0.05–0.15	[0.98, 1.00]	0.02	9,604

5.10.4 The cost of the distribution

Distributional propagation is the expensive form. Its cost is set by the discretisation level of the bounding functions and grows faster than linearly with it. Table 5.6 gives the time at four levels on the fifteen-node reference network and, as an independent measurement on a different network, on the 23-node power network of Section 5.7.2. On the reference network the cost rises by factors of about 5, 6.5 and 7.1 per doubling of the level; on the power network by 2.5, 6.75 and 7.8. The two agree at the upper levels and differ at the lowest, where the measurements are a few seconds and sensitive to timing noise. Identification does not depend on the level, so the whole of the cost is in propagation. The bounds are sound at every level, so the level buys precision and not correctness, and levels between 25 and 100 are the practical band on networks of the scale the exact method suits. The conditioning depth itself is unchanged across all three value forms, so imprecision never relaxes the structural cost of Section 5.8, and a network too reconvergent for exact point propagation is equally beyond exact interval or distributional propagation. Where a network supports exact propagation but not the distributional dial, the exact interval range together with sampling is the practical recourse.

One consequence of this cost is recorded because it was met in practice. The library the toolkit uses for probability-box arithmetic sets a default level of 200 when none is requested. A propagation on the 26-node KarlNetwork of Table 5.2 at that default

Table 5.6: Probability-box propagation time by discretisation level, on the fifteen-node reference network and on the 23-node power network. Identification is under 0.01 s at every level on both.

Network	Level 25	Level 50	Level 100	Level 200
counterexample-n15	1.2 s	6.0 s	39 s	276 s
power-network	3.05 s	7.76 s	52.4 s	407 s

ran for seventeen minutes without finishing, while the same propagation at level 50 completes in 546 seconds; the two figures describe one method at two settings. Chapter 9 returns to this point, since the interface at present exposes no control over the level.

5.11 Case Study: A Medical Drone Logistics Network

The three capabilities of this chapter are applied together on a real planning problem. Jones et al. designed a conceptual medical drone delivery network for Scotland [11], optimising station placement and network structure over real hospital, airport, and candidate-station locations, with two drone types, a short-range vertical take-off type with a nominal range of 70 km and a long-range fixed-wing type with a nominal range of 700 km, trading off delivery time, capital cost, and resilience. Their own resilience analysis works with point probabilities. The question their framework cannot answer, and this toolkit can, is how confidently each location in a candidate design can be called reachable from the supply hubs once the stated uncertainty in the component availabilities is carried through the network.

5.11.1 Inputs, traced to their source

Every input of the reliability model derives from the source study, with one stated extension. Location availability follows the study’s own resilience assumption, hub locations always available and every other location failing independently with probability 0.2. The hub availability is kept at exactly 1, and the non-hub availability becomes the interval $[0.75, 0.85]$ centred on the study’s figure, because that figure is stated in the study as an assumption, and an interval input makes the whole analysis a sensitivity

assessment of that assumption. A connection exists, per drone type, when the inter-location distance is within the type’s stated nominal range, the study’s own construction rule applied unchanged, and where both types serve a connection the two transmissions combine as independent alternatives.

The one extension concerns weather. The source study holds wind conditions constant in its own analysis and defers weather-driven range variation as an uncertainty that would need expert elicitation. The reliability model extends that deferred point and nothing else. A connection operating comfortably within a drone’s range is treated as reliable. A connection whose length falls in the final ten per cent of the range receives the vacuous interval $[0, 1]$, the statement that adverse weather may or may not close that route; the ten per cent is an illustrative derating margin, since the study quantifies none. Direction, finally, follows the supply logic. The study’s connections are undirected because a route can be flown either way, but the reliability question concerns supply reaching facilities from hubs, so the dependency graph is directed from the hub tier outward, matching the study’s own hub-and-spoke architecture.

5.11.2 Three designs and the redundancy boundary

The study’s actual optimised layouts are not published, so three configurations stand as proxies for the qualitative character of three of its described trade-off points: a fixed-wing-reliant centralised design serving remote locations through a hub backbone, 217 locations and 263 connections, a densely interconnected short-range design, 242 locations and 1,753 connections, and a minimal-investment design retaining only pre-existing infrastructure, 230 locations and 1,648 connections. They are proxies for a described character, not reproductions of an unavailable result, and every conclusion below attaches to them as such.

Constructing the dense designs reaches the boundary of exact analysis. Connecting every operationally plausible pair of locations produces a network whose identification alone did not complete, exhausting the memory of the analysis machine within minutes, so unrestricted connectivity sits beyond the practical range of exact analysis outright. The design parameter that controls the boundary is an engineering quantity, the num-

ber of alternate routes provisioned per location. It is a redundancy choice made by the designer, and the method has no parameter of its own. Provisioning sixteen routes per location brings the conditioning width of both dense designs to 16, inside the practical ceiling of around eighteen that earlier experience with the method located, and provisioning beyond sixteen changes neither the width nor the runtime materially, so the reconvergent structure of this network saturates there. The centralised design, tree-like by construction, carries width 10. Full exact interval propagation completes in under 25 seconds on the dense designs and in under a second on the centralised one.

The same boundary was measured for the decision-diagram alternative on the same networks, with identical inputs, and the two methods agree to floating-point precision everywhere both complete. At low redundancy both are comfortably fast, and the one-shot interval computation of the propagation is faster, by roughly a factor of fifty on the centralised design and fifteen on the minimal design provisioned at six routes. Raising the provision, the diagram construction completes up to twelve routes per location, at a build of some 123,000 nodes in 45 seconds, and did not complete within a documented budget from fourteen routes, under dynamic reordering, while the propagation’s runtime stays near five seconds across the same sweep. The claim this supports is narrow. Both methods are governed by the same structural width, and on this one real network family the propagation’s practical range, measured along a real redundancy parameter, extends further than the diagram’s, with the crossover between twelve and fourteen routes.

Distributional propagation has its own, tighter boundary on the same family, as Section 5.10.4 would predict. The full probability-box analysis completes in seconds on the centralised design and in minutes on the minimal design provisioned at six routes, and at eight routes it did not complete within an hour, so distributional propagation is affordable on the sparser designs, and interval propagation, which is exact and cheap, is what scales to the dense ones.

The structural figures of Table 5.7 were remeasured after the correction of the eligibility test described in Chapter 4, because the drone designs, unlike the synthetic corpus, contain hub nodes with prior exactly one. The correction reduced the maximum conditioning width of the dense decentralised design from 17 to 16 and left every

belief unchanged, which is the expected signature: the original test inflated the diamond count around such hubs without affecting the values.

5.11.3 What the propagated bounds say

Table 5.7 collects the propagated reliability bounds, and they differ in the way the designs differ. In the centralised design every location’s band is narrow, at most 0.10 wide with a mean of 0.089: reachability inherits the availability band assumed for the locations, because a tree accumulates little dependence, and it has no protection, since any single-route failure disconnects service. The dense designs spread. Well-connected locations are reachable with near-certainty, and the worst-served facility, Islay Hospital, an island site, carries a reachability probability bounded between 0.56 and 0.72 in both dense designs. That interval is the answer to the planning question under the stated input uncertainty, and it is a different kind of answer from the one a point analysis gives. A facility whose reachability is confidently 0.6 and one whose reachability is anywhere between 0.56 and 0.72 call for different interventions: better routes for the first, and better component data before committing capital for the second. No point-valued analysis of the same design can tell them apart. Figure 5.7 maps the dense decentralised design coloured by the lower reliability bound and by band width. The uncertainty concentrates at island and far-northern facilities, which is where additional redundancy, or better availability data, buys the most, and the map places the toolkit’s output on the geography the source study planned over.

Table 5.7: The three proxy designs: structure, conditioning width, and propagated interval reliability bounds. Every belief is contained in $[0, 1]$ with lower bound at most upper bound. The worst-served facility in both dense designs is Islay Hospital.

Design	$ V $	$ E $	width	mean band	max band	worst facility
fixed-wing centralised	217	263	10	0.089	0.100	[0.750, 0.850]
dense decentralised	242	1753	16	0.094	0.161	[0.562, 0.722]
concentrated minimal	230	1648	16	0.093	0.161	[0.562, 0.722]

The case study also gives the design process a measured trade-off. More provisioned redundancy improves resilience and simultaneously raises the cost of verifying it exactly,

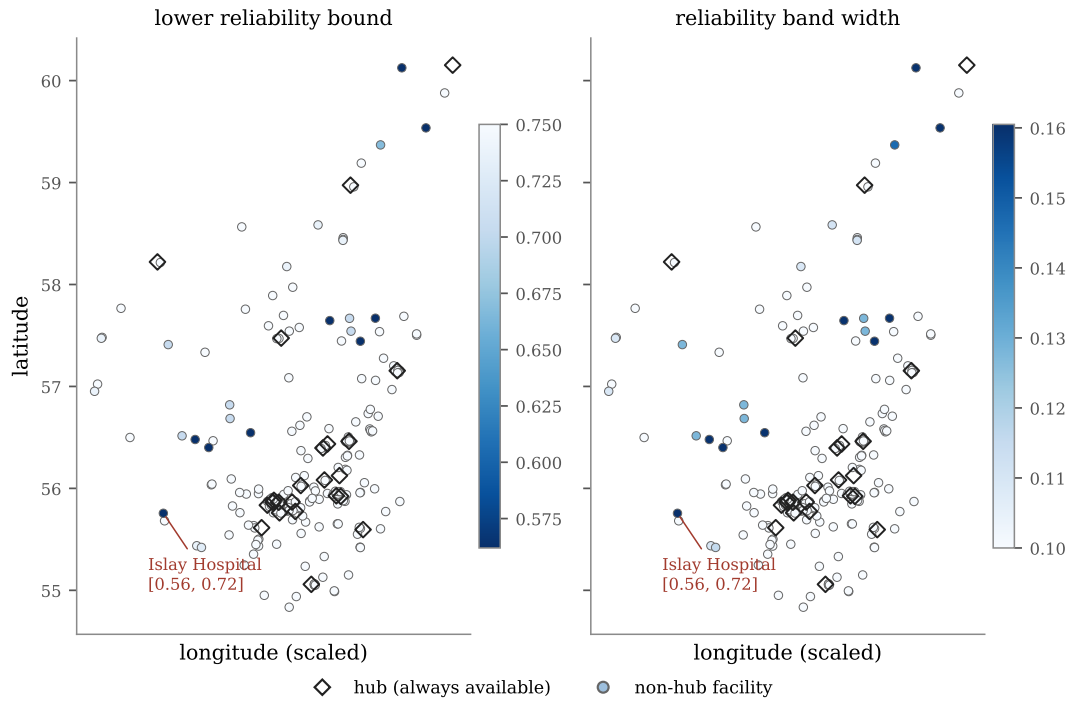


Figure 5.7: Reliability map of the dense decentralised design: each facility coloured by the lower bound of its propagated reachability (left, darker is less certainly reachable) and by the width of its reliability band (right, darker is more uncertain). Hubs, drawn as diamonds, are assumed always available. Uncertainty concentrates at island and far-northern facilities, and the worst-served facility, Islay Hospital, is bounded at $[0.56, 0.72]$.

and on this network that trade-off is a measured curve: the width saturates at sixteen routes per location, exact verification is practical there, and the point at which the alternative exact method stops being practical sits measurably earlier. Hence a designer choosing between these configurations can see, in the same analysis, which facilities are uncertain, why they are uncertain, and what verifying a more redundant design will cost.

5.12 Discussion

For exact point-valued reliability the toolkit offers no asymptotic advantage over a well-ordered decision diagram. Both are exponential in a width of the network, the conditioning width for the toolkit and the pathwidth for the diagram, and across the corpus neither dominates; the toolkit’s operation count tracks the sifted diagram’s node count to within a constant factor. Being exact, the toolkit is intended for the sparse to moderate regime, and on the corpus it is exact and practical to about fifty nodes, with dense or deeply reconvergent structure the limiting case, a ceiling shared with every exact method.

For interval inputs the toolkit returns the exact range at machine precision, because belief is monotone in every input, and it does so at lower one-shot cost than evaluating a diagram at the two corners. For probability-box inputs it returns sound bounds on the distribution of every belief. Their tightness is structure-dependent: tight for high-reliability or weakly reconvergent systems and conservative, though still sound, for strongly reconvergent systems with broadly uncertain components. Probability-box propagation is also costly, worse than quadratic in the discretisation level, and in the present implementation practical only for small networks; for larger networks the recommended combination is interval propagation together with Monte Carlo sampling. The bounds are sound at the discretisation level up to floating-point round-off, as is standard for Williamson–Downs arithmetic.

Two operators are provided for the recombination step. The Fréchet operator is sound for any dependence by the classical theorem. The positive-dependence operator

is tighter and is sound on every configuration tested, but its soundness is not proved as a theorem, and this is stated as an open problem. The published power-network reproduction of Section 5.7.2 carries a residual of order 10^{-3} at two of three regimes that is not a computation error and is not explained; it is also left open.

The toolkit addresses directed acyclic structures. A cyclic network is analysed only after the orientation of Chapter 3, and the general cyclic case is outside the present scope.

5.13 Summary

The belief of every node, the probability that it operates and is reached from a source, has been computed in one pass over the layered order, with the closed-form update wherever parent signals are independent and conditional enumeration on the diamonds of Chapter 4 wherever they are not. Lemmas 5.2 to 5.5 carry the exactness argument, and the supernode reduction with its store makes the recursion progressive and non-repeating. The propagation has been validated to machine precision against a decision-diagram oracle on 129 graphs and on named and adversarial families. It reproduces the published exact values of a grid benchmark at every sink, and of a power distribution network at one of three regimes, with a small open residual at the other two. Its cost is governed by the conditioning width and is competitive with, but not better than, a well-ordered decision diagram. Extended to intervals, the propagation is exact and cheaper than the diagram’s corner route; extended to probability boxes, it is sound, with a characterised tightness envelope and a cost that grows steeply with the discretisation level. On the medical drone network the three capabilities give a designer, in one analysis, the reachability bounds of every facility, the redundancy setting at which exact verification stops being practical, and the facilities whose uncertainty is structural.

Publications

The method of this chapter was first published as the Information Propagation Method at the 2023 International Conference on System Reliability and Safety [73]. The exact-

ness proofs, the interval and probability-box extensions and the corpus validation are the subject of a paper submitted to *Reliability Engineering & System Safety*.

Chapter 6

Information as Resources: Capacity Flow Toolkit

6.1 Introduction

A network that is connected can still fail to deliver. When a pipe is throttled or a feeder is derated, supply reaches every demand point and the amount that arrives is short of what is needed. The probabilistic interpretation of Chapter 5 asks whether supply reaches a node; the capacity interpretation asks how much, and what limits it. Under this interpretation the value attached to an edge is the most that can pass along it and the value attached to a node is the most that can pass through it. The Capacity Flow Toolkit is the set of modules that answers the second question. It reads the graph object of Chapter 3 directly, uses neither the layered iteration sets nor the Network Decomposition Module of Chapter 4, and is point-valued: every capacity is a single number, and no interval or probability-box propagation is claimed for this interpretation. Built on the maximum-flow minimum-cut theory reviewed in Chapter 2, the toolkit answers six questions about a capacitated DAG:

- Q1.** What maximum throughput can the network deliver under its current capacities?
- Q2.** Which cut sets form the binding bottleneck on that throughput, and how many equivalent cuts are there?
- Q3.** Which component failures or capacity reductions most reduce deliverable flow?
- Q4.** What capacity change on a given edge maintains, or first reaches, a target service level?
- Q5.** How much path-disjoint redundancy exists between sources and sinks?
- Q6.** Which components are single points of failure?

What is new in this chapter is the collection of these answers on one graph object, with every one of them checked. The flow model is stated with its reductions for several sources and sinks and for node capacities, each proved exact (Section 6.3). Three maximum-flow solvers are provided, with post-solve checks on every result (Section 6.4). The six questions are answered by modules that share one baseline solve, among them

an enumeration of every minimum cut through the residual lattice, failure analyses over brute-force and candidate-restricted sets, and a parametric threshold method that uses the piecewise-linear dependence of the maximum flow on a single capacity (Sections 6.5 and 6.6). Every module is validated against an independent computation on a corpus of twelve networks and on generated instances of up to 10^5 edges (Section 6.7), and the toolkit is applied to the IEEE Reliability Test System (Section 6.8). The chapter closes with the limitations of a point-valued flow model and the extensions that follow from it. Section 6.2 first recalls what the chapter builds on.

6.2 Background

Chapter 2 reviewed the maximum-flow minimum-cut theorem of Ford and Fulkerson [2, 33] and three results from the same theory that this toolkit uses. A node with its own capacity is handled by splitting it into an entry node and an exit node joined by an edge of that capacity. Any feasible flow decomposes into source-to-sink path flows that sum to the total. The minimum cuts of a network lie between the set of nodes reachable from the source in the residual graph and the set that can reach the sink in the reverse residual graph. Menger’s theorem, in its edge and node forms, gives the number of disjoint paths as a unit-capacity maximum flow. The Birnbaum importance of a component [34] is the model for the marginal measures of Section 6.5.3, with the difference that here it is a throughput range without a probability weighting.

Chapter 2 also reviewed the two treatments of uncertain capacities in the literature, the adversarial uncertainty set of robust optimisation [35] and the discrete random capacity of stochastic-flow reliability [36]. Neither is implemented here. The toolkit computes exact answers for a single point-valued capacity assignment, and a best-case and worst-case pair of assignments is two separate solves.

6.3 The Flow Model

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \rightarrow \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modelled, let

$$b : V \rightarrow \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v . Define the flow variable

$$f : E \rightarrow \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u, v))$ for each edge $(u, v) \in E$. For node v , define

$$\delta^+(v) := \{(v, w) \in E\}, \quad \delta^-(v) := \{(u, v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t , subject to edge capacities c_{uv} for $(u, v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. 6.1. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The performance measure is the deliverable throughput from s to t , a finer quantity than simple path connectivity.

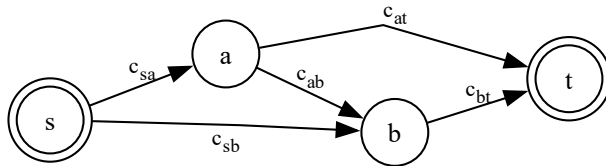


Figure 6.1: Minimal single-source single-sink capacitated DAG network.

For this base case, let $f : E \rightarrow \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t . The optimisation model is

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (6.1)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (6.2)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (6.3)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (6.4)$$

Taking F^* as the optimal (maximum feasible) value of F , if the sink node t has a required demand level D_t , the flow module quantifies service adequacy through the throughput margin

$$\Delta_t := F^* - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

6.3.1 Several sources and sinks

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure 6.2a shows the original multi-source multi-sink network.

By introducing a super-source s^* and a super-sink t^* , as shown in Fig. 6.2b, the reduced graph is

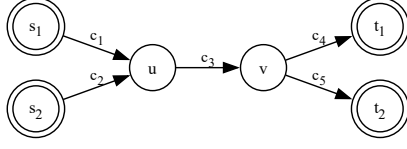
$$V' = V \cup \{s^*, t^*\},$$

$$E' = E \cup \{(s^*, s) : s \in S\} \cup \{(t, t^*) : t \in T\}.$$

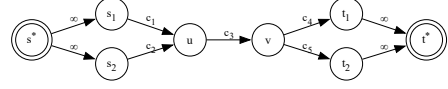
The connector capacities are set to non-binding values (implemented as Inf in the

solver):

$$c_{s^*s} = +\infty \quad \forall s \in S, \quad c_{tt^*} = +\infty \quad \forall t \in T.$$



(a) Original multi-source multi-sink network.



(b) Equivalent super-source/super-sink reduction.

Figure 6.2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

Proposition 6.1 (Exactness of the super-terminal reduction). *The optimal value of the multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals s^* and t^* .*

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F .

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^*s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \quad \forall s \in S,$$

and

$$f'_{tt^*} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \quad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, all capacity constraints from the original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from s^* and the net inflow to t^* are both equal to F . Therefore f' is a feasible s^* - t^* flow of value F .

Conversely, let f' be any feasible s^* - t^* flow on G' : then, restricting f' to the original edge set E gives a flow f on G . Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited

directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f' . Since feasible flows map in both directions without changing objective value, the two optimal values are equal. $\square$

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (6.5)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus (S \cup T), \quad (6.6)$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \quad (6.7)$$

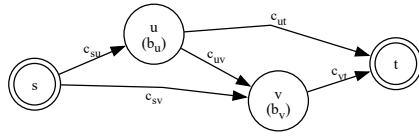
$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \quad (6.8)$$

6.3.2 Node capacities by node splitting

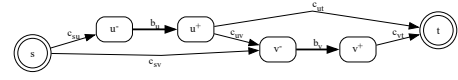
The framework accepts values on nodes as well as on edges, so a through-flow limit on a component can be modelled directly. As shown in Figure 6.3, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+ , connected by an internal edge:

$$(v^-, v^+), \quad c_{v^--v^+} = b_v.$$

The original incoming edges of v are redirected to v^- , and every original outgoing edge is emitted from v^+ .



(a) Original network with edge and node capacities.



(b) Equivalent node-split edge-capacitated network.

Figure 6.3: Node-splitting transformation.

This extends the edge-capacitated flow model with

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (6.9)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c. \quad (6.10)$$

Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimisation model then becomes

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (6.11)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (6.12)$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (6.13)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (6.14)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (6.15)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (6.16)$$

6.4 Maximum-Flow Solvers

The toolkit provides three solvers for F^* , summarised in Table 6.1; the algorithms are standard and are described in Ahuja, Magnanti and Orlin [74]. All three read the graph object of Chapter 3 directly, apply the super-terminal reduction of Section 6.3 when the network has more than one source or sink, and return the flow, the min-cut partition and the flow value. When node capacities are supplied the node-splitting transformation is applied before the solve and the results are mapped back to the original nodes and edges, so that edge flows, sink allocations and cut-side node sets are reported on the network as given.

Solver	Complexity	Best Fit
:edmonds_karp	$O(VE^2)$	Small networks
:dinic	$O(V^2E)$	Larger layered networks
:push_relabel	$O(V^2E) - O(V^3)^a$	Dense, high fan-out networks

^a Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 6.1: The three maximum-flow solvers.

6.4.1 Augmenting-path solvers

By the max-flow min-cut theorem of Ford and Fulkerson [2, 33], the maximum flow from s to t equals the smallest total capacity of any set of edges whose removal separates them. The augmenting-path solvers compute both at once.

Definition 6.2 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$ and a feasible flow f , the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E, f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad (\text{forward residual}), \\ f_{vu} & (v, u) \in E \quad (\text{backward residual}). \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

Algorithm 3 Ford-Fulkerson Min-Cut Max-Flow

```

1: Initialise residual graph  $G_f \leftarrow G$  with zero flow on all edges
2: while there exists an augmenting path  $P$  from source  $s$  to sink  $t$  in  $G_f$  do
3:    $\Delta \leftarrow \min_{e \in P} c_f(e)$  ▷ Bottleneck capacity along  $P$ 
4:   for each edge  $e = (u, v) \in P$  do
5:      $f(u, v) \leftarrow f(u, v) + \Delta$  ▷ Push flow forward
6:      $f(v, u) \leftarrow f(v, u) - \Delta$  ▷ Update reverse edge
7:   end for
8:   Update  $G_f$  to reflect new residual capacities
9: end while
10: Let  $A$  be the set of nodes reachable from  $s$  in  $G_f$ 
11: Min-cut =  $\{(u, v) \mid u \in A, v \notin A\}$ 
12: return Max-flow value  $|f|$ , min-cut set  $A$ 

```

As Algorithm 3 shows, the augmenting-path solvers repeatedly find a path and push flow along it. Most of the execution time is spent in four lookups:

- Capacity on any edge $u-v$
- Current flow on any edge $u-v$
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Fulkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver [75] uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a blocking-flow depth-first search. The level graph restricts traversal to edges $u \rightarrow v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor `ptr[u]` to track each node’s position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework’s default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

6.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

Algorithm 4 Push-Relabel Min-Cut Max-Flow

```

1: height[ $s^*$ ]  $\leftarrow |V|$ ; height[ $v$ ]  $\leftarrow 0 \forall v \neq s^*$ 
2: Push maximum admissible flow from  $s^*$  to all neighbours; initialise excess[ $v$ ]
3: while  $\exists$  active node  $u$  with excess[ $u$ ]  $> 0$  do
4:   if  $\exists$  neighbour  $v$  with  $r(u, v) > 0$  and height[ $u$ ] = height[ $v$ ] + 1 then
5:      $\delta \leftarrow \min(\text{excess}[u], r(u, v))$ 
6:     Push  $\delta$  units from  $u$  to  $v$ ; update excess[ $u$ ], excess[ $v$ ],  $r(u, v)$ ,  $r(v, u)$ 
7:   else
8:     height[ $u$ ]  $\leftarrow 1 + \min_{v: r(u, v) > 0} \text{height}[v]$   $\triangleright$  Relabel
9:   end if
10: end while
11: Traverse final flow state from  $s^*$  to identify min-cut partition
12: return Max-flow value  $|f|$ , min-cut set

```

6.5 The Questions the Toolkit Answers

The six questions of Section 6.1 are answered by modules that share the solvers of Section 6.4. Figure 6.4 shows how the modules are called from one baseline solve. This section takes the questions in turn; the throughput question is the baseline solve itself and the margin Δ_t of Section 6.3.

6.5.1 The binding cut and its lattice of equivalent cuts

The flow module solvers all compute the max flow F^* by constructing a representative valid min-cut S^* in the given network G . This output by itself neither indicates nor guarantees that S^* is the only possible min-cut set in G .

The distinction matters for engineering decisions. Infrastructure systems are designed with redundancy, and in such networks several structurally distinct partitions of V commonly have the same cut capacity F^* .

While the baseline max flow value directly addresses the first analysis objective, the utility module uses the already-computed min-cut and residual graph to build the extended partition S^{**} and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from t^* in G_f .

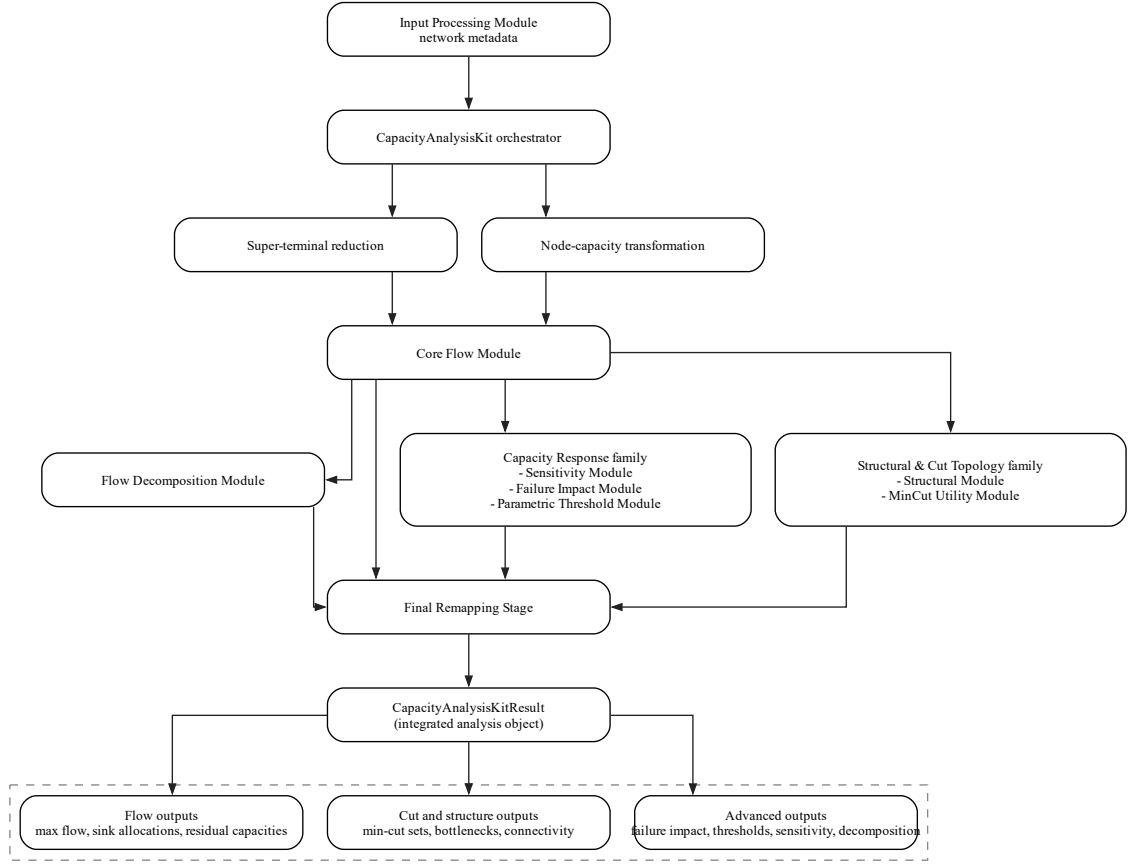


Figure 6.4: Runtime workflow for the flow-capacity toolkit with available modules.

Definition 6.3 (Extended Source Partition). *The extended source-side region is*

$$S^{**} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity. Consequently $S^ \subseteq S^{**}$.*

Definition 6.4 (Free Zone). *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{**} \setminus S^*.$$

Nodes in $\mathcal{F}$ can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of G under $f^\star$ takes the form $S = S^\star \cup R$ for some $R \subseteq \mathcal{F}$.

The free zone size $|\mathcal{F}|$ itself determines the degeneracy of the bottleneck structure. When $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and $S^\star$ is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition $S^{\star\star}$ and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. It then follows that two distinct edge sets can be derived directly from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u, v) \in E : f_{uv}^\star = c_{uv}, u \in S^{\star\star}, v \notin S^\star\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u, v) \in E : f_{uv}^\star = c_{uv}, u \in S^\star, v \in T^{\star\star}\},$$

where $T^{\star\star} = V \setminus S^{\star\star}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^\star \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq \text{cut_limit}$ all minimum cuts are returned and `is_complete = true`; otherwise, the first `cut_limit` cuts are returned with `is_complete = false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

6.5.2 Redundancy and single points of failure

The structural module identifies components whose importance follows from their position in G alone, whatever the capacities.

Understanding the route-level structure of G is a prerequisite for any path-level fragility or flow-distribution analysis. The module exposes a base path enumerator function that performs a depth-first search on the topologically sorted iteration set of the graph object of Chapter 3. Since G is a DAG, exact enumeration is always finite; however, highly redundant networks can admit exponentially many routes, so a `path_limit` parameter bounds the number of enumerated paths and returns a sample and not the complete set $\mathcal{P}$.

Flow Decomposition. Two networks can share identical topology and the same F^* while concentrating flow through entirely different routes. During design decisions where multiple configurations are evaluated and per-route visibility is needed, this decomposition exposes how f^* is distributed across $\mathcal{P}$.

Given an optimal flow f^* and a set of flow-agnostic paths $\mathcal{P}$, for each path $P = (v_0, v_1, \dots, v_k) \in \mathcal{P}$, the bottleneck flow contribution is the minimum flow on any edge along P :

$$\phi(P) = \min_i f_{v_i v_{i+1}}^*,$$

Thus, the edge meeting this minimum must be the path's local bottleneck. Paths are sorted in descending order of $\phi(P)$ as a ranked decomposition of f^* into per-route contributions. Where the min-cut module's ranking informs global capacity upgrade decisions, this ranking informs targeted interventions at the route level. In a network supplying two distinct demand nodes via separate routes, the min-cut module points to the tightest cut edge across the whole network, whereas the decomposition identifies independently which component is constraining delivery to each demand node.

Structural Single Point of Failure Nodes.

Definition 6.5 (Structural Single Point of Failure). *A node $v \in V \setminus (S \cup T)$ is a SPOF node if $F^*(G \setminus \{v\}) = 0$, where removing v also removes all incident edges. A*

SPOF node is one through which every source-to-sink path must pass; its removal alone disconnects the network regardless of any capacity assignment.

The candidate set is restricted to nodes lying on at least one s - t path,

$$\mathcal{C} = (\text{fwd}(S) \cap \text{bwd}(T)) \setminus (S \cup T),$$

where $\text{fwd}(S)$ and $\text{bwd}(T)$ are the sets of nodes forward-reachable from S and backward-reachable from T in G respectively. Nodes outside $\mathcal{C}$ lie on no s - t path and cannot be SPOF nodes. For each $v \in \mathcal{C}$, a BFS is run on $G[V \setminus \{v\}]$; if no sink in T is reachable from any source in S , v is recorded as a SPOF node. The function requires one BFS per candidate and runs in $O(|\mathcal{C}| \cdot (|V| + |E|))$ time.

In a node-capacitated network, a node appearing in the min-cut is not necessarily a SPOF node in this structural sense. A capacity-constrained node enters the cut-space by way of its split edges $(v_{\text{in}}, v_{\text{out}})$ with capacity b_v and is not necessarily a structural choke-point in G . In these networks, a node can simultaneously be a flow-limiting bottleneck and a SPOF node.

6.5.3 Throughput-limiting edges and their marginal values

At optimality, the flow f^* partitions the edge set into two classes: edges carrying strictly less than their capacity, $f_{uv}^* < c_{uv}$, and edges that carry exactly their capacity,

$$f_{uv}^* = c_{uv}.$$

The latter are referred to as *saturated edges* in this work. By the max-flow min-cut theorem, every minimum cut set is made up only of saturated edges; therefore, any augmenting path from S to T must cross at least one saturated edge. Since the binding constraint lies elsewhere, *unsaturated edges* carry slack and are transparent to perturbation; increasing or decreasing their capacity by a small enough amount has little to no effect on F^* . The Flow Sensitivity module focuses only on saturated edges, since they collectively form the capacity-critical boundary of the network. This module exposes

the following three functions.

Critical Edge Ranking. A direct measure of an edge’s failure impact is how much the max flow drops when that edge is disconnected from the network, referred to as edge criticality in this module. The function re-runs the flow solver on a modified network for each candidate. For each saturated edge (u, v) , the critical impact is:

$$\Delta_{uv}^* = F^*(G, c) - F^*(G, c_0^{(uv)}),$$

where $c_0^{(uv)}$ denotes the capacity function with c_{uv} set to zero and all other capacities unchanged. The function returns the edges in descending order of Δ_{uv}^* .

Marginal Capacity Values. Similar to the critical edge ranker, the function is scoped to saturated edges. For each saturated edge (u, v) , it quantifies the incremental gain in F^* per unit of additional capacity,

$$\mu_{uv} = \frac{F^*(G, c_\delta^{(uv)}) - F^*(G, c)}{\delta},$$

where $c_\delta^{(uv)}$ increases c_{uv} by δ with all other capacities unchanged.

In highly redundant networks, bypass routes often exist that can absorb rerouted flow and upgrade priorities are not always clear. In these systems, an edge can rank high for failure impact but have a lower marginal value. The critical edge function identifies which edges are most damaging to lose while the marginal capacity function complements this by identifying which ones are most worth upgrading.

Marginal Range. Both Δ^* and μ are snapshot evaluations relative to the current capacity assignment c . Depending on the network structure, an edge with a low μ under the current inputs may become highly influential under different capacity values. This Marginal Range function evaluates the total range of influence a single edge (u, v) can exert on F^* :

$$\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)}),$$

where $c_\infty^{(uv)}$ sets $c_{uv} = \infty$ and $c_0^{(uv)}$ sets $c_{uv} = 0$, and all other capacities fixed. This function is analogous to the classical Birnbaum importance measure $I_B(i) = \partial h(\mathbf{p}) / \partial p_i$ (see [34]). However, the analogy is conceptual: $\mathcal{B}_{uv}$ is a deterministic throughput range, while the Birnbaum measure is a probability-weighted sensitivity.

6.5.4 Failure impact and capacity degradation

The failure impact module models outright component failures and degraded operating states.

Single and Combined Edge Failures. For a single failed edge (u, v) , the throughput impact is:

$$\Delta_{uv} = F^*(G, c) - F^*(G, c_0^{(uv)}),$$

where $c_0^{(uv)}$ sets $c_{uv} = 0$ with all other capacities unchanged and candidate edges are restricted to E_{some} .

Real failure events are not always isolated to a single component. A storm in a power network can take down multiple edges simultaneously. Since the interaction between affected edges may open or close re-routing options, the combined impact of k simultaneous failures is not in general the sum of their individual Δ_{uv} impacts.

The k -edge failure function generalises the single failure case to any set of k edges removed simultaneously. For a combination $\mathcal{K} \subseteq E_{\text{some}}$ with $|\mathcal{K}| = k$, the combined impact is:

$$\Delta_{\mathcal{K}} = F^*(G, c) - F^*(G, c_0^{(\mathcal{K})}),$$

where $c_0^{(\mathcal{K})}$ sets $c_{uv} = 0$ for all $(u, v) \in \mathcal{K}$. All $\binom{|E_{\text{some}}|}{k}$ combinations require one re-solve each. In practice, because $\binom{|E_{\text{some}}|}{k}$ grows rapidly with k , this function is most tractable for $k = 2$ or $k = 3$ depending on the size and density of the network. The function returns a complete ranking of k -edge failure scenarios by throughput impact.

Capacity Degradation Scenarios. Infrastructure components do not always fail completely. Partial blockage or degraded components are more likely to present as a

reduction in c_{uv} without complete loss. Additionally, a domain expert working across multiple network states may want to compare the effect of varying input capacities and operating conditions. This function is intended to be used to efficiently compute the full F^* picture across all scenarios without needing to re-run the input processing or baseline solve steps for each. The degradation function accepts either an explicit override dictionary specifying new capacities for individual edges, or a uniform scale factor $\alpha \in [0, 1]$ applied to all finite capacities,

$$c'_{uv} = \alpha \cdot c_{uv}, \quad \forall (u, v) \in E.$$

6.5.5 Degradation and upgrade thresholds

The previous modules treat edge capacities as fixed and analyse the network's response to perturbations around them. The threshold module inverts this perspective. Given a required demand level $D_t \leq F^*$ to be met, it determines what capacity each edge must have for the network to meet that target delivery level. It is therefore a planning tool.

This module uses a structural property of F^* : it is piecewise linear in any single edge capacity c_e with all others fixed. Each linear segment maps to a fixed min-cut partition S^* , over which the threshold satisfying $F^*(c_e) = D_t$ can be recovered analytically. Therefore, breakpoints must occur exactly where the min-cut partition shifts, i.e. where a change in c_e causes a different set of edges to become the binding bottleneck.

Degradation Threshold. For a target edge (u, v) and a required flow level D_t , the degradation threshold $c_{uv}^\dagger$ is the minimum capacity the edge must retain for F^* to remain at least D_t :

$$c_{uv}^\dagger = \inf \{c \in [0, c_{uv}] : F^*(G, c_c^{(uv)}) \geq D_t\},$$

where $c_c^{(uv)}$ denotes the capacity function with c_{uv} replaced by c . The degradation margin $c_{uv} - c_{uv}^\dagger$ is the slack available before the service level is breached. The module ranks edges by ascending degradation margin, so the most vulnerable edges appear first. A small margin can be interpreted as the edge is close to becoming a binding constraint

on D_t , while a large margin means the edge can absorb substantial capacity loss without service impact.

The threshold is located by a partition-change recursion. Starting from the known endpoints $c_e = 0$ and $c_e = c_{uv}$, the algorithm evaluates F^* at the midpoint and compares the resulting min-cut partition to those at the endpoints. If both endpoints share the same partition, the segment is linear and the threshold is solved analytically. If the partitions differ, the interval contains a breakpoint and the recursion descends into the sub-interval containing D_t . This continues until the threshold is isolated to a partition-constant segment, where it is computed exactly. The result is exactly the capacity value at which F^* crosses D_t .

Two degenerate cases are handled explicitly. If $F^*(c_e = 0) \geq D_t$, the edge can degrade to zero without breaching the target. If $F^*(c_e = c_{uv}) < D_t$, the target is unachievable even at the current capacity, meaning the bottleneck lies entirely elsewhere and this edge cannot help.

Upgrade Threshold. If $F^* < D_t$ under the current capacity assignment, the upgrade threshold is the minimum capacity (u, v) must have for F^* to first reach D_t . Since the required capacity may exceed c_{uv} , the upper search bound is not known in advance. The algorithm uses a doubling search to bracket the threshold interval, repeatedly doubling the trial capacity until $F^* \geq D_t$, then applies the same partition-change recursion to locate it exactly.

An edge case arises when the edge is not on any minimum cut: expanding its capacity cannot raise F^* regardless of how large c_{uv} becomes. The doubling search detects this when F^* stops increasing before reaching D_t and returns `upgrade_ineffective = true`, indicating that reaching D_t requires addressing a different bottleneck.

When no explicit target is supplied, the module defaults to $D_t = 0.9 \cdot F^*$.

6.6 Cost

The total computational cost of any derived analysis is the product of the per-solve cost of the chosen max-flow algorithm and the number of solver calls required. Repeated-

solve tasks scale with solve count, which is the dominant cost driver in practice. Per-solve costs are summarised in Table 6.1, and Table 6.2 gives the per-task cost of every module function in units of one solve.

Module	Function	Operation	Complexity
<i>Base Solver</i>			
Base	Edmonds-Karp	BFS augmenting path	$O(VE^2)$
Base	Dinic	Level graph + blocking	$O(V^2E)$
Base	Push-Relabel	Preflow relabeling	$O(V^3)$
Node-Cap	Split transformation	Graph transform	$O(V + E)$
Node-Cap	Solve (split)	Solver on $2V, 2E$	$O(T_{\text{solver}})$
<i>Min-Cut Utility</i>			
Min-Cut	cut_edges	Extract partition	$O(E)$
Min-Cut	edges_some	BFS + filter	$O(E + T_s)$
Min-Cut	edges_every	Reachability	$O(V + E)$
Min-Cut	enumerate	Lattice ($2^{ \mathcal{F} }$)	$O(2^{ \mathcal{F} } \cdot E)^1$
<i>Structural</i>			
Struct	spof_nodes	BFS per candidate	$O(\mathcal{C} (V + E))$
Struct	enumerate_paths	DFS on DAG	$O(\mathcal{P})^2$
Struct	path_flow	Per-path scan	$O(\mathcal{P} \ell)$
Struct	redundancy	Menger (unit cap)	$O(E_c T_s)$
<i>Flow Sensitivity</i>			
Sens	critical_edges	Zero-cap solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_capacity	$+\delta$ solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_range	Range evals	$O(E_{\text{some}} T_s)$
<i>Failure Impact</i>			
Fail	single_edge	Per-edge solves	$O(E_{\text{some}} T_s)$
Fail	k_edge	Per-combination	$O(\binom{ E_s }{k}T_s)^3$
Fail	degradation	Per-scenario	$O(S T_s)$

Module	Function	Operation	Complexity
<i>Parametric Threshold</i>			
Param	degrade	Partition recursion	$O(\lg(c/\varepsilon)T_s)$
Param	upgrade	Doubling + recurse	$O(\lg^2(c/\varepsilon)T_s)$
Param	all_degrade	Per edge	$O(E_c \lg(c/\varepsilon)T_s)$

Table 6.2: Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.

6.7 Validation

Every output of the toolkit was checked against a computation that does not use its solvers. The checks were run on a set of twelve networks, the ten DAGs of the Chapter 7 validation set with the three published drone designs in place of the larger drone network, plus the PSPLIB instance of Chapter 7, each with integer capacities drawn uniformly from $[1, 20]$ per edge from a recorded seed, and four of them with node capacities drawn in the same way. The networks, seeds and files are listed in Appendix B.

The maximum-flow value was computed by all three solvers and by the `GraphsFlows.jl` library [76] as an external oracle. On every network the three solvers agree with each other and with the oracle exactly, and every returned flow passes the three post-solve checks: capacity feasibility on every edge, conservation at every transit node, and equality of the flow value and the reported cut capacity.

The structural outputs were checked by brute force. Every enumerated minimum cut was checked to have capacity equal to the maximum flow and to disconnect the sources from the sinks, and on networks with at most twenty candidate edges every edge subset that disconnects was enumerated and the reported cuts were confirmed to be exactly the minimal ones of minimum capacity. The two derived edge sets of Section 6.5.1 were checked against the same enumeration: the edges reported in every

minimum cut must equal the intersection of the enumerated cuts and the edges reported in some minimum cut their union. This check was added after the interface of Chapter 9 showed, on a 23-node network, a set of edges in every cut that lay entirely on the source side of the unique cut; the function had taken the complement of the sink-reachable set, and the earlier checks had tested the enumerated cuts without testing the two sets derived from them. Every reported saturated edge was checked to carry flow equal to its capacity. Single points of failure and the single-edge failure ranking were checked by removing each node and each edge in turn and recomputing the maximum flow with the oracle, exhaustively on the small networks and on a sample of between fifteen and forty removals on the large ones. Each degradation threshold was checked by evaluating the oracle just below and just above it, and by locating the threshold independently by bisection. The edge-disjoint path count was checked against a unit-capacity oracle flow, which is Menger’s theorem. The node-capacitated flow was checked against the oracle applied to the split graph, and the several-source and several-sink construction is exercised by every network in the set. All checks passed on all twelve networks.

The last of these checks found an error. On the IEEE network of Section 6.8.2, where ten of twenty-four buses carry a node capacity and the rest do not, the node-splitting transformation assigned the entry and exit nodes identifiers $2v$ and $2v + 1$, which collide with the identifiers of unsplit nodes when only some nodes are split and the identifiers are small and consecutive. None of the twelve corpus networks had exposed it, because in each of them every node or no node carried a capacity. The transformation now assigns identifiers above the largest identifier in the network, and the corrected code gives identical results on every network that had passed before. The change is described in Chapter 8.

6.8 Case Studies

Timings in this section are wall-clock measurements on a single core of the second call in a warm process, so that just-in-time compilation is excluded. The convention is stated in Appendix B.

6.8.1 A worked example on a synthetic network

The modules are first shown together on a synthetic network built so that each of them has something to report. It is a worked example; the case study follows in Section 6.8.2. The network is a multi-source, multi-sink capacitated DAG with 43 nodes, 107 directed edges. The network represents a distribution system that moves a resource between dispersed locations. As shown in Figure 6.6, it is structured as an 8-layer hierarchy:

- **Origin nodes** (blue): five source nodes, over-provisioned so the supply zone does not form the binding bottleneck.
- **Hub nodes** (green): eight intermediate relay nodes with high fan-out capacities.
- **Processing nodes** (red, node-capacitated) and **bypass nodes** (orange, node-capacitated): the processing corridor carries the bulk of flow; the bypass is a lightweight alternative with tighter node constraints.
- **Trunk and secondary nodes** (purple, brown): distribute flow through a re-converging structure toward the gateway layer.
- **Dispatch gateways** (light blue, node-capacitated): two gateways assigned equal edge capacity of 17 by design.
- **Distribution combiner** (pink): aggregates all gateway flow before fan-out to the sinks.
- **Sink nodes** (grey): seven demand nodes, each with incoming edge capacity 6.

Based on this layered structure, edge and node capacity assignments are chosen so that binding bottlenecks emerge at specific layers under the different analysis methods. Edge capacities in Layers 1–5 are scaled by a factor of 2 relative to the gateway layer. In this way, the gateway layer governs F^* under the edge-only solution. Similarly, to compute a measurable reduction in F^* , the processing node capacities are set to bind to the node-capacitated solve.

Two configurations of the same system are compared to show the effect of feeder reinforcement (Figures 6.6a and 6.6b). Both configurations keep the same topology and

differ in their feeder capacities. The comparison tests whether the bottleneck structure changes when the throughput does not.

Configuration A is a symmetric design with balanced feeder-edge capacities and a limited direct bypass to sink 4. The capacity is set below the sink’s 6-unit limit so that the bypass delivers partial service. Configuration B’s asymmetric gateway structure produces non-trivial parametric upgrade thresholds and non-zero marginal-range values. This version has larger feeder capacities from secondary nodes into both gateways.

Table 6.3 summarises the key structural differences between the two. The expectation is that two designs may yield the same max-flow value while differing in saturation pattern and in the set of minimum cuts, because their feeder capacities differ.

Two configurations compared

The base flow solver run returns $F^* = 37.0$, with `mincut_capacity` = 37.0 for both networks, which is the maximum feasible flow for the system as built. However, F^* alone does not distinguish the two designs. As shown in Table 6.3, the two networks still differ in structure: Configuration A carries more saturated edges, has a non-empty free zone and admits several equivalent minimum cuts, while the minimum cut in Configuration B is unique. The two designs have the same throughput under different constraints, which is what the remaining modules distinguish.

Table 6.3: Configuration comparison summary.

Metric	Configuration A	Configuration B
F^*	37.0	37.0
<code>saturated_edges</code>	41	35
<code>free_zone_size</code>	2	0
<code>min_cuts</code>	4	1
<code>SPOF_nodes</code>	0	0
<code>upgrade_ineffective</code> (gateway edges)	true	false

The difference is one of structural flexibility against bottleneck localisation. In Configuration A the free-zone nodes are {134, 135}. The cut lattice is therefore wider, and E_{every} and E_{some} differ. The gateway edges $134 \rightarrow 136$ and $135 \rightarrow 136$ appear in E_{some} but not in E_{every} , so neither is a universal failure mode on its own. Cut 4 has only

3 crossing edges while Cut 1 has 13, both at capacity 37 (Table 6.8). In Configuration B, feeder-capacity reinforcement pins the cut to a unique partition: free-zone size collapses to 0 and the gateway edges move into E_{every} . The choice between configurations depends on the optimisation target: Configuration A offers more structural redundancy and cut diversity, while Configuration B offers tighter bottleneck localisation and actionable upgrade thresholds.

Bypass edge and single points of failure. The direct bypass edge $132 \rightarrow 140$ improves `sink_4` service with capacity 3, below the sink-side limit of 6. The same edge raises redundancy scores for all secondary-to-gateway connections from 2 to 3, so redundancy improves across the whole gateway-feeding layer, well beyond the nodes it directly connects.

Without this edge, node 136 is the only SPOF node: every source-to-sink path passes through it and $F^*(G \setminus \{136\}) = 0$. Adding $132 \rightarrow 140$ gives $F^*(G \setminus \{136\}) = 3 > 0$, removing node 136 from the SPOF set, and both configurations report `spof_nodes` = $\emptyset$. However, only `sink_4` has an alternative path, so all other flow still passes through the combiner. Running the capacity degradation module with node 136 removed confirms this: F^* drops to 3, exposing the network’s continued vulnerability to combiner failure despite the absence of a formal SPOF classification.

Node-capacitated analysis. When node capacities are enabled, both configurations still return the same result: a 7-unit loss down to `node_cap_max_flow` = 30.0. The same six Layer 3 nodes {114, 115, 116, 117, 118, 119} saturate in both cases. This tells the designer where to act: once these node limits are active, changing gateway-edge capacities does not change the outcome.

The flow loss is not shared evenly across demand nodes (Figure 6.8). `sink_1` (137) falls from 4.0 to 0.0, so it loses service completely. `sink_2` (138) falls from 6.0 to 3.0, while sinks 139–143 stay at their previous levels. Total flow alone does not show this.

To test a direct fix, a single-node upgrade sweep was run on nodes 114–119 (Table 6.4). Upgrading any one of these nodes restores `sink_1` to its edge-only service level and returns total flow to $F^* = 37.0$ in both configurations. The required increment is identical

for all six nodes, while the three node pairs reach different absolute targets because they start from different capacities. The six upgrades therefore have the same effect on throughput, and the choice between them is settled by delivery constraints such as cost and installation effort.

Table 6.4: Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore `sink_1` service under the node-capacitated solve.

Node	Current b_v	Required Δb_v	New b_v
114 (<code>processing_A</code>)	5.0	7.0	12.0
115 (<code>processing_B</code>)	5.0	7.0	12.0
116 (<code>processing_C</code>)	7.0	7.0	14.0
117 (<code>processing_D</code>)	7.0	7.0	14.0
118 (<code>bypass_E</code>)	3.0	7.0	10.0
119 (<code>bypass_F</code>)	3.0	7.0	10.0

Flow decomposition results. Flow decomposition shows that both configurations achieve the same total throughput ($F^* = 37.0$) with very similar route-level allocation. Configuration A decomposes into 27 components and Configuration B into 25, so the difference is small and does not change the main delivery pattern. In both cases, the largest single component carries 3.0 units on a path bottlenecked at $(114 \rightarrow 120)$, and most remaining components are paths carrying 2.0 or 1.0 units distributed across the processing and bypass corridors.

This agrees with the structural results above. The A/B edge-capacity changes alter which secondary-to-gateway edges appear as local path bottlenecks, but they do not create a different high-level flow-allocation picture. The dominant route family still passes through the same Layer 3 processing/bypass region and then through the combiner to sinks, with the direct bypass to sink 4 contributing only partial service. At decomposition level, the two designs differ in local edge pressure more than in overall route usage.

Sensitivity analysis. Sensitivity ranking shows a tiered pattern in Configuration A (Table 6.5). Single-edge impact falls into four tiers, from the two gateway-to-combiner

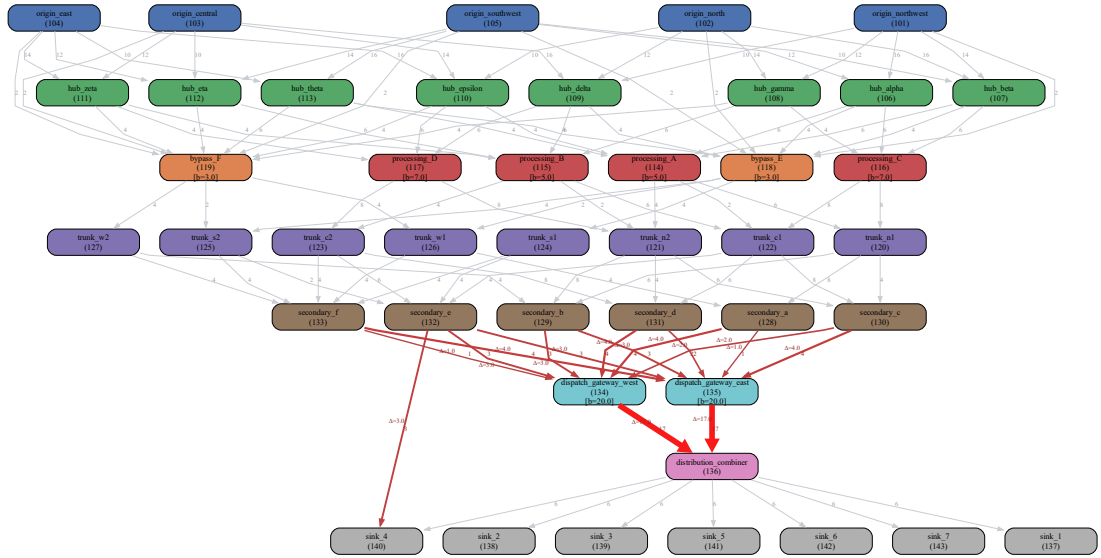
edges $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$ at the head of the ranking down to the outer feeders at the foot. Removing either gateway edge reduces throughput by about 46 per cent. Below the gateway pair sits a tier of secondary-to-gateway feeders, and the tier below that contains the direct bypass edge $(132 \rightarrow 140)$, which therefore carries the same single-edge impact as several of those feeders. In total, 15 edges have non-zero single-edge impact in Configuration A.

Table 6.5: Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).

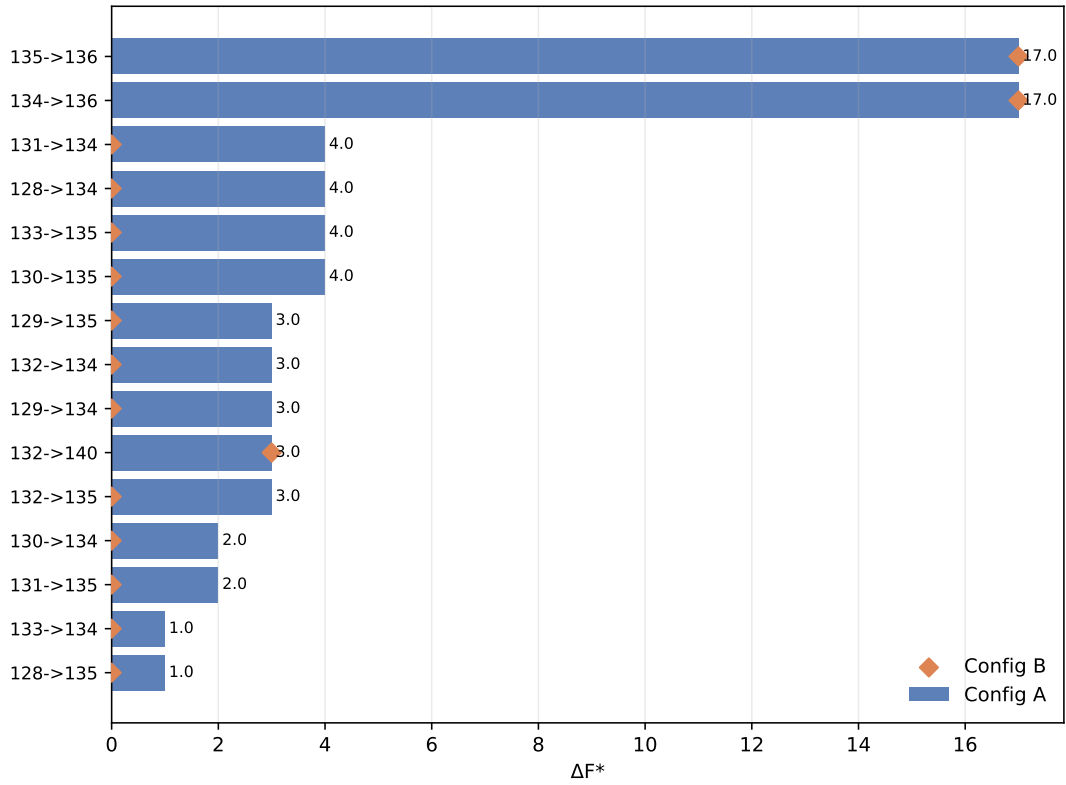
Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_a->dispatch_gateway_west	4.0	33.0
4	secondary_c->dispatch_gateway_east	4.0	33.0
5	secondary_d->dispatch_gateway_west	4.0	33.0
6	secondary_f->dispatch_gateway_east	4.0	33.0
7	secondary_b->dispatch_gateway_west	3.0	34.0
8	secondary_b->dispatch_gateway_east	3.0	34.0
9	secondary_e->dispatch_gateway_west	3.0	34.0
10	secondary_e->dispatch_gateway_east	3.0	34.0
11	secondary_e->sink_4	3.0	34.0
12	secondary_c->dispatch_gateway_west	2.0	35.0
13	secondary_d->dispatch_gateway_east	2.0	35.0
14	secondary_a->dispatch_gateway_east	1.0	36.0
15	secondary_f->dispatch_gateway_west	1.0	36.0

Configuration B is more concentrated: only three edges have non-zero impact, $(134 \rightarrow 136)$, $(135 \rightarrow 136)$, and $(132 \rightarrow 140)$, while all other evaluated edges give $\Delta^* = 0$ (Table 6.6). This means the feeder-capacity reinforcement in B absorbs any single feeder-edge loss through rerouting, so feeder removals no longer reduce F^* individually. The non-zero set therefore collapses from 15 edges in A to 3 in B.

Failure impact. The two-edge failure ranking confirms that the dominant pair is simultaneous loss of $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$, with drop 34.0 and perturbed flow 3.0. This is much larger than the next group (drop 21.0), and follows from the single-edge pattern shown in Figure 6.5. In Configuration B, the same gateway-pair



(a) Configuration A critical-edge overlay. Edge emphasis (colour/width) is proportional to non-zero ΔF^* .



(b) Ranked non-zero single-edge sensitivity for Configurations A and B.

Figure 6.5: Single-edge sensitivity in Configuration A and cross-configuration rank comparison (A vs B).

Table 6.6: Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).

Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_e->sink_4	3.0	34.0

dominance remains, while the non-zero failure set is more concentrated because feeder reinforcement absorbs most single-edge feeder losses through rerouting.

Degradation trajectory. The degradation trajectories are identical for Configurations A and B at all tested scale factors. The solved values are: $\alpha = 1.0 \rightarrow 37.0$, $\alpha = 0.9 \rightarrow 33.3$, $\alpha = 0.8 \rightarrow 29.6$, $\alpha = 0.7 \rightarrow 25.9$, $\alpha = 0.6 \rightarrow 22.2$, and $\alpha = 0.5 \rightarrow 18.5$. These points follow the linear relation

$$F^*(\alpha) = 37\alpha$$

over the tested interval.

The A/B curves that coincide point-for-point (Figure 6.7) indicate that the feeder-capacity differences that separate A and B under single-edge sensitivity do not produce a different global response under a uniformly scaled network-wide degradation.

Parametric thresholds. Parametric thresholds separate an edge’s criticality from its upgradeability, and Table 6.7 sets the two measures side by side for the three edges on which the configurations differ. In Configuration A, the gateway edges (134 $\rightarrow$ 136) and (135 $\rightarrow$ 136) need an infinite capacity increase to raise throughput at all, so they are upgrade-ineffective despite their high impact in the sensitivity ranking. In Configuration B, the same gateway edges become upgrade-effective at a finite capacity. Both configurations have zero margin on these edges: any capacity loss below the current level immediately reduces F^* .

This difference is explained by upstream supply. In Configuration A, supply into each gateway is already capped at the current gateway-edge capacity, so increasing

gateway-edge capacity alone cannot raise throughput. In Configuration B, feeder reinforcement creates upstream headroom, so gateway-edge upgrades can increase F^* .

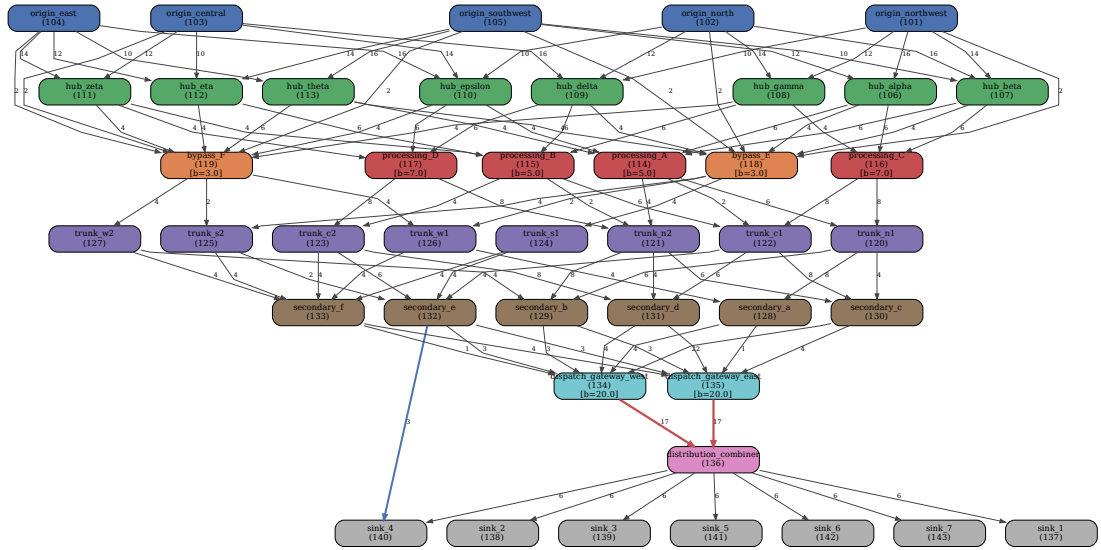
The bypass edge ($132 \rightarrow 140$) is upgrade-effective in both configurations, unlike the gateway edges. The contrast between the two configurations in upgrade feasibility is therefore concentrated at the gateway edges.

Table 6.7: Parametric threshold and marginal range comparison for key edges.

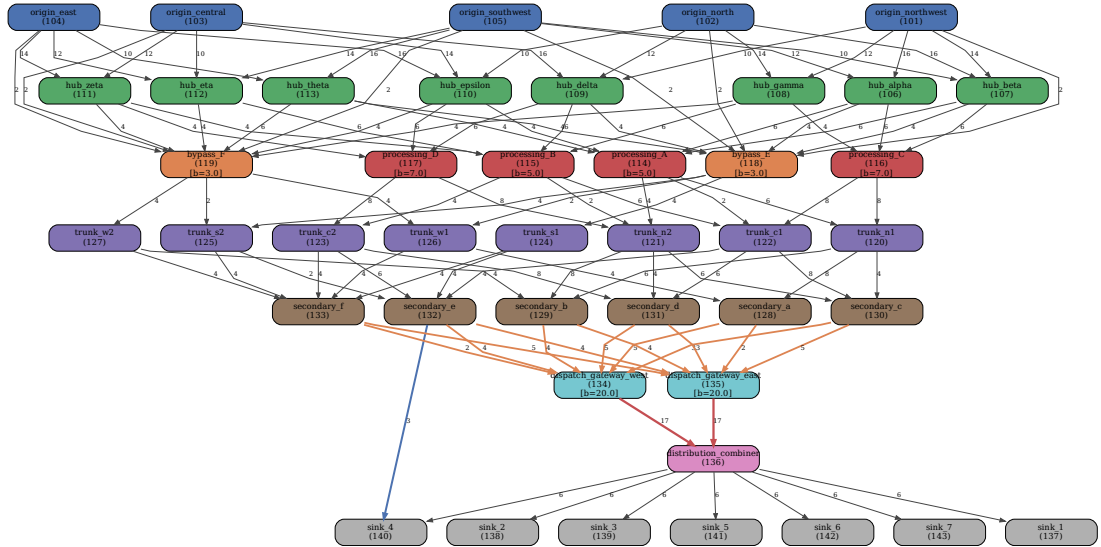
Edge	Configuration A		Configuration B	
	Upgrade	$\mathcal{B}_{uv}$	Upgrade	$\mathcal{B}_{uv}$
$134 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$135 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$132 \rightarrow 140$	+1.0	10.0	+1.0	16.0

Table 6.8: Min-cut enumeration for Configuration A.

Cut	Crossing edges
1	($128 \rightarrow 134$), ($128 \rightarrow 135$), ($129 \rightarrow 134$), ($129 \rightarrow 135$), ($130 \rightarrow 134$), ($130 \rightarrow 135$), ($131 \rightarrow 134$), ($131 \rightarrow 135$), ($132 \rightarrow 134$), ($132 \rightarrow 135$), ($132 \rightarrow 140$), ($133 \rightarrow 134$), ($133 \rightarrow 135$)
2	($128 \rightarrow 135$), ($129 \rightarrow 135$), ($130 \rightarrow 135$), ($131 \rightarrow 135$), ($132 \rightarrow 135$), ($132 \rightarrow 140$), ($133 \rightarrow 135$), ($134 \rightarrow 136$)
3	($128 \rightarrow 134$), ($129 \rightarrow 134$), ($130 \rightarrow 134$), ($131 \rightarrow 134$), ($132 \rightarrow 134$), ($132 \rightarrow 140$), ($133 \rightarrow 134$), ($135 \rightarrow 136$)
4	($132 \rightarrow 140$), ($134 \rightarrow 136$), ($135 \rightarrow 136$)



(a) Configuration A: symmetric gateway design. Blue edge $132 \rightarrow 140$ is the direct bypass to `sink_4`.



(b) Configuration B: orange edges show capacity-boosted secondary-to-gateway feeders

Figure 6.6: Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.

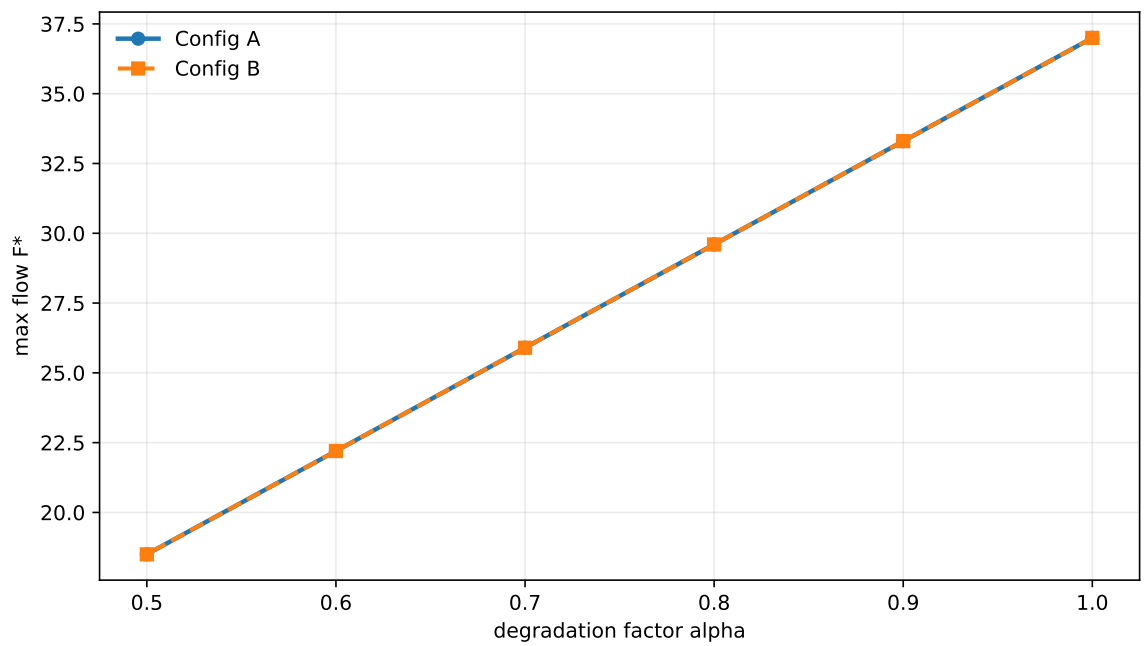


Figure 6.7: Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.

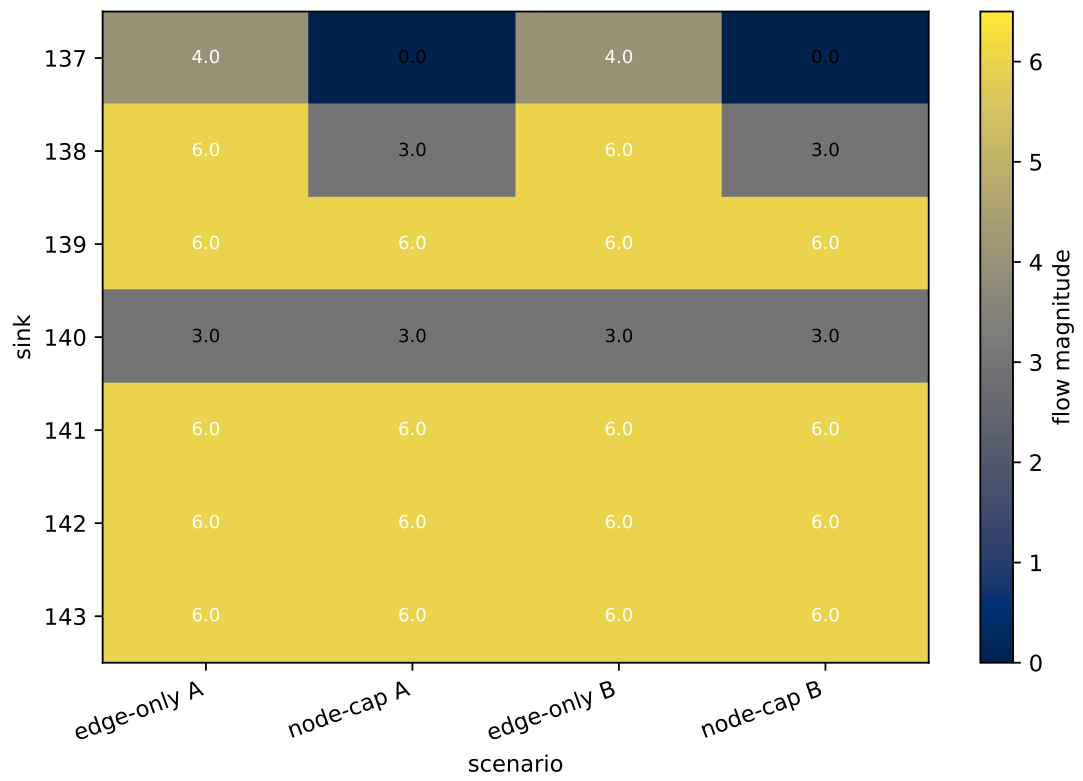


Figure 6.8: Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.

6.8.2 The IEEE Reliability Test System

The case study is the 24-bus IEEE Reliability Test System [77, 78], taken from the MATPOWER case file `case24_ieee_rts` [79]. The system has 24 buses, 33 generating units at eleven buses with a total nameplate capacity of 3405 MW, 38 transmission lines of which four are double-circuit pairs, and a peak demand of 2850 MW at seventeen buses. Line ratings in MVA are the edge capacities and generator nameplate capacities are the node capacities. The physical network is meshed, and the graph object of Chapter 3 requires a DAG, so each line was oriented by the direction of its real-power flow in a DC power flow at peak load, with generation dispatched at a uniform participation factor so that total generation equals total demand and bus 13 as the slack bus. Twenty-one of the 34 lines carry flow against the order in which the file lists their end buses. The four double-circuit pairs, which carry equal flow because their parameters are identical, become one edge each with the summed rating. No directed cycle remained after orientation, so no line was dropped. The result is a 24-node, 34-edge DAG (Figure 6.9) whose zero-in-degree nodes (buses 7, 21, 22 and 23) are exactly the buses with generation and no load, and whose zero-out-degree nodes (buses 4, 5, 6, 8 and 19) are exactly the buses with load and no generation, which is consistent with the orientation being physical. The orientation log and the input files are in Appendix A.

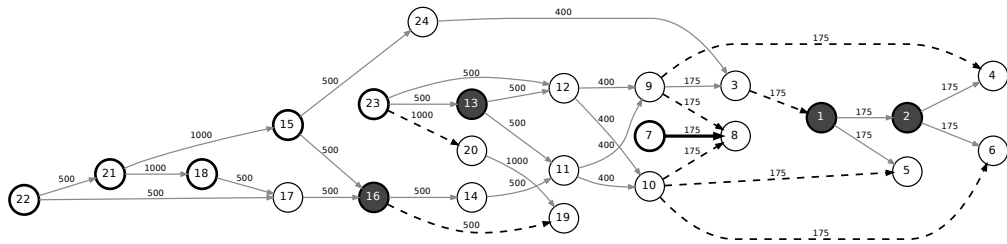


Figure 6.9: IEEE RTS-24 oriented by DC power flow at peak load. Edge labels are line ratings in MVA; double-circuit pairs are one edge with the summed rating. Bold outlines mark the eleven generator buses, and filled nodes mark the four generators (buses 1, 2, 13 and 16) that are at their nameplate limit in the maximum flow found under the net-injection model. The bold line 7–8 has 4 MVA of headroom, the least of any line. Dashed lines form the nine-line binding cut of the structural-subset model.

Seven buses carry both generation and load and are therefore neither a source nor a sink of the DAG. To measure deliverable throughput against the whole system demand, a super-source is connected to every bus whose net injection (generator capacity minus local load) is positive, with capacity equal to that net injection, and every bus whose net injection is negative is connected to a super-sink with capacity equal to its net load. A bus with both generation and load appears once, under its net sign; bus 15, for instance, has 215 MW of generation and 317 MW of load and is a net importer of 102 MW. The nine net-export buses offer 2162 MW to the network, the eleven net-import buses require 1607 MW from it, and the remaining 1243 MW of demand is served locally at export buses without entering the network.

Under this model the delivered throughput equals the net import demand, so the whole of the system peak demand is met once the load served locally at the export buses is counted. The same cut binds with generator capacities applied and with generation unconstrained, so at peak load neither the lines nor the generators limit delivery, and the maximum flow is set by the demand. No bus is a single point of failure, and the single-line and single-generator failure rankings are both empty, because every line and every generator has slack in the flow found. This is a statement about aggregate throughput under a flow model. Operational security is a separate question, since a remaining path could overload under a redispatch that the model does not represent. The minimum cut is unique: the lattice of Section 6.5.1 has an empty free zone and the enumeration is complete, and no generator edge lies in it. Table 6.9 collects the figures. Should demand grow, the system would bind first at the four generators already sitting at their nameplate limit and at the one line with a few MVA of headroom.

For contrast, the first analysis of the same DAG took only the four generation-only buses as sources and the five load-only buses as sinks, ignoring the mixed buses. Under that model the network delivers 2725 MVA with edge capacities alone and 1135 MVA with generator capacities, the binding cut is the nine dashed lines of Figure 6.9, and the most damaging single-line failures are the two double circuits at bus 20 (1000 MVA each), line 16–19 (500 MVA) and lines 3–1 and 7–8 (175 MVA each). The two models answer different questions, and the difference between them is the reason the net-injection

model is the one reported: the structural-subset model measures a transmission bottleneck that appears only when the mixed buses are excluded from supply and demand, and those buses do not exist as pure sources and sinks in the real system.

Table 6.9: IEEE RTS-24 under the net-injection model at peak load.

Quantity	Value
System peak demand	2850 MW
Demand served locally at export buses	1243 MW
Net import demand	1607 MW
Deliverable throughput, generator capacities applied	1607 MW (100 per cent)
Deliverable throughput, generation unconstrained	1607 MW
Binding cut	the eleven import-bus edges, 1607 MW
Single points of failure	none
Single-line failures reducing throughput	none
Single-generator failures reducing throughput	none
Generators at nameplate limit in the flow found	buses 1, 2, 13, 16
Line with least headroom	7–8, 171 of 175 MVA

6.8.3 Scale

The solvers were timed on four generated instances of the **genrmf** mesh family used in the DIMACS implementation challenge [80]. The generator here produces the same layered mesh topology with every edge directed forward, so that the instances are DAGs, and they are therefore a variant of the published family and not its output. Sizes are 360 nodes and 924 edges, 3,332 and 9,324, 27,040 and 78,364, and 36,992 and 107,644, with capacities up to 1000 from a recorded seed. Table 6.10 gives the warm second-call times. Dinic is the fastest of the three on every instance and completes the largest in under a second; push-relabel is within a factor of four; Edmonds-Karp was not run above 50,000 edges, where its $O(VE^2)$ bound makes it impractical. The external oracle, which agrees with all three on every instance, takes 29 s on the largest. The comparison is between implementations on the same machine and says nothing about the algorithms in general; it establishes that the toolkit’s default solver handles networks two orders of magnitude larger than any in the corpus in interactive time.

Table 6.10: Warm second-call solve times on DAG variants of the **genrmf** mesh family. Edmonds-Karp was not run above 50,000 edges.

Instance	Nodes	Edges	Dinic (s)	Push-relabel (s)	Edmonds-Karp (s)
small	360	924	0.002	0.097	0.007
medium	3,332	9,324	0.017	0.019	0.179
large	27,040	78,364	0.961	1.520	–
xlarge	36,992	107,644	0.657	2.277	–

6.9 Discussion

The toolkit is point-valued. An interval or probability-box capacity is not propagated, and running the best-case and worst-case capacities gives two solves whose relation to the true range of the maximum flow is a bound only because the maximum flow is monotone in every capacity. Extending the toolkit to intervals by that monotonicity argument is direct for the flow value; the cut structure, the failure rankings and the thresholds are not monotone and would need the same kind of treatment that the floats received in Chapter 7.

Exactness in this chapter is on the evaluated candidates. The sensitivity and failure modules restrict their candidates to the saturated edges and to the edges of some minimum cut, and their results are exact on those sets. An edge outside the candidate set cannot change the maximum flow by a small change in its capacity, but a large enough change can, and the k -edge failure analysis is exhaustive only over the candidate set.

The toolkit computes no probability. It does not answer the stochastic-flow question of Chapter 2, the probability that a stated demand can be met, and it does not weight a failure by its likelihood. That combination is a natural extension, since the probability toolkit of Chapter 5 gives the probability that a component operates, but it is not built.

All analyses are on a single fixed-capacity instance. Time-varying degradation and adaptive rerouting are outside the model, and the degradation scenarios of Section 6.5.4 are a sequence of separate instances.

The global edge and node connectivity of a DAG, defined as the minimum over all ordered node pairs of the unit-capacity maximum flow, is zero, because a sink has no outgoing edge and every DAG has a sink. The implementation computed exactly that

definition and therefore returned zero on every network, which the validation confirmed independently. The whole-graph definition is the one the module documents and computes, and it is left as it is. The quantity that answers the redundancy question on a DAG is the minimum over declared source and sink pairs, which the per-pair disjoint-path counts of Section 6.5.2 already give; a named metric for that minimum is listed in Chapter 11.

Two further extensions follow from the model. Minimum-cost flow, in which each edge carries a cost per unit as well as a capacity, would answer how a given demand is met most cheaply, and a time-expanded network would carry the flow model across a planning horizon. Both are listed in Chapter 11.

6.10 Summary

The Capacity Flow Toolkit answers six questions about a capacitated DAG. The flow model has been stated, with the reductions for several sources and sinks and for node capacities proved exact, and three maximum-flow solvers have been provided on the graph object of Chapter 3, each result checked for feasibility, conservation and agreement between flow and cut. Six questions have been answered by modules that share one baseline solve: throughput and margin, the binding cut and the lattice of every equivalent cut, the most damaging failures and degradations, the marginal value and range of each edge, degradation and upgrade thresholds by a partition-change search, and redundancy and single points of failure. The toolkit has been validated against an external oracle and brute-force enumeration on twelve networks, timed on generated instances of up to 10^5 edges, and applied to the IEEE Reliability Test System, on which neither lines nor generators bind at peak load and the nearest constraints are four generators and one line. The toolkit is point-valued, and its extension to interval capacities and to probability-weighted measures is left to future work.

Chapter 7

Information as Schedule and Cost: Critical Path Toolkit

7.1 Introduction

The Critical Path Toolkit implements the schedule and cost interpretation of the network model. Under this interpretation the value attached to a node is a quantity that accumulates along dependency paths, such as a duration or a cost, and the value attached to an edge is the amount added or applied in the transfer between two components. The toolkit reads the graph object of Chapter 3 directly. Every analysis in it is a linear-time pass over the layered iteration sets, and the Network Decomposition Module of Chapter 4 is not used.

For a network carrying such a quantity, a forward pass gives the best or worst value that a complete chain of dependencies can produce. The margin of a component is how far it can move before that value changes. The margin identifies the critical components, and it can be computed only when the arithmetic of the quantity can be run backwards.

The contributions of this chapter are the following. The classical critical path method is generalised to a family of analysis modes, each defined by a pair of operators, and the condition under which a mode has a backward pass is stated and used to select the modes provided. Two propagation kernels are given that serve every mode. For interval durations, the forward quantities are computed exactly from two corner runs, and the floats are computed exactly by a domination split that reduces the enumeration from every uncertain duration to those that can bypass the node in question. Every result is checked against an independent oracle.

The rest of the chapter is structured as follows. Section 7.2 recalls the results from Chapter 2 on which the toolkit builds. Section 7.3 defines the analysis modes and Section 7.4 the two kernels. Section 7.5 describes the treatment of interval durations, and Section 7.6 presents the domination split and proves its exactness. Section 7.7 describes the validation of the toolkit. Section 7.8 applies it to a benchmark project network and locates the boundary of the split on a dense mesh. Section 7.9 discusses limitations and extensions, and Section 7.10 concludes the chapter.

7.2 Background

Chapter 2 reviewed the critical path method [3], its float calculus, and the treatment of duration uncertainty from PERT [37] to the interval results of Chanas, Zieliński and their co-authors. When each duration is an interval, the forward quantities are monotone in the durations, and their exact bounds are found at the two corners of the input box. The backward quantities are not monotone. Deciding whether an activity is possibly critical is NP-complete and computing float bounds is NP-hard [10, 38–40]. The interval treatment of Sections 7.5 and 7.6 rests on these two facts.

The forward and backward passes of the classical method are linear algebra over an idempotent semiring, and the backward pass is a residuation [42]. Hence a forward recursion exists for any pair of combination and compounding operations, but a backward pass exists only when the algebra of the pair supports one. A slack reported for a pair without such a backward pass is a number with no meaning. The modes of Section 7.3 rest on this fact.

7.3 Analysis Modes

The forward recursion of the critical path method combines the values arriving at a node from its parents and compounds the result with the node’s own value. Writing $\oplus$ for the combination and $\otimes$ for the compounding, the classical method has $\oplus = \max$ and $\otimes = +$. Other pairs give other analyses: $\min$ and $+$ give the fastest chain, $\max$ and $\times$ give the best multiplicative chain, and $+$ and $+$ give a roll-up in which every route contributes. Whether a margin is defined is a property of the pair.

When $\oplus$ is an order ($\max$ or $\min$) and $\otimes$ is monotone with a residual, a margin is defined. A bound on the combined value then binds every argument, and the bound inverts across each edge, so a constraint of the form “finish by K ” travels backwards through the network in the same way that values travel forwards. This is residuation in an idempotent semiring [42], and it is what the classical backward pass is. The backward object is a margin: the distance between the best complete path and the best complete path forced through the node in question, expressed as a difference for the

additive pairs and as a ratio for the multiplicative one. The zero set of the margin is the critical structure of the analysis.

When $\oplus$ is a sum, a total is not a deadline and no residuation exists. However, a summed system is linear, and a linear map has an adjoint. The backward object is therefore a sensitivity, and the margin it defines is an allowance against a stated budget. It is not a slack against a deadline. A pair with neither structure supports a forward recursion and nothing else, and no margin is defined for it.

A configuration of the toolkit is called a mode. A mode is an operator pair together with the backward semantics its algebra supports. Table 7.1 lists the four modes provided. The MaxScaling mode is applied in Chapter 10. The three order-based modes share one kernel and one backward mechanism, described in Section 7.4.1. The accumulation mode has its own kernel, described in Section 7.4.2.

Table 7.1: The modes provided by the toolkit. In the three order-based modes the backward quantities are computed by the same fold on the reversed graph and the margin is the gap between the project value and the value through the node. In the accumulation mode the backward quantity is the adjoint, a path count.

Mode	$\oplus$	$\otimes$	Backward object	Margin
LongestPath	max	+	reverse fold, margin $P - t_v$	slack
ShortestPath	min	+	reverse fold, margin $t_v - P$	margin over optimum
MaxScaling	max	$\times$	reverse fold, margin $P/t_v - 1$	ratio slack
Accumulation	+	+	adjoint (path multiplicity)	allowance under a budget

7.4 The Propagation Kernels

7.4.1 The order-based kernel

For the order-based modes one fold serves both directions. Let d_v be the value of node v , w_{uv} the value of edge (u, v) , $\text{pa}(v)$ the parents of v and $\text{succ}(v)$ its successors. Read forwards over the layered iteration sets, the fold computes

$$F_v = \left(\oplus_{u \in \text{pa}(v)} F_u \otimes w_{uv} \right) \otimes d_v, \quad (7.1)$$

the best value that a chain of dependencies can deliver into each node, with each source seeded from an initial value. Read over the reversed graph, the same fold computes

$$R_v = \oplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs}, \quad (7.2)$$

the best completion from each node onwards, with each sink seeded at the neutral element of $\otimes$. The two arrays meet at each node. Their compound

$$t_v = F_v \otimes R_v \quad (7.3)$$

is the best complete path forced through v . The project value P is the combination of the forward values at the sinks, and the margin of v is the gap between the two,

$$m_v = P - t_v \text{ (LongestPath)}, \quad m_v = t_v - P \text{ (ShortestPath)}, \quad m_v = P/t_v - 1 \text{ (MaxScaling)}. \quad (7.4)$$

A node is critical when its margin is zero. For the additive modes the textbook schedule quantities are recovered from the same two arrays,

$$\text{ES}_v = F_v - d_v, \quad \text{LF}_v = P - R_v, \quad \text{LS}_v = \text{LF}_v - d_v. \quad (7.5)$$

Algorithm 5 gives the kernel.

On the four-node network of Figure 7.1, with durations 1, 5, 2, 1 and no edge delays, LongestPath completes in 7 along $1 \rightarrow 2 \rightarrow 4$ and node 3 carries slack 3. ShortestPath completes in 4 along $1 \rightarrow 3 \rightarrow 4$ and node 2 carries margin 3. The two modes select different chains from the same inputs.

7.4.2 The linear kernel

In the accumulation mode the combination is a sum, so

$$F_v = \sum_{u \in \text{pa}(v)} (F_u + w_{uv}) + d_v. \quad (7.6)$$

Algorithm 5 Order-based kernel (one mode)

Require: layered iteration sets L , incoming and outgoing indices, sources S , node values d , edge values w , mode $(\oplus, \otimes)$ with neutral element e

Ensure: F , R , P , t , m , critical set

```

1: for all layers  $L_i$  in order, nodes  $v \in L_i$  do
2:   if  $v \in S$  then  $F_v \leftarrow e \otimes d_v$ 
3:   else  $F_v \leftarrow (\oplus_{u \in \text{pa}(v)} F_u \otimes w_{uv}) \otimes d_v$ 
4:   end if
5: end for
6: for all layers  $L_i$  in reverse order, nodes  $v \in L_i$  do
7:   if  $\text{succ}(v) = \emptyset$  then  $R_v \leftarrow e$ 
8:   else  $R_v \leftarrow \oplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs}$ 
9:   end if
10: end for
11:  $P \leftarrow \oplus_{v \in \text{sinks}} F_v$ 
12: for all nodes  $v$  do
13:    $t_v \leftarrow F_v \otimes R_v$ ;  $m_v \leftarrow \text{margin}(P, t_v)$  by Eq. (7.4)
14: end for
15:  $\text{critical} \leftarrow \{v : m_v = 0\}$ 

```

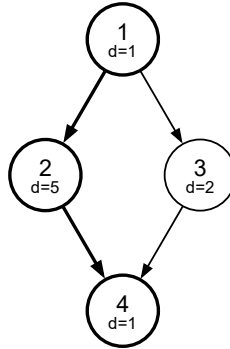


Figure 7.1: The running example. Node durations 1, 5, 2, 1. LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4.

A node's value therefore reaches a downstream target once per directed route. The total at a target t expands to

$$F_t = \sum_v m_v d_v + \sum_{(u,v) \in E} m_v w_{uv}, \quad (7.7)$$

where m_v is the number of directed routes from v to t . The counts are computed by a reverse traversal seeded with $m_t = 1$,

$$m_v = \sum_{s \in \text{succ}(v)} m_s, \quad (7.8)$$

which is the adjoint of the forward map. The count m_v is the derivative of the total with respect to the node's value, and the product $m_v d_v$ is the node's contribution to the total. Under a budget B the allowance of node v is

$$a_v = \frac{B - F_t}{m_v}, \quad (7.9)$$

the headroom divided across the node's routes. Algorithm 6 gives the kernel. On the running example the total at node 4 is 10 and node 1 counts twice, once through each branch. If no target is given the toolkit takes the sink with the largest forward value.

Algorithm 6 Linear kernel (accumulation mode)

Require: layered iteration sets L , indices, sources S , node values d , edge values w , target t (default: sink with largest F), optional budget B

Ensure: F , total F_t , multiplicities m , contributions, allowances

```

1: for all layers  $L_i$  in order, nodes  $v \in L_i$  do
2:    $F_v \leftarrow d_v$  if  $v \in S$ , else  $\sum_{u \in \text{pa}(v)} (F_u + w_{uv}) + d_v$ 
3: end for
4:  $m_t \leftarrow 1$ 
5: for all layers  $L_i$  in reverse order, nodes  $v \in L_i, v \neq t$  do
6:    $m_v \leftarrow \sum_{s \in \text{succ}(v)} m_s$ 
7: end for
8: for all nodes  $v$  with  $m_v > 0$  do
9:    $\text{contribution}_v \leftarrow m_v d_v$ ;   if  $B$  given,  $a_v \leftarrow (B - F_t)/m_v$ 
10: end for
```

7.4.3 Cost

Both kernels touch each node once and each edge a constant number of times, so every mode costs $O(|V| + |E|)$ whatever the operators are. The kernels are generic over the value type they propagate, and the concrete types are described in Chapter 8.

7.5 Interval Durations

When the durations are intervals, the quantities of the toolkit divide into two classes according to monotonicity.

Every forward quantity, F_v , R_v , t_v and P , is monotone in every input. Hence its exact bounds are obtained from two scalar runs of the kernel, one with every input at its lower endpoint and one with every input at its upper endpoint, and interval arithmetic is not needed. The toolkit computes the forward quantities in this way for every mode.

A margin is a difference of two monotone quantities that share their inputs, so it is not monotone in anything, and the hardness results of Section 7.2 apply to it. The toolkit provides three methods for the margins, and each result is tagged with the method that produced it, so that an exact range and an enclosure cannot be confused.

1. *Conservative enclosure.* From the exact bounds on P and t_v , the LongestPath margin lies within

$$[\max(0, \underline{P} - \bar{t}_v), \bar{P} - \underline{t}_v], \quad (7.10)$$

and correspondingly for the other modes. The enclosure is sound. It costs no more than the two corner runs, and it is not exact, because the lower corner of P and the upper corner of t_v are not in general attained by the same configuration of durations.

2. *Exhaustive corner enumeration.* The margin extremes are attained at corner configurations (Proposition 7.1 below), so enumerating every corner of the non-degenerate node and edge intervals and running the kernel at each gives the exact margin range. The cost is 2^k runs for k interval inputs. Beyond a stated cap the toolkit refuses unless the cap is raised.

3. *Domination split.* For the LongestPath mode with point-valued edge values, the exact margin range is obtained from a much smaller enumeration, described in Section 7.6.

From the margin bounds the toolkit reports the two criticality sets of the interval PERT literature [10]. A node is *necessarily critical* when the upper bound of its margin is zero, that is, it is critical in every configuration of the box. A node is *possibly critical* when the lower bound of its margin is zero, that is, it is critical in at least one configuration. Under the conservative enclosure the necessarily critical set is certain and the possibly critical set is a superset that the exact methods tighten. Degenerate intervals, whose two endpoints coincide, are held fixed by every method. They do not enter the enumeration.

An earlier implementation of the toolkit ran the scalar kernels on an interval type with overloaded arithmetic, so that $\max$, $+$ and $-$ were applied to intervals directly. It was withdrawn because it produced wrong results at the combination and margin steps. In the combination step, $\max$ of two overlapping intervals is not defined, since neither contains the other. Taking the endpoint-wise maximum, which is what the overloaded operator did, returns an interval such as $[5, 7]$ from $[3, 7]$ and $[5, 6]$, whose lower end comes from one parent and whose upper end from the other, so the value no longer describes any single path and the critical chain read from it can be wrong. In the margin step, the float $P - t_v$ subtracts two quantities that depend on the same durations. Interval subtraction treats them as independent, which is the dependency problem of Chapter 2. For the dummy start node of a project, for example, P and t_v are the same interval, say $[30, 46]$, and interval subtraction gives $[-16, 16]$ when the true float is exactly 0. The margins were therefore wider than the true range, and negative floats appeared where none exist. The present toolkit computes each interval quantity by the scheme its structure permits, and never by running the kernel on an interval type.

7.6 Exact Interval Floats by the Domination Split

Throughout this section the mode is LongestPath, the durations d_u lie in intervals, the edge values are fixed, and $f_v = P - t_v \geq 0$ is the float of node v . The question is the exact range of f_v over the box of duration configurations.

Proposition 7.1 (Corner sufficiency). *With all other durations fixed, f_v is a monotone function of any single duration d_u . Consequently the extremes of f_v over the box are attained at corner configurations.*

Proof. A path visits a node at most once, so as a function of $x = d_u$ every path length is either constant or affine with slope one. Hence $P(x) = \max(C_0, C_1 + x)$, where C_1 is the best path through u and C_0 the best avoiding it, and likewise $t_v(x) = \max(A_0, A_1 + x)$ over the paths through v . Each has one breakpoint with slope 0 then 1. If P 's breakpoint comes first their difference has slope pattern $0, +1, 0$, otherwise $0, -1, 0$, and in both cases f_v is monotone in x on the whole line. For the corner claim, push any coordinate to whichever endpoint does not worsen the objective, which monotonicity permits, and repeat per coordinate until the configuration is a corner. $\square$

Lemma 7.2 (Incomparable coordinates). *If no complete path contains both u and v , then f_v is nondecreasing in d_u , whatever the other durations are.*

Proof. No path through v contains u , so t_v does not depend on d_u , while P is nondecreasing in it. $\square$

Lemma 7.3 (Dominated coordinates). *If every complete path through u also passes through v , and in particular if $u = v$, then f_v is nonincreasing in d_u , whatever the other durations are.*

Proof. Suppose increasing d_u increases P . The new maximiser must contain u , hence v , hence it is a path through v , so t_v rises to equal P and the float drops to zero. If P does not increase, the float cannot rise because t_v is nondecreasing in every duration. $\square$

The two lemmas cover every duration except those in the *bypass set*

$$H_v = \{u : \text{some complete path contains both } u \text{ and } v, \text{ and some complete path contains } u \text{ but not } v\} \quad (7.11)$$

the nodes that share a route with v and can also route around it. Figure 7.2 shows the three classes around one node. In the implementation the classes are found from the ancestor and descendant closures of the graph object together with two traversals: the set of nodes that can reach a sink without passing v , and the set of nodes reachable from a source without passing v . An ancestor of v that cannot reach a sink without v is dominated, as is a descendant that cannot be reached from a source without v ; a node that is neither ancestor nor descendant is incomparable; the rest form H_v .

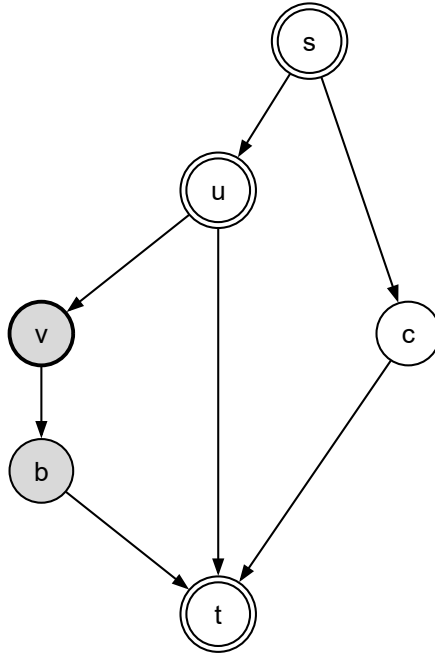


Figure 7.2: Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.

Theorem 7.4 (Exactness of the domination split). *The maximum of f_v over the box*

is attained at a corner with every incomparable duration at its upper bound, every dominated duration at its lower bound, and the bypass durations at some corner of H_v . The minimum is attained dually. Enumerating the $2^{|H_v|}$ bypass corners with the remaining durations pinned therefore yields the exact float range of v .

Proof. By Proposition 7.1 an optimum lies at a corner. The three classes partition the durations. At an optimal corner, moving an incomparable duration to its upper bound cannot decrease the maximum by Lemma 7.2, and moving a dominated duration to its lower bound cannot decrease it by Lemma 7.3, so some optimal corner has the pinned pattern and lies within the enumerated set. $\square$

Algorithm 7 gives the procedure. Its cost is

$$2 \sum_v 2^{|H_v|} \tag{7.12}$$

scalar propagations, one maximising and one minimising sweep per node, against 2^k for exhaustive enumeration of all k interval durations. The exponent now depends on the reconvergent width around each node, and no longer on the size of the problem. One exhaustive sweep serves every node at once. The implementation first screens on the number of non-degenerate interval inputs: below a fixed threshold it computes both counts before running anything and uses whichever is cheaper, so that within that regime it is never worse than exhaustive enumeration and refuses only when both counts exceed a stated limit; above the threshold it declines without pricing the split at all and returns the conservative enclosure. The screen is a fixed constant and not a function of the network's bypass sets, so a network with many interval inputs and small bypass sets is declined although the split would have been cheap. Chapter 10 meets this case on a real network. Some instances must remain hard, because the general problem is NP-hard [10], and the split shows where the hardness sits, in networks whose bypass sets are large.

The bypass set is a measure of reconvergence, as the conditioning set of Chapter 4 is, though the toolkit does not use that module. Both analyses pay for routes that divide and meet again: the decomposition module per join, through the nodes it must

Algorithm 7 Domination split for exact interval floats (LongestPath, point-valued edge values)

Require: graph object, interval durations $[\underline{d}_u, \bar{d}_u]$, scalar edge values, run limit

Ensure: exact float range $[\underline{f}_v, \bar{f}_v]$ for every node v

```

1:  $K \leftarrow$  set of nodes with non-degenerate intervals;  $k \leftarrow |K|$ 
2: for all nodes  $v$  do
3:   classify each  $u \in K$  as incomparable, dominated, or bypass ( $u \in H_v$ )
4: end for
5: total  $\leftarrow 2 \sum_v 2^{|H_v|}$ 
6: if  $2^k \leq$  total then run exhaustive enumeration instead and return
7: end if
8: if total  $>$  run limit then refuse
9: end if
10: for all nodes  $v$  do
11:   for objective in (maximise, minimise) do
12:     pin incomparable durations at upper (resp. lower) and dominated at lower
        (resp. upper)
13:     for all corners of  $H_v$  do
14:       run Algorithm 5; record  $f_v$ 
15:     end for
16:   end for
17:    $\underline{f}_v \leftarrow$  minimum recorded;  $\bar{f}_v \leftarrow$  maximum recorded
18: end for
19: necessarily critical  $\leftarrow \{v : \bar{f}_v = 0\}$ ; possibly critical  $\leftarrow \{v : \underline{f}_v = 0\}$ 

```

fix, and the split per node, through H_v . The two sets index reconvergence at different granularities. Neither contains the other in general, and neither computation uses the other.

7.7 Validation

The toolkit was checked against three computations that do not use its kernels. The checks were run on a set of ten networks, listed with their sizes in Appendix B, and they apply to every result reported in Section 7.8.

The first check is a path oracle. It lists every complete path from a source to a sink, evaluates each path under the mode’s operators, and from the list obtains the project value, the best path through each node, each node’s margin and the critical set. On the seven networks small enough to list every path (32 to 306 nodes, 128 to 2,450 paths), the toolkit and the oracle agree to within 3×10^{-14} in every quantity for all four modes, and the critical sets are identical. On the three networks with too many paths to list (2.7×10^6 and more), two weaker checks are used instead: two thousand randomly sampled paths never give a better value than the project value the toolkit reports, and the accumulation total equals the sum of Eq. (7.7) evaluated from the multiplicities.

The second check is for the interval methods. The path oracle is run at every corner of the input box, which gives the exact margin range directly. The conservative enclosure of Eq. (7.10) is checked to contain that range, the exhaustive method is checked to equal it, and the domination split is checked to give the same margins and the same necessarily and possibly critical sets as the exhaustive method on every network on which both can run.

The third check is Monte Carlo sampling, used where the exhaustive method is out of reach. Fifty thousand configurations of durations are drawn uniformly from the box, the scalar kernel is run on each, and the resulting floats are checked to lie within the split’s bounds and to reach them.

7.8 Case Studies

Timings in this section are wall-clock measurements on a single core of the second call in a warm process, so that just-in-time compilation is excluded. The convention is stated in Appendix B.

7.8.1 A benchmark project network

The first case study is instance j301_1 of the PSPLIB project scheduling library [81]. PSPLIB instances are activity-on-node networks with a dummy start and a dummy end, each activity carrying a published duration, and they are the standard benchmark set for project scheduling. The instance has 32 nodes, of which 30 are real activities, and 48 precedence edges, with one source, one sink, twelve forks and twelve joins in eleven layers. Its resource requirements are not used: the toolkit computes the precedence-only schedule, which the PSPLIB instance file records as the MPM-Time of the instance. The durations are listed in Appendix A.

Under LongestPath the project completes in 38 units, which matches the MPM-Time of 38 published with the instance, along the chain $1 \rightarrow 3 \rightarrow 8 \rightarrow 12 \rightarrow 14 \rightarrow 17 \rightarrow 22 \rightarrow 23 \rightarrow 24 \rightarrow 30 \rightarrow 32$ (Figure 7.3). Eleven nodes are critical, nine of them real activities, and three further activities (4, 10 and 16) carry a slack of one unit (Table 7.2). Under ShortestPath the fastest complete chain takes 18 units, through activities 2, 4, 6, 10, 25 and 30. The two modes share activity 30 and nothing else, so the chain that bounds the project from above and the chain that bounds it from below run through different parts of the network. Each of the three deterministic passes completes in under 0.2 milliseconds.

Read as cost with the same values, the accumulation mode gives a total of 362 at the end node. The activities with the largest contribution are not those with the largest durations. Activity 2, with duration 8, reaches the end node along five routes and contributes 40; activity 3, with duration 4, reaches it along nine routes and contributes 36; activity 16, with duration 10, lies on only two routes and contributes 20 (Table 7.3). Under a budget of 398.2, ten per cent above the current total, activity 3 can grow by

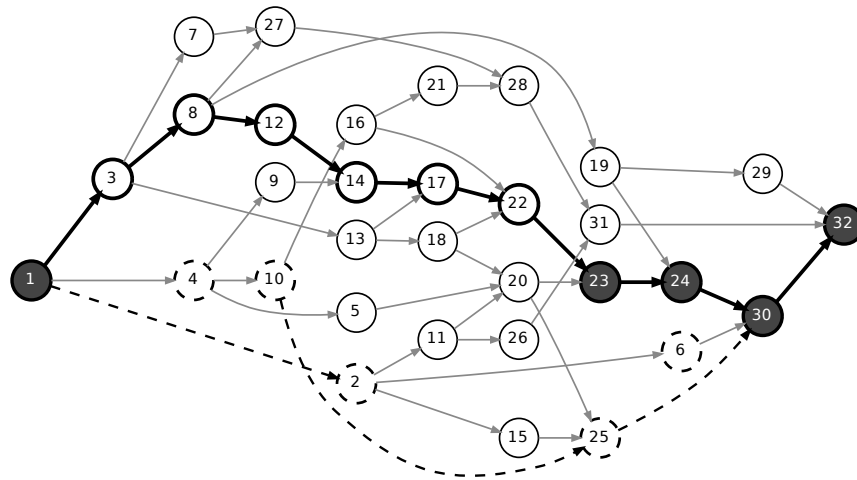


Figure 7.3: PSPLIB j301_1. Bold nodes and edges form the LongestPath critical chain (completion 38); dashed nodes and edges form the ShortestPath chain (completion 18). The filled nodes are those the interval analysis of Figure 7.4 finds necessarily critical under ± 20 per cent duration uncertainty.

Table 7.2: LongestPath on PSPLIB j301_1: the critical chain and the three nearest non-critical activities.

Node	Duration	Completion	Slack
3	4	4	0
8	9	13	0
12	2	15	0
14	3	18	0
17	6	24	0
22	7	31	0
23	2	33	0
24	3	36	0
30	2	38	0
4	6	6	1
10	7	13	1
16	10	23	1

4.0 units before the budget is reached, activity 16 by 18.1, and activity 6, which lies on a single route, by 36.2. Contribution and allowance order the activities differently.

Table 7.3: Accumulation on PSPLIB j301_1 under a budget of 398.2 at the end node (current total 362). Allowance is headroom divided by path multiplicity. The first eight rows are the largest contributors; node 6 is appended below the rule because the text discusses its allowance despite a smaller contribution.

Node	Duration	Multiplicity	Contribution	Allowance
2	8	5	40	7.2
3	4	9	36	4.0
4	6	6	36	6.0
8	9	4	36	9.1
11	9	3	27	12.1
13	6	4	24	9.1
10	7	3	21	12.1
16	10	2	20	18.1
6	8	1	8	36.2

With every one of the 30 activity durations widened to ± 20 per cent and the two dummies held at zero, the project value lies in $[30.4, 45.6]$. Exhaustive corner enumeration of the floats would require 2^{30} propagations, about 1.1×10^9 . The domination split required 50,524, a factor of about 21,000 fewer, with the largest bypass set holding 13 of the 30 durations and the mean bypass set 6.8. It completed in 4.2 seconds, about 82 microseconds per corner, each corner being one run of Algorithm 5. Five nodes are necessarily critical, the two dummies and activities 23, 24 and 30, so the final three activities of the deterministic chain are critical in every configuration of the box while the first six are not. Nineteen nodes are possibly critical, and thirteen activities cannot become critical anywhere in the box (Figure 7.4).

The conservative enclosure of Section 7.5, obtained from the two corner runs alone, certifies no node as necessarily critical and reports 26 nodes as possibly critical. The exact split therefore identifies three necessarily critical activities that the enclosure does not, and removes seven activities from the possibly critical set. The deterministic pass names eleven nodes, the enclosure names 26, and the exact interval analysis names 19 and identifies the three that are certain. Monte Carlo sampling of fifty thousand

configurations within the box produced no violation of the bounds. The sampled floats reached the lower and the upper bound at 30 of the 32 nodes, and at the remaining two the bounds are attained by the corner configurations the split itself identified.

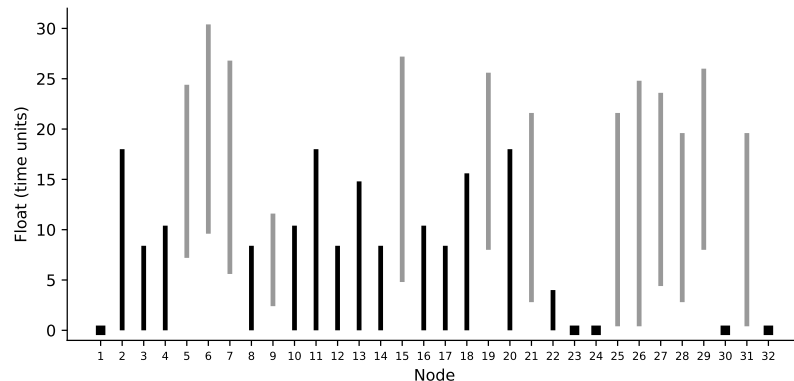


Figure 7.4: Exact float bounds on PSPLIB j301_1 with every duration at plus or minus 20 per cent. Black bars are the possibly critical nodes, whose lower bounds reach zero. Grey bars are the activities whose floats stay positive everywhere in the box. The necessarily critical nodes are drawn as points at zero, since both bounds of their float are zero.

7.8.2 The boundary of the split

The advantage of the split disappears under dense mesh reconvergence. On a five-by-five grid DAG with all 25 durations interval-valued, the bypass sets hold up to 23 of the 25 durations, so the split would need 3.9×10^7 propagations against 3.4×10^7 for one shared exhaustive sweep. The never-worse rule therefore selects the exhaustive sweep, and the instance is at the boundary of the method. The hardness results guarantee that some regime behaves this way, and it is the same regime in which the conditioning sets of the probability toolkit grow.

7.9 Discussion

The domination split is proved and implemented for the LongestPath mode only. The ShortestPath dual mirrors Section 7.6 but has not been derived in full, and the toolkit refuses the split for any other mode. For those modes the exact interval margins are

available only by exhaustive corner enumeration.

The split requires point-valued edge values. An interval-valued edge reduces to an interval duration by subdividing the edge with a node that carries the value, so the reduction is known, but the toolkit does not yet perform it and refuses interval edge values on the split path.

Probability-box durations are not supported. The forward quantities would propagate by the p-box arithmetic of Chapter 2, but the margins would meet the same dependence problem as the intervals, and no corner argument applies to them. Therefore no method for them is claimed.

The cost of the split grows with the bypass sets, and on dense meshes the sets approach the whole network. The never-worse rule keeps the cost at or below that of exhaustive enumeration, and no lower. Separately, the screening threshold that decides whether the split is attempted is a fixed count of interval inputs, chosen for safety and not derived from the network's bypass-set cost; computing that cost first, which is cheap, would let more networks reach the exact method.

The boundary of the toolkit is set by the algebra. A new quantity is added by giving its operator pair, and the pair obtains a backward pass in the same way as the four provided modes, by carrying a residual or by being linear. Whether a proposed pair has this property is a mathematical question that the implementation does not check, and showing it is the responsibility of whoever adds the mode. Two extensions follow the existing arguments: neighbouring nodes have overlapping bypass sets and therefore overlapping corner evaluations, which could be shared in the way the hash-consed store of Chapter 4 shares diamonds, and the edge subdivision above would admit interval edge values. These are listed in Chapter 11.

7.10 Summary

The classical critical path method has been generalised to a family of analysis modes, each an operator pair with the backward semantics its algebra supports, and four modes have been provided: LongestPath, ShortestPath and MaxScaling on one order-based

kernel, and Accumulation on a linear kernel whose backward object is the adjoint. Under interval durations the forward quantities have been computed exactly from two scalar runs, and the floats exactly by the domination split, whose correctness rests on three short proofs and whose cost grows with the reconvergent width around each node instead of with the number of uncertain inputs. The toolkit has been validated against a path oracle, exhaustive corner enumeration and Monte Carlo sampling, and applied to a benchmark project network. The MaxScaling mode is demonstrated on the integrated case study of Chapter 10. On a dense mesh the split reaches its boundary and the toolkit falls back to exhaustive enumeration, as the hardness of the general problem requires.

Chapter 8

Package Implementation in Julia

8.1 Introduction

Every method in Chapters 3 to 7 is implemented once, in one Julia package, and no analysis logic exists anywhere else in the framework. The interface of Chapter 9 calls the package through a local server and adds nothing to the computation. This chapter describes the package as a piece of engineering. It covers the choice of language and what it did and did not deliver; the types that realise the definitions of Chapter 3; the module structure and how an analysis is added to it; the three value forms in code; the caches that make the probability toolkit tractable; the validation scripts that stand in for a test suite, and what they found; the server; and how the package is distributed and its results reproduced.

The reader this chapter is written for is an engineer who would extend or embed the package, and an examiner who wants to know whether the software is a faithful and verifiable realisation of the methods. Code is quoted only where a design decision is clear in no other way.

8.2 The Language

The package is written in Julia [69], version 1.12. Three properties of the language were the reasons for choosing it. Multiple dispatch allows one function name to carry a different method for each value form, so that `multiply_values` on two `Float64` values, two intervals or two probability boxes is written once at each call site and resolved by the argument types (Section 8.5). Compiled performance without a second language means that the propagation loops, which run millions of times on the larger networks of Chapter 5, are the same code that is read and edited, with no C or Fortran layer beneath them. Furthermore, the probability-box arithmetic of Chapter 2 exists as a maintained Julia library, `ProbabilityBoundsAnalysis.jl` [45], so the framework did not have to implement p-box convolution itself.

Two expectations did not hold. Julia’s threading was expected to parallelise the propagation, and it did not: recursive task spawning over nested diamonds overflowed the task stack on deep networks, and the shared diamond cache would have needed

locking that removed the gain. Propagation is therefore single-threaded, and the iteration sets of Chapter 3, which are a natural boundary for parallel work, are used for ordering only. Similarly, just-in-time compilation, which makes the second call of a method fast, makes the first call slow, so that a fresh server process answers its first request in seconds, where the second takes milliseconds. This is why every timing in the thesis is a second-call timing, and why Appendix B states the convention.

8.3 Package Structure

The package is organised as a facade over five toolkit-facing modules plus two internal support modules, shown in Figure 8.1 and summarised in Table 8.1. The facade, `InformationPropagationAnalysis`, includes every module and re-exports twelve names: the five module names themselves (`Input`, `Diamonds`, `Probability`, `CriticalPath`, `Flow`), the two value types (`Interval`, `pbox`), and the four top-level entry points (`new_identify`, `update_beliefs_iterative`, `critical_path`, `analyze_all`), so that a user writes `using InformationPropagationAnalysis` and nothing else. The two support modules, `GraphValidation` and `GraphTraversal`, are included in the facade but not re-exported, since nothing outside the package calls them directly; they realise Chapter 3 together with `Input`, which reads structure and value files into the graph object. `Diamonds` realises Chapter 4, and `Probability`, `CriticalPath` and `Flow` realise Chapters 5 to 7 in turn.

The module names above are the registered package’s own, and they differ from the names used during development, from which the chapters of this thesis were written. The retired critical path implementation of Chapter 7, called `CriticalPath` during development to distinguish it from the mode-based `CriticalPathV2`, was never carried into the registered package: only the mode-based toolkit exists there, and it now carries the plain name `CriticalPath` with no clash, since nothing of the retired version remains to clash with. The same renaming, mapped in Table 8.1, applies to every other module.

The sizes in Table 8.1 show where the work went. The decomposition module is four times the size of the propagation module it serves, because the identification, the

Table 8.1: Modules of the registered package, the development name each carried, the chapter each realises, and their size.

Registered name	Development name	Realises	Files	Lines
Input	Shared (part)	Ch. 3		
GraphValidation	Shared (part)	Ch. 3	3	1,440
GraphTraversal	Shared (part)	Ch. 3		
Diamonds	DiamondDecomposition	Ch. 4	6	3,355
Probability	ProbabilityPropagation	Ch. 5	6	747
Flow	FlowCapacity	Ch. 6	12	6,101
CriticalPath	CriticalPathV2	Ch. 7	6	626

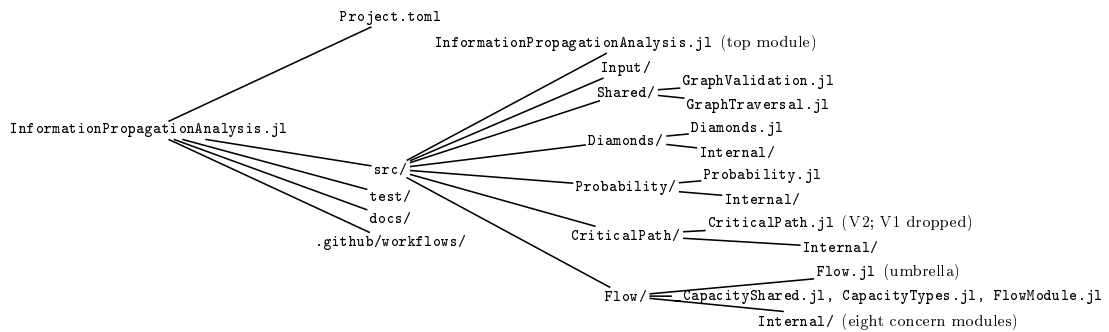


Figure 8.1: Structure of the registered package. Input builds the graph object; GraphValidation and GraphTraversal support it internally; Diamonds, Probability, CriticalPath and Flow are the four analysis-facing modules described in Section 8.3.

context-aware identity and the store of Chapter 4 carry most of the difficulty, and the propagation itself is a short recurrence once the diamonds are known. The flow toolkit is the largest module because it answers six questions with nine sub-modules and three solvers: `Sensitivity`, `FailureImpact`, `Structural`, `FlowDecomposition`, `ParametricThreshold`, `NodeCapacitatedFlow`, `MinCutUtilities` and `GlobalConnectivity` are the eight concern modules named in Figure 8.1’s `Internal/` folder, alongside the base solver module itself.

A fourth analysis is added by writing a module that reads the graph object and, if it needs the reconvergent structure, the `unique_subgraphs` field the decomposition module adds. No module has to be modified for the new one to run. The three toolkits are independent of one another and share only the graph object, which is what Chapter 1 claimed for the framework and what the code enforces by having no imports between toolkits.

8.4 The Graph Object

The graph object of Chapter 3 is a Julia named tuple whose fields are listed in Table 8.2. It is built once by `read_complete_network`, and every analysis takes it as its first argument. Two representation choices determine the performance of everything above it.

Table 8.2: Fields of the graph object and their types.

Field	Type	Content
<code>edgelist</code>	<code>Vector{Tuple{Int64,Int64}}</code>	directed edges
<code>outgoing_index</code>	<code>Dict{Int64,Set{Int64}}</code>	children of each node
<code>incoming_index</code>	<code>Dict{Int64,Set{Int64}}</code>	parents of each node
<code>source_nodes, sink_nodes</code>	<code>Set{Int64}</code>	nodes with no parent, no child
<code>fork_nodes, join_nodes</code>	<code>Set{Int64}</code>	fan-out and fan-in ≥ 2
<code>iteration_sets</code>	<code>Vector{Set{Int64}}</code>	the layers of Chapter 3
<code>ancestors, descendants</code>	<code>Dict{Int64,Set{Int64}}</code>	self-inclusive closures
<code>unique_subgraphs</code>	<code>Dict{key, Diamond}</code>	added by decomposition

First, adjacency is held as dictionaries from a node identifier to a set of identifiers, and edges are keyed by a tuple of two identifiers. Sets make membership tests and the

union operations of the closure algorithm constant-time on average, and tuple keys let the value maps of every analysis (a probability, a capacity, a duration per edge) use the same key as the structure without conversion. Second, the ancestor and descendant closures are stored in full. This is the space-for-time trade stated in Chapter 3: the decomposition algorithm asks whether one node is an ancestor of another many thousands of times on a large network, and each answer is one set lookup. On the densest networks of the corpus the closures are the largest structure in memory, and Section 8.6 returns to what that costs.

Julia has a mature general graph library, `Graphs.jl`, with weighted and property-graph companions, and it was not used, for four reasons. Its vertex identifiers must be the contiguous integers 1 to n , while infrastructure data arrives with arbitrary integer identifiers and gaps, so every input would need a renumbering layer and every output a reverse one. Its topological sort returns a flat order, not the layers that every algorithm in the framework iterates over. It has no directed-acyclic-graph type, so acyclicity would be checked after construction. Chapter 3 requires it to be enforced during ingestion. And attaching per-edge values of a user-defined type through the property-graph companion is not type-stable, which in Julia means the inner loops would allocate. A purpose-built object with the fields of Table 8.2 avoids all four and adds no dependency.

8.5 The Three Value Forms in Code

A deterministic value is a `Float64`, and its helpers are plain Base arithmetic. An interval is a two-field immutable struct, `Interval(lower, upper)`, defined in the input module. A probability box is the `pbox` type of `ProbabilityBoundsAnalysis.jl` [45], re-exported so that a user sees one set of names; its own binary operators, `convIndep`, `convFrechet` and `convPerfect`, and its own constructors, `interval`, `makepbox` and the parametric family (`normal`, `uniform` and others), are what the interval and p-box branches of every helper call. The package itself depends directly on `ProbabilityBoundsAnalysis`, `JSON`, `DataStructures` and `DelimitedFiles` and nothing else; an earlier implementation also depended on `Combinatorics`, `DataFrames` and `Distributions` directly, none of which

is called anywhere in the current source, and all three were removed in the restructure that produced the module layout of Table 8.1. Where the parametric p-box constructors reach a named distribution, they reach it through `ProbabilityBoundsAnalysis`’s own use of `Distributions.jl`, one layer below the package’s own dependency graph, not through a call the package makes itself.

The propagation of Chapter 5 is written against a small set of helper functions, listed in Table 8.3, each with a method per form. The recurrence calls `multiply_values`, `complement_value`, `prod_values` and `pbox_conditional_combine` and never names a type, so the same source computes exact beliefs, exact interval bounds and sound p-box bounds according to what it is given. This is the one place where multiple dispatch carries the design, and it is the reason the framework can claim one propagation for three forms.

Table 8.3: Value helpers over which the probability propagation is generic.

Helper	Float64	Interval	pbox
<code>multiply_values</code>	product	corner product	independent convolution
<code>complement_value</code>	$1 - a$	$[1 - \bar{a}, 1 - \underline{a}]$	$1 \ominus a$
<code>sum_values,</code> <code>prod_values</code>	fold	fold of the above	fold of the above
<code>pbox_conditional_combine</code>	$wA + (1 - w)B$	corner enumeration	mixture over w , positive-dependence blend

The last row is the operator behind the finding recorded in Chapter 5. The total-probability step at a diamond join, $wA + (1 - w)B$ for a conditioning belief w , is a convex combination. The first p-box implementation treated it as a convolution, which gave bounds wider than the truth with mass above one. The present operator integrates over the discretisation levels of w , combines the two branches under a positive-dependence bound, since both branches are monotone in their inputs, and envelopes over the imprecision of w . It then intersects the result with the unit box, and refuses, with an error naming the cause, if the excursion outside $[0, 1]$ before that intersection exceeds

floating-point noise. That refusal is deliberate. A silent clamp would have hidden the original error.

However, the critical path toolkit does not use dispatch for its interval form, for the reason recorded in Chapter 7: running the scalar kernels on an interval type gives wrong minima and over-wide margins. Its interval results are therefore computed by scheme, from scalar runs at chosen corners, and its own `ValueInterval` type exists only to carry a result. That type checks `lower <= upper` on construction with no tolerance, and the check is kept strict because a constructor that silently reorders bounds hides errors. The consequence, found on the PSPLIB instance of Chapter 7, is that two independent sweeps that both compute an exactly zero float can differ by a few units in the last place and trip the check. The fix is in the caller: a float within the tolerance of zero is set to zero before the interval is built, and the constructor stays strict.

8.6 Caches and Memory

The decomposition of a network is computed once and stored, keyed by the content of the structure file and the prior file, so that a second scenario on the same network reuses it and an edit to either file invalidates only what depends on it. Within a propagation, every solved diamond is memoised under a compact key of hashes, the diamond’s edge list and the priors it was solved under, so that a diamond posed again by a different enclosing recursion is looked up. It is not recomputed. Chapter 5 reports that this deduplication reduces the work to between one and five per cent of the a priori bound on the larger networks.

However, the store’s memory, not its time, is what limits the probability toolkit. Under interval inputs the number of distinct contexts a diamond is posed under grows with the width of the intervals, and on one benchmark network the store reached two million entries and 24 gigabytes, where the same network under point values needed three thousand. A lean mode of the store, off by default, holds for each entry only the belief at its own join in place of the full per-subgraph dictionaries. An entry is about one hundred times smaller. The compute path reads a cached entry only at the join,

so the results are unchanged, and the rare collision of two diamonds with the same key but different joins is caught by a lookup miss that recomputes. The mode was gated on identical results across the eleven-network regression set before it was kept.

Identification, as distinct from propagation, is bounded by memory in the same way, and its failures on the largest networks of Chapter 5 are out-of-memory errors at identification and not time-outs. The single-threaded propagation of Section 8.2 is therefore not the binding constraint on scale.

8.7 Validation Infrastructure

The registered package carries a formal test suite (214 assertions, run through `Pkg.test`) and continuous integration on GitHub Actions, matrixed across Julia 1.12 on Ubuntu and Windows. Neither, however, is where the exactness claims of the analysis chapters come from. The test suite exercises the package’s own correctness on small synthetic instances, chiefly a brute-force exact-reliability oracle over diamonds with up to seventeen state bits. Running it is slow, for a reason that is recorded here: on Julia 1.12, `ProbabilityBoundsAnalysis.jl` cannot be precompiled, because an upstream method overwrites a Fréchet-bound function in a way the compiler cannot cache, so the whole dependency chain loads interpreted on every fresh process. A warm depot completes the suite in under a minute, and almost all of that is the load; the assertions themselves run in under ten seconds. A cold depot, which is what a continuous-integration runner starts from, spends several minutes precompiling roughly a hundred transitive packages before the same assertions run, and the workflow sets `JULIA_PKG_PRECOMPILE_AUTO=0` to skip attempting to precompile the one package that cannot be. This is a property of an upstream dependency, and it affects only how long a contributor waits.

What the test suite and continuous integration certify is narrower than what the chapters of this thesis claim. The evidence for those claims, the exactness of the propagation on real and adversarial networks up to the scale reported, the validity of the interval and p-box bounds, and the correctness of the flow and critical path toolkits on their case studies, is a set of validation scripts, one family per toolkit, each of which runs

the toolkit and an independent computation on the same inputs and compares them. They are listed with their oracles in Appendix B. For the probability toolkit the oracles are a binary decision diagram library, exhaustive path enumeration and Monte Carlo sampling (Chapter 5); for the critical path toolkit a brute-force path oracle, exhaustive corner enumeration and Monte Carlo (Chapter 7); for the flow toolkit three solvers against one another, an external flow library and brute-force removal, bisection and subset enumeration (Chapter 6). The scripts are run on a fixed corpus of networks with recorded seeds, and their logs are kept with the results they certify. Converting them into a test suite that runs on every change is listed in Chapter 11; the difference between the two is when they run, not what they check.

These scripts found six errors that the methods' proofs did not prevent. Two were in identification: a collision between the context and the key of a diamond on reconvergent grids under non-unit priors, and an eligibility test that excluded every node with prior one and so found no diamonds in a network whose priors were all one (Chapter 4 proves the identity that replaced the first). One was the floating-point residual of Section 8.5, found on a published scheduling benchmark and not on any synthetic network, because only the benchmark had a degenerate node whose float two sweeps reached by different arithmetic. One was in the node-splitting transformation of the flow toolkit, which assigned the entry and exit nodes of a split node the identifiers $2v$ and $2v + 1$; these collide with unsplit nodes when only some nodes carry a capacity and the identifiers are small, which no corpus network exercised because each gave every node or no node a capacity, and which the IEEE test system of Chapter 6 exercised at once. One was in the minimum-cut lattice, whose set of edges present in every minimum cut was computed against the wrong side of the residual reachability and so returned saturated edges inside the source side; the error was visible on the smallest network in the interface of Chapter 9, and the validation check that should have caught it, comparing the set against the enumerated cuts, is now part of the flow scripts. And one was in the validation of a probability-box input, where the library's minimum and maximum of an imprecise box return a box where a number was expected, so a valid input failed with a type error; the bounds are now read from the box's own discretised curves.

Each of the six is a boundary the synthetic corpus did not reach and a real or benchmark instance did, which is the argument for benchmark instances in a validation set. Each was fixed at its cause, and each fix was gated on unchanged results everywhere the code had passed before.

8.8 The Server

The interface of Chapter 9 reaches the package through a small HTTP server, **InfoPropServer**, built on **HTTP.jl** and **JSON**. These are the only two dependencies of the package that the analysis modules do not use, and they are confined to the server directory, so that the package can be split from the server without change.

The server binds the loopback interface and accepts cross-origin requests only from the local interface’s origin, which is the mechanism behind the data-sovereignty claim of Chapter 9: the client and the server share a machine, and no request can arrive from elsewhere. An upload creates a session, which is a folder holding the network’s files as uploaded, byte for byte, plus a metadata record and, once computed, the persisted decomposition. Every analysis is a route with a handler of its own (structure, capacity and flow, diamond identification and subgraph analysis, probability propagation, critical path), so that adding an analysis means adding a handler and a route and touching nothing else. Each response carries a request identifier and, on failure, the exception message and a bounded stack trace, which is how the errors of Section 8.7 were diagnosed from the interface. The contract between server and client is written as an OpenAPI document, version 2.0.0, from which the interface’s client library is generated (Chapter 9).

One handler decision follows from Chapter 7. The exact interval floats by the domination split can decline an instance whose run count exceeds the limit, and the handler must then fall back to the conservative enclosure. Declining and failing were at first caught by the same clause, so that the floating-point error of Section 8.5 was reported to the user as intractability. The split now declines with an exception type of its own that carries the run counts, the handler catches only that type and reports the

counts, and every other exception propagates.

8.9 Distribution and Reproducibility

The package is published in the Julia General registry as `InformationPropagationAnalysis.jl` [82], version 0.2.1, registered by pull request #166556 at version 0.2.0 and updated to 0.2.1 shortly after, and the results in this thesis were produced under Julia 1.12.6. Its direct dependencies, as stated in Section 8.5, are `ProbabilityBoundsAnalysis`, `JSON`, `DataStructures` and `DelimitedFiles`, all with version bounds, and a manifest fixes every transitive version. Version 0.2.1 supersedes the five-file, reliability-only package registered as the same name’s first release: it carries the full module structure of Table 8.1, including the flow and critical path toolkits, and every fix described in this chapter (the p-box validation fix of Section 8.5, the minimum-cut lattice fix and the domination-split exception of Section 8.8) is in the tagged version, not a later, unregistered patch. The retired critical path implementation of Section 8.3 was not carried into either registered release. The second release renamed the modules to the names of Table 8.1 and added the test suite and continuous integration of Section 8.7. Three code changes were made between 0.2.0 and 0.2.1. Two are not observable to a caller: a rename from deprecated queue methods to their replacements, and a type check widened from a concrete dictionary type to the abstract dictionary interface, which was already satisfied in every case the package exercises. The third removed four duplicated helper functions from the flow toolkit’s sensitivity module, with the effect that the edges reported in some or every minimum cut are now returned in sorted order; the set of edges is unchanged. The same 214 assertions pass identically on both versions, including the exact belief, the power-network golden vector, maximum flow against minimum cut, and the PSPLIB makespan of Chapter 7. The package was confirmed by installing it from the registry into a clean depot, separate from the development checkout, and running one case from each toolkit against it: exact belief propagation, maximum flow agreeing with the minimum cut, and a critical path makespan with its critical activities, all matching the development checkout. This check was run against the registry install

available at the time, version 0.2.0. A repeat of the same check, stamped explicitly at 0.2.1, gave $\text{belief}(5) = 0.5832878525190001$ on the power network, a maximum flow of 10.0 agreeing with a minimum cut of 10.0, and a critical path longest-path value of 10.0, each identical to the corresponding 0.2.0 result. The two releases are confirmed equivalent for every purpose this thesis makes of the package.

The server and interface of Chapter 9 are developed in their own public repository, `information-propagation-no-code` [83]. The corpus, the validation scripts and their logs, the case-study inputs and every figure’s generating script are kept separately, in `thesis-data` [84], a repository shared with the journal paper of Chapter 5. Its README maps every chapter, figure and table of this thesis to the script and data that produced it. This repository is what is archived and cited: the tagged release `v1.0` (DOI 10.5281/zenodo.22180227) fixes the version of every script and dataset the thesis’s results depend on. Reproducing a chapter’s numbers is a matter of installing the package at the version stated in Section 8.9, checking out the archived repository at the cited tag, and running the scripts named for that chapter; the timing convention of Appendix B applies.

8.10 Limitations

Propagation is single-threaded, for the reasons in Section 8.2; a lock-free store or a work-stealing scheme over the iteration sets would be the route to parallelism and is not built. The package’s own test suite certifies correctness on small synthetic instances and does not cover the scale or the real networks this thesis reports. The package reads only the two structure formats of Chapter 3 and performs no conversion from domain formats. The critical path toolkit carries no probability-box form and the flow toolkit no interval or probability-box form.

8.11 Summary

The package realises the methods of Chapters 3 to 7 in seven Julia modules behind one facade, on one graph object whose types were chosen for the lookups the algorithms

make and for identifiers as infrastructure data supplies them. Multiple dispatch carries the probability toolkit across three value forms through a small set of helpers, and the critical path toolkit does not use it. Two caches, one of decompositions keyed by file content and one of solved diamonds keyed by hashes, make the probability toolkit tractable, and memory is what limits it, not time. Its test suite certifies correctness on small instances; a set of oracle-backed validation scripts on a fixed corpus certifies every result reported in this thesis, and the six errors those scripts found and fixed are recorded here. A loopback-only server exposes the package to the interface of the next chapter through a handler per analysis and a published contract.

Chapter 9

The No-Code Web Interface

9.1 Introduction

Chapter 2 closed on a gap that has nothing to do with algorithms. The analyses of Chapters 5 to 7 exist as a Julia package, and a package is used by writing code. The engineer who holds the system model does not write code. The tools that do reach such a person are either cloud services, which are not acceptable for a proprietary system model, or licensed desktop suites that carry none of these analyses. That gap is closed by the interface described here. It is a browser application that runs on the engineer's own machine, takes the same files the package reads, runs the three toolkits against them through the local server of Chapter 8, and returns every result in the form the package computed it.

Each decision in the interface is described with the reason behind it. Section 9.2 states who the interface is for and what follows from that. Section 9.3 walks through the workflow in the order a user meets it, with the screens of the power distribution network of Chapter 5 shown as illustrations of the controls. Section 9.4 lists the rules the interface enforces on how results are shown, and one rule it does not yet enforce. Section 9.5 gives the architecture and Section 9.6 the contract between the interface and the package. Section 9.7 states what evidence the chapter can offer for usability and what it cannot, and Section 9.8 the limits of the interface. The full application of the interface to a real network, with results, is the subject of Chapter 10; nothing in this chapter reports a number from a screen.

9.2 Who the Interface Is For

An engineer with a system model and a question is the user. The model may be an edge list exported from an asset register or drawn up by hand. The question is one of the three of Chapter 1: with what probability does supply reach this point, how much can the system deliver, or when does this sequence complete. The user is assumed to know the system and its data, to be comfortable with a spreadsheet and a browser, and not to program.

Four requirements follow. The user must not have to write code, so every analysis

is reached by uploading files and pressing a button. The user must not have to learn a second data format, so the files the interface accepts are the files the package reads, byte for byte. The user's system model must not leave the user's machine, since the models in question describe critical infrastructure and in some sectors cannot be sent to a third party at all, so the interface runs against a local server and nothing is transmitted elsewhere. And the user must be able to trust what is on the screen, so the interface shows what the package computed and adds no interpretation of its own. The last requirement is the one that shapes the interface most, and Section 9.4 returns to it.

9.3 The Workflow

A session begins with an upload (Figure 9.1). The user selects a folder that holds the network's structure file at its root and one sub-folder per scenario, each scenario carrying any subset of the four input files of Chapter 3. The upload page reads the folder before anything is sent and lists what it found: the network name, the structure file, the scenarios, and for each scenario which toolkits its files support and in which value form. A scenario folder may be named for its value form (`float`, `interval`, `pbox`) or for an operating case (`Baseline`, `Degraded`); in the second case the value form is read from each file's own `data_type` field. The expected layout is printed beside the chooser, so that a user with a folder in the wrong shape sees the difference before uploading.

On upload the server creates a session, a folder holding the files as uploaded and a metadata record, and returns the network's structure. Figure 9.2 gives the shape of the whole workflow before the screenshots that follow walk through it step by step. The network page (Figure 9.3) shows the graph object of Chapter 3 as the package built it: node and edge counts, sources and sinks, forks and joins, the layered order, and the network drawn by layer. This page is where a user first sees whether the structure file was read as intended, since a mis-oriented edge or a missing node shows here as a wrong source or an unexpected layer.

Each toolkit has a page of its own, reached from the navigation rail, and the three pages share one layout. A row of scenario chips at the top selects the scenario, each

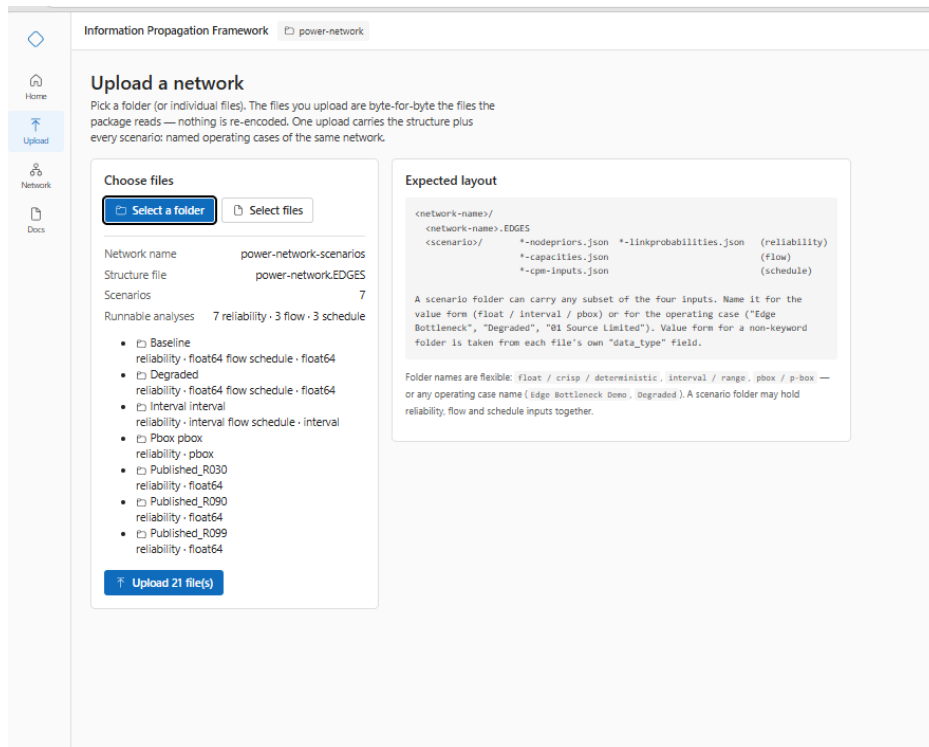


Figure 9.1: The upload page. Left, the folder chosen, with the scenarios it holds and the analyses each supports listed before upload. Right, the expected folder layout.

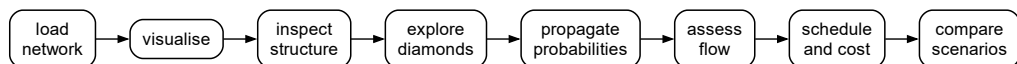


Figure 9.2: The workflow a session follows: load the network, then read its structure, explore its reconvergence, and apply the three toolkits, before comparing scenarios.

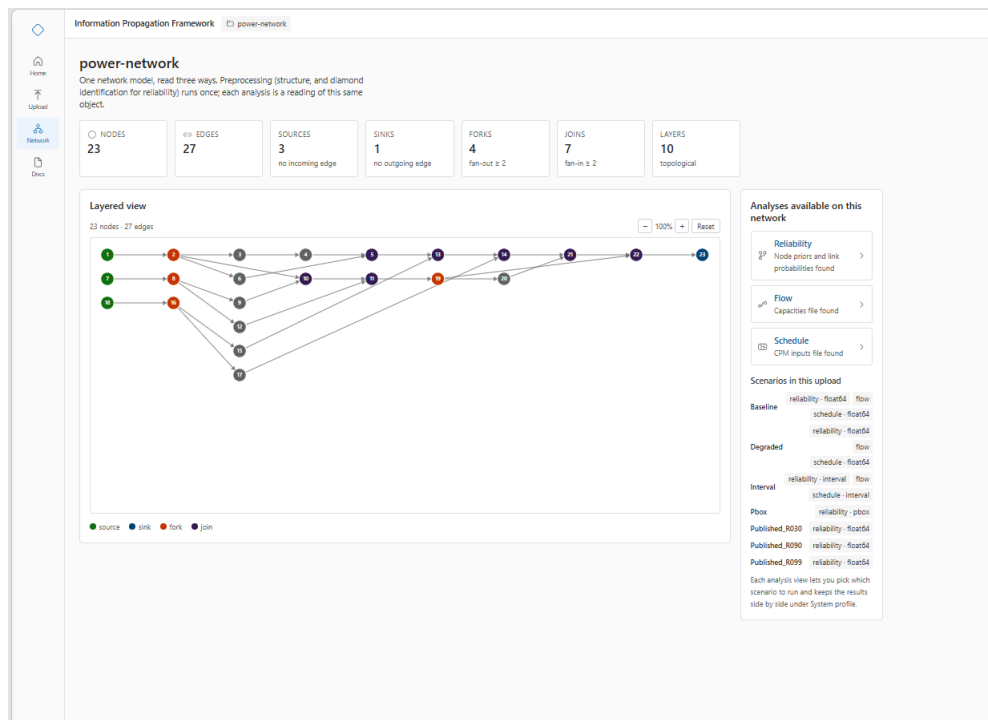


Figure 9.3: The network page after upload: structural summary of the graph object and the network drawn by layer.

chip showing the scenario's value form, and a value-form filter above the chips narrows the row to deterministic, interval or probability-box scenarios. Below the chips are the toolkit's tabs. For the probability toolkit (Figure 9.4) these are Belief, the per-node table with summary tiles; Diamonds, the reconvergence structure the decomposition identified; Visualisation, the network drawn with the conditioning set highlighted; and Compare. The Belief table can be filtered by node role and restricted to diamond joins, and every summary tile names what it summarises. Similarly, the flow and schedule pages have their own tabs: configuration, summary, bottlenecks and visualisation for flow (Figure 9.6); time, cost, visualisation and compare for schedule (Figure 9.7).

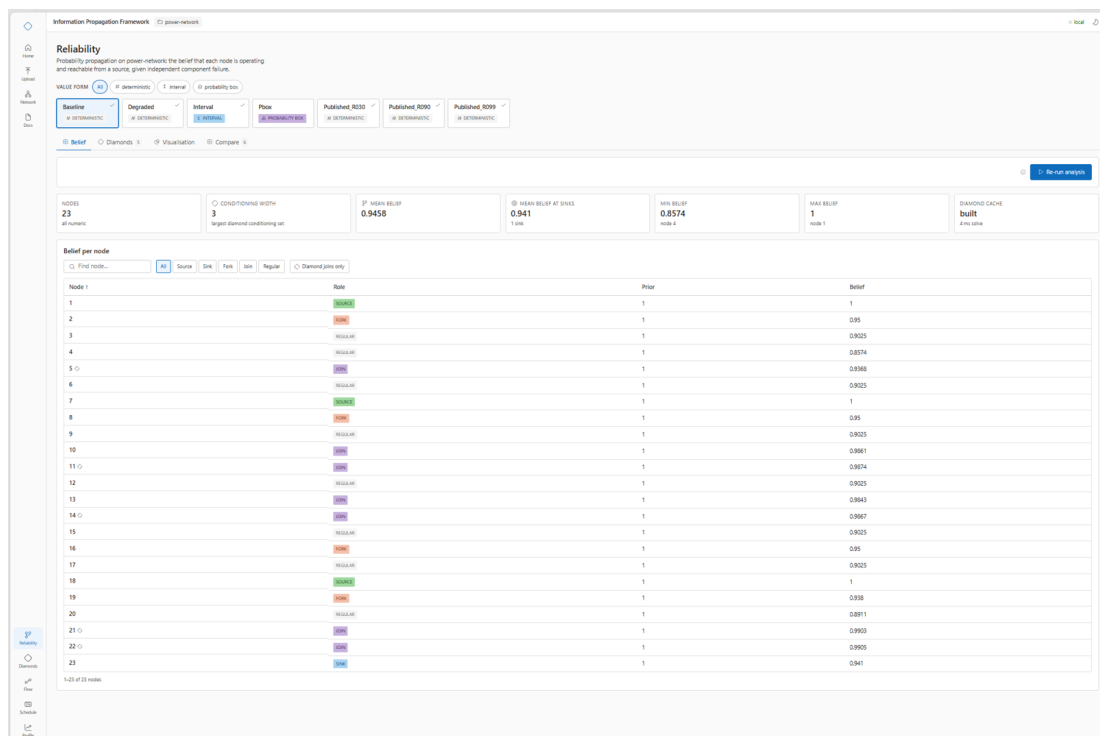


Figure 9.4: The reliability page, Belief tab. Top, the value-form filter and the scenario chips, each chip labelled with its value form. Below the tabs, summary tiles and the per-node belief table with its role filters.

A diamond in the Diamonds tab opens a detail view (Figure 9.5) that draws the diamond as a graph of its own, lists its conditioning set, local sources and size, and offers two actions. The first runs the reliability analysis on the diamond alone, as a standalone network, with the local sources' priors overridable, which lets a user study a

reconvergence that dominates the network’s uncertainty without the rest of the network in view. The second promotes the diamond: it is uploaded as a new, independent network and becomes a session of its own. Both actions are possible because Chapter 4 returns every diamond as a graph object of the same form as the network.

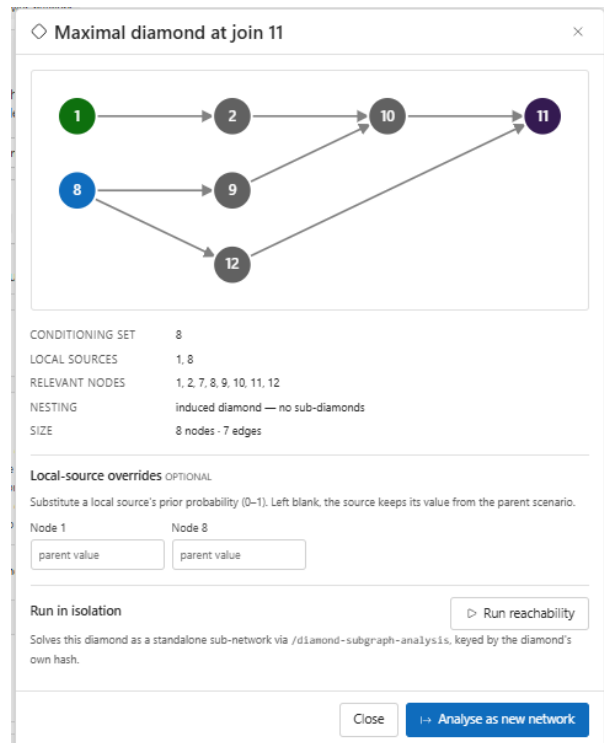


Figure 9.5: A diamond’s detail view: the subgraph, its conditioning set and local sources, the override fields for the local sources, and the two actions, run in isolation and analyse as a new network.

The Compare tab (Figure 9.8) selects any subset of the network’s scenarios for the toolkit, runs those not yet run, one at a time, and places the results side by side in one table with an optional baseline for differences. The system-profile page (Figure 9.9) is the one view that crosses toolkits: every result set that any analysis has produced on the network is listed, and any one of them can be drawn on the network with a second overlaid, so that a schedule’s critical path and a flow’s binding cut can be seen on the same nodes. The page computes nothing; it draws what the analyses already found.

Chapter 9. The No-Code Web Interface

Flow
Maximum deliverable throughput from a source set to a sink set under the active edge and, when modeled, node capacity constraints — with minimum cut diagnostics, structure, topology, flow sensitivity, failure impact and parameters. Throughput: Flow4Ed capacitors only; multi-source / multi-sink and node capacities are handled internally.

Capacities scenario
Each scenario is one $\rightarrow$ Capacities (json: operating case found on this network).
Scenario: **Baseline**
Capacities: **Baseline/feasible-capacity.json**
Network: **power-network**
Edges: **power-network-capacity/power-network-edges**

CAPACITY VALUE TYPE
☒ Deterministic ☐ Interval unmodeled ☐ Probability box unmodeled
The flow solver computes with feasible capacities only.

Run options
Max flow solver: **Dinic** Tolerance: **1e-10** $\rightarrow$ for k -edge failure analysis
Minimum cut enumeration limit: **2** Path enumeration limit: **10000** k -edge combination limit: **10000**
New search depth: **64** Target throughput (optional): **none**
Uniform capacity scale factors (if optional, comma-separated): **eg: 0.5, 0.75, 0.5**
Each k is value flow flow with only flow capacity related to k .
☒ Apply node capacities when the file provides them (handled internally by node splitting)

Run flow analysis **Get test Baseline**

Figure 9.6: The flow page, configuration tab: scenario, solver, tolerance, the failure combination size, enumeration limits, and the optional target throughput and degradation multipliers.

Scenario
Interval Interval — Interval/Interval-cap-inputs.json

EXPECTED VALUE TYPE
☒ Deterministic ☒ Interval ☐ Probability box unmodeled
The value type comes from the CPM file (see data_type and this selection — this is a per selection from the response's initial_page is authoritative).
Interval — pass modes
A mode is an operator put together with a failure mode and a probability box. A mode is a pass mode. Each mode is a pass mode. The CPM file declares the mode's length (mode / $\rightarrow$ A mode is only a pass mode if it is a mode).

MODES
Auto — the operator gain the CPM file declares **MODES**
Auto — the operator gain the CPM file declares

Run flow analysis **Get test Baseline**

Time
No run analysis in this CPM file. **Visualization** **Compare**

TIME PAGE **FORWARD PAGE** **BACKWARD PAGE**

PROJECT VALUE
18.162 **Interval**
Interval complete path — the value path

CRITICAL ACTIVITIES
3 / 22
Activity: 1 priority

COMPLETION RUNS
4180
Interval/Interval

MODE
LongestPath
mode

METHOD
Exact — domination split

COMPLETION
0.266 s

NECESSARY CRITICAL **1** **2** **3** **4** **5** **6** **7** **8** **9** **10** **11** **12** **13** **14** **15** **16** **17** **18** **19** **20**

ACTIVITY	MODE	COMPLETION	FAILURE	STATUS	CRITICAL
1	Source	05.05	05.165	05.05	Possible
2	Path	11.25	05.165	05.05	Possible
3	Interval	02.45	05.165	05.05	Possible
4	Interval	05.05	05.165	05.05	Possible
5	Join	04.05	05.165	05.05	Possible
6	Interval	02.45	07.145	05.75	Possible
7	Source	05.05	05.165	05.05	Possible
8	Path	11.25	05.165	05.05	Possible
9	Interval	02.45	05.165	05.05	Possible
10	Join	05.05	05.165	05.05	Possible
11	Join	04.05	05.165	05.05	Possible
12	Interval	02.45	07.145	05.75	Possible
13	Join	05.105	05.165	05.05	Possible
14	Join	05.125	05.165	05.05	Possible
15	Interval	02.45	05.125	05.05	Possible
16	Path	11.25	05.125	05.05	Possible
17	Interval	02.45	05.125	05.05	Possible
18	Source	05.05	05.125	05.05	Possible
19	Path	05.105	05.165	05.05	Possible
20	Interval	05.125	05.165	05.05	Possible
21	Join	07.145	05.165	05.05	At Necessity
22	Join	05.165	05.165	05.05	At Necessity
23	Sink	05.165	05.165	05.05	At Necessity

Completion 01 — the last value is a value of dependencies can deliver into the activity.
Completion 02 — the last value is a value of dependencies can deliver into the activity.
Completion 03 — the last value is a value of dependencies can deliver into the activity.
Completion 04 — the last value is a value of dependencies can deliver into the activity.
Completion 05 — the last value is a value of dependencies can deliver into the activity.
Completion 06 — the last value is a value of dependencies can deliver into the activity.
Completion 07 — the last value is a value of dependencies can deliver into the activity.
Completion 08 — the last value is a value of dependencies can deliver into the activity.
Completion 09 — the last value is a value of dependencies can deliver into the activity.
Completion 10 — the last value is a value of dependencies can deliver into the activity.
Completion 11 — the last value is a value of dependencies can deliver into the activity.
Completion 12 — the last value is a value of dependencies can deliver into the activity.
Completion 13 — the last value is a value of dependencies can deliver into the activity.
Completion 14 — the last value is a value of dependencies can deliver into the activity.
Completion 15 — the last value is a value of dependencies can deliver into the activity.
Completion 16 — the last value is a value of dependencies can deliver into the activity.
Completion 17 — the last value is a value of dependencies can deliver into the activity.
Completion 18 — the last value is a value of dependencies can deliver into the activity.
Completion 19 — the last value is a value of dependencies can deliver into the activity.
Completion 20 — the last value is a value of dependencies can deliver into the activity.
Completion 21 — the last value is a value of dependencies can deliver into the activity.
Completion 22 — the last value is a value of dependencies can deliver into the activity.
Completion 23 — the last value is a value of dependencies can deliver into the activity.

Figure 9.7: The schedule page under an interval scenario. The value-form selector shows the form the file declares and marks the probability-box form unavailable for this toolkit; the advanced control exposes the pass modes; the result tiles carry the method tag.

Chapter 9. The No-Code Web Interface

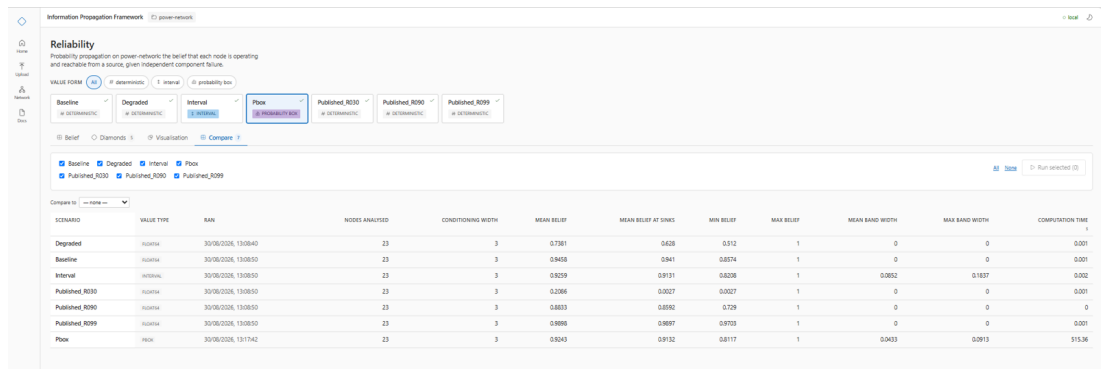


Figure 9.8: The Compare tab: scenario selection, the run control for scenarios not yet run, and the side-by-side table with a baseline selector.

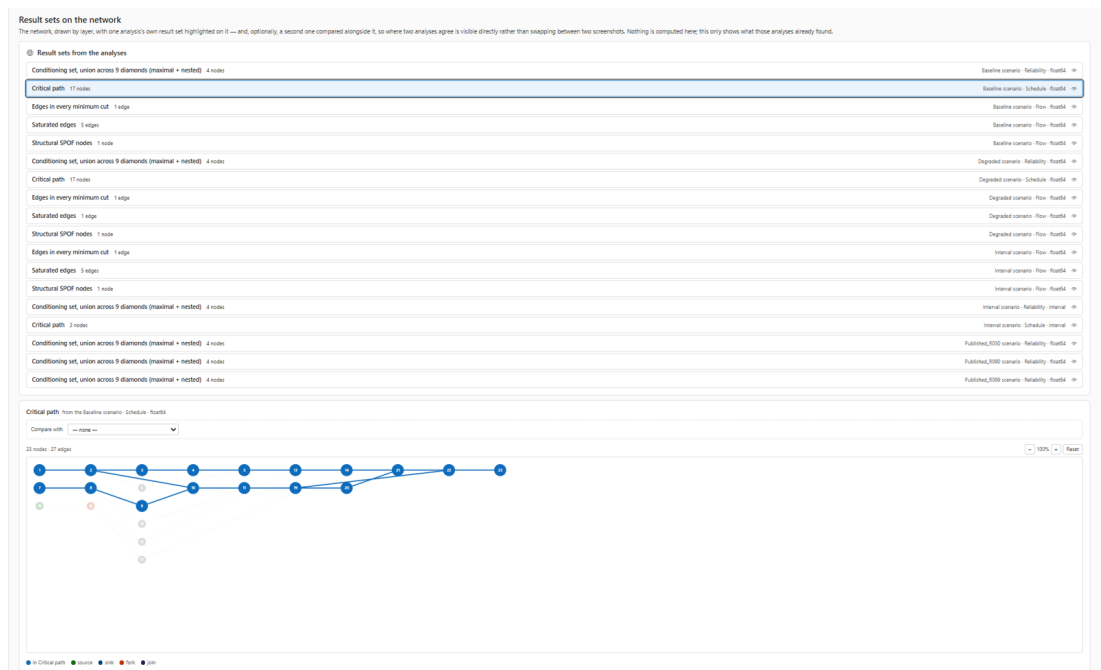


Figure 9.9: The system-profile page: the result sets available on the network, one selected and drawn by layer, with a second selectable for overlay.

9.4 What the Interface Enforces

The fourth requirement of Section 9.2, that the screen shows what the package computed and nothing else, is met by four rules that the interface applies everywhere, and they are stated here with the reason for each.

A value keeps its form. A belief under an interval scenario is shown as a range wherever it appears, and a belief under a probability-box scenario as its bounding summary, never as a midpoint. One component, `ValueDisplay`, renders every value of every form in every table and detail view, so that no view can invent its own rendering. The one exception is stated on the screen: a few summary tiles, such as the mean belief, use a midpoint so that a single summary number is possible, and each such tile says so in its label. A per-node value is never flattened. The reason is Chapter 5: the range is the result, and its midpoint is a number the toolkit never computed.

A result set is what the toolkit returned. The bottleneck table reports the edges in every minimum cut as the flow toolkit computed them; the profile page draws a critical path as the schedule toolkit reported it; no score, ranking or threshold is computed in the interface. Where a table is sorted, the sort key is a field the toolkit returned. A number computed in the interface would be a number the thesis has not validated.

Every result carries its method, since what a result guarantees depends on the method that produced it (Chapter 7). The schedule toolkit returns interval margins from one of three methods, and the probability toolkit returns beliefs from an exact or a decomposition-only call; in each case the response names the method, and the interface shows the tag beside the result. A conservative enclosure and an exact range are never displayed the same way, because a reader who cannot tell them apart cannot use either.

One analysis runs at a time. Because the propagation is single-threaded and its cache is not locked (Chapter 8), a second concurrent request would either wait or, at worst, corrupt a shared store. The Compare tab therefore runs unrun scenarios in sequence, and no view issues two requests to the server together.

However, one rule the interface does not yet enforce is recorded here because it was found in use. Probability-box propagation has a cost that grows steeply with the

discretisation level of the bounding functions, and Chapter 5 recommends levels between 25 and 100 for networks of the scale the method suits. The server does not set the level, so the library’s own default of 200 applies to every request, which is the expensive end of the range. A run on the 26-node KarlNetwork at that default ran for seventeen minutes and blocked the single-threaded server, while the same run at level 50 completes in nine. The interface exposes no control over the level, applies no default of its own, and gives no estimate or warning before a run that may take minutes. What a user meets as intractability is therefore a setting the theory chapter already documents, and the fix is a control and a default in the interface, not a change to the method. It is listed in Chapter 11.

9.5 Architecture

The interface is an Angular application in an Nx workspace, built on Microsoft’s Fluent web components for its controls and on d3 for the network drawing. The workspace holds eight projects: the application shell, which owns routing, the navigation rail and the pages that are not toolkit-specific; five feature libraries, one per toolkit page (reliability, which also carries the standalone Diamonds page, flow, schedule), one for the system profile and one for the session-inputs editor; and three shared libraries, an API client generated against the server’s published contract, a data-access library holding the scenario cache and the folder convention, and a presentational library of cards, tables and the network graph.

Two dependency rules are enforced by the build, not by convention. A layer rule allows the shell to depend on features, features on the shared libraries, and presentational components only on other presentational components and the API client, never on data access. Thus a component in the presentational library that needs a scenario’s shape declares a small structural type of its own. A scope rule allows a feature to depend on its own scope and the shared scope only, so that no feature imports another feature; a view that needs another toolkit’s components, such as the standalone Diamonds page, lives inside that toolkit’s library. Both rules are checked by the linter on every build,

which is how the claim of Chapter 1 that the three analyses are independent is made to hold in the interface as well as in the package.

The Fluent component set has no card, table or data-grid component, so the interface’s cards are its own and its tables are native HTML tables styled with the Fluent design tokens. About forty components are registered once at start-up, and about fifty icons are drawn from the Fluent system icon set.

Client and server share one machine. The server binds the loopback address and accepts requests only from the interface’s own origin, as Chapter 8 describes, and the interface has no other network access. Every request names files by their path in the session; no request carries the network in its body, and no file is re-encoded on the way. Figure 9.10 shows the whole deployment inside the boundary of the user’s machine: the browser talks to the analysis service over HTTP on localhost, and the service alone reaches the framework’s modules and the session’s files and decomposition store. A local-first application in this sense keeps the usability of the web without the custody question of Chapter 2, since there is no remote party to hold the data [64].

9.6 The Contract as the Boundary

The boundary between interface and package is a written contract: an OpenAPI document, version 2.0.0, describing fourteen endpoints, from which the interface’s API client is generated. Each analysis has one endpoint and one handler, and each endpoint takes a JSON body naming the session and the files and returns a JSON response whose values keep their form on the wire: a deterministic value is a number, an interval is an object with a lower and an upper field, and a probability box is a fuller typed object. Because the same files that the interface uploads are the files the package reads, anything the interface does can be repeated from the command line against the same folder, and the case study of Chapter 10 records every server request and response beside its screenshots for that reason.

The contract has changed once during the work, when three endpoints were renamed to match the toolkit names of this thesis. The old names were kept as aliases, so

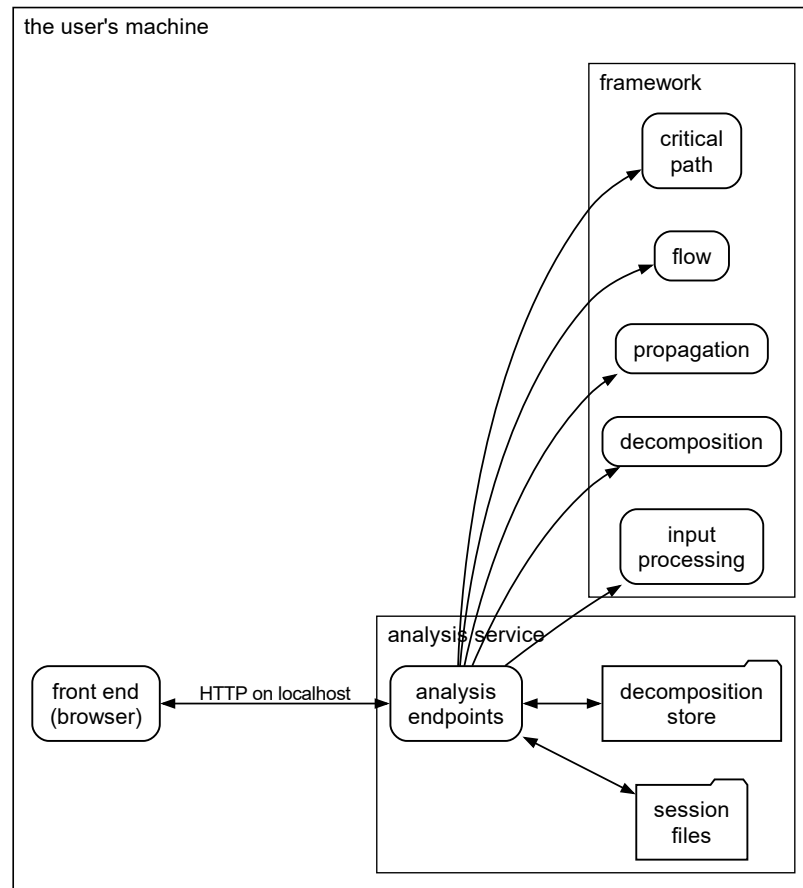


Figure 9.10: The deployment, contained entirely within the user's machine. The front end reaches the analysis service over HTTP on localhost; the service alone calls the framework's modules and reads and writes the session's files and decomposition store.

the interface built against the earlier contract continued to work while the client was regenerated. That is the ordinary benefit of a published boundary, and it is recorded because it was tested.

9.7 Evaluation

This interface has been used by its author and demonstrated to others, and it has not yet been used independently by an engineer who does not program. Hence the evaluation that can be offered is a task-based walkthrough, and it is labelled as such: it shows that the tasks a user would set are reachable without code, and it does not show how a user found them.

The tasks are the ones a planner with a system model would set. Load a model that exists as an edge list and three input files. See whether the structure was read as intended. Find the belief at each demand point under the current component data, and under a stated range on that data. Find which reconvergence in the network carries the uncertainty, and study it alone. Find the deliverable throughput and the binding cut, and what fails first. Find the completion time of a dependent sequence and which activities are certainly critical under duration uncertainty. Compare two operating cases. See where the schedule's critical path and the flow's binding cut coincide.

Every one of these is a click sequence in Section 9.3, and every result it produces is a result of Chapters 5 to 7 in the form those chapters state. The demonstration of the sequence on a real network, with every request and response recorded, is Chapter 10. However, the walkthrough cannot show the time a new user takes, where the labels are unclear, or which of the four rules of Section 9.4 a user notices; that is the study that remains to be done, and it is listed in Chapter 11.

9.8 Limitations

The interface has no control over the probability-box discretisation level and no warning before a long run, as Section 9.4 records. It has no user study behind it. It accepts the two structure formats of Chapter 3 and no domain format, so a user with an EPANET

or MATPOWER file converts it first. It runs one analysis at a time. The session is a folder on disk and there is no export beyond the response JSON, which the user can save from the browser. Finally, the interface is a client of the package, so every limit of the package, single-threaded propagation, no probability-box schedule, no interval flow, is a limit of the interface as well.

9.9 Summary

An engineer who does not program can run the three toolkits on a system model by uploading the files the package reads and pressing a button, on a machine the model never leaves. The interface shows what the package computed and enforces that in four rules: a value keeps its form, a result set is what the toolkit returned, every result carries its method, and one analysis runs at a time. It is built as an Angular workspace whose module boundaries are checked by the build, against a published contract from which its client is generated, so that anything done in the interface can be repeated at the command line. One cost control is missing and has been located. The interface has been demonstrated and not yet studied in use, and its application to a real network follows in the next chapter.

Chapter 10

An End-to-End Application: One Network, Three Analyses

10.1 Introduction

Each of Chapters 5 to 7 applied its toolkit to a network of its own. None of them could show the claim of Chapter 1 that one model of a system serves all three analyses without the topology being entered again, because each demonstrated one analysis. This chapter shows it. A single water distribution network is converted once, uploaded once through the interface of Chapter 9, and analysed by all three toolkits under several scenarios, and every number reported here is the one the server returned, with the request and response recorded beside it.

The network is Net3, the third example network distributed with EPANET [85]. Section 10.2 describes how it was oriented and what the graph object found in it. Section 10.3 derives the inputs for the three interpretations and states which values are from a source and which are assumed. Sections 10.4 to 10.6 give the results of each toolkit, Section 10.7 places two of them on the same drawing, and Section 10.8 records what the run showed about the framework itself: two boundaries reached and three errors found. Section 10.9 closes.

10.2 The Network

Net3 models a water distribution system with two reservoirs, a lake and a river, three storage tanks, ninety-two junctions, 117 pipes and two pumps. It is distributed with EPANET as a test case and its input file is public, so the structure has a citable source and the conversion can be repeated. The file used is the copy bundled with WNTR 1.5.0; its hash is recorded in Appendix A.

The physical network is meshed, and the model of Chapter 3 requires an acyclic graph, so each link was oriented by the direction of its flow in a hydraulic simulation. The simulation is the input file's own seven-day run, and the orientation is taken at its final time step. Every link received a direction from the sign of its flow; three links carried zero flow at that step and kept the direction the file declares. No link had to be dropped or merged: the flow-oriented network was already acyclic. This is the operating-snapshot assumption of Chapter 3 applied to a real system, and it holds here

without adjustment.

The graph object built from the oriented edge list has 97 nodes and 119 edges. Its sources are the two reservoirs and one tank that the simulation shows discharging at the chosen step; its eighteen sinks are sixteen demand junctions and the two tanks that are filling. There are 35 forks, 23 joins and 28 layers. The decomposition of Chapter 4 finds 21 maximal diamonds and 307 unique diamonds including nested ones, with a maximum conditioning width of 12 (Figure 10.1). An earlier conversion of the same file was oriented by breadth-first search from a chosen root, as the metro network of Chapter 5 was. It has almost the same number of maximal diamonds and a width of 5. The hydraulic orientation keeps more of the network’s real loop structure, and the deeper nesting follows from it.

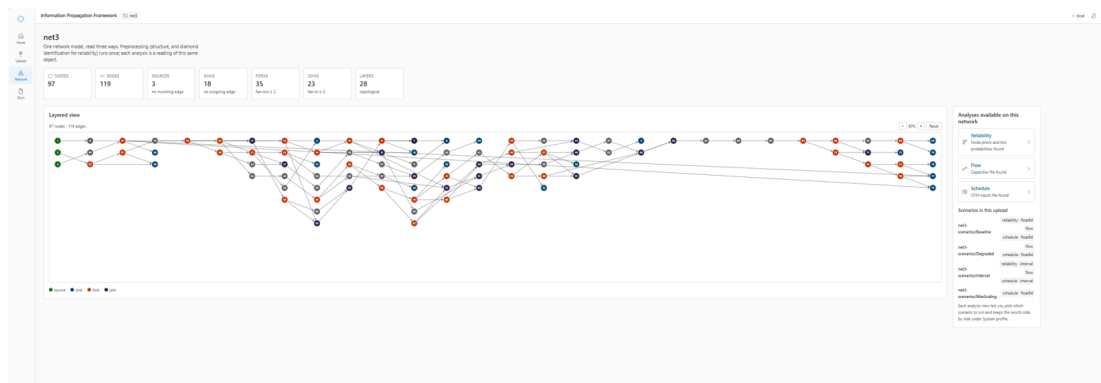


Figure 10.1: The network page after upload: 97 nodes, 119 edges, 3 sources, 18 sinks, 35 forks, 23 joins and 28 layers, with the four scenario folders and the analyses each supports.

10.3 Inputs

One structure file and four scenario folders were uploaded. Every value below is either taken from a source, computed from the input file, or assumed and named as such.

10.3.1 Reliability

Every node prior is exactly one. A junction is a demand point, and the reservoirs and tanks have no failure mode in this model, so reliability lives on the edges. The two

pumps carry 0.99, the level the pumping literature treats as appropriate for a critical asset where failure between overhauls is not acceptable [86]; this is general guidance; no figure exists for these pumps. Three links are placeholder tank connectors in the input file, with a length of 99 feet, a diameter of 99 inches and a roughness of 199, and they carry exactly one. The remaining 114 pipes carry the probability of no break in one year under an annual break-rate model. The rates are breaks per hundred mile-years by diameter class from the Utah State University survey of North American mains [87]: 13.3 for 3 to 12 inches, 3.1 for 14 to 24 inches and 0.2 for 30 to 36 inches, all materials combined since the file records no material. The rate times the pipe length gives the expected number of breaks, and the Poisson probability of none is the edge value. The resulting pipe probabilities range from 0.891, on the longest small-diameter main, to 1.000. An interval scenario widens every value that is not exactly one by five per cent either side.

A probability-box scenario was not run. The sum over the 307 diamonds of $2^{|C|}$ is 5.5×10^4 , beyond the point at which a comparably costed drone design in Chapter 5 did not complete within its budget, and the tractability boundary of the probability-box form is characterised in that chapter.

10.3.2 Capacity

Capacities are in litres per second throughout. Each pipe’s capacity is the flow at a velocity of 1.5 metres per second through its stated diameter, the conservative end of the range in Section 10.3’s convention; each pump’s is the largest flow point of its own curve in the input file, 252.4 for the lake pump and 883.3 for the river pump; the reservoir edges and the three placeholder connectors are unbounded. To measure throughput against demand, the construction of Chapter 6’s IEEE case is used: a super-sink with one edge from each of the 58 demand junctions, capacity equal to that junction’s demand at the simulation step. A degraded scenario derates the network’s dominant transmission main, an 8.6-mile 30-inch pipe that is by a wide margin the longest large-diameter link, to half its capacity, and takes the river pump, the larger of the two, out of service.

10.3.3 Schedule

The schedule interpretation is a restoration programme: the recommissioning of every pipe, pump and tank after a shutdown, in the order the network's dependencies impose. Durations follow the procedure a North American utility specifies for new and repaired mains [88–90]: one hour to isolate (assumed), a two-hour pressure test, a flush of three pipe volumes at three feet per second, a 24-hour chlorination hold and 40 hours of bacteriological sampling. Every pipe therefore carries 67 hours plus its flush time, which ranges from minutes to 12.6 hours on the trunk main. The two regulatory holds dominate every pipe's duration; this is how restoration is in practice, gated by disinfection procedure. Pumps carry eight hours and tanks 24, both assumed, since the standards cover neither. Reservoirs and junctions carry zero. An interval scenario widens every duration by twenty per cent either side. A fourth scenario carries the reliability edge probabilities as multiplicative factors with unit node values, for the MaxScaling mode of Chapter 7.

10.4 Reliability

Under the deterministic inputs, the belief that supply reaches a node ranges from 0.806 at node 76 to 1 at the sources, with a mean of 0.934 over the network and 0.928 over the eighteen sinks (Figure 10.2). The lowest values sit at the far end of the long small-diameter mains, where the product of many pipe probabilities accumulates and no second route compensates. The five nodes reported in Appendix B as a spot check, 32, 29, 81, 54 and 78, carry 0.856, 0.930, 0.977, 0.810 and 0.843. The propagation took 4.6 seconds in a warm process; identification was reused from the store after its first computation.

Under the interval inputs the picture changes (Figure 10.3). A five per cent widening on each pipe produces a mean band of 0.36 across the network and a maximum of 0.64 at node 76, whose belief lies in [0.354, 0.990]. The widening compounds along the same long chains that lowered the point beliefs, and the 307 diamonds mean that an interval on one pipe enters many nodes' ranges through several routes. The upper bounds stay

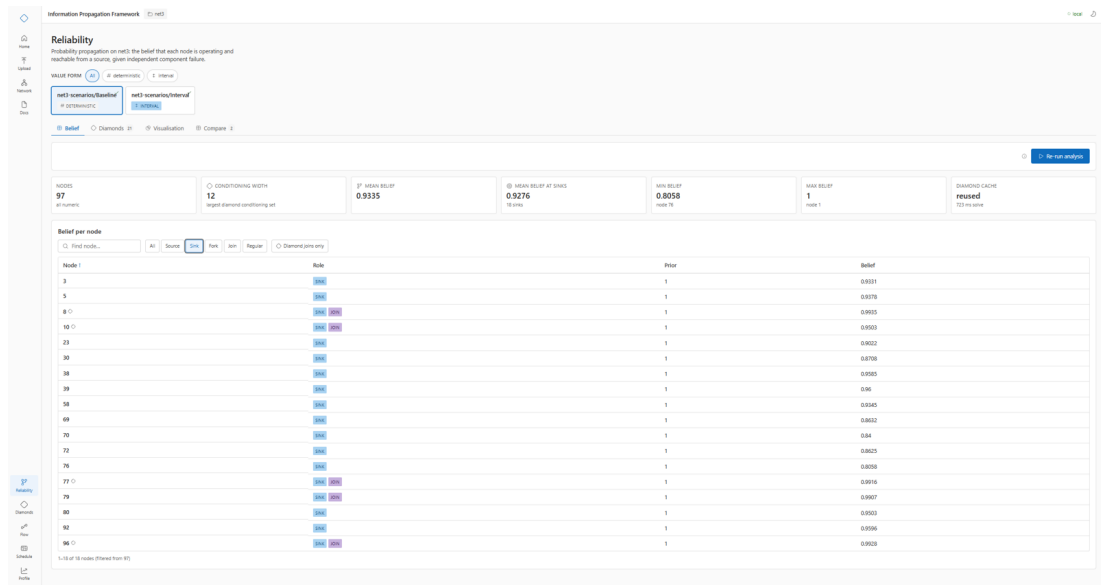


Figure 10.2: Reliability, deterministic scenario, filtered to the eighteen sinks: the belief at each, with the summary tiles above. Conditioning width 12; minimum belief 0.806 at node 76.

near one at most sinks, because the corner in which every pipe is at its upper probability is a highly reliable network; the lower bounds fall sharply, because the opposite corner is not. A point analysis at the central values reports 0.934; the exact range says that the same network, under uncertainty a utility would regard as modest, cannot be certified above 0.35 at its worst-served junction. Which of the two a planner should act on is the question Chapter 5 posed on the drone network, and this is its answer on a real one.

The Diamonds tab (Figure 10.4) lists the 21 maximal diamonds with their conditioning sets and nesting. The largest, at join 77, spans 46 nodes and 56 edges and holds eleven sub-diamonds (Figure 10.5); the width-12 conditioning set of the summary tile belongs to a diamond nested inside it, which the detail view reaches through its sub-diamond list. Every one of the 21 conditions on node 95 alone at its own level, which is the junction immediately downstream of the river pump: the whole network's reconvergence is anchored on the point where the two supplies first meet.

Chapter 10. An End-to-End Application: One Network, Three Analyses

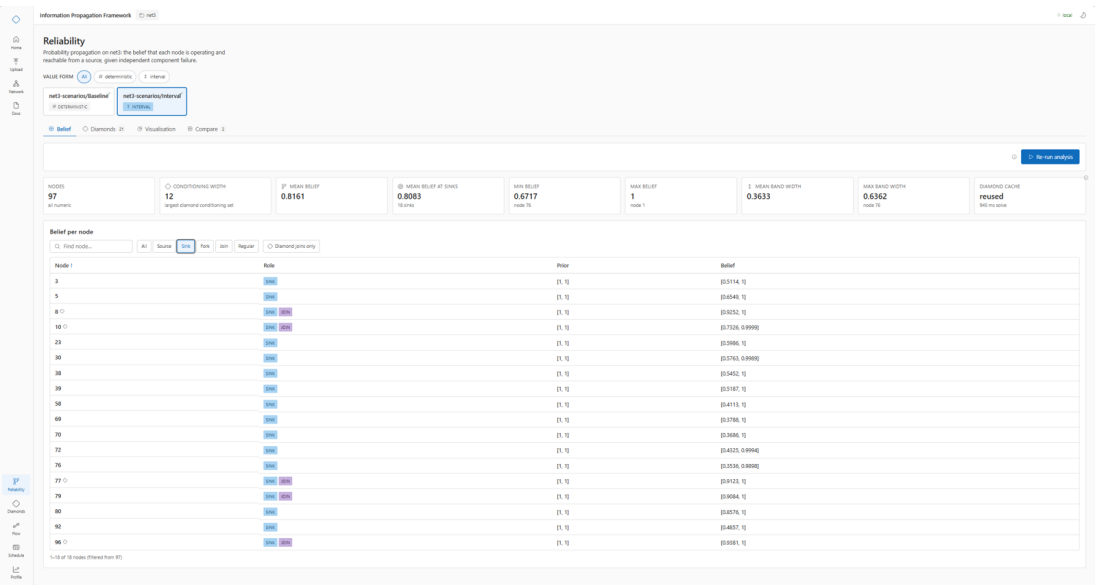


Figure 10.3: Reliability, interval scenario, the same eighteen sinks: each belief as a range. Mean band 0.363, maximum band 0.636 at node 76.

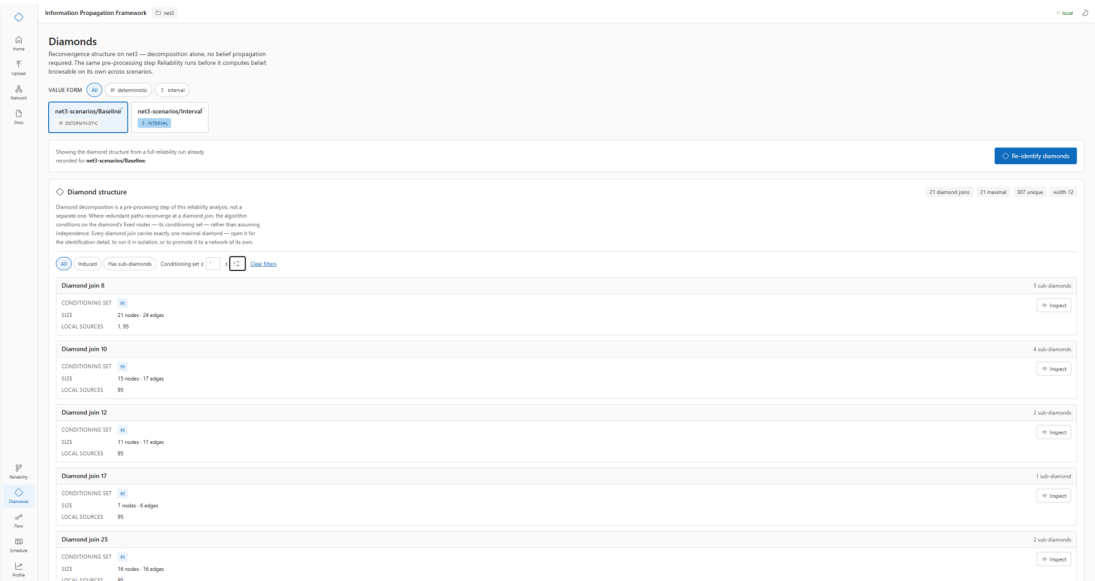


Figure 10.4: The Diamonds tab: 21 maximal diamonds, 307 unique, width 12. Each entry gives the conditioning set, size and local sources, and the number of nested sub-diamonds.

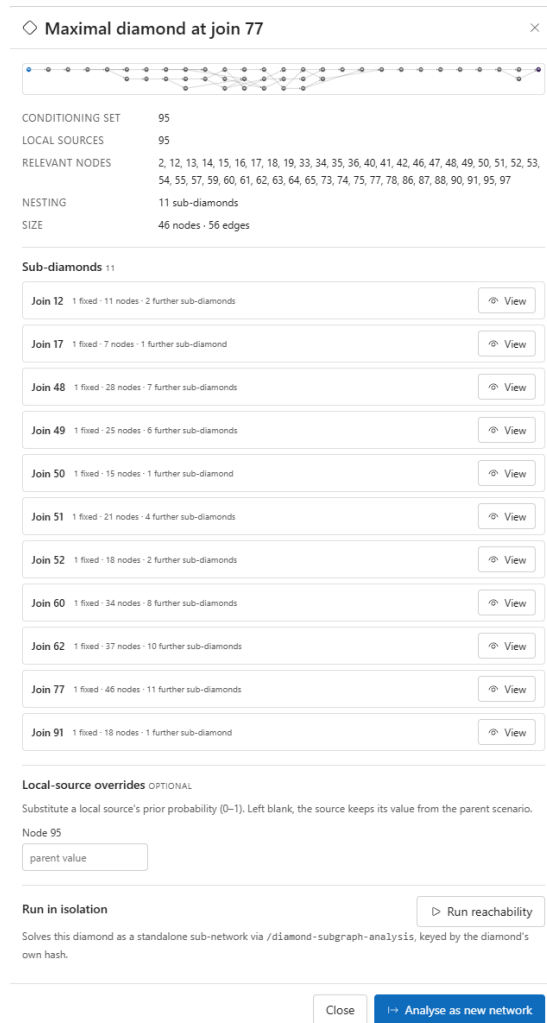


Figure 10.5: The detail view of the diamond at join 77: 46 nodes, 56 edges, conditioning set {95}, eleven sub-diamonds, and the controls to run it in isolation or promote it.

10.5 Capacity

The network delivers 1,837.9 litres per second against the demand of the 58 junctions at the simulation step (Figure 10.6). The minimum cut is unique: the free zone is empty and the enumeration is complete. Its four edges are the two pump discharges, 95 to 96 and 95 to 97 from the river pump’s junction and 7 to 8 from the lake’s, and the outlet of the source tank, 94 to 81, carrying 684.1, 883.3, 194.6 and 76.0 litres per second (Figure 10.7). Hence the binding constraint of the network at this demand is its supply: the pumps and the tank are at their limits, and every pipe downstream has headroom. Seven edges are saturated, 158 source-to-sink paths exist, and no node is a single point of failure.

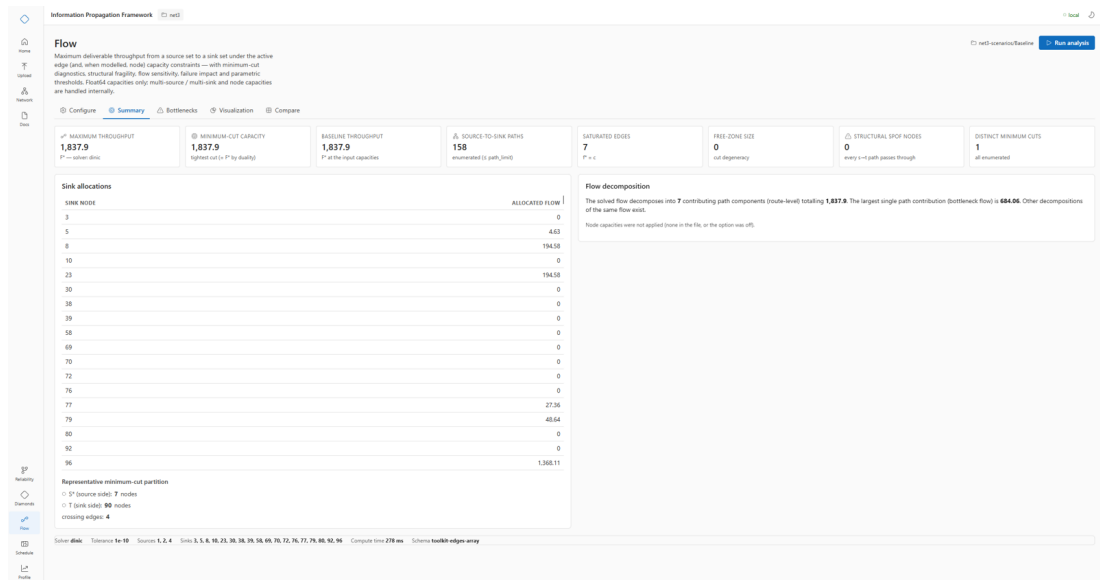


Figure 10.6: Flow summary, baseline: throughput 1,837.9 litres per second, unique minimum cut of four edges, no single point of failure, and the allocation per sink.

The sensitivity table (Figure 10.8) ranks the same four edges by the throughput lost on their removal, 883.3, 684.1, 194.6 and 76.0, and reports each one’s marginal range, the throughput the edge can influence between zero capacity and unbounded capacity. For edge 95 to 96 that range is unbounded, because the edge sits immediately downstream of the unbounded reservoir edge and its own capacity is the only limit on that route, and the table shows the infinity symbol. Under the degraded scenario the

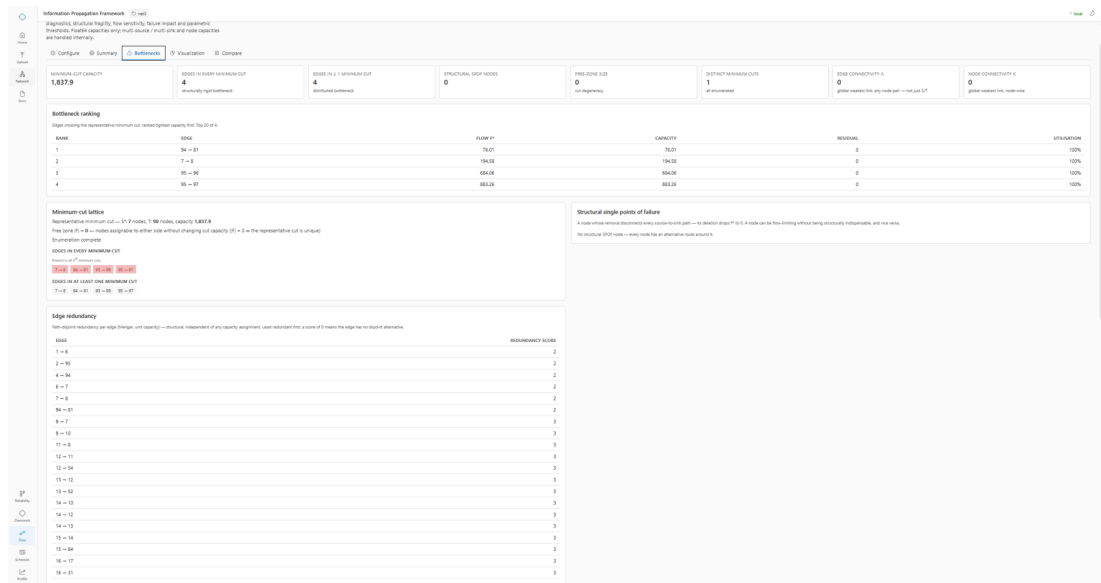


Figure 10.7: Flow bottlenecks, baseline: the four edges of the minimum cut with their flows, the lattice’s empty free zone, and the per-edge redundancy scores.

throughput falls to 954.6 litres per second, a 48 per cent loss (Figure 10.9). The cut is the same four edges, one of them now carrying the derated flow, and a fifth edge saturates. No single line failure was reported as critical in the degraded network either, for the same reason as in the baseline: supply binds before transmission.

10.6 Schedule

The restoration programme completes in 1,773.3 hours, about 74 days, along a chain of 28 activities that runs from the river reservoir through the trunk main and the eastern loop to junction 69 (Figure 10.10); 27 further activities are within ten per cent of the project value. The chain is long because the regulatory holds add 64 hours to every pipe it passes, and it runs where it does because the trunk main’s 12.6-hour flush falls on it. Under the interval durations the project value lies in [1,418.7, 2,128.0] hours, the two corners of the input box (Figure 10.11). The exact floats by the domination split were declined: the 122 non-degenerate interval durations exceed the fixed screening threshold of Chapter 7, so the toolkit returned the conservative enclosure, tagged as such on the screen, under which no activity is necessarily critical and 61 are possibly

Chapter 10. An End-to-End Application: One Network, Three Analyses

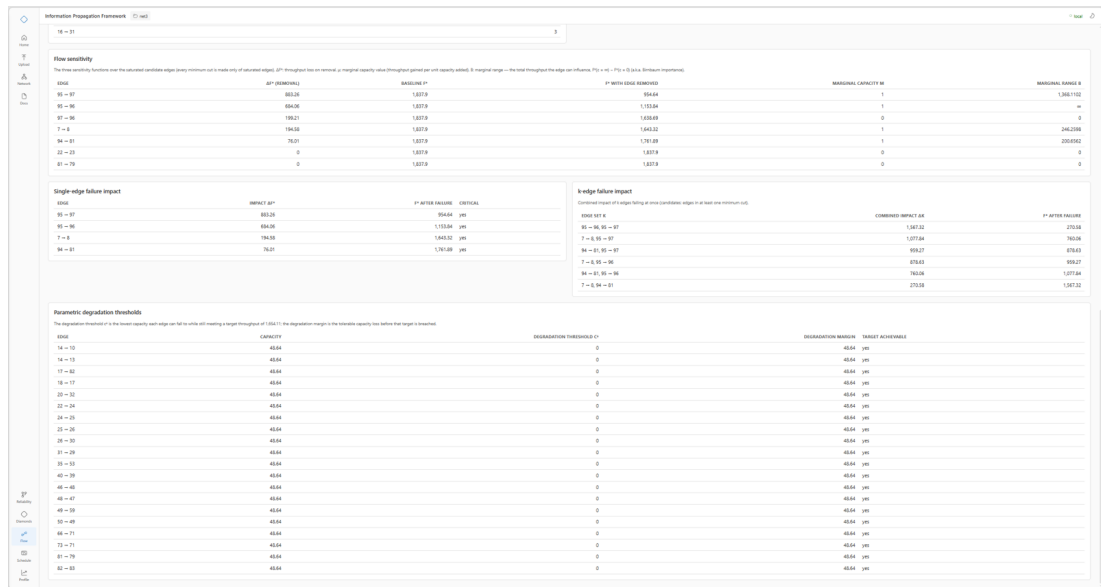


Figure 10.8: Flow sensitivity and failure impact, baseline. The marginal range of edge 95 to 96 is unbounded. The k -edge table gives the combined loss of each pair of cut edges; the degradation table gives the capacity each edge can lose before a target of 1,654 litres per second is missed.

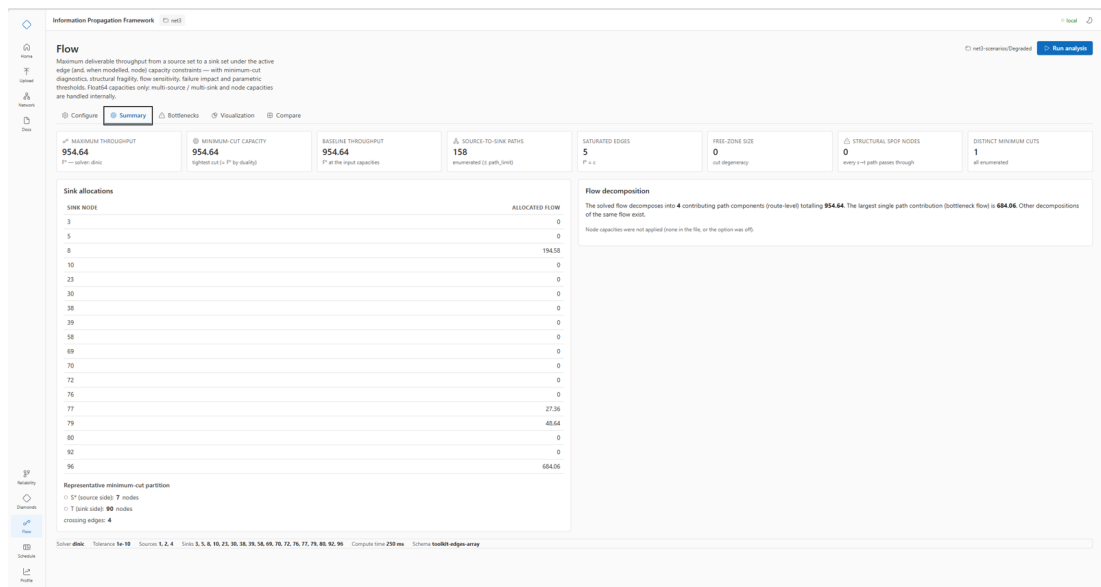


Figure 10.9: Flow summary, degraded scenario: 954.6 litres per second, a 48 per cent drop, with the same four-edge cut and five saturated edges.

critical. Section 10.8 returns to this.

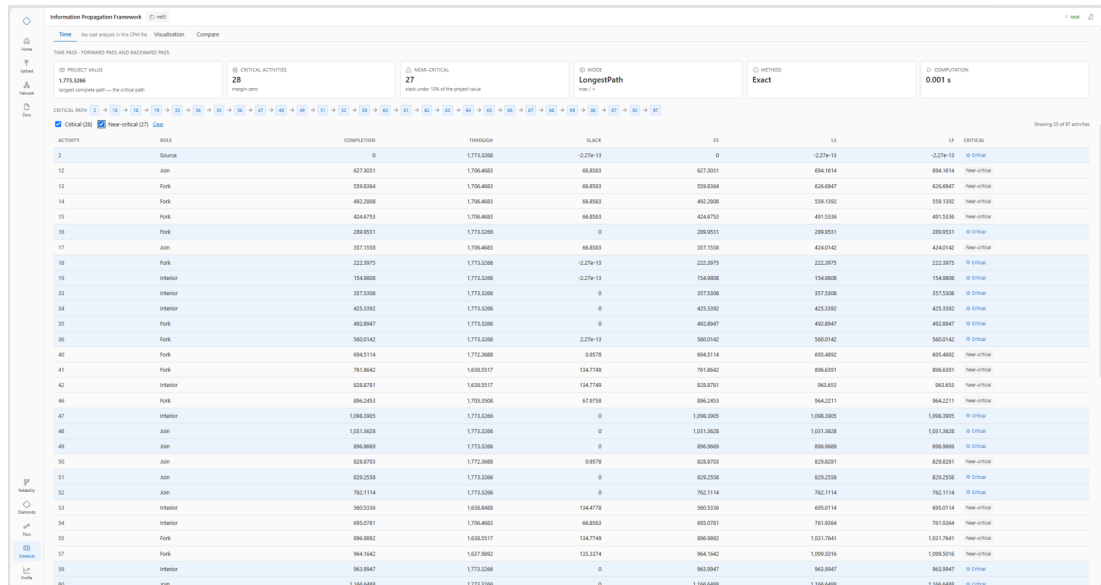


Figure 10.10: Schedule, baseline, LongestPath: project value 1,773.3 hours, 28 critical and 27 near-critical activities, with the critical chain and the per-activity schedule quantities.

The multiplicative mode answers a different question on the same topology: with the reliability probabilities as the factors, the best single route to each node is the most reliable route to it. The best route in the network compounds to 0.9928, and the least reliable best route reaches node 76 at 0.7515 (Figure 10.12). Node 76 is the worst-served node under both interpretations, which is expected, since its low belief is the product of the same long chain of pipe probabilities that makes its best route weak; the two analyses agree on where the problem is and differ in what they say about it, one giving the probability that any route works and the other the reliability of the best one. The Compare tab (Figure 10.13) places the four schedule scenarios side by side, with the interval row carrying its two criticality counts in separate columns.

10.7 Two Result Sets on One Drawing

The profile page draws any result set on the network and overlays a second. The four edges of the minimum cut and the 28 nodes of the critical path have no node

Chapter 10. An End-to-End Application: One Network, Three Analyses

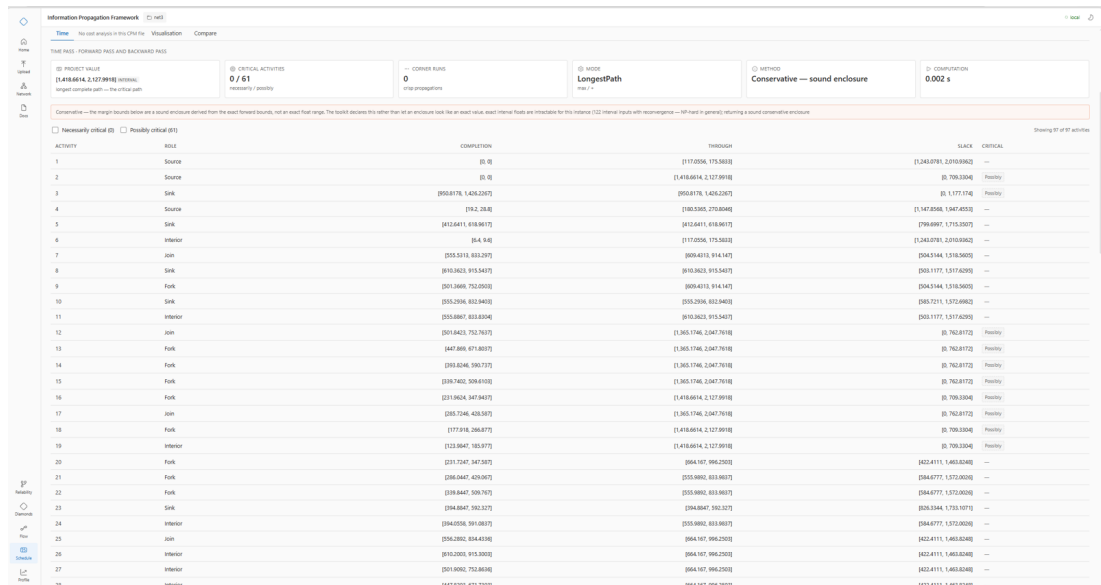


Figure 10.11: Schedule, interval scenario: project value [1,418.7, 2,128.0] hours. The method tag reads conservative enclosure and the banner states why the exact split was declined; 0 activities necessarily critical, 61 possibly critical.

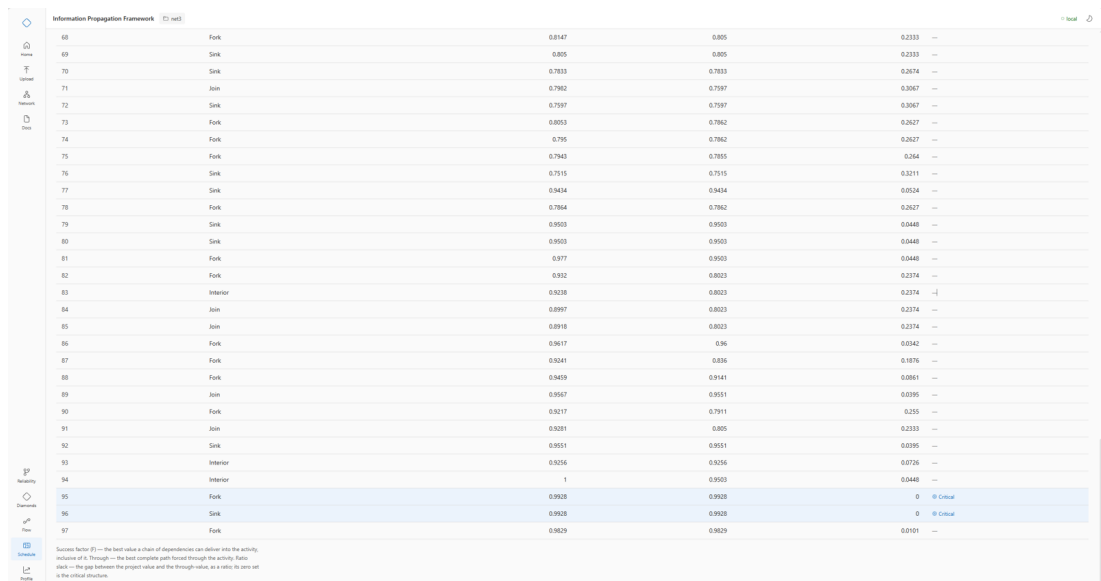


Figure 10.12: Schedule, MaxScaling mode with the reliability probabilities as factors, showing the table rows around node 76: the best route in the network compounds to 0.9928 and the weakest best route reaches node 76 at 0.7515.

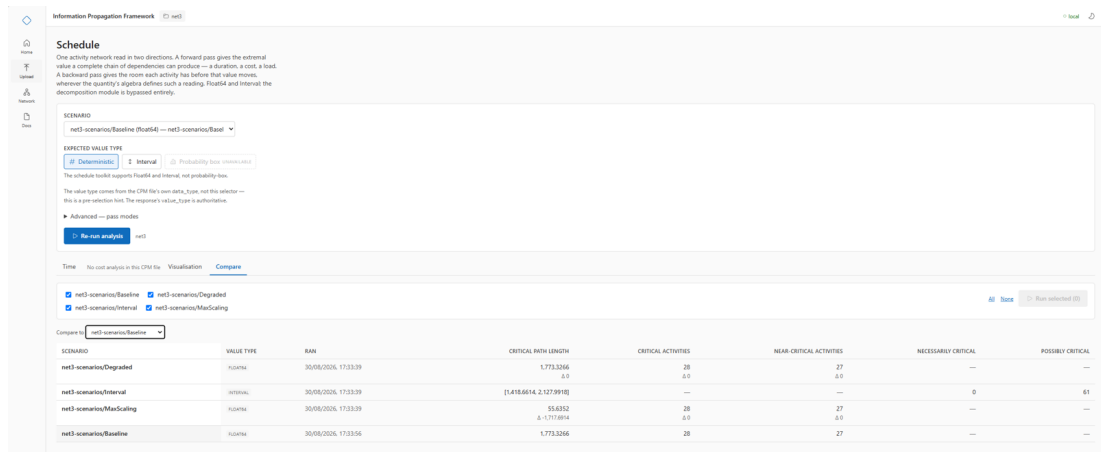


Figure 10.13: The schedule Compare tab across the four scenarios, with the baseline selected for differences. The interval row reports necessarily and possibly critical counts separately.

in common (Figure 10.14). The flow bottleneck sits at the supply end, at the pump discharges and the tank outlet, and the restoration critical chain runs through the trunk main and the distant loop. Therefore a planner who reinforced the network at its flow bottleneck would shorten no part of its restoration, and one who shortened the critical chain would add no throughput. The two analyses are answering different questions about the same components, and it takes both to see that the components are different. The conditioning set of the reliability analysis, the union across all 307 diamonds, is a third set of 26 nodes, and it too can be drawn.

10.8 What the Run Showed About the Framework

Three toolkits ran on one upload, and the network file was entered once. The reliability, capacity and schedule analyses each read the graph object the structure file produced and each added its own input file to it; no topology was re-entered and none of the three required a change to the others' files. The whole sequence, including a second scenario for each toolkit, took under a minute of computation in a warm process: 4.6, 2.3 and 2.1 seconds for the three baseline runs.

However, two boundaries of the framework were reached on this network, and both are the boundaries the earlier chapters located. The probability-box form was not

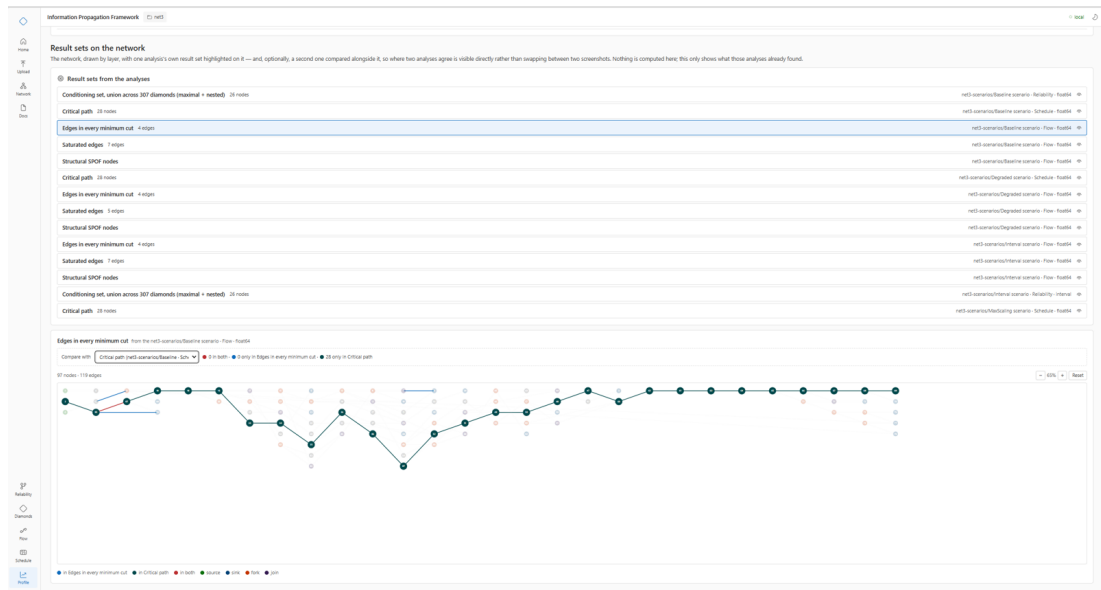


Figure 10.14: The profile page: the edges of the minimum cut (baseline flow) overlaid with the critical path (baseline schedule) on the network drawn by layer. Zero nodes in both.

run, because the network’s conditioning cost is past the point Chapter 5 identified as practical. The exact interval floats were declined, because 122 interval durations exceed the screening threshold of Chapter 7; the split was not priced, and on a network whose bypass sets might have been small that is a limitation of the screen, which Chapter 11 lists as future work. The PSPLIB instance of Chapter 7 never approached either boundary. A real network of moderate size reached both.

Three errors were found and fixed in the course of the run, and they are added to the record of Chapter 8. The unbounded reservoir capacities were the first inputs to produce an infinite value in an analysis result, and the server’s serialiser refused it; non-finite values are now converted to string tokens at the response boundary. The interface then rendered that token as not-a-number in one column, which was a second error, since an unbounded marginal range is a finding. In addition, the schedule Compare table reported the interval scenario’s two criticality counts as a single number, zero, which collapsed a value form in the way the rules of Chapter 9 forbid. Each was found because a real network exercised a path the corpus had not, which is the pattern of every error in Chapter 8’s list.

10.9 Summary

A public water distribution network was oriented by its own hydraulics, converted once, and analysed under the three interpretations of Chapter 3 from one upload. Supply reaches its worst-served junction with probability 0.81 at the stated values and with a certified range of $[0.35, 0.99]$ under five per cent input uncertainty; the network delivers 1,838 litres per second against demand, bound by its pumps and source tank, and 955 with the trunk main derated and the larger pump out; and its restoration takes 74 days along a 28-activity chain, between 59 and 89 days under twenty per cent duration uncertainty. The flow bottleneck and the restoration critical chain share no component. The run reached two boundaries the earlier chapters had located and found three implementation errors, each recorded.

Chapter 11

Conclusions and Future Work

11.1 What Was Set Out and What Was Done

Chapter 1 began with a water treatment works and the observation that a goal-oriented process system, once its direction of flow is fixed, is a directed acyclic graph. It asked five questions. Can one such model serve the three analyses an engineer runs on the system? Can the reconvergence that makes a system robust be identified once and reused? Can exact reliability be computed for interval and probability-box inputs, and at what cost? Which schedule quantities survive interval durations exactly? And can all of this reach an engineer who does not program, without the system model leaving the engineer's machine? This chapter answers each question from the work of the preceding ones, restates what the thesis contributes, collects the limitations that the chapters stated individually, and sets out the work that follows from them.

11.2 Answers to the Research Questions

11.2.1 One model, three analyses

Chapter 3 defined the network model as a finite directed acyclic graph with sources, sinks, forks and joins. From that definition the chapter built a graph object that carries the topology, a layered processing order whose existence Theorem 3.4 guarantees, and the self-inclusive ancestor and descendant closure of every node. Every analysis in the thesis reads that object as its first argument, and Chapter 8 showed that the code enforces the claim: the three toolkits have no imports between them and share only the graph object. The three interpretations of the values attached to the model, probabilistic, capacity, and schedule and cost, were each given a toolkit that answers its question exactly on the same topology, and Chapter 10 ran all three on Net3 from one upload. Hence the answer to the first question is yes, on the model's four assumptions: independent component behaviour, binary component states under the probabilistic interpretation, a static topology over the analysis horizon, and acyclicity as a property of the operating snapshot.

11.2.2 Reconvergence identified once

Chapter 4 defined the diamond, a fork whose paths reconverge at a downstream join, as the default pattern of a generic decomposition module, and proved the identification complete, deterministic and terminating. Each unique diamond is stored once, as a graph object of the same form as the network, under an identity that includes the conditioning context. A diamond posed under two different contexts is therefore kept apart, and a diamond posed twice under one context is stored once. The Probability Propagation Toolkit consumes the module, and Chapter 9 showed a diamond being isolated and promoted to a network of its own through the interface. However, the Critical Path Toolkit computes its own bypass sets and does not use the module; the two structures index reconvergence at different granularities and neither contains the other. The answer to the second question is yes for the probabilistic interpretation, and the module is available to any further analysis that a user adds.

11.2.3 Exact reliability under imprecision, and its cost

Chapter 5 computed the belief of every node, the probability that it operates and is reached from a source, by conditioning on the diamonds and recombining by total probability, and proved the recursion exact. Against a sifted decision-diagram oracle the beliefs agree to 1.1×10^{-16} on 129 graphs and on named and adversarial families. The toolkit reproduces the published exact values of a sixteen-node grid at every sink and of a 23-node power distribution network at one of three regimes. At the other two regimes a residual of order 10^{-3} remains, which is not a computation error and is left open. For interval inputs the toolkit returns the exact range, because belief is monotone in every input, at lower one-shot cost than the decision-diagram route. For probability-box inputs it returns sound bounds on the distribution of every belief, tight for high-reliability or weakly reconvergent networks and conservative for deeply reconvergent networks with broadly uncertain components, at a cost that grows faster than quadratically with the discretisation level.

The cost question has a plain answer. The toolkit is exponential in the largest conditioning set the network forces, which is the same structural width that governs

a well-ordered decision diagram. Across the corpus neither method dominates. On independent structure the two tie, and on dense correlated structure the diagram is smaller. Therefore the toolkit’s case rests on the two capabilities the diagram does not offer, native interval and probability-box propagation.

11.2.4 Schedule quantities under interval durations

Chapter 7 generalised the critical path method to a family of modes, each an operator pair with the backward semantics its algebra supports, and provided four. Under interval durations the forward quantities are monotone and their exact bounds follow from two scalar runs at the corners of the input box. The floats are not monotone, and the general problem is NP-hard. Nevertheless, the domination split proved that the exact float of a node is attained with the incomparable durations at one bound, the dominated durations at the other, and only the bypass durations enumerated, so that the cost is $2 \sum_v 2^{|H_v|}$ against 2^k . On the PSPLIB benchmark instance the split needed 50,524 propagations against 2^{30} , and it found three necessarily critical activities that the conservative enclosure does not identify and removed seven from its possibly critical set. On a dense mesh the bypass sets approach the whole network and the toolkit falls back to exhaustive enumeration, as the hardness of the problem requires. The answer to the fourth question is that every forward quantity is computed exactly at the cost of two runs, and the floats are computed exactly at a cost set by reconvergent width, with a boundary that the toolkit locates and refuses beyond.

11.2.5 Delivery without code and without leaving the machine

Chapter 8 described the package that realises every method, registered in the Julia General registry at version 0.2.1 and verified end to end from a clean install, and the loopback-only server through which the interface reaches it. Chapter 9 described the interface, in which an engineer uploads the files the package reads, byte for byte, and runs the three toolkits by pressing a button. The interface enforces that a value keeps its form, that a result set is what the toolkit returned, that every result carries its method, and that one analysis runs at a time. Client and server share one machine and

no request leaves it. Chapter 10 demonstrated the whole sequence on Net3. The answer to the fifth question is yes. The interface has been demonstrated and not yet studied in use.

11.3 Contributions

The six contributions listed in Chapter 1 have each been delivered. Chapter 3 gave the network model and its graph object. Chapter 4 gave the decomposition module with the diamond as its default pattern, proved complete, deterministic and terminating. Chapter 5 gave the Probability Propagation Toolkit, exact for point inputs, exact in range for intervals and sound for probability boxes, validated against decision diagrams, path enumeration and Monte Carlo and against two published benchmarks. Chapter 6 gave the Capacity Flow Toolkit, exact for point-valued capacities and validated against an external library and brute-force enumeration. Chapter 7 gave the Critical Path Toolkit, generalised over its operator pair and exact for interval floats by the domination split. Chapters 8 and 9 delivered all of it as a registered package and a local no-code interface.

In addition to the list, the thesis contributes a way of working that its chapters practise. Every result reported is produced by the toolkit and checked against a computation that shares none of its machinery. Timings are second-call measurements under one stated convention, networks carry their provenance, and assigned inputs are named as assigned. The six errors that the validation scripts found are recorded with their fixes, each at a boundary the synthetic corpus did not reach and a real or benchmark instance did. The registered package, the application repository and the validation repository together let a reader reproduce any chapter's numbers from the cited versions.

11.4 Limitations

The model excludes common-cause failure, degraded component states, reconfiguration during the analysis horizon, and feedback; a cyclic network is analysed only after orientation. These are the assumptions of Chapter 3. Every result is conditional on

them.

The Probability Propagation Toolkit offers no asymptotic advantage over a well-ordered decision diagram for exact point-valued reliability, and it is practical to about fifty nodes of moderate reconvergence, with dense or deeply reconvergent structure the limiting case that every exact method shares. Its probability-box form is practical only for small networks. Moreover, its tighter recombination operator is validated sound on every configuration tested but not proved, and the power-network reproduction carries an unexplained residual at two of three regimes.

The Capacity Flow Toolkit is point-valued. A best-case and worst-case pair of capacities is two solves, not an interval result. Its exactness is on the evaluated candidates where pruning applies, it computes no probability, and its whole-graph connectivity measures are zero on any DAG and are left as defined.

The Critical Path Toolkit's domination split is proved and implemented for the LongestPath mode only and requires point-valued edge values, and the toolkit has no probability-box form. Whether a proposed new mode has a backward pass is a mathematical question the implementation does not check.

The package is single-threaded in propagation, performs no conversion from domain formats, and has a test suite that covers small instances only. It does not cover the scale or the real networks the thesis reports. The interface exposes no control over the probability-box discretisation level and gives no warning before a long run, and it has not been used independently by an engineer who does not program.

11.5 Future Work

The work below follows from the limitations, and each item is stated with the chapter that motivates it.

- A soundness proof for the positive-dependence recombination operator, or a tight operator whose soundness follows from a theorem (Chapter 5).
- A depth-limited hybrid for the probability toolkit, conditioning exactly to a stated depth and bounding beyond it; an interval outer bound beyond the depth is sound

as it stands, and the probability-box form would need the Fréchet combination there (Chapter 5).

- The residual between the toolkit’s exact values and the published values of the power distribution network at two regimes, which is not a computation error and whose source has not been located (Chapter 5).
- The shortest-path dual of the domination split, the reduction of interval edge values to interval durations by edge subdivision, and the sharing of corner evaluations across overlapping bypass sets (Chapter 7).
- Replacing the fixed interval-input threshold that gates the domination split with the split’s own bypass-set cost, computed before deciding, so that a network with many interval inputs but small bypass sets reaches the exact method. The fixed threshold sent the water network of Chapter 10 to the conservative enclosure (Chapter 7).
- A probability-box form for the critical path toolkit and interval and probability-box forms for the flow toolkit, the latter direct for the flow value by monotonicity and open for the cut structure and thresholds (Chapters 6 and 7).
- A source-to-sink connectivity measure for DAGs, minimum-cost flow, and a time-expanded flow model (Chapter 6).
- Cyclic networks, by unrolling or by strongly connected components (Chapter 3).
- A second decomposition pattern, obtained by supplying the module with its own three discriminators in place of the diamond’s, with no change to the module itself (Chapter 4).
- Parallel propagation over the iteration sets with a lock-free diamond store, and a test suite that runs the validation corpus on every change (Chapter 8).
- A control and a default for the probability-box discretisation level in the interface, with an estimate before a long run, and a study of the interface in use by engineers who do not program (Chapter 9).

11.6 Closing Remark

The framework began from a practical observation about infrastructure data: the topology of a system is entered three times for three analyses and kept consistent by hand, and the uncertainty in every input is collapsed to a number before any analysis runs. It ends with one model, three exact analyses that carry that uncertainty through, and an interface that puts them in the hands of the engineer who holds the data. The case study of Chapter 10 showed, on a public water network analysed from one upload, that the components which bound its throughput and the components which set its restoration time were different ones, which no single analysis could have said. Each chapter records the limitations of its own method, and this chapter collects them. The boundaries they mark, of conditioning width, of bound tightness and of network scale, are among the results of this thesis, because they state the networks on which each of the three analyses is exact.

Appendices

Appendix A

Case-Study Network Specifications

Appendix A. Case-Study Network Specifications

This appendix describes every network presented in a chapter as a case study, a named reproduction target, or a distinct benchmark family. Each chapter cites one entry here rather than re-describing a network in prose. For each network it gives the source and citation, the node and edge counts, the rule by which an undirected or domain-specific network was oriented into a directed acyclic graph, and, for the networks used as named case studies, which input values are taken from a source and which are assigned for demonstration. The flow and critical-path validation summaries of Appendix B additionally exercise a set of synthetic networks generated purely for structural diversity and scale, among them `grid-graph-5x5`, `continental_medical_network`, `ergo-proxy-dag-network`, `glasgow_to_shetland_extreme` and `highland_to_lowland_full_network`; these carry no independent provenance, are not case studies, and are not catalogued individually here. Full edge lists, the per-node input tables, the converters, and the recorded server responses are archived in the thesis data repository (Appendix B); the file named at the end of each entry is its location there.

A.1 Networks at a glance

Network	Nodes / edges	Source	Used in
<code>power-network</code>	23 / 27	Tong and Tien [32]	Ch. 5
<code>grid-graph</code>	16 / 24	Tong and Tien [32]	Ch. 5
<code>KarlNetwork</code>	26 / 74	Synthetic (colleague-generated)	Ch. 4, 5
<code>counterexample-n15</code>	15 / 23	This project's adversarial instance	Ch. 4
<code>metro_directed_dag</code>	306 / 350	Berlin U/S-Bahn [91]	Ch. 5
<code>PSPLIB j301_1</code>	32 / 48	Kolisch and Sprecher [81]	Ch. 7
<code>IEEE RTS-24</code>	24 / 34	MATPOWER [79], IEEE RTS [77]	Ch. 6
Drone: FW-reliant centralised	217 / 263	Jones <i>et al.</i> [11]	Ch. 5

Appendix A. Case-Study Network Specifications

Network	Nodes / edges	Source	Used in
Drone: VTOL dense decentralised	242 / 1753	Jones <i>et al.</i> [11]	Ch. 5
Drone: concentrated minimal	230 / 1648	Jones <i>et al.</i> [11]	Ch. 5
munin, water, 17 bnlearn benchmarks	8/8 to 1041/1397	bnlearn repository [92]	Ch. 5
Net3	97 / 119	EPANET Example Network 3 [85]	Ch. 10
Random and mutated corpus	10–28 nodes, 129 graphs	This project's harness	Ch. 5
Adversarial families	fan-in- k , mesh- w	This project's harness	Ch. 5

Table A.1: Every named case-study network, reproduction target and benchmark family in the thesis. Counts are of the directed acyclic graph after orientation. The drone designs use non-contiguous node identifiers; the count is of nodes actually present.

A.2 power-network (23 nodes, 27 edges)

Four-substation power distribution network from Tong and Tien [32], Figure 11. Already directed in the source. Sources are nodes $\{1, 7, 18\}$ (the three solid black circles in the figure); the single sink is node 23. Component reliabilities for the reproduction study (Ch. 5) are the paper's own values, applied under its uniform-link-reliability scheme ($R_l \in \{0.9, 0.99, 0.3\}$, node priors 1.0). The interval, probability-box, capacity and schedule scenarios use values assigned for demonstration in this thesis; the five published-reliability scenario folders (Published_R030, Published_R090, Published_R099) hold the inputs for the reproduction. File: `corpus/named-networks/power-network/`.

A.3 grid-graph (16 nodes, 24 edges)

The 4×4 grid benchmark from Tong and Tien [32], Figure 10. Directed in the source. Sources are nodes $\{1, 3, 13\}$. Used for the exactness reproduction against the paper's Tables 2 and 3 ($R_l = 0.9$ and $R_l = 0.1$); the reproduction reaches the published values

Appendix A. Case-Study Network Specifications

to within the paper’s own five-decimal print precision. File: `corpus/named-networks/grid-graph/`.

A.4 KarlNetwork (26 nodes, 74 edges)

A synthetic network built by a colleague as an 18-diamond stress test for the decomposition module: dense reconvergence in a small graph. Not a model of any physical system; all reliability, capacity and schedule values are assigned for demonstration. Used for the diamond-count and conditioning-width figures in Ch. 4–5 and for the probability-box discretisation timing study. File: `corpus/named-networks/KarlNetwork/`.

A.5 counterexample-n15 (15 nodes, 23 edges)

A hand-constructed adversarial instance from this project’s own validation corpus, built to exercise a specific failure mode of an earlier diamond-identification routine (a non-induced diamond whose conditioning set is undercounted if nested diamonds are not unioned in). Synthetic by design. File: `corpus/named-networks/counterexample-n15/`.

A.6 PSPLIB j301_1 (32 activities, 48 precedences)

Instance `j301_1` of the single-mode resource-constrained project-scheduling set `j30` from PSPLIB [81]. 30 real activities plus a dummy start and end (durations 0). The activity-on-node precedence graph is already a directed acyclic graph. Durations and the successor list are in `validation/critical-path/thesis_writing_pack/appendix_a_durations_successors.csv` (30 rows, dummy nodes omitted); the file’s recorded MPM-Time is 38, which the LongestPath pass reproduces. The interval scenario applies a $\pm 10\%$ relative half-width to every duration. Conversion: `converters/psplib_to_ipf.py`, which consumes `j30/j301_1.sm` from the PSPLIB distribution. Files: `case-studies/psplib-j301_1/`.

A.6.1 Durations and successors

Appendix A. Case-Study Network Specifications

Activity	Duration	Immediate successors
2	8	6, 11, 15
3	4	7, 8, 13
4	6	5, 9, 10
5	3	20
6	8	30
7	5	27
8	9	12, 19, 27
9	2	14
10	7	16, 25
11	9	20, 26
12	2	14
13	6	17, 18
14	3	17
15	9	25
16	10	21, 22
17	6	22
18	5	20, 22
19	3	24, 29
20	7	23, 25
21	2	28
22	7	23
23	2	24
24	3	30
25	3	30
26	7	31
27	8	28
28	3	31
29	7	32
30	2	32
31	2	32

Table A.2: PSPLIB j301_1 activity durations and precedence successors. Dummy start (node 1) and end (node 32) carry duration 0 and are omitted. Source: `validation/critical-path/thesis_writing_pack/appendix_a_durations_successors.csv`, parsed directly from `j301_1.sm`.

A.7 IEEE RTS-24 (24 buses, 34 edges as oriented)

The 24-bus IEEE Reliability Test System, taken from the MATPOWER case file `case24_ieee_rts.m` [79], which traces to the 1979 original [77]. 24 buses, 38 transmission lines (four double-circuit pairs), 33 generating units at 11 generator buses, 17 load buses, slack bus 13, total peak demand 2850 MW.

Orientation. The mesh is oriented into a directed acyclic graph by the sign of the DC power flow at peak load under uniform-participation-factor dispatch. Each double-circuit line pair is merged into one edge carrying the summed rating. The result is 24 nodes and 34 edges with no directed cycles; nothing was dropped for acyclicity. Line ratings in MVA are the edge capacities. The orientation log records every line’s flow sign and resulting direction.

Net-injection formulation. The primary flow answer uses a net-injection model: each bus’s net injection is its generator nameplate capacity minus its local peak load; a super-source (node 0) feeds every net-export bus and every net-import bus feeds a super-sink (node 25), each edge capacity equal to the bus’s net injection or net load. A bus with both local generation and local load appears once, under its net sign. This adds two nodes and gives 26 nodes, 54 edges.

Files, in `case-studies/rts24/`: both `.EDGES` variants, the three capacity files, `orient_rts24.py`, the orientation and net-injection logs, and the recorded server request and response.

A.8 Medical drone delivery designs (217/263, 242/1753, 230/1648)

Three candidate network designs for a Scotland-wide medical drone logistics system, with structure derived from the conceptual study of Jones *et al.* [11]: a fixed-wing-reliant centralised design, a VTOL-dense decentralised design, and a concentrated-minimal design. Node identifiers are non-contiguous (facilities, hubs and waypoints share a

Appendix A. Case-Study Network Specifications

numbering space); the counts above are of nodes actually present. Node reliabilities are demonstration values assigned in this thesis; hubs are treated as always available. The designs are used for the conditioning-width and interval-reliability comparison in Ch. 5 (Islay Hospital is the worst-served facility in both dense designs, bounded at $[0.56, 0.72]$). Generators: `converters/drone_network_to_dag*.jl`, from the source facility data in `case-studies/drone-networks/`.

A.9 metro_directed_dag (306 nodes, 350 edges)

The Berlin U-Bahn and S-Bahn network from Behrendorf *et al.* [91], oriented into a directed acyclic graph by breadth-first search from node 18. Structure only; node and edge reliabilities are assigned. Used as a larger structural test for the decomposition and propagation pipeline in Ch. 5. File: `corpus/named-networks/metro_directed_dag_for_ipm/`.

A.10 bnlearn structural benchmarks

Seventeen Bayesian-network structure benchmarks from the bnlearn repository [92]: `asia`, `alarm`, `andes`, `barley`, `cancer`, `child`, `diabetes`, `earthquake`, `hailfinder`, `hepar2`, `insurance`, `link`, `mildew`, `pathfinder`, `pigs`, `sachs`, `survey` and `win95pts`, plus `munin` and `water`, converted from their BIF topology by a structure-only parser. These are directed acyclic by construction. Every reliability value is synthetic and is *not* a decision-relevant claim about the domain each network models; the benchmarks are used only to test the decomposition and propagation on a range of sizes (8 to over 1000 nodes) and reconvergence structures. Files: `corpus/bnlearn/`, with `corpus/bnlearn/CORPUS_PROVENANCE.md` recording the per-network provenance.

A.11 Net3 (97 nodes, 119 edges)

EPANET Example Network 3 [85], the integrated case study of Ch. 10. The `Net3.inp` file is the copy bundled with WNTR 1.5.0 (sha256 recorded in `case-studies/net3/`

Appendix A. Case-Study Network Specifications

`RESULTS.md`), not a hand-edited one.

Orientation. Each link is oriented by the sign of its flow at the final timestep of the `.inp` file’s own seven-day simulation (604,800 s). No link was dropped for acyclicity, because the flow-direction orientation is naturally a directed acyclic graph. Three links carried exactly zero flow at that timestep and keep the model’s own declared direction, which is the orientation rule’s stated tie-break. Full log: `case-studies/net3/net3_orientation_log.txt`.

Structure. 97 nodes (2 reservoirs, 3 tanks, 92 junctions) and 119 edges (117 pipes and 2 pumps). The EPANET-id to integer-id to node-type mapping is in `case-studies/net3/net3-node-mapping.txt`.

Inputs. The reliability input classification (which edges are pumps, dummy connectors or real pipes, each one’s `.inp` length and diameter, and the derived Baseline probability) is in `case-studies/net3/net3-scenarios/reliability_input_classification.csv`. Schedule durations follow the restoration-procedure gloss recorded in `RESULTS.md` §3 (pipe, pump and tank node durations, with each value marked as sourced or assumed). Capacities are the pipe and pump ratings; reservoir and tank-connector capacities are unbounded and exercise the non-finite-value serialisation path.

A separate, older breadth-first-oriented conversion of the same `.inp` file (`net3-water`) exists in the corpus and is *not* used in the case study; it is kept in `case-studies/net3/superseded-bfs-conversion/` with a pointer to `RESULTS.md` §1, which explains the difference.

A.12 The random and adversarial corpus

The bulk of the probability toolkit’s exactness validation (Ch. 5) runs on a corpus of 129 synthetic graphs, not stored as files but reconstructed exactly from their names and recorded seeds:

Appendix A. Case-Study Network Specifications

- **120 random_nX\$_p\$Y\$_s\$Z\$ graphs:** 10 to 28 nodes, generated by a seeded `MersenneTwister` through `graph_gen.jl`'s `scaling_random`. Node and edge probabilities uniform at 0.9 (never exactly 0 or 1 on any node).
- **8 mutant_rand28_sN\$ graphs:** mutations of a 28-node, $p = 0.12$ base graph (`MersenneTwister(7), scaling_mutants($\protect\global\let\T1\tellipsis.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font\T1\tellipsis$;seeds=1:8,adds=5,dels=2))`).
- **counterexample-n15:** the one real persisted file (§A.5).

Two adversarial families are used for the complexity study: **fan-in- k** , in which factorisation keeps the cost linear, and **mesh- w** , in which the cost grows with the treewidth. Both are generated from recorded seeds in `corpus/adversarial/adversarial_scaling.jl`. Generators, seeds, the reconstruction script, and the per-graph results are in `corpus/random-and-mutated/` and `corpus/adversarial/`; `corpus/random-and-mutated/README.md` gives the exact reconstruction parameters.

Appendix B

Full Numerical Results

Appendix B. Full Numerical Results

This appendix holds the complete validation tables that the toolkit chapters summarise but do not reproduce in full, and a map from every figure and table in the thesis to the script and data file that produced it. Every network named here is specified in Appendix A. The raw logs, per-graph results, request and response JSON, and the scripts themselves are archived in the thesis data repository (§B.6 gives the DOI and the path of each artefact).

B.1 Timing convention

Every runtime reported in this thesis is wall-clock, measured on a single core, on the *second* call within a warm process. A discarded warm-up run precedes every measurement, so no figure includes just-in-time compilation. Where a comparison is made against another method, the ratio of run counts is reported rather than raw seconds wherever the comparison would otherwise depend on the machine. This convention is stated once here and referenced, not restated, by each chapter.

B.2 Probability propagation

B.2.1 Exactness on the 129-graph corpus

The probability toolkit was checked against an independently built reduced ordered binary decision diagram oracle on the full corpus of 129 synthetic graphs (Appendix A.12). **All 129 agree exactly**, worst absolute deviation 1.1×10^{-16} for point beliefs and 2.78×10^{-16} for interval bounds, both at machine precision, and none changed against the pre-existing recorded results (tolerance 10^{-9}). The corpus is reconstructed exactly from recorded seeds; its node priors lie in a range that never places a non-source node at exactly 0 or 1, so it does not exercise the degenerate-prior path. Script: `corpus/random-and-mutated/rerun_129_corpus.jl`; per-graph results: `rerun_129_corpus_results.csv`.

B.2.2 Named-network reproductions

Appendix B. Full Numerical Results

Network	Published source	Result	Worst diff
grid-graph , $R_l = 0.9$	Tong and Tien [32], Table 2	reproduced	4.75×10^{-6} (node 12)
grid-graph , $R_l = 0.1$	Tong and Tien [32], Table 3	reproduced	4.29×10^{-6} (node 16)
power-network , $R_l = 0.99$	Tong and Tien [32], Table 5	reproduced	0.00000
power-network , $R_l = 0.9$	Tong and Tien [32], Table 5	small residual	1.76×10^{-3}
power-network , $R_l = 0.3$	Tong and Tien [32], Table 5	small residual	5.1×10^{-4}

Table B.1: Reproduction of published exact reliabilities. The grid residuals are within the paper’s own five-decimal print precision. The two power-network residuals at $R_l = 0.9$ and 0.3 are order 10^{-3} , larger than print rounding; the toolkit and the decision diagram agree with each other to machine precision at all three points, so the gap is a modelling difference from the paper, flagged and not resolved. Scripts: `validation/probability/asce_grid_reproduction.jl`, `asce_power_reproduction.jl`.

B.2.3 Adversarial families

On the fan-in- k family the deduplicated operation count stays linear in k ; on the mesh- w family it grows with the treewidth, matching the decision diagram’s own node count.

Full sweep: `validation/probability/data/adversarial_factored.csv`; script: `corpus/adversarial/adversarial_scaling.jl`.

B.2.4 Interval and probability-box bounds

Per-node interval and probability-box belief tables for the drone designs, the certified requirement-violation bound against the Monte Carlo sample count it replaces, and the probability-box propagation time by discretisation level are in `validation/probability/data/(certified_bound_vignette.csv, timing_imprecise.csv, drone_ksweep_remeasure_results.csv)`.

B.3 Capacity flow

B.3.1 Twelve-network validation summary

Every network was solved by all three built-in max-flow solvers (Dinic, Edmonds–Karp, push–relabel) and checked against the external `GraphsFlows` library; the extended suite additionally checks min-cut structure, saturated edges, single-point-of-failure nodes, single-edge failure impact, the degradation threshold, and Menger edge connectivity against brute-force or bisection oracles.

Network	Max flow F^*	All checks
<code>grid-graph-5x5</code>	35	PASS
<code>metro_directed_dag_for_ipm</code>	29	PASS
<code>KarlNetwork</code>	39	PASS
<code>water</code>	154	PASS
<code>PSPLIB j301_1</code>	14	PASS
<code>continental_medical_network</code>	501	PASS
<code>ergo-proxy-dag-network</code>	479	PASS
<code>glasgow_to_shetland_extreme</code>	762	PASS
<code>highland_to_lowland_full_network</code>	3326	PASS
Drone: FW-reliant centralised	755	PASS
Drone: concentrated minimal	5939	PASS
Drone: VTOL dense decentralised	6044	PASS

Table B.2: Flow toolkit validation. F^* is the maximum flow; every solver and every extended check agrees. Full per-check results: `validation/flow/flow_validation_summary.csv` and `flow_validation_extended_summary.csv`; script: `validation/flow/run_flow_validation.jl`.

B.3.2 DIMACS-scale timing

Instance	Nodes	Edges	Dinic (warm s)	Oracle (warm s)
<code>genrmf_dag_small</code>	360	924	0.002	1.181
<code>genrmf_dag_medium</code>	3,332	9,324	0.017	0.085
<code>genrmf_dag_large</code>	27,040	78,364	0.961	9.891
<code>genrmf_dag_xlarge</code>	36,992	107,644	0.657	28.906

Appendix B. Full Numerical Results

Instance	Nodes	Edges	Dinic (warm s)	Oracle (warm s)
----------	-------	-------	----------------	-----------------

Table B.3: Generated GENRMF directed-acyclic max-flow instances [80]. Warm second-call times, Dinic solver. Full table (all three solvers): `validation/flow/dimacs/timings.csv`; generator: `validation/flow/dimacs/generate_genrmf_dag.py` (seeded).

B.3.3 Configuration comparison, sensitivity and min-cut enumeration

The full single-edge sensitivity rankings for Configurations A and B of the flagship benchmark, the node-upgrade sweep, the parametric threshold comparison, and the complete min-cut enumeration are in `case-studies/flow-configs-ab/`; the driver is `check_flagship_benchmark.jl`. The IEEE RTS-24 line-model and net-injection results are in `case-studies/rts24/` with `run_rts24_analysis.jl` and `run_rts24_netinjection_analysis.jl`.

B.4 Critical path

B.4.1 Ten-network float validation

Ten networks were checked in all four modes (LongestPath, ShortestPath, MaxScaling, Accumulation) against a path oracle where tractable and against the mode’s structural invariants otherwise.

Network	Check	Result (all 4 modes)
<code>water</code>	oracle	PASS
<code>KarlNetwork</code>	oracle	PASS
<code>grid-graph-5x5</code>	oracle	PASS
<code>metro_directed_dag_for_ipm</code>	oracle	PASS
<code>ergo-proxy-dag-network</code>	invariants	PASS
<i>... and five more. Full table: <code>validation/critical-path/float_validation_summary.csv</code>.</i>		

Appendix B. Full Numerical Results

Network	Check	Result (all 4 modes)
---------	-------	----------------------

Table B.4: Critical-path float validation. Script: `validation/critical-path/run_float_validation.jl` with `oracle.jl`. The interval-split engine is checked separately by `run_interval_validation.jl` and, on PSPLIB j301_1, by a 50,000-sample Monte Carlo soundness and attainment check (`run_mc_check_psplib_j301_1.jl`, 0 bound violations).

B.4.2 PSPLIB j301_1 per-node float bounds

The per-node necessarily-critical and possibly-critical float bounds under $\pm 10\%$ duration uncertainty are in `validation/critical-path/thesis_writing_pack/psplib_j301_1_float_bounds.csv`, with the earlier `water` $k = 32$ precedent in `water_k32_float_bounds.csv`.

B.5 Net3 integrated case study

Toolkit and scenario	Headline result	Timing (warm, s)
Reliability, Baseline	source-to-sink reachability, all 18 sinks	4.61
Reliability, Interval ($\pm 5\%$)	certified requirement range [0.35, 0.99]	N/A
Flow, Baseline	throughput 1,838 litres/s	2.33
Flow, Degraded	throughput 955 litres/s (48% drop)	N/A
Schedule, Baseline Longest-Path	project value 1,773.3 hours (≈ 74 days), 28 of 97 critical	2.08
Schedule, Interval Longest-Path	conservative enclosure [1,418.66, 2,127.99] hours	N/A
Schedule, MaxScaling	least-reliable supply route 0.7515 (node 76)	N/A

Table B.5: Net3 headline numbers. The interval schedule is the *conservative enclosure*: the exact domination split is screened out before any cost is computed because Net3’s 122 non-degenerate interval durations exceed the fixed threshold at which the split is attempted (a stated limitation, Ch. 7 and 11). Every request and response JSON, the run summary, and the full write-up with provenance are in `case-studies/net3/`; driver: `net3-scenarios/run_net3_scenarios.py`.

The flow toolkit’s non-finite-value serialisation path was first exercised by this case

Appendix B. Full Numerical Results

study’s unbounded reservoir and tank-connector capacities; the fix is recorded in `case-studies/net3/thesis_writing_pack/INDEX.md`.

B.6 Figure and table to script and data map

Results in this thesis were produced with `InformationPropagationAnalysis.jl` v0.2.1 (Julia General registry, git-tree-sha1 `adcdacd7cc306d8ffe15e440ff0c7b72ce6f024a`) and the `information-propagation-no-code` application v1.0 (DOI 10.5281/zenodo.22180253) [83]. The complete data set (every script, seed, input file and recorded server response) is archived at DOI 10.5281/zenodo.22180227 [84]; the paths below are relative to that archive. This section is the full version of the mapping; the repository `README.md` carries the same table with per-row package and application versions.

Figure / table	Script	Data
Fig. (width vs diagram size), Ch. 5	<code>complexity_validate.jl</code> , <code>rerun_129_corpus.jl</code>	<code>figures/ch05_probability/</code> <code>*.csv</code>
Fig. (p-box tightness envelope)	<code>grid_envelope.jl</code> (+ Fréchet)	<code>grid_envelope{,_frechet}.csv</code>
Fig. (fan-in / mesh crossover)	<code>adversarial_scaling.jl</code>	<code>adversarial_factored.csv</code>
Fig. (hero CDF)	<code>hero_figure.jl</code>	<code>hero_figure.csv</code>
Fig. (drone reliability map)	<code>drone_beliefs_check.jl</code>	Tab. B.1 data
Tab. (worked-network trace)	<code>prob_chapter_example_trace.jl</code>	N/A
Tab. (4 nets vs BDD)	<code>bdd_oracle.jl</code> , <code>oracles.jl</code>	<code>validation/probability/</code>
Tab. B.1 (published reproductions)	<code>asce_grid_reproduction.jl</code> , <code>asce_power_reproduction.jl</code>	N/A
Tab. (adversarial complexity)	<code>adversarial_scaling.jl</code> , <code>graph_families.jl</code>	<code>data/adversarial*.csv</code>
Tab. (certified bound vs MC)	<code>certified_bound_vignette.jl</code>	<code>data/certified_bound_vignette.csv</code>
Tab. (p-box time vs discretisation)	p-box timing scripts	<code>data/timing_imprecise.csv</code>
Tab. (3 drone designs)	<code>drone_ksweep_remeasure.jl</code>	<code>drone_ksweep_remeasure_results.csv</code>

Appendix B. Full Numerical Results

Figure / table	Script	Data
Tab. B.2 (12-network flow)	<code>run_flow_validation.jl</code>	<code>flow_validation_summary.csv</code> (+ extended)
Tab. B.3 (DIMACS timing)	<code>run_dimacs_validation.jl</code> , <code>generate_genrmf_dag.py</code>	<code>dimacs/timings.csv</code>
Figs / tabs (flagship A/B: overlay, bars, sweep, mincut)	<code>check_flagship_benchmark.jl</code> , <code>generate_case_study_artifacts.py</code>	<code>case-studies/flow-configs-ab/</code>
Fig. / tab (IEEE RTS-24)	<code>run_rts24_netinjection_analysis.jl</code>	<code>case-studies/rts24/</code>
Fig. (exact float bounds), Ch. 7	<code>figures/ch07_cpm/make_fig04_floats.py</code>	<code>psplib_j301_1_float_bounds.csv</code>
Tabs (j301_1 LongestPath, Accumulation)	server runs per <code>psplib_j301_1_RESULTS.md</code>	<code>case-studies/psplib-j301_1/_analysis-results/</code>
Tab. B.4 (10-network CPM)	<code>run_float_validation.jl</code> , <code>oracle.jl</code>	<code>float_validation_summary.csv</code>
Tab. (j301_1 per-node bounds)	<code>run_mc_check_psplib_j301_1.jl</code>	<code>psplib_j301_1_float_bounds.csv</code>
Tab. B.5 (Net3 headline)	<code>run_net3_scenarios.py</code>	<code>case-studies/net3/net3-scenarios/</code>
Ch. 9 interface figures	N/A (screenshots of the deployed application)	N/A

Table B.6: Every data-driven figure and validation table in the thesis, its generating script, and its source data, all relative to the archived data repository. Diagrams and diamond renders are regenerated from the network data, not archived; interface figures are screenshots and are not reproducible artefacts. Full logs and scripts are archived at the DOI above.

Bibliography

- [1] R. Billinton and R. N. Allan, ‘Network modelling and evaluation of simple systems,’ in *Reliability Evaluation of Engineering Systems: Concepts and Techniques*, Boston, MA: Springer US, 1992.
- [2] L. R. Ford and D. R. Fulkerson, ‘Maximal flow through a network,’ *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: 10.4153/CJM-1956-045-5
- [3] J. E. Kelley and M. R. Walker, ‘Critical-path planning and scheduling,’ in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA: ACM, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318
- [4] S. Ferson, V. Kreinovich, L. Ginzburg, D. S. Myers and K. Sentz, ‘Constructing probability boxes and Dempster–Shafer structures,’ Sandia National Laboratories, Albuquerque, NM, Tech. Rep. SAND2002-4015, 2003.
- [5] R. C. Williamson and T. Downs, ‘Probabilistic arithmetic I: Numerical methods for calculating convolutions and dependency bounds,’ *International Journal of Approximate Reasoning*, vol. 4, no. 2, pp. 89–158, 1990.
- [6] L. G. Valiant, ‘The complexity of enumeration and reliability problems,’ *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979. DOI: 10.1137/0208032
- [7] M. O. Ball, ‘Computational complexity of network reliability analysis: An overview,’ *IEEE Transactions on Reliability*, vol. R-35, no. 3, pp. 230–239, 1986. DOI: 10.1109/TR.1986.4335422

Bibliography

- [8] D.-H. Huang, C.-F. Huang and Y.-K. Lin, ‘Exact project reliability for a multi-state project network subject to time and budget constraints,’ *Reliability Engineering & System Safety*, vol. 195, p. 106 744, 2020. DOI: 10.1016/j.ress.2019.106744
- [9] Y. Tong, ‘New approaches for modeling and reliability assessment of infrastructure flow networks,’ Ph.D. dissertation, Georgia Institute of Technology, 2021.
- [10] S. Chanas and P. Zieliński, ‘The computational complexity of the criticality problems in a network with interval activity times,’ *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: 10.1016/S0377-2217(01)00048-0
- [11] D. Jones et al., ‘Conceptual design of a medical drone logistics network for Scotland,’ *International Journal of Logistics Research and Applications*, 2025, Submitted.
- [12] L. Xing and S. V. Amari, ‘Fault tree analysis,’ in *Handbook of Performability Engineering*, London: Springer London, 2008.
- [13] M. Ouyang, ‘Review on modeling and simulation of interdependent critical infrastructure systems,’ *Reliability Engineering & System Safety*, vol. 121, pp. 43–60, 2014. DOI: 10.1016/j.ress.2013.06.040
- [14] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley and S. Havlin, ‘Catastrophic cascade of failures in interdependent networks,’ *Nature*, vol. 464, no. 7291, pp. 1025–1028, 2010. DOI: 10.1038/nature08932
- [15] W. Liu and J. Li, ‘An improved cut-based recursive decomposition algorithm for reliability analysis of networks,’ *Earthquake Engineering and Engineering Vibration*, vol. 11, no. 1, pp. 1–10, 2012.
- [16] Y. Kim and W.-H. Kang, ‘Network reliability analysis of complex systems using a non-simulation-based method,’ *Reliability Engineering & System Safety*, vol. 110, pp. 80–88, 2013.

Bibliography

- [17] A. Satyanarayana and R. K. Wood, ‘A linear-time algorithm for computing K-terminal reliability in series-parallel networks,’ *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: 10.1137/0214057
- [18] E. F. Moore and C. E. Shannon, ‘Reliable circuits using less reliable relays,’ *Journal of the Franklin Institute*, vol. 262, no. 3, pp. 191–208, 1956. DOI: 10.1016/0016-0032(56)90559-2
- [19] A. Satyanarayana and M. K. Chang, ‘Network reliability and the factoring theorem,’ *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: 10.1002/net.3230130107
- [20] L. B. Page and J. E. Perry, ‘A practical implementation of the factoring theorem for network reliability,’ *IEEE Transactions on Reliability*, vol. 37, no. 3, pp. 259–267, 1988. DOI: 10.1109/24.3752
- [21] R. K. Wood, ‘A factoring algorithm using polygon-to-chain reductions for computing K-terminal network reliability,’ *Networks*, vol. 15, no. 2, pp. 173–190, 1985. DOI: 10.1002/net.3230150204
- [22] N. Robertson and P. D. Seymour, ‘Graph minors. II. Algorithmic aspects of treewidth,’ *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. DOI: 10.1016/0196-6774(86)90023-4
- [23] A. K. Goharshady and F. Mohammadi, ‘An efficient algorithm for computing network reliability in small treewidth,’ *Reliability Engineering & System Safety*, vol. 193, p. 106665, 2020. DOI: 10.1016/j.ress.2019.106665
- [24] G. S. Fishman, ‘A comparison of four Monte Carlo methods for estimating the probability of s–t connectedness,’ *IEEE Transactions on Reliability*, vol. 35, no. 2, pp. 145–155, 1986.
- [25] R. E. Bryant, ‘Graph-based algorithms for Boolean function manipulation,’ *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819
- [26] A. Rauzy, ‘New algorithms for fault trees analysis,’ *Reliability Engineering & System Safety*, vol. 40, no. 3, pp. 203–211, 1993.

Bibliography

- [27] R. E. Bryant and C. Meinel, ‘Ordered binary decision diagrams,’ in *Logic Synthesis and Verification*, Boston, MA: Springer, 2002.
- [28] A. P. Ershov, ‘On programming of arithmetic operations,’ *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, The origin of the hash-consing technique. DOI: 10.1145/368892.368907
- [29] J.-C. Filliâtre and S. Conchon, ‘Type-safe modular hash-consing,’ in *Proceedings of the ACM SIGPLAN Workshop on ML*, ACM, 2006, pp. 12–19. DOI: 10.1145/1159876.1159880
- [30] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Francisco, CA: Morgan Kaufmann, 1988.
- [31] S. L. Lauritzen and D. J. Spiegelhalter, ‘Local computations with probabilities on graphical structures and their application to expert systems,’ *Journal of the Royal Statistical Society, Series B*, vol. 50, no. 2, pp. 157–224, 1988.
- [32] Y. Tong and I. Tien, ‘Probability propagation method for reliability assessment of acyclic directed networks,’ *ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering*, vol. 5, no. 3, p. 04019011, 2019. DOI: 10.1061/AJRUA6.0001017
- [33] L. R. Ford and D. R. Fulkerson, *Flows in Networks*. Princeton, NJ: Princeton University Press, 1962.
- [34] Z. W. Birnbaum, ‘On the importance of different components in a multicomponent system,’ in *Multivariate Analysis II*, P. R. Krishnaiah, Ed., New York: Academic Press, 1969, pp. 581–592.
- [35] D. Bertsimas, E. Nasrabadi and S. Stiller, ‘Robust and adaptive network flows,’ *Operations Research*, vol. 61, no. 5, pp. 1218–1242, 2013. DOI: 10.1287/opre.2013.1200
- [36] P. Doulliez and E. Jamouille, ‘Transportation networks with random arc capacities,’ *Revue française d’automatique, informatique, recherche opérationnelle. Recher-*

Bibliography

- che opérationnelle*, vol. 6, no. V3, pp. 45–59, 1972. [Online]. Available: https://www.numdam.org/item/R0_1972__6_3_45_0/
- [37] D. G. Malcolm, J. H. Roseboom, C. E. Clark and W. Fazar, ‘Application of a technique for research and development program evaluation,’ *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: 10.1287/opre.7.5.646
- [38] S. Chanas and P. Zieliński, ‘On the hardness of evaluating criticality of activities in a planar network with duration intervals,’ *Operations Research Letters*, vol. 31, no. 1, pp. 53–59, 2003.
- [39] D. Dubois, H. Fargier and J. Fortin, ‘Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks,’ *Journal of Intelligent Manufacturing*, vol. 16, pp. 407–421, 2005. DOI: 10.1007/s10845-005-1654-5
- [40] J. Fortin, P. Zieliński, D. Dubois and H. Fargier, ‘Criticality analysis of activity networks under interval uncertainty,’ *Journal of Scheduling*, vol. 13, pp. 609–627, 2010. DOI: 10.1007/s10951-010-0163-3
- [41] J. Valdes, R. E. Tarjan and E. L. Lawler, ‘The recognition of series parallel digraphs,’ *SIAM Journal on Computing*, vol. 11, no. 2, pp. 298–313, 1982. DOI: 10.1137/0211023
- [42] F. Baccelli, G. Cohen, G. J. Olsder and J.-P. Quadrat, *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Chichester: John Wiley & Sons, 1992.
- [43] S. Ferson, *Pba.r: Probability bounds analysis in R*, <https://github.com/ScottFerson/pba.r>, 2019.
- [44] N. Gray, S. Ferson, M. De Angelis, A. Gray and F. Baumont de Oliveira, ‘Probability bounds analysis for Python,’ *Software Impacts*, vol. 12, p. 100246, 2022. DOI: 10.1016/j.simpa.2022.100246

Bibliography

- [45] A. Gray, S. Ferson and E. Patelli, *ProbabilityBoundsAnalysis.jl: Arithmetic with sets of distributions*, Proceedings of JuliaCon, 2021. [Online]. Available: <https://github.com/Institute-for-Risk-and-Uncertainty/ProbabilityBoundsAnalysis.jl>
- [46] L. A. Zadeh, ‘Fuzzy sets,’ *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [47] G. Shafer, *A Mathematical Theory of Evidence*. Princeton, NJ: Princeton University Press, 1976.
- [48] L. Thurner et al., ‘Pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems,’ *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018. DOI: 10.1109/TPWRS.2018.2829021
- [49] Esri, *ArcGIS utility network: Trace types*, <https://doc.esri.com/en/arcgis-pro/latest/help/data/utility-network/utility-network-trace-types.html>, 2024.
- [50] Hottinger Brüel & Kjær (ReliaSoft). ‘BlockSim: System reliability, availability, and maintainability analysis.’
- [51] Isograph Ltd. ‘Availability Workbench.’
- [52] ITEM Software. ‘ITEM ToolKit: Reliability analysis and safety software tools.’
- [53] S. Balakrishnan and B. Cassottana, ‘InfraRisk: An open-source simulation platform for resilience analysis in interconnected power–water–transport networks,’ *Sustainable Cities and Society*, vol. 83, p. 103 963, 2022. DOI: 10.1016/j.scs.2022.103963
- [54] J. E. Hannay, C. MacLeod, J. Singer, H. P. Langtangen, D. Pfahl and G. Wilson, ‘How do scientists develop and use scientific software?’ In *Proceedings of the 2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, IEEE, 2009, pp. 1–8. DOI: 10.1109/SECSE.2009.5069155

Bibliography

- [55] P. Prabhu et al., ‘A survey of the practice of computational science,’ in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11, New York, NY: ACM, 2011, pp. 1–12. DOI: 10.1145/2063348.2063374
- [56] A. Abendroth, ‘The democratization of software engineering: Evolution, definition, and the future of low-code and no-code platforms,’ *Management Review Quarterly*, vol. 67, 2026. DOI: 10.1007/s11301-026-00603-2
- [57] SimScale GmbH, *SimScale: AI-native engineering simulation software in the cloud*, <https://www.simscale.com/>, 2026.
- [58] M. Daud, F. M. Ugliotti and A. Osello, ‘Comprehensive analysis of the use of web-gis for natural hazard management: A systematic review,’ *Sustainability*, vol. 16, no. 10, p. 4238, 2024. DOI: 10.3390/su16104238
- [59] M. P. Repetto, M. Burlando, G. Solari, P. De Gaetano, M. Pizzo and M. Tizzi, ‘A web-based GIS platform for the safe management and risk assessment of complex structural and infrastructural systems exposed to wind,’ *Advances in Engineering Software*, vol. 117, pp. 29–45, 2018. DOI: 10.1016/j.advengsoft.2017.03.002
- [60] M. Bastian, S. Heymann and M. Jacomy, ‘Gephi: An open source software for exploring and manipulating networks,’ in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, AAAI Press, 2009, pp. 361–362. DOI: 10.1609/icwsm.v3i1.13937
- [61] P. Shannon et al., ‘Cytoscape: A software environment for integrated models of biomolecular interaction networks,’ *Genome Research*, vol. 13, no. 11, pp. 2498–2504, 2003. DOI: 10.1101/gr.1239303
- [62] E. Patelli, H. George-Williams, J. Sadeghi, R. Rocchetta, M. Broggi and M. de Angelis, ‘OpenCossan 2.0: An efficient computational toolbox for risk, reliability and resilience analysis,’ in *Proceedings of the Joint ICVRAM ISUMA UNCERTAINTIES Conference*, A. T. Beck, G. F. M. de Souza and M. A. Trindade, Eds., 2018.

Bibliography

- [63] K. Hashizume, D. G. Rosado, E. Fernández-Medina and E. B. Fernández, ‘An analysis of security issues for cloud computing,’ *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1–13, 2013. DOI: 10.1186/1869-0238-4-5
- [64] M. Kleppmann, A. Wiggins, P. van Hardenberg and M. McGranaghan, ‘Local-first software: You own your data, in spite of the cloud,’ in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, ser. Onward! 2019, ACM, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737
- [65] The MathWorks, Inc., *Graph and network algorithms*, <https://www.mathworks.com/help/matlab/graph-and-network-algorithms.html>, 2026.
- [66] G. Csárdi and T. Nepusz, ‘The igraph software package for complex network research,’ *InterJournal, Complex Systems*, p. 1695, 2006.
- [67] A. A. Hagberg, D. A. Schult and P. J. Swart, ‘Exploring network structure, dynamics, and function using NetworkX,’ in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, G. Varoquaux, T. Vaught and J. Millman, Eds., Pasadena, CA, 2008, pp. 11–15. DOI: 10.25080/TCWV9851
- [68] petgraph contributors, *Petgraph: Graph data structure library for Rust*, <https://github.com/petgraph/petgraph>, 2026.
- [69] J. Bezanson, A. Edelman, S. Karpinski and V. B. Shah, ‘Julia: A fresh approach to numerical computing,’ *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: 10.1137/141000671
- [70] L. J. M. Aslett, *ReliabilityTheory: Tools for structural reliability analysis*, <https://cran.r-project.org/package=ReliabilityTheory>, 2024.
- [71] V. Gaur, O. P. Yadav, G. Soni and A. P. S. Rathore, ‘A literature review on network reliability analysis and its engineering applications,’ *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, vol. 235, no. 2, pp. 167–181, 2021.

Bibliography

- [72] H. Pérez-Rosés, ‘Sixty years of network reliability,’ *Mathematics in Computer Science*, vol. 12, pp. 275–293, 2018. DOI: 10.1007/s11786-018-0345-5
- [73] T. Ohiani and E. Patelli, ‘The Information Propagation Method for Efficient Network Reliability Analysis,’ in *2023 7th International Conference on System Reliability and Safety (ICSRS)*, IEEE, Nov. 2023, pp. 580–584. DOI: 10.1109/ICSRS59833.2023.10381157
- [74] R. Ahuja, R. Ahuja, T. Magnanti and J. Orlin, *Network Flows: Theory, Algorithms, and Applications* (Always learning). Pearson, 2014, ISBN: 978-1-292-04270-1. [Online]. Available: <https://books.google.co.uk/books?id=rFuLNgEACAAJ>
- [75] J. Edmonds and R. M. Karp, ‘Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems,’ *Journal of the ACM (JACM)*, vol. 19, pp. 248–264, 1972. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6375478>
- [76] JuliaGraphs contributors, *GraphsFlows.jl: Flow algorithms on top of Graphs.jl*, <https://github.com/JuliaGraphs/GraphsFlows.jl>, 2024.
- [77] Reliability Test System Task Force of the Application of Probability Methods Subcommittee, ‘IEEE reliability test system,’ *IEEE Transactions on Power Apparatus and Systems*, vol. PAS-98, no. 6, pp. 2047–2054, 1979. DOI: 10.1109/TPAS.1979.319398
- [78] C. Grigg et al., ‘The IEEE reliability test system-1996. a report prepared by the reliability test system task force of the application of probability methods subcommittee,’ *IEEE Transactions on Power Systems*, vol. 14, no. 3, pp. 1010–1020, 1999. DOI: 10.1109/59.780914
- [79] R. D. Zimmerman, C. E. Murillo-Sánchez and R. J. Thomas, ‘MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education,’ *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, 2011. DOI: 10.1109/TPWRS.2010.2051168

Bibliography

- [80] D. S. Johnson and C. C. McGeoch, Eds., *Network Flows and Matching: First DIMACS Implementation Challenge* (DIMACS Series in Discrete Mathematics and Theoretical Computer Science). American Mathematical Society, 1993, vol. 12.
- [81] R. Kolisch and A. Sprecher, ‘PSPLIB – a project scheduling problem library,’ *European Journal of Operational Research*, vol. 96, no. 1, pp. 205–216, 1997. DOI: 10.1016/S0377-2217(96)00170-1
- [82] T. Ohiani, *InformationPropagationAnalysis.jl*, Julia General registry, git-tree-sha1 adcdacd7cc306d8ffe15e440ff0c7b72ce6f024a, version 0.2.1, <https://juliahub.com/ui/Packages/General/InformationPropagationAnalysis>, 2026.
- [83] T. Ohiani, *Information-propagation-no-code: No-code workbench for exact information-propagation analysis on DAGs — server + UI over InformationPropagationAnalysis.jl*, GitHub repository, version 1.0, <https://github.com/Temi-Tory/information-propagation-no-code>, 2026. DOI: 10.5281/zenodo.22180253
- [84] T. Ohiani, *Thesis-data: Validation and case-study data for Information Propagation Methods for Reliability, Reachability and Flow Analysis in Directed Acyclic Process Networks*, GitHub repository, archived on Zenodo, version 1.0, <https://github.com/Temi-Tory/thesis-data>, 2026. DOI: 10.5281/zenodo.22180227
- [85] L. A. Rossman, ‘EPANET 2 users manual,’ U.S. Environmental Protection Agency, National Risk Management Research Laboratory, Cincinnati, OH, Tech. Rep. EPA/600/R-00/057, 2000.
- [86] E. Butts, *Reliability in water and pumping systems*, *Water Well Journal*, Jul. 2022.
- [87] S. L. Barfuss, ‘Water main break rates in the USA and Canada: A comprehensive study,’ Utah Water Research Laboratory, Utah State University, Tech. Rep., Dec. 2023, Successor to the 2018 Folkman study (same series); surveyed 800+ utilities, 400,000 miles of water main.
- [88] *AWWA C600-17: Installation of ductile-iron water mains and their appurtenances*, American Water Works Association, 2017.

Bibliography

- [89] *AWWA C605-17: Underground installation of polyvinyl chloride (PVC) and molecularly oriented polyvinyl chloride (PVCO) pressure pipe and fittings*, American Water Works Association, 2017.
- [90] *AWWA C651-14: Disinfecting water mains*, American Water Works Association, 2014.
- [91] J. Behrendorf, T.-E. Regenhardt, M. Broggi and M. Beer, ‘Numerically efficient computation of the survival signature for the reliability analysis of large networks,’ *Reliability Engineering & System Safety*, vol. 216, p. 107935, 2021. DOI: 10.1016/j.ress.2021.107935
- [92] M. Scutari, ‘Learning Bayesian networks with the bnlearn R package,’ *Journal of Statistical Software*, vol. 35, no. 3, pp. 1–22, 2010. DOI: 10.18637/jss.v035.i03